

Spec-Driven Development

Professional AI Edition — a 300-page guide to EARS, DDD, BDD, Gherkin, TDD, architecture, coding agents, independent verification, security, and delivery evidence.



Spec-Driven Development

How structured intent, domain language, executable contracts, AI coding agents, and independent evidence can form a traceable software-delivery system.

Core thesis. AI does not remove the need for requirements, architecture, or tests. It increases the value of making those artifacts explicit because weak intent can now be transformed into a large amount of plausible code at machine speed.

INTENT

EARS requirements, domain rules, examples, non-functional constraints, and explicit exclusions.

EXECUTION

Plans, vertical tasks, controlled tools, sandboxes, and coding-agent work packets.

PROOF

Independent tests, mutation analysis, review evidence, and operational confirmation.

Hekuran Islamaj

Author · software engineering research and practitioner guide

AUTHOR STATEMENT

Author's note

This edition keeps the end-to-end pipeline and research basis of the previous book, but replaces the repetitive AI section with a practical, evidence-aware operating model.

The starting problem remains simple: a short ticket can appear precise while hiding authority, policy, timing, data, and failure assumptions. The PLT-4821 walkthrough follows one partial-refund feature from ambiguous request to EARS, Domain-Driven Design, tasks, architecture, BDD, TDD, implementation, verification, and delivery measurement.

The redesigned edition treats AI as a participant in that lifecycle—not as a magical final step. Coding agents receive bounded context, declared permissions, deterministic gates, and an independent test oracle. Humans remain accountable for domain truth, risk acceptance, and merge decisions.

“Spec first” is not a request for more prose. It is a demand that intent become reviewable, executable, and capable of proving the implementation wrong.

What changed

- ✓ Professional cover, typography, navigation, and part openers
- ✓ New AI-agent engineering section with worked prompts and controls
- ✓ Explicit treatment of oracle leakage, groupthink, permissions, and security
- ✓ Balanced productivity evidence rather than universal speed claims
- ✓ NIST-aligned governance and a 30/60/90 adoption roadmap

How to read it

Use the early chapters for foundations; the middle for detailed methods; Part XII for AI-enabled execution; the case studies for application; and the closing parts for research design, measurement, templates, and sources.

RESEARCH SYNTHESIS

Abstract

Spec-Driven Development (SDD) is examined as an emerging umbrella for workflows in which structured specifications guide planning, task decomposition, implementation, and verification, increasingly with AI coding agents. The book separates this modern packaging from established requirements engineering, EARS, Domain-Driven Design, Behavior-Driven Development, Gherkin, Specification by Example, Test-Driven Development, traceability, and mutation testing.

The central proposal is to treat SDD as a governed evidence graph. Domain decisions and requirements are given stable identifiers; examples and contracts make behavior concrete; architecture constrains the solution space; tasks bound change; agents execute inside declared permissions; and independent tests, review, and operational signals challenge the result. The PLT-4821 partial-refund case is retained and extended with healthcare, fintech, retail, SaaS, logistics, and brownfield examples.

The AI section develops context packets, agent-role separation, deterministic gates, tool security, conformance scoring, productivity measurement, and governance. Empirical findings are qualified by task, user, codebase, and evaluation design: fixed-task experiments, repository benchmarks, field studies, and vendor reports answer different questions. The result is a practical framework for using AI to accelerate bounded engineering work without transferring authority from approved artifacts to generated output.

REQUIREMENTS ENGINEERING

EXECUTABLE CONTRACTS

CODING AGENTS

INDEPENDENT VERIFICATION

SOURCE TRANSPARENCY

Primary source basis

The book is grounded in two author-supplied documents and the earlier 121-page guide. The first document is an end-to-end walkthrough of PLT-4821; the second is a slide-by-slide speech that teaches the same method to delivery teams. Their terminology and narrative spine are retained. Research additions are separated by section labels and evidence bands.

SOURCE A

SDD-walkthrough-01.md

Feature -> EARS -> DDD -> tasks -> architecture -> BDD -> TDD -> implementation -> verification -> DORA/SPACE/DevEx/GSM.

SOURCE B

SDD-speech-per-slide(1).md

A 30-slide teaching narrative on ambiguity cost, workflow, roles, agent context, frameworks, measurement, and commitments.

PRIOR EDITION

Spec-Driven Development Deep Dive

The 121-page research and practitioner guide whose strongest body chapters are preserved as pages 13-129 of this edition.

EVIDENCE POLICY

Research integrity and evidence policy

Claim class	How it is treated in this book
Standard / official specification	Used to define normative processes, syntax, or current product behavior; not treated as outcome proof.
Primary empirical study	Reported with design, population, task context, and uncertainty where available.
Systematic review / meta-analysis	Used to synthesize patterns and disagreements; heterogeneity remains visible.
Book / foundational article	Used for concepts and practice lineage, not for universal causal claims.
Vendor documentation	Used to describe a workflow or tool capability at the stated version.
Worked example	Clearly labeled illustrative; numbers demonstrate calculation or instrumentation, not real organizational performance.
Inference / recommendation	Presented as reasoned synthesis and separated from directly observed evidence.

CRITICAL REVIEW

Evidence corrections and qualifications

Simplified claim	Research-edition treatment
SDD is one established method	Modern AI-oriented SDD is an emerging umbrella. Toolchains differ in artifacts, gates, persistence, and authority.
TDD reduces defects by 40-90%	That range comes from four industrial case studies. Meta-analytic evidence is more mixed and context-dependent.
AI always makes developers faster	Controlled studies report different effects by task, developer, tooling, and study design; later estimates also face selection effects.
DORA has four metrics	The classic four remain historically important. DORA moved to a five-metric model in 2024 by adding deployment rework rate.
BDD equals Gherkin automation	BDD is collaboration and discovery; Gherkin is one representation; automation is a later practice.
EARS removes ambiguity	EARS improves sentence structure but cannot guarantee correct policy, completeness, or stakeholder agreement.
A prompt can enforce scope	Only permissions, hooks, sandboxes, and CI gates enforce. A prompt can request or persuade.
PLT-4821 dashboard proves improvement	The numbers are a worked pilot illustration unless backed by real, retained organizational data.

CONCEPTUAL MODEL

The integrated method map

CONCEPT DIAGRAM



Method	Primary uncertainty removed	Primary artifact
DDD	What the domain means and where responsibility belongs	Ubiquitous language, context map, model
EARS	What event/state/condition requires what response	Structured requirement
BDD/Gherkin	Which examples make behavior concrete across roles	Rules and scenarios
TDD	Whether the implementation satisfies a small executable expectation	Failing/passing test and design increment
SDD	Whether the whole artifact chain remains aligned	Plan, tasks, contracts, verification, trace links

DIRECT ANSWER

Executive summary

- 1 SDD is most useful as a governed lifecycle, not a fashionable synonym for planning before coding.
- 2 Requirements quality is the first control point. EARS exposes trigger, state, exception, and optionality slots, but domain truth still requires people.
- 3 DDD gives words stable meaning; BDD and examples reveal missing rules; TDD supplies rapid local feedback; traceability holds the chain together.
- 4 AI agents amplify both clarity and ambiguity. Use small task packets, independent oracles, path/tool permissions, and non-bypassable gates.
- 5 Verification must include operational evidence, not only tests: rollout, observability, security, accessibility, reconciliation, and recovery.
- 6 Measure trends and mechanisms. Use DORA for delivery, SPACE and DevEx for human/system balance, and GSM to keep metrics subordinate to goals.
- 7 For research, specification mutation is a promising way to test whether semantic changes are detected and propagated accurately.

DETAILED TABLE OF CONTENTS

Contents I

FRONT MATTER

Author's note · Abstract · Source basis · Evidence policy · Critical review · Method map · Executive summary 1–9

PART I — FOUNDATIONS

- 1. What Spec-Driven Development actually is 13
- 2. The integrated method map 16
- 3. Requirements quality and artifact thinking 19

PART II — RELATED METHODS IN DEPTH

- 4. Domain-Driven Design 23
- 5. EARS requirements syntax 27
- 6. Behavior-Driven Development 32
- 7. Gherkin as a scenario language 37
- 8. Specification by Example 43
- 9. Test-Driven Development 47

PART III — INTEGRATED SDD PRACTICE

- 10. A complete SDD lifecycle 53
- 11. End-to-end case study: returns and refunds 59
- 12. Brownfield change and regression control 79

PART IV — QUALITY, EVIDENCE, AND RESEARCH

- 13. Traceability and change-impact analysis 84
- 14. Specification mutation analysis 89
- 15. Metrics and experimental design 96
- 16. Governance and adoption 102

PART V — REUSABLE MATERIAL

- 17. Templates and review checklists 109
- 18. Synthesis and glossary 120
- 19. Annotated reading list 125

DETAILED TABLE OF CONTENTS

Contents II

PART VI — EVIDENCE AND DEFINITIONS		
Evidence classes, definitions, and reading rules	130	
PART VII — ADVANCED REQUIREMENTS AND EARS		
Authority, ambiguity, EARS patterns, and measurable constraints	142	
PART VIII — DOMAIN MODELING AND DDD		
Ubiquitous language, boundaries, aggregates, and context maps	159	
PART IX — BDD, GHERKIN, AND EXAMPLES		
Discovery, scenarios, executable examples, and living documentation	175	
PART X — TDD AND TEST STRATEGY		
Red–green–refactor, seams, oracles, and layered evidence	191	
PART XI — ARCHITECTURE, CONTRACTS, AND TRACEABILITY		
Hexagonal design, APIs, events, consumer contracts, and change impact	208	
PART XII — AI-ERA SDD AND AGENT GOVERNANCE		
Why AI changes the bottleneck	228	
AI-assisted SDD control loop	229	
Context engineering and the PLT-4821 task packet	230–231	
Prompts, permissions, hooks, and gates	232	
Agent roles · single-agent · multi-agent workflows	233–235	
Independent oracles · generated requirements · generated tests	236–238	
MCP, least privilege, and agent security	239–240	
Conformance evaluation and productivity evidence	241–242	
Governance and 30/60/90 adoption	243–244	
PARTS XIII–XVIII		
PLT-4821 case · additional cases · mutation research · metrics · templates · sources	245–300	

READING PATHS

How to use this edition

The book is intentionally modular. Start with the path that matches the decision you need to make, then follow identifiers and page references into the rest of the evidence chain.

Delivery leader

Pages 7–9, 52–58, 96–108, and 287–294. Focus on entry gates, task size, review capacity, metrics, and adoption.

Product owner / analyst

Pages 19–46 and 142–190. Focus on authority, EARS, examples, open questions, and business acceptance.

Engineer / architect

Pages 23–58, 191–244, and 245–262. Focus on boundaries, contracts, test seams, task packets, and agent controls.

QA / test engineer

Pages 32–51, 84–101, 175–244, and 277–286. Focus on independent oracles, traceability, mutation, and conformance.

AI engineering lead

Pages 227–244, then 84–101 and 287–294. Focus on context packets, permissions, role separation, security, evaluation, and governance.

Researcher / student

Pages 5–9, 89–101, 130–141, 227–244, and 277–300. Focus on evidence classes, mixed empirical findings, research gaps, and methods.

Fastest practical route: read the integrated lifecycle, the PLT-4821 case, and Part XII. Then use the templates to create one feature packet and evaluate whether the artifacts actually constrain implementation.

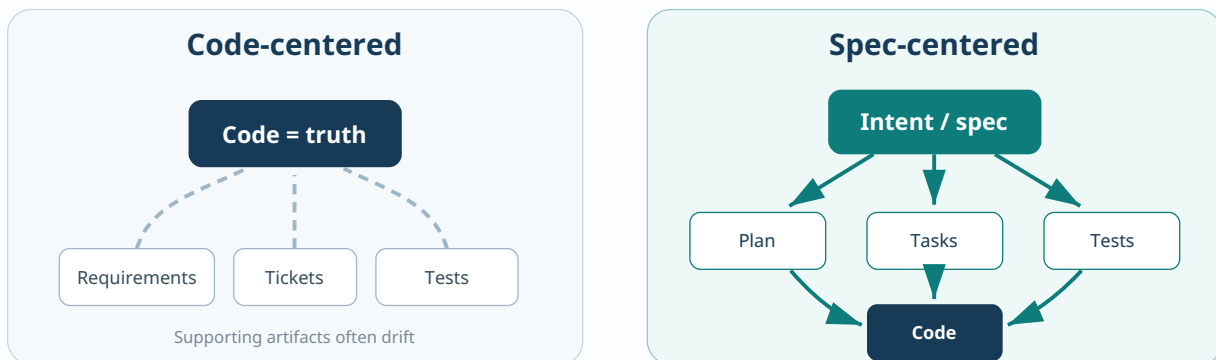
1 What Spec-Driven Development actually is

Spec-Driven Development (SDD) is a way of organizing software work so that an explicit, reviewable specification drives downstream design, implementation, and verification. The specification is not merely a document written before coding; it is maintained as a living control surface for the system.

1.1 The power inversion

Traditional projects often claim that requirements are important, yet day-to-day truth gradually moves into code, tickets, chat threads, and the memories of individual engineers. When the written intent and the running system diverge, people usually trust the running system. Modern SDD attempts to reverse that hierarchy: intent is made explicit enough that plans, tasks, tests, and implementations can be checked against it. GitHub Spec Kit describes a core process of **Spec → Plan → Tasks → Implement**, with each phase producing an artifact that becomes structured context for the next phase [1].

Diagram 1. Code-centered development versus spec-centered development



The goal is not to diminish code. It is to make the reasons for the code visible, reviewable, and testable.

1.2 SDD is a lifecycle, not a file format

A project does not become spec-driven because it contains a file named `spec.md`. The method requires a chain of disciplined activities:

1. **Discover intent.** Understand the business problem, actors, policies, constraints, risks, and definitions.
2. **Express behavior.** State what must be observable, including unhappy paths, boundaries, and non-functional expectations.

3. **Resolve ambiguity.** Ask questions before architecture or code silently chooses an interpretation.
4. **Design a solution.** Record data, interfaces, boundaries, trade-offs, and quality strategies.
5. **Decompose work.** Generate tasks whose completion can be demonstrated against acceptance criteria.
6. **Implement with feedback.** Use tests and small increments to reduce the distance between intent and working software.
7. **Verify and maintain alignment.** Retain traceability as requirements change and production evidence accumulates.

A useful test

Ask: “When requirement R-17 changes, can we identify the affected rule, scenarios, design decisions, tasks, tests, code components, and operational controls without relying on memory?” If not, the process may be document-heavy, but it is not yet strongly spec-driven.

1.3 What SDD is not

NOT THIS

- A giant up-front document that freezes learning.
- Unreviewed AI-generated prose.
- A prompt transcript treated as a contract.
- Automatic code generation without verification.
- Every decision written at the same level of detail.

PREFER THIS

- Progressive elaboration and versioning.
- Explicit review gates and named owners.
- Stable identifiers and traceable artifacts.
- Tests as evidence, not merely coverage numbers.
- Detail proportional to risk and reversibility.

1.4 Three levels of specification

Level	Question answered	Typical artifacts	Main failure when missing
Intent	Why does this capability exist, and what outcome matters?	Problem statement, goals, actors, domain glossary, policies, constraints	Teams deliver a technically correct solution to the wrong problem.
Behavior	What must users and collaborating systems observe?	EARS requirements, business rules, examples, Gherkin scenarios, NFRs	Different stakeholders carry different interpretations of “done.”
Realization	How will the behavior be implemented and proven?	Architecture, APIs, data models, task plan, tests, observability and rollout plan	Implementation choices are opaque and cannot be connected to requirements.

1.5 A minimal working definition

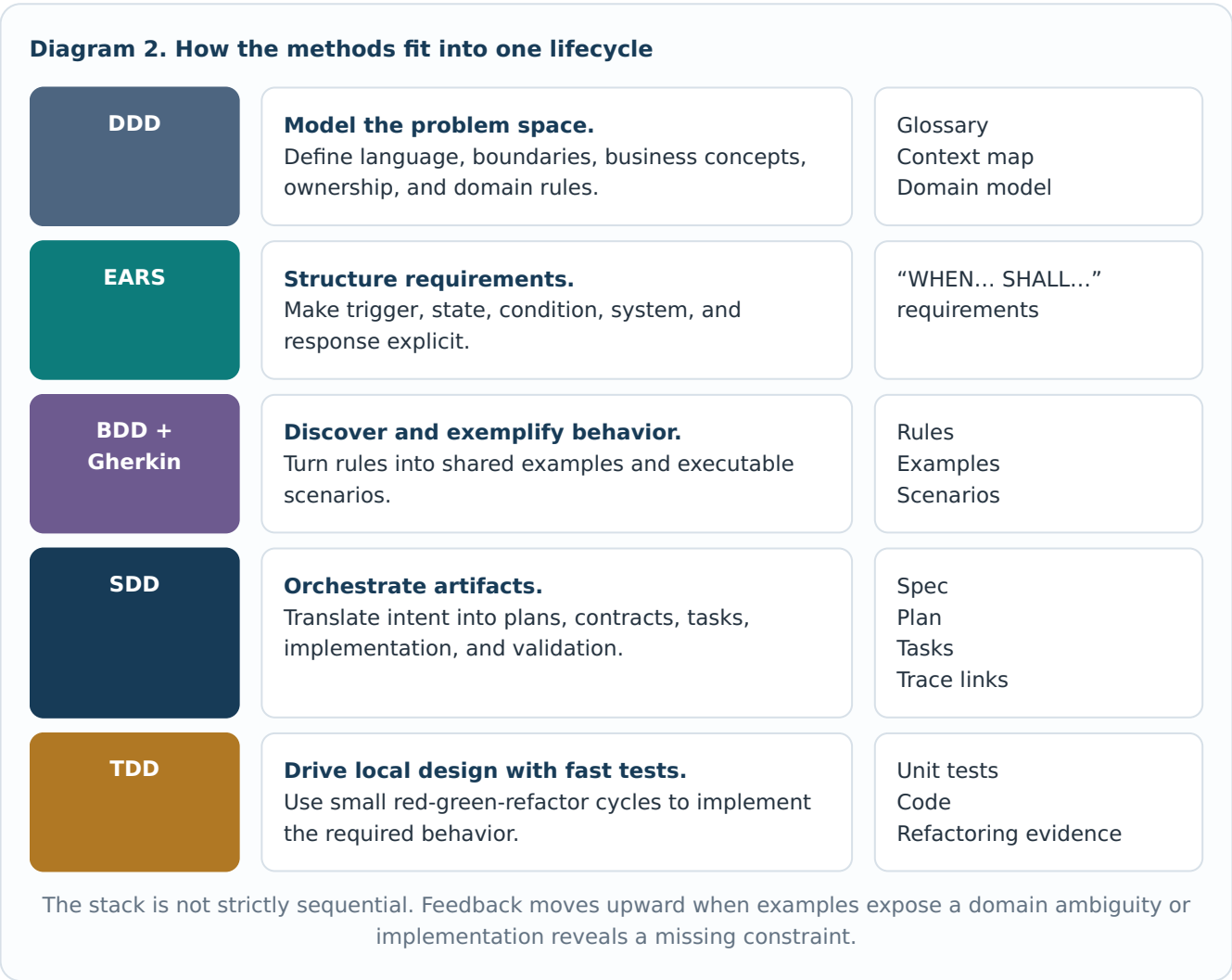
Spec-Driven Development is a development system in which explicit, versioned intent is progressively refined into behavior, design, implementation work, and verification evidence, while preserving enough traceability to evaluate alignment and the impact of change.

This definition intentionally does not require a specific tool, AI model, programming language, or notation. It allows EARS, Gherkin, formal models, API schemas, examples, property-based tests, and ordinary prose to coexist when each is used for the problem it expresses best.

2 The integrated method map

DDD, EARS, BDD, Gherkin, Specification by Example, TDD, and SDD are related, but they are not interchangeable. Each solves a different class of problem. Combining them is valuable precisely because their responsibilities are distinct.

2.1 The stack from domain to evidence



2.2 Comparison by primary concern

Approach	Primary concern	Primary artifact	Typical granularity	Contribution to SDD
DDD	Business complexity and model boundaries	Domain model and ubiquitous language	Product / bounded context	Prevents the spec and AI agent from inventing inconsistent concepts.

Approach	Primary concern	Primary artifact	Typical granularity	Contribution to SDD
EARS	Clear natural-language requirements	Structured “shall” statement	Individual requirement	Improves precision and supports systematic review or transformation.
BDD	Shared understanding of observable behavior	Rules and concrete examples	Feature / business rule	Connects product intent to acceptance evidence through conversation.
Gherkin	Consistent scenario expression	Feature / Rule / Scenario / Given-When-Then	Example or scenario	Provides a parseable, tool-friendly representation of examples.
Specification by Example	Concrete, validated, living specifications	Illustrative examples and automated acceptance checks	Business rule / workflow	Reduces abstract ambiguity and keeps examples close to verification.
TDD	Implementation feedback and design	Failing unit-level test	Class / function / component	Turns selected behaviors into small design increments and regression protection.
SDD	End-to-end intent orchestration	Connected specification artifact set	Feature / system / program	Maintains the chain from desired outcome to implementation and evidence.

2.3 Why no single technique is sufficient

Precise but meaningless

A perfectly formatted EARS requirement can still encode the wrong business policy. DDD discovery and stakeholder validation remain necessary.

Readable but shallow

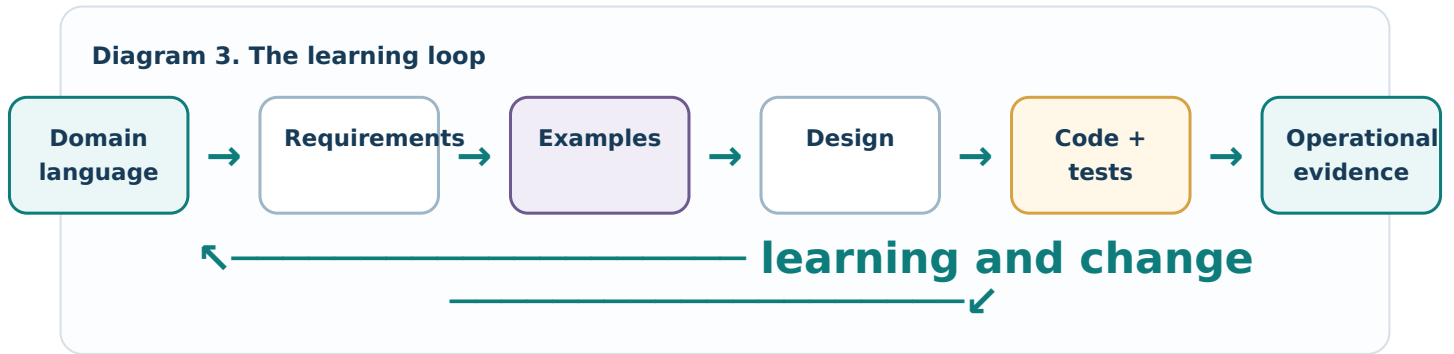
A Gherkin scenario can be syntactically valid yet omit business rules, boundary values, security constraints, or architectural implications.

Tested but misaligned

TDD can produce clean, well-tested code that faithfully implements an incorrect assumption. Tests prove their assertions, not the correctness of the requirement.

2.4 Feedback loops are a feature, not a process failure

Linear diagrams are convenient, but effective specification work is iterative. A concrete example may reveal a missing domain term. A design spike may show that a requirement is too expensive. A unit test may reveal that one concept has too many responsibilities. Production telemetry may show that the acceptance threshold was unrealistic. SDD should make these discoveries visible and route them back to the right artifact.



3 Requirements quality and artifact thinking

An AI coding agent can produce a fluent plan from a weak requirement. Fluency is not evidence of correctness. SDD therefore needs explicit quality criteria for both individual requirements and the relationships among artifacts.

3.1 Quality dimensions for an individual requirement

Dimension	Review question	Typical defect	Improvement technique
Necessary	Does this requirement contribute to an agreed outcome or constraint?	Inherited feature with no current purpose	Link to goal, stakeholder, regulation, or risk
Unambiguous	Could two reasonable readers implement it differently?	“quickly,” “appropriate,” “normally”	EARS structure, glossary, examples, thresholds
Atomic	Can it be implemented, changed, and verified independently?	Multiple behaviors joined by “and”	Split and assign stable identifiers
Feasible	Can it be satisfied under known technical, legal, and economic constraints?	Absolute availability or latency promise	Architecture validation and measurable budget
Verifiable	What observable evidence would demonstrate satisfaction?	“The interface shall be intuitive”	Usability measure, scenario, test oracle
Consistent	Does it contradict another requirement or domain rule?	Refund in 5 days vs 7 days elsewhere	Cross-artifact analysis and rule catalog
Traceable	Can we identify its origin and downstream realization?	Requirement exists only in a meeting note	IDs, source links, trace matrix, repository history

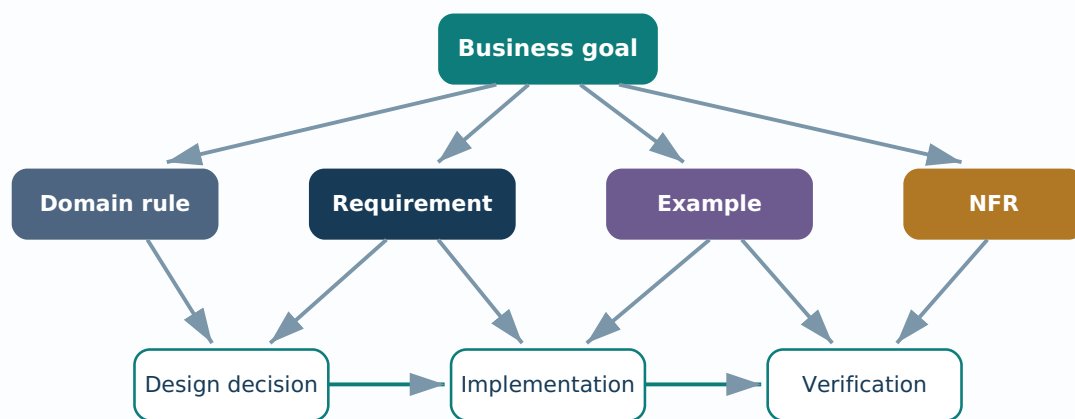
ISO/IEC/IEEE 29148 provides a standards-based foundation for requirements engineering and remains current after review in 2024 [6]. SDD does not replace requirements engineering; it raises the value of requirement quality because downstream AI-assisted transformations can amplify both clarity and defects.

3.2 Artifact quality is relational

A requirement can be well written and still be ineffective if no test checks it. A test can be excellent and still be irrelevant if it is not connected to an agreed behavior. This means quality must be evaluated at two levels:

- **Intrinsic quality:** Is the artifact clear, internally coherent, and fit for its notation?
- **Relational quality:** Are the links to upstream intent and downstream evidence complete, correct, and current?

Diagram 4. A specification is an evidence graph



3.3 Stable identifiers and controlled vocabulary

Stable IDs allow tools and people to discuss the same object even when wording changes. A practical naming scheme might be:

Prefix	Artifact	Example
GOAL	Business outcome	GOAL-RET-01: reduce manual return handling
TERM	Domain definition	TERM-RET-04: Return Window
RULE	Business policy	RULE-RET-07: eligibility period is 30 calendar days
REQ	System requirement	REQ-RET-012: reject late requests
SCN	Example/scenario	SCN-RET-018: request on day 31
ADR	Architecture decision	ADR-RET-003: event-driven refund initiation
TASK	Implementation unit	TASK-RET-024: add eligibility policy object
TEST	Automated or manual evidence	TEST-RET-031: boundary test for day 30/day 31

3.4 Progressive elaboration

Not every artifact should be complete on day one. A safer pattern is to elaborate just before irreversible or high-risk decisions. Early specifications should be precise about *outcomes*, *policies*, *boundaries*, and *risks*, while postponing low-level implementation details. Before implementation begins, the selected slice should be sufficiently detailed to support planning, examples, and test oracles.

Definition of “ready for planning”

The problem and actors are named; terms are defined; core rules and boundary conditions are explicit; unresolved questions are visible; acceptance examples cover happy, unhappy, and edge paths; major non-functional constraints are measurable; and the team agrees on what is intentionally out of scope.

PART II

Related methods in depth

Each method strengthens a different layer of the specification-to-evidence chain.

4 Domain-Driven Design (DDD)

Domain-Driven Design focuses software development on a rich understanding of the business domain. Its most important contribution to SDD is not a particular code pattern; it is the discipline of building a shared language and preserving meaningful boundaries before requirements are translated into architecture or generated code.

4.1 Why DDD matters to SDD

An AI agent is highly capable of producing plausible names. That is useful when the domain is simple, but dangerous when terms have contractual, financial, medical, or operational meaning. Consider the words *return*, *refund*, *reversal*, and *chargeback*. In casual conversation they may overlap. In a marketplace they can belong to different workflows, owners, state machines, accounting treatments, and legal obligations.

DDD reduces this ambiguity through a **ubiquitous language**: a deliberately maintained vocabulary used in conversation, specifications, tests, interfaces, and code. Eric Evans' foundational text treats the model and language as mutually reinforcing; modern SDD can extend the same idea into AI context, artifact templates, and automated checks [14].

4.2 Strategic DDD: boundaries before components

Subdomains

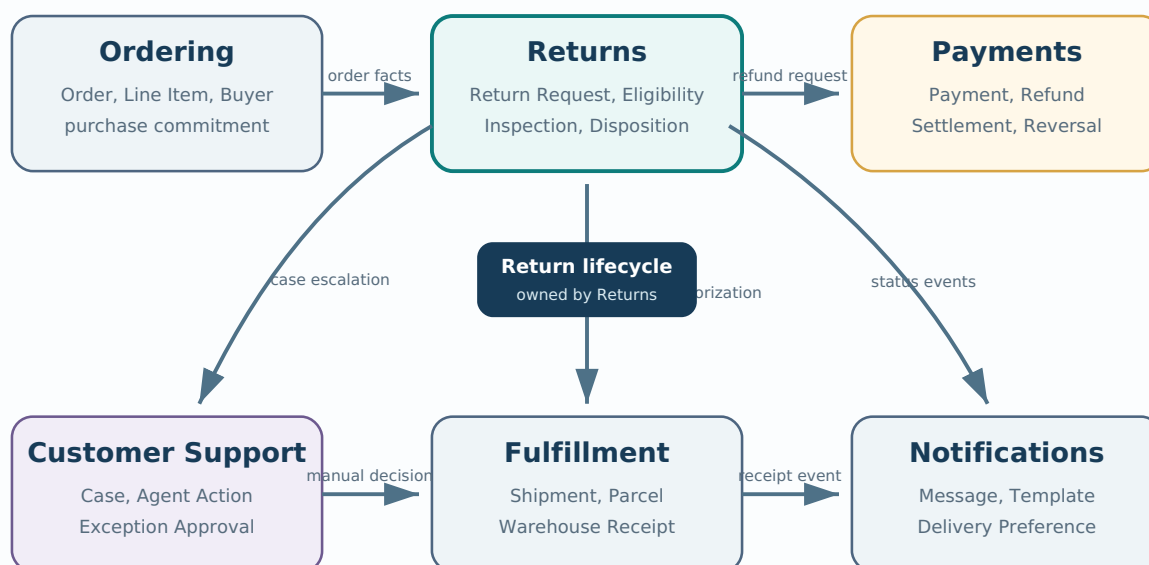
A subdomain is a part of the business problem space. A useful classification is:

- **Core:** differentiates the organization and deserves focused modeling.
- **Supporting:** necessary for the business but not the main differentiator.
- **Generic:** commonly solved by standard products or services.

For a marketplace, matching buyers to sellers may be core, while identity verification might be generic or supporting depending on the business model. This classification influences how much specification detail, custom implementation, and validation effort should be invested.

Bounded contexts

A bounded context is a boundary within which a model and its language have a consistent meaning. The same word may legitimately mean something different across contexts. An *Order* in Sales may represent commercial commitment; in Fulfillment it may be a shipment workload; in Finance it may be a set of accounting entries. SDD should avoid flattening these differences into one universal data model.

Diagram 5. Marketplace bounded-context map

The Returns context owns eligibility and lifecycle decisions. Other contexts provide facts or execute outcomes through explicit contracts.

4.3 A domain glossary for the case study

Term	Definition inside the Returns context	Do not confuse with
Return Request	A buyer's request to send one or more fulfilled items back for a stated reason.	A warehouse receipt or a payment refund.
Return Window	The interval during which a normal return request may be submitted, measured from confirmed delivery.	The time allowed to ship an authorized parcel.
Return Authorization	A decision that the request is eligible and may proceed under specified instructions.	A guarantee that money will be refunded.
Inspection Outcome	The warehouse assessment of received items against expected identity and condition.	The buyer-provided reason code.
Refund Instruction	A command sent to Payments stating the approved amount and financial references.	The actual settlement outcome returned by the payment provider.
Exception Approval	A documented support decision that overrides one or more normal eligibility rules.	Editing or bypassing a rule without traceability.

4.4 Tactical DDD: making rules explicit in the model

Tactical patterns are useful only when they clarify domain behavior. For the return capability:

- **Entity:** `ReturnRequest` has identity and changes state over time.
- **Value object:** `ReturnWindow`, `Money`, and `ReturnReason` are defined by values and invariants.
- **Aggregate:** a `ReturnRequest` aggregate protects lifecycle rules such as “a rejected request cannot later be shipped.”
- **Domain service:** an `EligibilityPolicy` may combine facts from the order, product, geography, and exception policy.
- **Domain event:** `ReturnAuthorized`, `ReturnReceived`, and `RefundApproved` communicate completed domain facts.

Conceptual domain model

```
ReturnRequest
  id: ReturnId
  orderId: OrderId
  requestedItems: List<ReturnItem>
  requestedAt: Instant
  state: ReturnState
  authorization: ReturnAuthorization?
  inspection: InspectionOutcome?
```

Behavior:

```
request(...)
authorize(policyDecision)
reject(reason)
markShipped(trackingNumber)
recordReceipt(receiptFacts)
approveRefund(amount)
```

Invariant examples:

- only an eligible request can be authorized
- only an authorized request can be marked shipped
- approved refund amount cannot exceed captured item value
- every exception approval records actor, reason, and timestamp

4.5 DDD artifacts inside an SDD repository

DDD artifact	Suggested location	How downstream artifacts use it
Ubiquitous-language glossary	<code>/domain/glossary.md</code>	Requirements, prompts, API names, scenario wording, code review checks
Context map	<code>/domain/context-map.md</code>	Architecture plan, ownership, integration contracts, task decomposition
Business-rule catalog	<code>/domain/rules/</code>	EARS requirements, BDD examples, policy tests, operational controls

DDD artifact	Suggested location	How downstream artifacts use it
Domain decision record	/domain/decisions/	Explains terminology and modeling choices that should not be silently changed by an agent

4.6 Example: detecting a language defect before code

AMBIGUOUS REQUEST

“When the return is completed, refund the order.”

Problems: What does completed mean? Whole order or selected item? Gross or net amount? Who owns the decision? What about failed settlement?

DOMAIN-ALIGNED STATEMENT

“When the Returns context records an approved refund amount for a received return, it shall issue a Refund Instruction to Payments for the approved item amount and tax allocation.”

The contexts, fact, command, and financial basis are now visible. More detail may still be needed, but the model is no longer implicit.

4.7 DDD anti-patterns in AI-assisted SDD

- **One model for everything:** the agent creates a universal `Order` object shared by Sales, Fulfillment, Returns, and Finance.
- **Database-first vocabulary:** business concepts are named after tables or vendor fields rather than domain meaning.
- **Pattern theater:** entities, repositories, and aggregates are added without invariants or business complexity.
- **Generated synonym drift:** the spec says “Return Authorization,” tasks say “RMA,” the API says “approval,” and tests say “accepted.”
- **Boundary leakage:** Returns directly edits payment records instead of requesting a refund through a defined contract.

SDD review gate: domain alignment

Before requirements are finalized, check that every important noun and state belongs to a named context, uses the glossary definition, and has an owner. Flag new synonyms as review questions rather than allowing an AI agent to normalize them silently.

5 EARS requirements syntax

EARS—the Easy Approach to Requirements Syntax—is a small set of patterns for writing structured natural-language requirements. It was introduced by Alistair Mavin and colleagues to reduce common defects such as ambiguity, vagueness, complexity, and omission while preserving the accessibility of ordinary language [4].

5.1 Why EARS is valuable in SDD

SDD often begins with human language because stakeholders can read and review it. Natural language is inclusive, but unconstrained prose frequently hides triggers, states, exceptions, and system boundaries. EARS adds a lightweight grammar without requiring every stakeholder to learn a formal specification language.

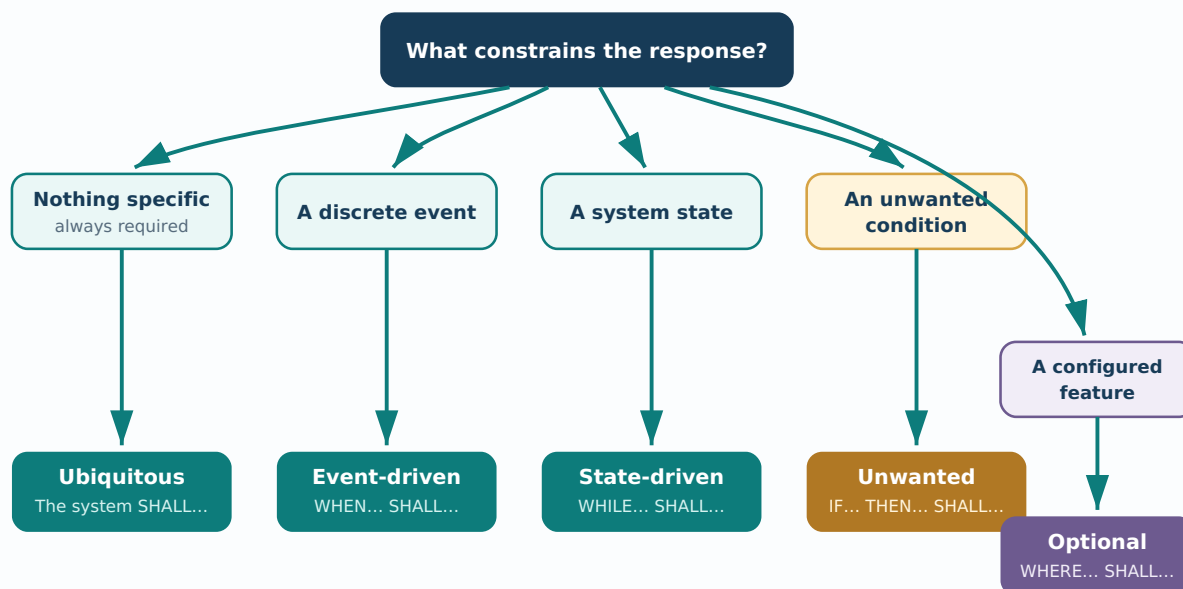
For AI-assisted development, this structure has a second benefit: it separates semantic roles. A tool can more reliably identify the *trigger*, *precondition*, *system*, and *required response*, then propose scenarios or tests. This does not make the requirement correct; it makes its structure more visible and easier to inspect.

5.2 The five foundational EARS patterns

Pattern	Canonical shape	Use when	Marketplace example
Ubiquitous	The <system> shall <response>.	The behavior is always required and is not tied to a specific trigger or state.	The Returns Service shall retain an audit record for every eligibility decision.
Event-driven	WHEN <trigger>, the <system> shall <response>.	A discrete event causes a response.	WHEN a buyer submits a return request, the Returns Service shall evaluate eligibility.
State-driven	WHILE <state>, the <system> shall <response>.	Behavior is required continuously or whenever the system is in a defined state.	WHILE a return is awaiting inspection, the Returns Service shall prevent refund approval.
Unwanted behavior	IF <undesired condition>, THEN the <system> shall <response>.	The requirement describes detection, mitigation, or safe handling of an abnormal situation.	IF the payment provider rejects a refund instruction, THEN the Payments Adapter shall record the failure and schedule a retry.
Optional feature	WHERE <feature is included>, the <system> shall <response>.	Behavior applies only to a product configuration, market, tenant, or enabled capability.	WHERE instant store credit is enabled, the Returns Service shall offer eligible buyers store credit as a refund method.

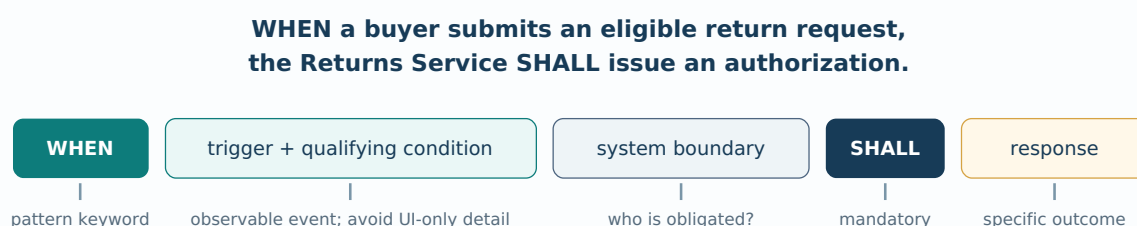
A complex requirement can combine clauses, for example: *WHERE instant credit is enabled, WHILE the buyer is in the Trusted tier, WHEN an eligible return is authorized, the Returns Service shall offer immediate store credit.* Use combinations carefully; excessive nesting can make one requirement difficult to review and test.

Diagram 6. Choosing an EARS pattern



5.3 Anatomy of a well-formed EARS requirement

Diagram 7. Semantic roles in an event-driven requirement



5.4 Worked transformation: vague prose to testable requirements

ORIGINAL PROSE

“Customers can usually return products for about a month, and they should be told if there is a problem.”

PROBLEMS MADE VISIBLE

- Who counts as a customer?
- What starts and ends the month?
- Calendar or business days?
- Which products are excluded?
- What is “a problem”?
- What notification channel and timing?

RULE-RET-007 / DOMAIN POLICY

The standard Return Window begins at the confirmed delivery timestamp and ends after 30 calendar days.

This is a business rule, not yet a system response. It defines the time semantics shared by multiple requirements.

REQ-RET-012 / EVENT-DRIVEN

WHEN a buyer submits a return request within the applicable Return Window, the Returns Service shall evaluate the request against product, order, geography, and fraud eligibility rules.

The request does not incorrectly promise authorization; it promises an evaluation.

REQ-RET-013 / UNWANTED BEHAVIOR

IF a buyer submits a return request after the applicable Return Window and no active exception approval exists, THEN the Returns Service shall reject the request with reason code `RETURN_WINDOW_EXPIRED`.

The abnormal condition and deterministic system response are explicit.

REQ-RET-014 / EVENT-DRIVEN

WHEN an eligibility decision is completed, the Returns Service shall make the decision, reason code, and next permitted action available to the buyer within 5 seconds for at least 99.5% of requests measured monthly.

The notification expectation is converted into an observable response with a measurable service objective.

5.5 Boundary values: the requirement is not the example

EARS gives a requirement structure, but examples are still needed to settle interpretation. For the 30-day policy, questions include whether delivery day is day 0 or day 1, how time zones are selected, and what happens during clock changes. A requirement should state the policy; examples should demonstrate representative and boundary cases.

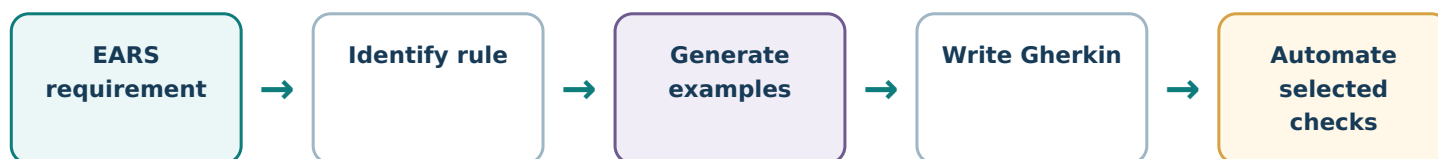
Delivery	Request	Expected result	Why this example matters
1 March, 10:00 UTC	1 March, 10:05 UTC	Within window	Immediate request
1 March, 10:00 UTC	31 March, 09:59:59 UTC	Within window	Just before boundary
1 March, 10:00 UTC	31 March, 10:00 UTC	Policy decision required	Exact boundary must be defined
1 March, 10:00 UTC	31 March, 10:00:01 UTC	Outside window	Just after boundary

5.6 Common EARS defects

Defect	Weak example	Why it fails	Correction
Passive or missing system	WHEN received, a refund shall be processed.	It is unclear what is received and which system is obligated.	WHEN Fulfillment records the authorized parcel as received, the Returns Service shall evaluate refund eligibility.
UI detail as trigger	WHEN the user clicks the blue button...	Couples business behavior to one interface.	WHEN the buyer confirms the return request...
Compound response	The system shall validate, authorize, notify, update, and refund...	Hard to trace, implement, and verify independently.	Split into atomic requirements with explicit ordering or events.
Unmeasurable quality	The system shall respond quickly.	No test oracle or service objective.	State percentile, threshold, population, and measurement period.
Implementation leakage	The system shall insert a row into <code>returns_v2</code> .	Specifies a current solution rather than required behavior.	State the durable business fact; move storage details to design.
Exception hidden in prose	Returns are rejected after 30 days unless support says otherwise.	The override, evidence, and ownership are undefined.	Add an explicit exception approval concept and requirement.

5.7 EARS-to-scenario transformation

A practical transformation pipeline is:



Requirement REQ-RET-013

IF a buyer submits a return request after the applicable Return Window and no active exception approval exists,
THEN the Returns Service shall reject the request with reason code RETURN_WINDOW_EXPIRED.

Derived questions

- Which clock and time zone define “after”?
- Is the boundary inclusive or exclusive?
- What makes an exception approval active?
- Can support create an exception after rejection?
- Is the reason code externally visible or translated?

5.8 EARS review checklist

- ☐ The obligated system or component is explicitly named.
- ☐ The trigger, state, unwanted condition, or optional feature is observable and defined.
- ☐ The response describes an outcome, not unnecessary implementation detail.
- ☐ The requirement contains one primary obligation.
- ☐ Every ambiguous noun appears in the domain glossary.
- ☐ Quantities include units, boundaries, populations, and measurement windows.
- ☐ Negative behavior and failure handling are not omitted.
- ☐ Representative examples and boundary examples exist separately.
- ☐ The requirement has an owner, source, rationale, and stable identifier.

Limit of EARS

EARS improves syntax; it does not automatically provide completeness, domain validity, conflict detection, prioritization, or proof of feasibility. Treat it as a disciplined doorway into deeper analysis, not as a substitute for analysis.

6 Behavior-Driven Development (BDD)

Behavior-Driven Development is a collaborative discovery and delivery practice centered on examples of externally observable behavior. Gherkin is often used to document those examples, but BDD is broader than Gherkin and broader than test automation.

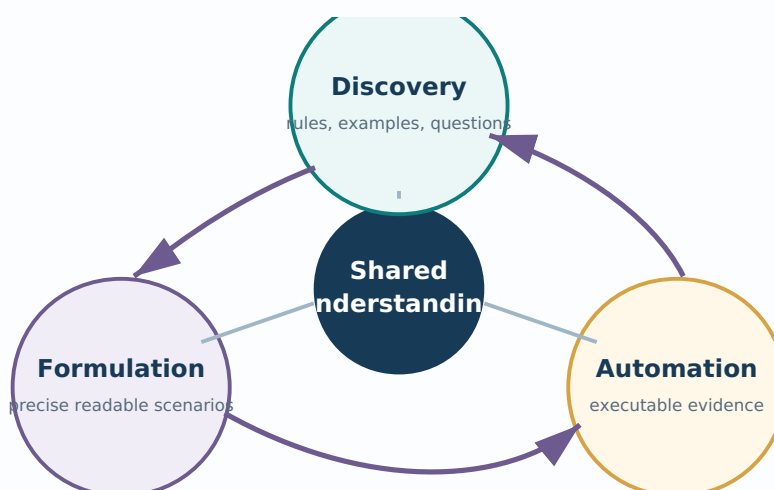
6.1 The original motivation

Dan North introduced BDD as a response to recurring confusion in Test-Driven Development: what to test, how to name tests, and how to connect technical work to business value [8]. The language of *behavior* reframed tests as examples of what a system should do in a particular context. Later practice developed a recognizable loop of **discovery, formulation, and automation**.

A 2023 systematic literature review describes BDD as using structured and testable behavior descriptions and reports its role in clarifying requirements and reducing ambiguity, while also identifying open areas for research [10]. These findings should be interpreted as a synthesis of a heterogeneous evidence base, not as proof that adding Gherkin files guarantees success.

6.2 The three activities of BDD

Diagram 8. BDD as discovery, formulation, and automation



Discovery

People with different perspectives explore a feature before implementation. A common workshop composition is sometimes called the “Three Amigos”: product or domain perspective, development perspective, and testing/quality perspective. The point is cognitive diversity, not a fixed number of attendees.

Discovery should surface:

- business rules and policy decisions;
- examples that illustrate ordinary behavior;
- counterexamples and failure paths;
- questions that remain unresolved;
- assumptions, risks, dependencies, and data needs;
- which outcomes require automated, manual, or operational evidence.

Formulation

Important examples are rewritten into precise, readable scenarios. Formulation is editorial and analytical work: remove irrelevant UI detail, choose stable domain language, identify one rule per scenario, make preconditions explicit, and ensure the expected outcome is observable.

Automation

Selected scenarios are connected to executable checks. Not every discussion example must become a UI-level automated test. Automation should occur at the lowest reliable layer that demonstrates the behavior: domain unit test, API/component test, contract test, or full workflow test. The scenario remains a specification; the automation is one implementation of evidence.

6.3 Example Mapping

Example Mapping is a lightweight workshop structure that organizes a story or capability into four types of cards:

Story / goal

The user outcome or slice being discussed.

Rules

Statements that constrain or govern behavior.

Examples

Concrete cases illustrating each rule.

Questions

Unresolved issues that prevent confident delivery.

Worked Example Mapping board: “Buyer requests a return”

Card type	Content
Story	As a buyer, I want to request a return for an eligible delivered item so that I can receive an appropriate remedy.
Rule 1	A normal request must be submitted within the applicable Return Window.
Examples	Request on day 1; request just before the day-30 boundary; request just after the boundary; request with an active support exception.
Rule 2	Some product categories are final sale.
Examples	Standard clothing item; personalized item; hygienic item with seal intact; hygienic item with seal broken.
Rule 3	A buyer cannot request more units than were fulfilled minus units already returned.
Examples	Return 1 of 2 units; attempt to return 3 of 2; second partial return after one unit was already refunded.
Questions	Which time zone defines the window? Who may approve an exception? Can damaged final-sale goods be returned? Is a replacement considered a remedy?

6.4 BDD’s relationship to EARS

EARS and BDD complement each other:

EARS contribution	BDD contribution	Combined result
Provides a consistent sentence structure.	Challenges the meaning through examples and questions.	A requirement that is both structured and concretely understood.
Makes trigger/state/response explicit.	Explores context, boundary values, and observable outcomes.	Scenarios can be systematically derived without assuming syntax equals completeness.
Supports catalogs and automated linting.	Supports collaborative conversation across roles.	Machine-assistable artifacts that remain human-centered.

6.5 A discovery conversation in detail

Product: Buyers have 30 days to return an item.

Tester: Is the 30th day included?

Developer: From purchase or delivery? Which delivery timestamp if an order has several parcels?

Operations: *Some carriers confirm delivery late. Support currently accepts proof from the buyer.*

Finance: *Do not promise a refund at authorization; refund amount can change after inspection.*

Product: *Then the capability is return authorization, not immediate refund. The window starts per fulfilled item at confirmed delivery, and support can record a documented exception.*

This conversation changes the model, the requirement, and likely the architecture. BDD's value lies in discovering these distinctions before they are embedded in code.

6.6 Business-readable versus implementation-coupled scenarios

IMPLEMENTATION-COUPLED

```
Given the user is on /orders/1729
And CSS selector #return-btn is visible
When the user clicks #return-btn
Then table returns_v2 has one new row
And HTTP 201 is returned
```

BEHAVIOR-FOCUSED

```
Given a delivered item is eligible for return
When the buyer submits a return request
Then the request is authorized
And return instructions are provided
```

The first example may be useful as a UI or API test, but it is brittle as a business specification. The second survives interface and storage changes. Lower-level tests can separately verify URLs, selectors, persistence, and status codes.

6.7 Test levels for BDD scenarios

Behavior	Best initial automation layer	Why
Return-window policy	Pure domain unit/property tests	Fast, deterministic, many boundary values, no infrastructure required.
Returns publishes a valid refund instruction	Component or contract test	Proves the interface and mapping without a full payment environment.
Buyer completes a return request through the web journey	Small set of end-to-end tests	Proves critical integration and presentation, but should remain selective due to cost and brittleness.
Monthly 99.5% response objective	Production telemetry/SLO evaluation	Cannot be fully established by pre-production functional tests.

6.8 BDD anti-patterns

- **Scenario factory:** converting every acceptance criterion into Given-When-Then without collaborative discovery.
- **Automation-first:** debating step definitions before agreeing on the rule and examples.
- **One giant scenario:** a feature's entire workflow in a single test with dozens of steps.
- **Conjunction abuse:** "And" chains that contain several distinct behaviors and failure causes.
- **Testing the framework:** scenarios assert implementation mechanics rather than business outcomes.
- **False executability:** a scenario is labeled executable but contains vague outcomes such as "works correctly."
- **Living-documentation decay:** scenarios pass while the wording no longer reflects current domain language.

6.9 BDD review checklist

- ☐ A cross-functional discovery conversation occurred for non-trivial behavior.
- ☐ Rules are separated from examples and unanswered questions.
- ☐ Each scenario illustrates one rule or one meaningful variation.
- ☐ Scenarios use domain language rather than UI or storage vocabulary.
- ☐ Happy paths, rejection paths, failure handling, and boundaries are represented.
- ☐ Automation is placed at the lowest reliable layer that proves the behavior.
- ☐ Passing automation is not treated as the only evidence for non-functional behavior.
- ☐ Scenarios remain readable to domain stakeholders after implementation.

7 Gherkin as a scenario language

Gherkin is a structured plain-text language used by Cucumber and related tools. Its keywords make scenarios parseable and consistent, while the sentences remain readable to people. Gherkin can represent behavior discovered through BDD, but syntax alone does not create shared understanding [9].

7.1 Core structure

Feature: Return eligibility

A short description of the capability and its value.

Rule: Standard returns must be requested within 30 calendar days

Background:

Given the marketplace uses the standard return policy

Scenario: Request submitted within the return window

Given an item was delivered 10 calendar days ago

And the item is not final sale

When the buyer requests a return for the item

Then the return request is authorized

And return instructions are provided

Scenario: Request submitted after the return window

Given an item was delivered 31 calendar days ago

And no active exception approval exists

When the buyer requests a return for the item

Then the return request is rejected

And the reason is "Return window expired"

7.2 What the keywords mean

Keyword	Purpose	Good practice	Common misuse
Feature	Names a capability and provides business context.	Explain actor value and scope, not implementation architecture.	Using one feature file as a folder for unrelated scenarios.
Rule	Groups examples under a business rule.	State a rule that can be challenged and illustrated.	Using "Rule" as a technical module name.
Scenario / Example	Describes one concrete behavior example.	Name the distinguishing outcome or condition.	Names such as "Test 1" or "Happy path."
Given	Establishes relevant context and preconditions.	Use domain facts already true before the action.	Performing the main business action in Given.

Keyword	Purpose	Good practice	Common misuse
When	Identifies the event or action under examination.	Prefer one central When per scenario.	Long sequences of UI clicks and technical calls.
Then	States an observable outcome.	Assert business-visible state, event, decision, or message.	Checking internal methods or database schemas.
And / But	Improves readability within the current semantic role.	Use sparingly and keep the scenario focused.	Hiding multiple behaviors in a long chain.
Background	Shares context among scenarios in a feature or rule.	Keep short, visible, and business-relevant.	Creating an invisible setup program with many technical steps.
Scenario Outline	Runs the same scenario shape with example rows.	Use when rows express meaningful variations of one rule.	Replacing readable scenarios with huge combinatorial tables.

7.3 Scenario Outline: boundary values

Rule: The standard return window ends after 30 calendar days

Scenario Outline: Evaluate a request near the return-window boundary

Given an eligible item was delivered at "2026-03-01T10:00:00Z"

When the buyer submits the request at "<requested-at>"

Then the eligibility result is "<result>"

And the decision reason is "<reason>"

Examples:

requested-at	result	reason	
2026-03-31T09:59:59Z	eligible	WITHIN_RETURN_WINDOW	
2026-03-31T10:00:00Z	eligible	WITHIN_RETURN_WINDOW	
2026-03-31T10:00:01Z	ineligible	RETURN_WINDOW_EXPIRED	

The table records a policy decision: the exact boundary is inclusive. If the organization chooses the opposite interpretation, the requirement and examples must change together. A test suite should not make this decision silently.

7.4 Data Tables and Doc Strings

Gherkin supports structured data when a scenario needs a small business-readable dataset. Keep the data relevant to the rule.

```
Scenario: Refund is limited to the approved returned quantity
Given the order contains:
  | sku      | fulfilled-qty | unit-price |
  | SHIRT-RED | 2             | 40.00 EUR  |
And one unit was previously refunded
When the buyer requests a refund for two more units
Then the request is rejected
And the reason is "Quantity exceeds refundable units"
```

A Doc String is useful for a message body, contract fragment, or formatted content, but avoid placing a large technical payload into a scenario unless the payload itself is the behavior under discussion.

7.5 Declarative versus imperative style

IMPERATIVE JOURNEY

```
Given I open the home page
And I click "My orders"
And I select order 1729
And I scroll to the second item
And I click "Return"
And I select "Too small"
And I click "Continue"
When I click "Submit"
Then I see a green banner
```

DECLARATIVE BEHAVIOR

```
Given a buyer has an eligible delivered item
And the buyer selects "Too small" as the reason
When the buyer submits a return request
Then the request is authorized
And return instructions are provided
```

The imperative version can still be useful for a dedicated usability or navigation test. The declarative version is better living documentation because it describes the business transaction rather than one interface implementation.

7.6 Step-definition design

Step definitions connect scenario language to automation code. Poor step design can make readable scenarios technically unmaintainable.

Principle	Reason	Example
Use domain phrases	Preserves a common vocabulary and reduces UI coupling.	Given an item is eligible for return
Avoid regex synonym explosions	Dozens of equivalent phrases fragment the language.	Choose "submits a return request," not many variants such as sends/creates/opens.
Keep steps composable but meaningful	Very broad steps hide behavior; very small steps create noise.	Model a domain fact or action, not each field assignment.

Principle	Reason	Example
Separate setup from assertion	Improves failure diagnosis and semantic clarity.	A Given establishes state; a Then observes outcome.
Do not share mutable scenario state broadly	Reduces order dependence and hidden coupling.	Use scenario-scoped context objects.

Illustrative step definitions (TypeScript-like pseudocode)

```

Given('an item is eligible for return', async function () {
  this.item = await fixtures.deliveredItem({
    deliveredDaysAgo: 10,
    finalSale: false,
    remainingReturnableQuantity: 1
  });
});

When('the buyer submits a return request', async function () {
  this.response = await returnsApi.requestReturn({
    buyerId: this.item.buyerId,
    orderItemId: this.item.id,
    quantity: 1,
    reason: 'TOO_SMALL'
  });
});

Then('the request is authorized', function () {
  expect(this.response.status).toEqual('AUTHORIZED');
  expect(this.response.authorizationId).toBeDefined();
});

```

7.7 Tags and test organization

Tags can express metadata such as requirement links, test layer, risk, or execution profile:

```

@REQ-RET-012 @api @critical
Scenario: Eligible request is authorized
...

```

Use tags as metadata, not as a substitute for structure. A useful convention might include:

- `@REQ-...` for traceability;
- `@domain`, `@component`, `@contract`, `@journey` for automation layer;
- `@critical` or a risk ID for execution priority;
- `@manual` only when the evidence is intentionally not automated, with a stated reason.

7.8 Scenario quality heuristics

Heuristic	Healthy sign	Warning sign
Focus	One rule and one distinguishing variation	Several Whens, unrelated Then outcomes, long procedural narrative
Language	Words match the glossary and domain conversations	Framework, CSS, database, or transport details dominate
Observability	Outcome can be perceived through a public interface, event, or business state	Then calls an internal method or assumes private storage
Independence	Scenario creates or declares its relevant context	Depends on execution order or residue from another scenario
Signal	A failure points toward a violated behavior	One scenario has so many assertions that the cause is unclear
Durability	Survives redesign of UI and internals	Minor presentation change breaks most scenarios

7.9 Mapping EARS clauses to Gherkin roles

EARS element	Likely Gherkin role	Caution
WHERE optional feature	Given configuration or Rule scope	Do not repeat the feature flag in every scenario if a Rule or Background communicates it more clearly.
WHILE state	Given state	State may require several domain facts; keep the phrase meaningful.
WHEN trigger	When action/event	An EARS trigger sometimes includes conditions that belong in Given.
IF unwanted condition	Given abnormal context or When failure occurs	Choose based on whether the condition exists before or results from the action.
System SHALL response	Then outcome	One requirement may require several observable Then statements or several scenarios.

7.10 Gherkin anti-pattern catalog

1. **Feature files as test scripts:** they list clicks and selectors but contain no rules.
2. **Copy-paste scenarios:** many near-identical scenarios differ only in values that a focused outline could express.
3. **Mega-background:** a long Background hides essential context and makes each scenario difficult to understand alone.
4. **Conjunctive scenarios:** repeated And steps turn one scenario into a workflow suite.
5. **Vague Then:** “Then it succeeds” or “Then an error is shown” gives no stable oracle.

6. **Scenario without a rule:** an example exists but nobody can state the business policy it illustrates.
7. **Automation leakage:** retries, waits, mock configuration, or environment setup appear in the feature text.

SDD transformation rule

Do not automatically convert every EARS requirement into exactly one Gherkin scenario. First identify the underlying rule, then generate representative examples. A single requirement may require multiple scenarios; several requirements may jointly define one scenario.

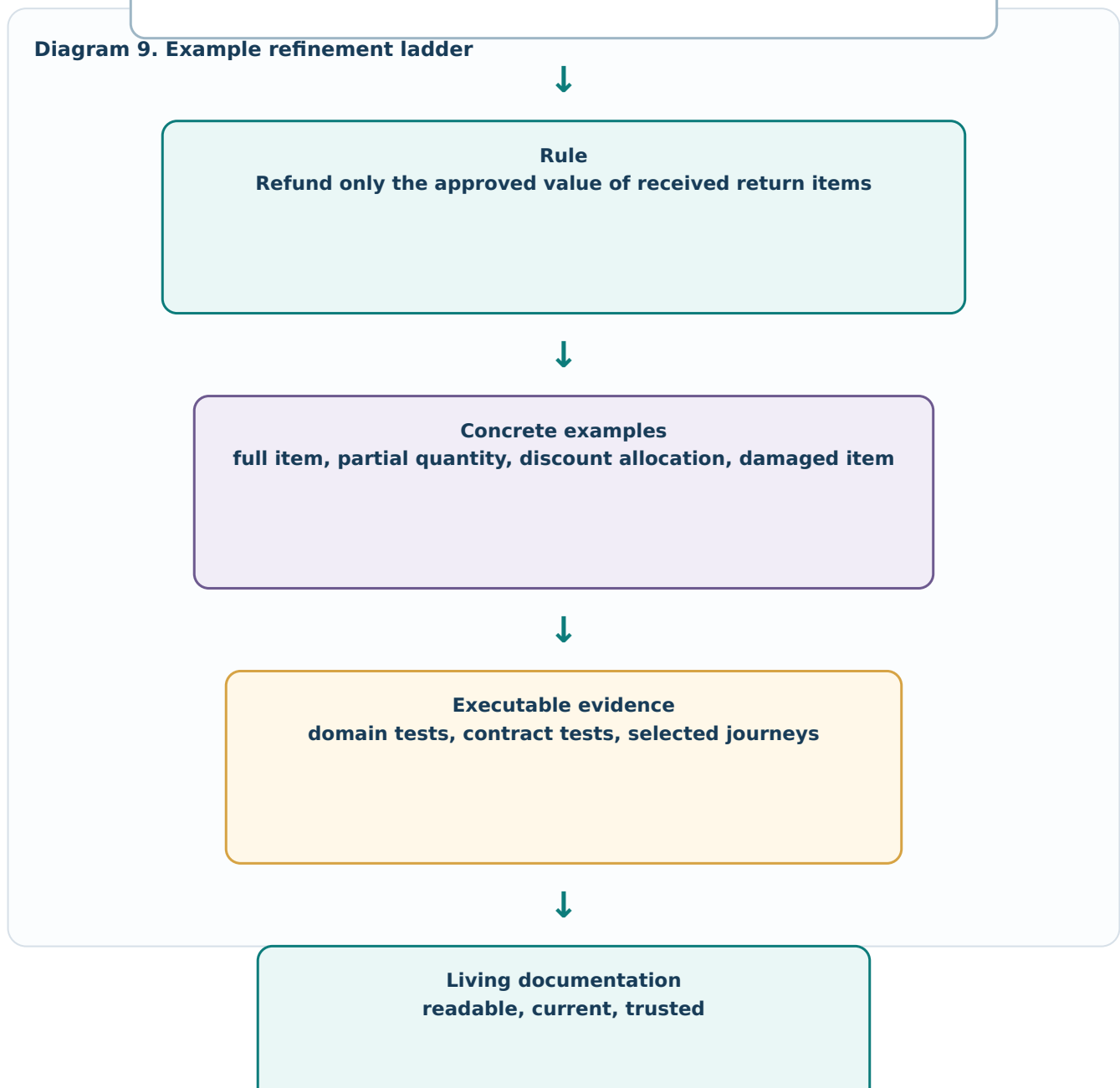
8 Specification by Example

Specification by Example is a collaborative approach in which realistic examples clarify requirements, guide implementation, become acceptance checks where useful, and remain accessible as living documentation. Gojko Adzic's book synthesized the practice from many project case studies [11].

8.1 From abstract rule to concrete understanding

Abstract statements are compact but underdetermine behavior. “Refund the customer after a valid return” leaves questions about amount, timing, method, taxes, shipping fees, discounts, currencies, partial quantities, and failed payment fails. Examples force the team to choose values and outcomes.

Diagram 9. Example refinement ladder



8.2 A worked refund-calculation example

Suppose an order contains two shirts at EUR 40 each and a EUR 20 order-level discount. A naive implementation may refund EUR 40 for one returned shirt. A domain rule might instead allocate the discount proportionally, producing a EUR 30 item refund. Concrete tables make the financial rule reviewable.

Case	Order composition	Return	Expected merchandise refund	Reason
EX-REF-01	2 × EUR 40; no discount	1 shirt	EUR 40	Direct unit price
EX-REF-02	2 × EUR 40; EUR 20 proportional discount	1 shirt	EUR 30	Returned item carries EUR 10 of allocated discount
EX-REF-03	2 × EUR 40; EUR 20 discount on first item only	First shirt	EUR 20	Discount attribution follows promotion result
EX-REF-04	2 × EUR 40; EUR 20 discount on first item only	Second shirt	EUR 40	Second item had no discount allocation
EX-REF-05	1 × EUR 40; refunded previously	Same item again	EUR 0 / reject	Approved cumulative refund cannot exceed captured value

8.3 Principles adapted to SDD

Specification by Example practice	Application inside SDD
Derive scope from goals	Every feature specification links to measurable outcomes and explicitly excludes unrelated work.
Specify collaboratively	Examples are reviewed by domain, product, engineering, quality, and operations perspectives before task generation.
Illustrate using examples	Rules have representative, boundary, error, and counterexamples—not only abstract acceptance criteria.
Refine the specification	Remove incidental detail, split compound rules, define terms, and expose unresolved questions.
Automate without changing meaning	Connect examples to appropriate tests while keeping implementation mechanics outside the business-readable artifact.
Validate frequently	Run automated checks and human reviews as specifications, contracts, and code evolve.
Evolve living documentation	Version examples with the feature and treat stale examples as defects.

8.4 Illustrative versus exhaustive examples

A living specification should communicate the rule with a small set of meaningful examples. It should not attempt to list every possible input. Exhaustive confidence can come from other techniques:

- **Equivalence partitioning:** choose representatives from classes expected to behave the same.
- **Boundary-value analysis:** test just below, at, and just above thresholds.
- **Property-based testing:** generate many values and assert invariants.
- **Model-based testing:** generate paths through a state model.
- **Mutation testing:** evaluate whether tests detect deliberately introduced faults.

The living example explains. The broader automated strategy challenges.

8.5 A decision table as specification

When several conditions interact, prose or individual scenarios can hide missing combinations. A decision table makes the rule space explicit.

Rule row	Within window?	Final sale?	Defective?	Active exception?	Decision
R1	Yes	No	Either	Either	Eligible, subject to remaining rules
R2	No	No	No	No	Reject: window expired
R3	No	No	No	Yes	Eligible by exception
R4	Either	Yes	No	No	Reject: final sale
R5	Either	Yes	Yes	Either	Route to defect policy

Missing policy revealed

The table exposes a business question: does an active exception override final-sale status, or only the return window? That question should be answered in the rule catalog before code generation.

8.6 When examples become dangerous

- **Example becomes the rule:** developers hard-code values from one example rather than implementing the general policy.
- **Only happy examples:** the specification appears clear but says nothing about rejection, retries, or partial failure.

- **Overfitted tables:** dozens of columns model implementation data instead of business distinctions.
- **Stale examples:** automation passes because code and test changed together, while the documented outcome no longer matches stakeholder intent.
- **Unowned oracles:** nobody is accountable for confirming expected values such as refund allocation.

8.7 Example portfolio for one rule

Example type	Purpose	Return-window example
Representative	Shows normal behavior	Request 10 days after delivery is eligible.
Boundary	Defines threshold semantics	Request exactly 30 days after delivery is eligible.
Counterexample	Shows what is not covered	Request 30 days after purchase but before delivery is not evaluated from purchase date.
Negative	Shows explicit rejection	Request 30 days and one second after delivery is rejected.
Exception	Shows authorized override	Late request with an active, scoped exception proceeds.
Failure	Shows dependency failure handling	Delivery timestamp is temporarily unavailable; decision is deferred, not guessed.
Non-functional	Shows quality expectation	Eligibility response is available within the service objective.

8.8 Living documentation governance

Living documentation is not merely generated documentation. It remains useful because:

- the examples are written in domain language;
- they are organized by capabilities and rules, not test framework packages;
- selected scenarios are linked to execution evidence;
- the publication process marks obsolete, pending, and accepted behavior clearly;
- ownership and review dates are visible;
- the documentation is easy to navigate by rule, requirement, context, or release.

SDD connection

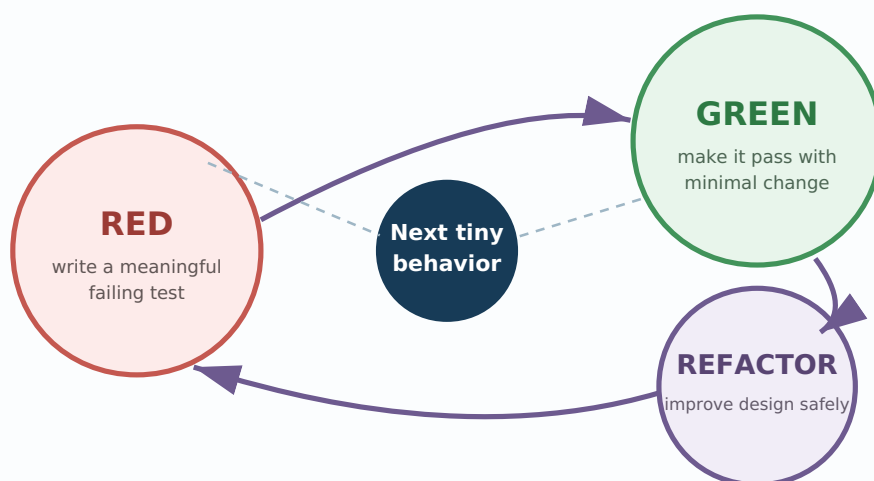
Examples should be first-class inputs to planning and task decomposition. A task list that covers the prose requirement but omits the boundary and failure examples is incomplete.

9 Test-Driven Development (TDD)

Test-Driven Development is a short implementation feedback cycle: write a test that fails for the intended reason, write the smallest implementation that makes it pass, then improve the design while preserving behavior. In SDD, TDD operates close to code; it does not replace domain discovery, requirements, or acceptance examples.

9.1 The Red-Green-Refactor cycle

Diagram 10. TDD feedback loop



Red: a test fails for the right reason

The test describes one missing behavior. A compilation failure may be acceptable at the first step, but the developer should soon observe a failure that clearly indicates the behavior is absent. A test that passes immediately provides no evidence that it can detect the missing capability.

Green: the smallest responsible implementation

“Minimal” does not mean careless or hard-coded for production. It means avoiding speculative design beyond the current behavior. The test constrains the next increment; other quality constraints, coding standards, and architectural decisions still apply.

Refactor: improve structure without changing behavior

Passing tests create a safety net for renaming, extracting concepts, removing duplication, clarifying dependencies, and improving design. Refactoring is not optional cleanup after “real work”; it is the step that prevents a sequence of small additions from becoming accidental complexity.

9.2 Where TDD sits inside SDD



An acceptance example such as “a request one second after the return-window boundary is rejected” may lead to several TDD increments: representing the window, selecting a clock, computing the boundary, returning a decision type, and mapping the result to an API response. The acceptance example defines the business outcome; TDD helps discover a maintainable implementation.

9.3 Worked TDD example: return-window policy

Step 1 - express a boundary behavior

```
def test_request_at_boundary_is_eligible():
    delivered_at = instant("2026-03-01T10:00:00Z")
    requested_at = instant("2026-03-31T10:00:00Z")

    decision = StandardReturnWindow(days=30).evaluate(
        delivered_at=delivered_at,
        requested_at=requested_at,
    )

    assert decision == Eligibility.ELIGIBLE
```

At first, `StandardReturnWindow` may not exist. The failing test establishes both the required behavior and an emerging interface.

Step 2 - implement enough behavior

```
class StandardReturnWindow:
    def __init__(self, days: int):
        self._duration = timedelta(days=days)

    def evaluate(self, delivered_at, requested_at):
        deadline = delivered_at + self._duration
        return (
            Eligibility.ELIGIBLE
            if requested_at <= deadline
            else Eligibility.EXPIRED
        )
```

Step 3 - add the adjacent failure

```
def test_request_after_boundary_is_expired():
    delivered_at = instant("2026-03-01T10:00:00Z")
    requested_at = instant("2026-03-31T10:00:00.001Z")

    decision = StandardReturnWindow(days=30).evaluate(
        delivered_at=delivered_at,
        requested_at=requested_at,
    )

    assert decision == Eligibility.EXPIRED
```

Step 4 - introduce domain information, not booleans

A boolean such as `True` says little about why a decision occurred. A richer result supports the EARS requirement, user response, audit, and traceability.

```
@dataclass(frozen=True)
class EligibilityDecision:
    eligible: bool
    reason: DecisionReason
    evaluated_at: datetime
    policy_version: str

class StandardReturnWindow:
    def evaluate(self, delivered_at, requested_at):
        deadline = delivered_at + self._duration
        if requested_at <= deadline:
            return EligibilityDecision(
                eligible=True,
                reason=DecisionReason.WITHIN_RETURN_WINDOW,
                evaluated_at=requested_at,
                policy_version="RET-WINDOW-2026-01",
            )
        return EligibilityDecision(
            eligible=False,
            reason=DecisionReason.RETURN_WINDOW_EXPIRED,
            evaluated_at=requested_at,
            policy_version="RET-WINDOW-2026-01",
        )
```

9.4 Triangulation and test selection

One test can invite a hard-coded implementation. A second, meaningfully different example forces generalization. A third may reveal a new concept. This is sometimes called triangulation.

Increment	New test	Design pressure
1	Request exactly at day 30 is eligible	Create policy boundary and evaluation result.
2	Request after day 30 is expired	Implement comparison rather than constant success.

Increment	New test	Design pressure
3	Active exception makes a late request eligible	Separate standard window from exception policy.
4	Missing delivery time defers the decision	Represent “unknown” rather than guessing or throwing a generic error.
5	Policy version is recorded	Make decisions auditable and reproducible.

9.5 Test doubles and architecture

TDD can help expose dependencies. If eligibility logic needs the current time, avoid directly calling the system clock. Inject a clock or pass the evaluation time. If it needs order facts, use an explicit port or value object rather than a hidden database query. However, excessive mocking can couple tests to implementation interactions.

INTERACTION-HEAVY

```
repo.get_order.assert_called_once()
clock.now.assert_called_once()
flags.is_enabled.assert_called_once()
notifier.send.assert_called_once()
logger.info.assert_called_once()
```

This can pass while the business decision is wrong and may break during harmless refactoring.

BEHAVIOR-CENTERED

```
decision = policy.evaluate(facts,
at=fixed_time)
assert decision.reason ==
RETURN_WINDOW_EXPIRED
assert decision.next_actions ==
[REQUEST_EXCEPTION]
```

Use interaction tests selectively at collaboration boundaries where the interaction is itself part of the contract.

9.6 TDD does not define the right requirement

Kent Beck’s *Test Driven Development: By Example* is the foundational practitioner reference [19]. Empirical literature is more nuanced than slogans. A meta-analysis by Rafique and Misisic examined effects on external quality and productivity and illustrates that outcomes vary with study context and rigor [22]. The responsible conclusion is not “TDD always improves quality” or “TDD slows teams”; it is that short test-first feedback can be valuable, but benefits depend on skill, task, test design, codebase, and adoption conditions.

9.7 TDD and acceptance tests

Dimension	Acceptance/BDD example	TDD unit/component test
Audience	Domain, product, quality, engineering	Primarily engineering
Question	Does the system exhibit the agreed behavior?	What is the next small design increment?

Dimension	Acceptance/BDD example	TDD unit/component test
Vocabulary	Domain language	Domain plus technical design language
Speed	Varies; often component or workflow level	Usually very fast and isolated
Quantity	Small set of illustrative and critical examples	Many focused cases and invariants
Change sensitivity	Should survive internal refactoring	May evolve with local design while preserving behavior

9.8 TDD anti-patterns

- **Test-after labeled TDD:** implementation is completed first and tests merely document it.
- **Green without red:** tests are written but never observed failing for the intended reason.
- **Micro-step theater:** tiny steps add ceremony without learning or risk reduction.
- **Implementation-coupled assertions:** tests encode private structure and resist safe refactoring.
- **Mock everything:** interaction expectations replace business outcomes.
- **No refactor phase:** passing code accumulates duplication and poor names.
- **Coverage as goal:** line coverage replaces evaluation of meaningful behavior and fault detection.
- **Unit-only confidence:** local tests pass while contracts, deployment, data migration, or operations fail.

9.9 TDD review checklist for an SDD task

- ☐ The task links to a requirement or scenario and names the behavior slice.
- ☐ The first failing test demonstrates absence of the behavior, not an unrelated setup failure.
- ☐ Tests use clear arrange-act-assert structure or an equally understandable form.
- ☐ Business decisions produce meaningful result types and reason codes.
- ☐ Boundary and invalid cases are present where risk warrants them.
- ☐ Refactoring occurs while tests remain green.
- ☐ Tests avoid unnecessary coupling to private implementation details.
- ☐ Higher-level contract and acceptance evidence still exists.

Combined principle

BDD asks, “Are we building the agreed behavior?” TDD asks, “What is the next small, testable design step?” SDD connects both questions to the domain, the plan, and the evidence trail.

PART III

Integrated SDD practice

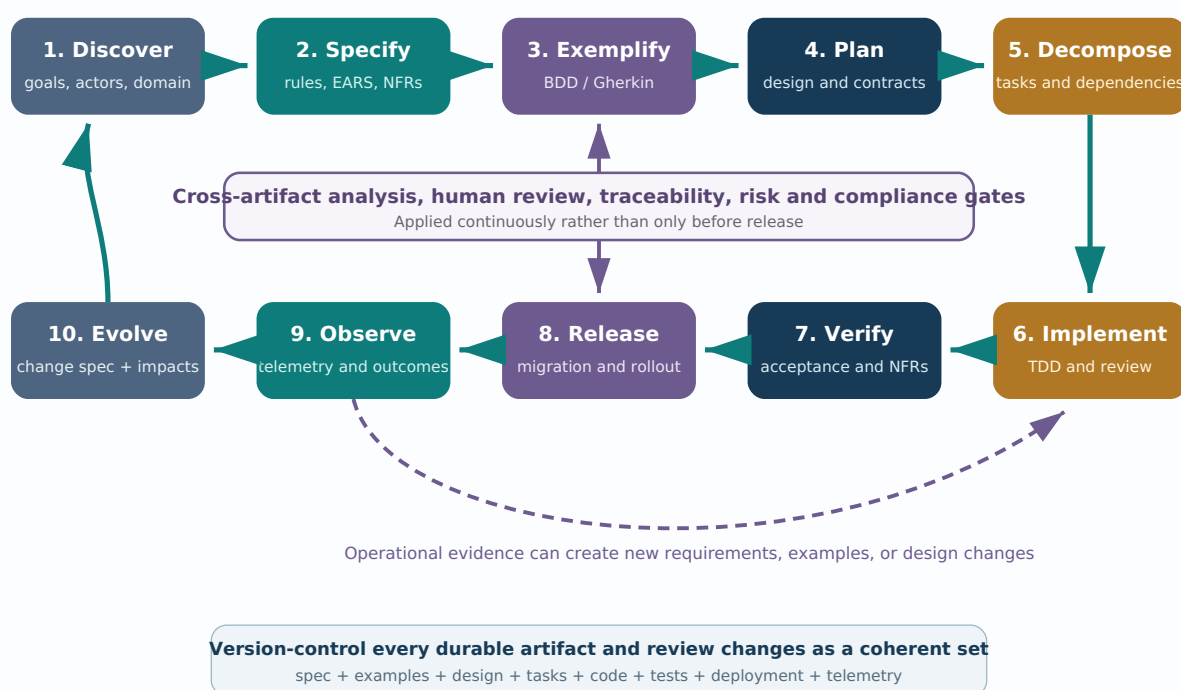
A complete worked lifecycle—from domain discovery and structured requirements to architecture, tasks, tests, rollout, and controlled change.

10 A complete SDD lifecycle

A practical SDD lifecycle turns an initial idea into a set of connected artifacts, with explicit review gates between transformations. AI agents can accelerate drafting, analysis, and implementation, but humans remain accountable for domain validity, priorities, risk acceptance, and evidence.

10.1 The end-to-end flow

Diagram 11. SDD lifecycle with feedback and quality gates



10.2 Phase 0 - establish project principles

Before feature-level work, define durable constraints that should apply across features. GitHub Spec Kit calls this a project constitution [1]. Other teams may use engineering principles, architecture guardrails, definition-of-done policies, or compliance controls.

Example project principles**P-01 Domain language**

All public APIs, domain tests, specifications, and business-facing events shall use terms from the approved glossary.

P-02 Traceability

Every implementation task shall reference at least one requirement or design decision. Every critical requirement shall have verification evidence.

P-03 Security and privacy

Personal data shall be minimized, access-controlled, encrypted in transit and at rest, and excluded from application logs unless explicitly approved.

P-04 Backward compatibility

Published integration contracts shall be changed through additive evolution or an approved versioned migration.

P-05 Testing strategy

Business policies shall be tested primarily at the domain/component layer. End-to-end tests shall be reserved for critical cross-system journeys.

P-06 Operational readiness

A feature is not complete without defined metrics, structured logs, alerts, failure recovery, and rollback or containment procedures.

10.3 Phase 1 - discover and frame

Start with a problem statement, not a proposed implementation. A useful framing artifact contains:

Field	Question	Returns example
Problem	What undesirable situation exists?	Buyers contact support for routine returns, creating delay and manual handling.
Outcome	What measurable change would indicate success?	Increase eligible self-service return authorizations while preserving policy and fraud controls.
Actors	Who experiences or influences the capability?	Buyer, support agent, warehouse operator, finance analyst, seller, carrier.
Scope	What behavior is included?	Eligibility, authorization, instructions, receipt, inspection, refund instruction, status visibility.
Out of scope	What is deliberately excluded?	Chargebacks, warranty repair, seller-to-platform disputes, physical disposal optimization.
Constraints	What cannot be freely chosen?	Existing Payments API, regional consumer rules, order data ownership, rollout by market.
Risks	What failure is costly?	Over-refunds, policy inconsistency, fraud, duplicate instructions, inaccessible return labels.

10.4 Phase 2 - specify behavior

A feature specification should organize information so that missing behavior is visible. The exact template can vary, but a strong structure includes:

1. problem, goals, and non-goals;
2. actors and domain terms;
3. business rules and policy versions;
4. functional requirements with IDs;
5. non-functional requirements and measurable objectives;
6. examples, scenarios, and decision tables;
7. data and integration expectations at a behavioral level;
8. failure, recovery, and unchanged behavior;
9. open questions, assumptions, and risks;
10. acceptance and rollout criteria.

10.5 Phase 3 - clarify before architecture

Clarification is not a cosmetic editing phase. It prevents the design or coding agent from choosing unstated policy. Questions should be classified and owned:

Question category	Example	Owner / evidence
Domain policy	Does a support exception override final-sale status?	Product/domain owner; policy record
Legal/compliance	Do regulated markets require a different return window?	Legal/compliance interpretation
User experience	What remedy choices are presented and when?	Product/design research
Technical feasibility	Can the order service provide item-level delivery time?	Architecture spike / contract owner
Operational	Who handles a return stuck in transit for 21 days?	Operations runbook owner
Measurement	How is manual handling rate calculated?	Analytics definition and source

No silent assumptions

When an unresolved question affects money, safety, privacy, legal obligation, authorization, irreversible data change, or public contract, the agent should stop that branch of implementation and record a decision request. “Most likely” is not an acceptable policy source.

10.6 Phase 4 - plan the realization

The technical plan translates behavior into an implementation strategy while preserving trace links. It should include:

Architecture

Context ownership, components, interfaces, data flows, consistency boundaries, scaling and availability approach.

Data

Entities, lifecycle, migration, retention, privacy classification, audit, reconciliation, and deletion behavior.

Contracts

API/event schemas, idempotency, ordering, error semantics, compatibility, retries, and versioning.

Quality strategy

Test layers, security checks, performance experiments, accessibility, observability, rollout, and rollback.

Architecture decision record example

ADR-RET-003: Refund initiation through an idempotent command

Status: Accepted

Context:

Returns decides the approved amount; Payments owns payment-provider interaction and settlement state.

Duplicate delivery can occur because the messaging system provides at-least-once delivery.

Decision:

Returns publishes RefundRequested with a stable refundInstructionId.

Payments treats refundInstructionId as an idempotency key and emits RefundAccepted or RefundRejected.

Consequences:

- + Preserves bounded-context ownership.
- + Supports retries without duplicate provider refunds.
- + Enables reconciliation between approved and settled amounts.
- Introduces asynchronous user-visible state.
- Requires dead-letter handling, monitoring, and replay controls.

Trace:

REQ-RET-030, REQ-RET-031, NFR-RET-006, RULE-RET-019

10.7 Phase 5 - generate executable tasks

Tasks should be independently understandable, demonstrably complete, and ordered by dependencies. Avoid tasks such as “implement backend” or “add tests.”

ID	Task	Depends on	Trace	Completion evidence
TASK-RET-001	Add glossary and policy constants for Return Window	-	TERM-RET-04, RULE-RET-007	Reviewed domain artifact and lint check
TASK-RET-004	Implement pure eligibility policy with boundary and property tests	001	REQ-RET-012, REQ-RET-013	Domain tests including exact boundary
TASK-RET-007	Expose eligibility evaluation through Returns API	004	SCN-RET-001..006	API component tests and schema validation
TASK-RET-010	Persist decision reason, input facts, and policy version	004	REQ-RET-016, P-02	Persistence and audit retrieval tests
TASK-RET-016	Publish idempotent RefundRequested event	010, contract approval	ADR-RET-003, REQ-RET-030	Contract test and duplicate-delivery test
TASK-RET-022	Add dashboards and alerts for decision latency and refund failures	007, 016	NFR-RET-001, NFR-RET-006	Dashboard review and alert simulation

10.8 Phase 6 - implement and continuously compare

Implementation agents should receive the smallest coherent context necessary: project principles, feature spec, relevant domain artifacts, approved plan, current task, contracts, and nearby code. They should not be asked to infer the entire project from an unstructured chat history.

A task execution loop can be:

1. read task and all trace-linked artifacts;
2. state the intended behavior and affected components;
3. identify unresolved contradictions or missing information;
4. write or update the smallest meaningful test;
5. implement and refactor;
6. run local and relevant regression checks;
7. update trace metadata and task evidence;
8. summarize deviations from the plan for human review.

10.9 Phase 7 - verify more than functional correctness

Evidence class	Questions	Examples
Functional	Does behavior match the rules and examples?	Domain, API, contract, and workflow tests
Structural	Does implementation respect architecture and boundaries?	Dependency rules, schema checks, architecture tests
Security/privacy	Are threats and data obligations addressed?	Threat-model review, authorization tests, secret scanning, log inspection
Performance/reliability	Does the system meet measurable objectives under realistic conditions?	Load tests, fault injection, retry/idempotency tests, capacity model
Accessibility/usability	Can intended users complete critical journeys?	Automated accessibility checks, keyboard journey, assistive-technology review, usability study
Operational	Can teams detect, diagnose, contain, and recover?	Dashboards, alerts, runbooks, replay procedure, rollback exercise
Outcome	Does the feature improve the business/user problem?	Adoption, completion, manual handling, complaint rate, experiment analysis

10.10 Contemporary tool patterns

GitHub Spec Kit's documented core process is Spec → Plan → Tasks → Implement, supported by templates and cross-artifact analysis [1]. Kiro documents requirements-first and design-first feature-spec workflows and produces `requirements.md`, `design.md`, and `tasks.md`; its requirements-first approach uses EARS-style requirements [3]. These tools illustrate a broader principle: the workflow should create durable, reviewable artifacts rather than relying on transient conversation context.

Tooling caveat

Tool commands and product capabilities evolve. Treat the cited official documentation as the source for current syntax. The architectural guidance in this report is intentionally tool-independent.

11 End-to-end case study: returns and refunds

This chapter applies the complete method to one feature. The goal is not to prescribe the only valid design, but to show how each artifact adds information, exposes ambiguity, and creates traceable implementation and verification work.

11.1 Problem statement and measurable outcomes

Problem

Routine return requests are handled through customer support because eligibility rules and next steps are not available through self-service. Buyers wait for a decision, agents apply policy inconsistently, and Finance receives refund requests with incomplete traceability.

ID	Outcome	Measure	Guardrail
GOAL-RET-01	Increase self-service resolution	Share of eligible return requests authorized without agent contact	No increase in incorrect authorizations
GOAL-RET-02	Reduce buyer uncertainty	Median time from request submission to decision and instructions	Accessible alternatives remain available
GOAL-RET-03	Improve financial control	Share of refund instructions with complete source, amount, and policy trace	No duplicate provider refunds
GOAL-RET-04	Reduce manual workload	Agent minutes per 1,000 completed returns	Exception cases remain reviewable by a human

Targets should be assigned after baseline measurement. Inventing a target to complete a template can create false precision. The specification may initially state the metric and decision owner, then add an approved target before rollout.

11.2 Scope and non-scope

Included

- Buyer initiates item-level return.
- Eligibility evaluation and reason.
- Exception approval reference.
- Authorization and return instructions.
- Shipment and warehouse receipt status.
- Inspection outcome.
- Approved refund instruction.
- Buyer-visible status and failure recovery.

Excluded from this feature

- Chargeback and payment dispute handling.
- Warranty repair and replacement logistics.
- Seller reimbursement accounting.
- Carrier contract optimization.
- Warehouse disposition and resale pricing.
- General promotions-engine redesign.
- Changing existing order-cancellation behavior.

11.3 Actors and responsibilities

Actor/system	Responsibility	Authority	Evidence produced
Buyer	Requests return, selects item/quantity/reason, follows instructions	May act only on own eligible purchases	Request intent, selected remedy, shipment details
Returns Service	Owns eligibility, authorization, lifecycle, and refund approval decision	Applies current policy and scoped exceptions	Decision, reason, policy version, events
Order Service	Provides immutable order and fulfillment facts	Owns purchase, line-item, price, and delivery facts	Order/fulfillment snapshot and version
Customer Support	Investigates exceptions and records approvals	May override only permitted rules within delegated limits	Approval actor, scope, reason, expiry
Fulfillment/Warehouse	Receives parcel and records inspection facts	May not decide financial settlement	Receipt time, item identity, condition, quantity
Payments	Executes approved refund instruction	Owns provider interaction and settlement status	Accepted/rejected/settled result and provider reference
Notifications	Delivers status communications	Uses approved content and preferences	Delivery status and channel

11.4 Domain rules

ID	Rule	Rationale/source	Owner
RULE-RET-001	A return request applies to one or more fulfilled order-item quantities, not an entire order by implication.	Supports partial shipments and partial returns.	Returns product owner
RULE-RET-007	The standard Return Window begins at confirmed item delivery and includes the instant exactly 30 calendar days later.	Approved standard policy; regional variants may exist.	Policy owner
RULE-RET-009	Requested quantity must not exceed fulfilled quantity minus quantities already authorized, received, rejected after receipt, or refunded according to the lifecycle policy.	Prevents duplicate or excessive return rights.	Returns domain owner
RULE-RET-011	Final-sale classification blocks preference returns but does not automatically block a verified defect remedy.	Distinguishes policy from defect obligations.	Legal + product
RULE-RET-014	An exception approval is scoped to specific rule IDs, item quantities, buyer, and expiry.	Prevents broad or reusable overrides.	Support operations
RULE-RET-019	The cumulative settled refund for an order item must not exceed its refundable captured value after promotion and tax allocation.	Financial invariant.	Finance domain owner
RULE-RET-022	Refund approval occurs only after required receipt/inspection evidence unless an approved “refund-before-return” policy applies.	Fraud and service trade-off.	Risk + product

11.5 Functional requirements in EARS

REQ-RET-001 / UBIQUITOUS

The Returns Service shall assign a globally unique identifier to each return request.

Supports idempotency, audit, buyer support, and cross-system correlation.

REQ-RET-002 / EVENT-DRIVEN

WHEN a buyer submits a return request, the Returns Service shall verify that the buyer is authorized to act on the referenced order items.

Authorization precedes disclosure of policy or order details.

REQ-RET-003 / UNWANTED BEHAVIOR

IF the buyer is not authorized to act on any referenced order item, **THEN** the Returns Service shall reject the request without revealing whether the order or item exists.

Prevents identifier enumeration and information leakage.

REQ-RET-012 / EVENT-DRIVEN

WHEN an authorized buyer submits a return request, the Returns Service shall evaluate each requested item quantity against the applicable eligibility rules and recorded exceptions.

REQ-RET-013 / UNWANTED BEHAVIOR

IF a requested item is outside the applicable Return Window and no active exception approval covers **RULE-RET-007**, **THEN** the Returns Service shall reject that item with reason code `RETURN_WINDOW_EXPIRED`.

REQ-RET-015 / UNWANTED BEHAVIOR

IF order or delivery facts required for a reliable eligibility decision are unavailable, **THEN** the Returns Service shall record the decision as deferred and shall not infer favorable or unfavorable facts.

Makes uncertainty a modeled outcome rather than a generic error or guess.

REQ-RET-016 / UBIQUITOUS

The Returns Service shall retain the input-fact versions, applied rule versions, decision outcome, reason codes, and decision timestamp for every eligibility evaluation.

REQ-RET-018 / EVENT-DRIVEN

WHEN all requested item quantities are eligible, the Returns Service shall create a Return Authorization and shall provide the buyer with the authorized quantities, shipment deadline, and return instructions.

REQ-RET-020 / STATE-DRIVEN

WHILE a return is authorized but not recorded as shipped, the Returns Service shall permit cancellation by the buyer and shall prevent refund approval.

REQ-RET-024 / EVENT-DRIVEN

WHEN Fulfillment records receipt of an authorized return parcel, the Returns Service shall reconcile received item identities and quantities with the authorization.

REQ-RET-026 / UNWANTED BEHAVIOR

IF received item identity or quantity differs from the authorization, **THEN** the Returns Service shall place the affected items in manual review and shall not approve a refund for them.

REQ-RET-030 / EVENT-DRIVEN

WHEN a refund amount is approved, the Returns Service shall issue an idempotent Refund Instruction to Payments containing the return identifier, order-item allocation, currency, amount, and policy trace.

REQ-RET-031 / UNWANTED BEHAVIOR

IF Payments rejects or cannot process a Refund Instruction, **THEN** the Returns Service shall retain the approved amount, record the payment failure state, and make the next recovery action visible to authorized operations users.

REQ-RET-034 / OPTIONAL FEATURE

WHERE instant store credit is enabled and the buyer satisfies the configured trust policy, the Returns Service shall offer store credit as an alternative remedy before physical receipt.

11.6 Non-functional requirements

ID	Requirement	Evidence strategy
NFR-RET-001	At least 99.5% of synchronous eligibility responses shall be available within 5 seconds, measured monthly at the public API boundary, excluding documented client-aborted requests.	Load model, production histogram, monthly SLO evaluation
NFR-RET-002	The submission endpoint shall be idempotent for repeated requests carrying the same buyer-scoped idempotency key and semantically equivalent payload for 24 hours.	Component tests, concurrency test, persistence constraint review
NFR-RET-003	Buyer-facing return journeys shall meet the organization's approved accessibility standard and support keyboard-only completion of critical actions.	Automated scan, keyboard test, assistive-technology review

ID	Requirement	Evidence strategy
NFR-RET-004	Decision records shall be tamper-evident, access-controlled, and retained according to the approved financial and privacy schedule.	Access tests, audit design review, retention job evidence
NFR-RET-005	No payment card data or sensitive authentication information shall be processed or logged by the Returns Service.	Data-flow review, log scanning, contract inspection
NFR-RET-006	Duplicate delivery of a Refund Instruction shall not create more than one provider refund for the same instruction ID.	Idempotency test with repeated and concurrent delivery
NFR-RET-007	Operators shall be able to correlate a buyer-visible return identifier with eligibility, fulfillment, refund, and notification events without accessing raw personal data in logs.	Observability walkthrough and incident simulation

11.7 BDD examples and Gherkin scenarios

Rule: the return window is based on confirmed delivery

```
@RULE-RET-007 @REQ-RET-012 @domain
```

```
Feature: Evaluate the standard return window
```

```
Rule: A normal request is permitted through the instant 30 calendar days after confirmed item delivery
```

```
Scenario: Request at the inclusive deadline
```

```
Given an eligible item was confirmed delivered at "2026-03-01T10:00:00Z"
```

```
And no later delivery correction exists
```

```
When the buyer requests a return at "2026-03-31T10:00:00Z"
```

```
Then the return-window decision is eligible
```

```
And the reason is "WITHIN_RETURN_WINDOW"
```

```
And policy version "RET-WINDOW-2026-01" is recorded
```

```
Scenario: Request immediately after the deadline
```

```
Given an eligible item was confirmed delivered at "2026-03-01T10:00:00Z"
```

```
And no active exception covers the return-window rule
```

```
When the buyer requests a return at "2026-03-31T10:00:00.001Z"
```

```
Then the item is ineligible
```

```
And the reason is "RETURN_WINDOW_EXPIRED"
```

Rule: an exception is explicit and scoped

```
@RULE-RET-014 @REQ-RET-013 @component
Scenario: A late request proceeds under a scoped active exception
Given the item is outside the standard return window
And support approval "EXC-8042" covers:
  | buyer      | BUYER-21 |
  | order-item  | ITEM-93  |
  | quantity    | 1        |
  | rule        | RULE-RET-007 |
  | expires-at  | 2026-04-15T00:00:00Z |
When the buyer requests one unit before the approval expires
Then the return-window decision is eligible by exception
And approval "EXC-8042" is included in the decision trace
```

Rule: missing facts never become guessed facts

```
@REQ-RET-015 @failure @component
Scenario: Delivery evidence is temporarily unavailable
Given the buyer is authorized for the order item
And the Order Service cannot provide a confirmed delivery timestamp
When the buyer submits a return request
Then the eligibility decision is deferred
And the request is not authorized or rejected as out of window
And the buyer is told that eligibility could not yet be confirmed
And a retryable evaluation is scheduled
```

Rule: refund instructions are idempotent

```
@NFR-RET-006 @REQ-RET-030 @contract @critical
Scenario: The same refund instruction is delivered more than once
Given refund instruction "RFI-7001" for 30.00 EUR was accepted by Payments
When the identical instruction is delivered two more times
Then Payments reports the original accepted result for each delivery
And exactly one provider refund exists for instruction "RFI-7001"
```

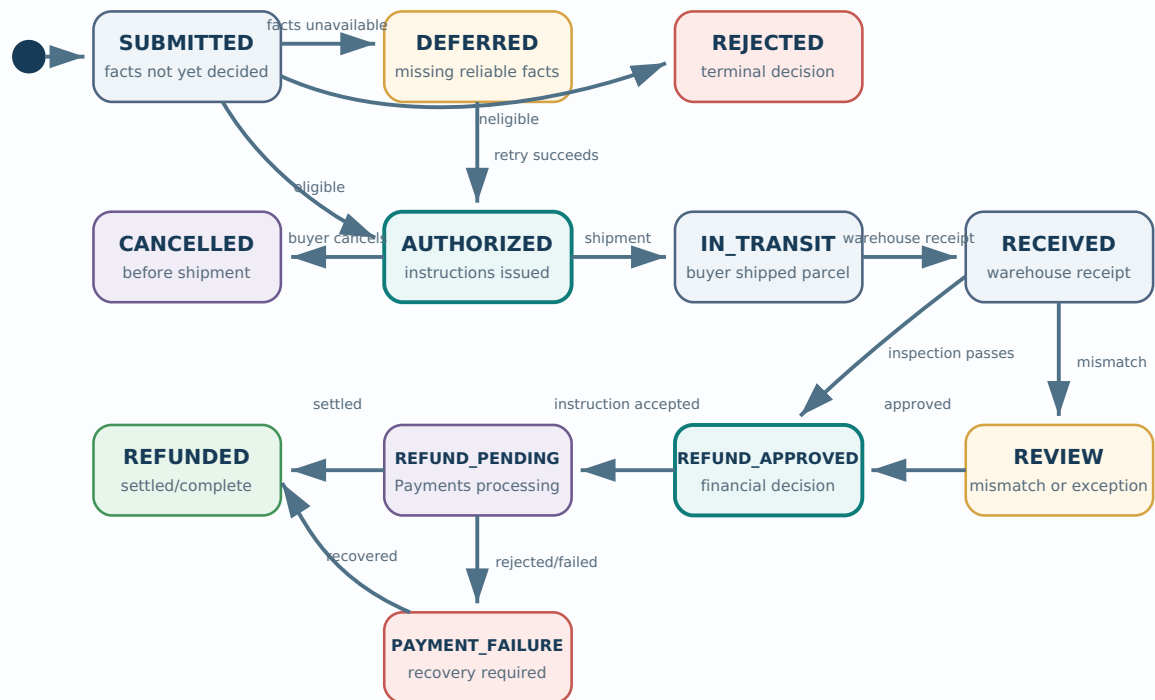
11.8 Eligibility decision table

Case	Authorized buyer	Facts available	Within window	Final sale	Defect path	Exception	Outcome
D1	No	Either	Either	Either	Either	Either	Reject without existence disclosure
D2	Yes	No	Unknown	Unknown	Unknown	Either	Defer; do not guess
D3	Yes	Yes	Yes	No	N/A	N/A	Continue to remaining

Case	Authorized buyer	Facts available	Within window	Final sale	Defect path	Exception	Outcome
							eligibility rules
D4	Yes	Yes	No	No	N/A	No	Reject: window expired
D5	Yes	Yes	No	No	N/A	Window override	Continue; record exception
D6	Yes	Yes	Yes	Yes	No	No	Reject: final sale
D7	Yes	Yes	Either	Yes	Yes	Either	Evaluate defect-remedy policy

11.9 State model

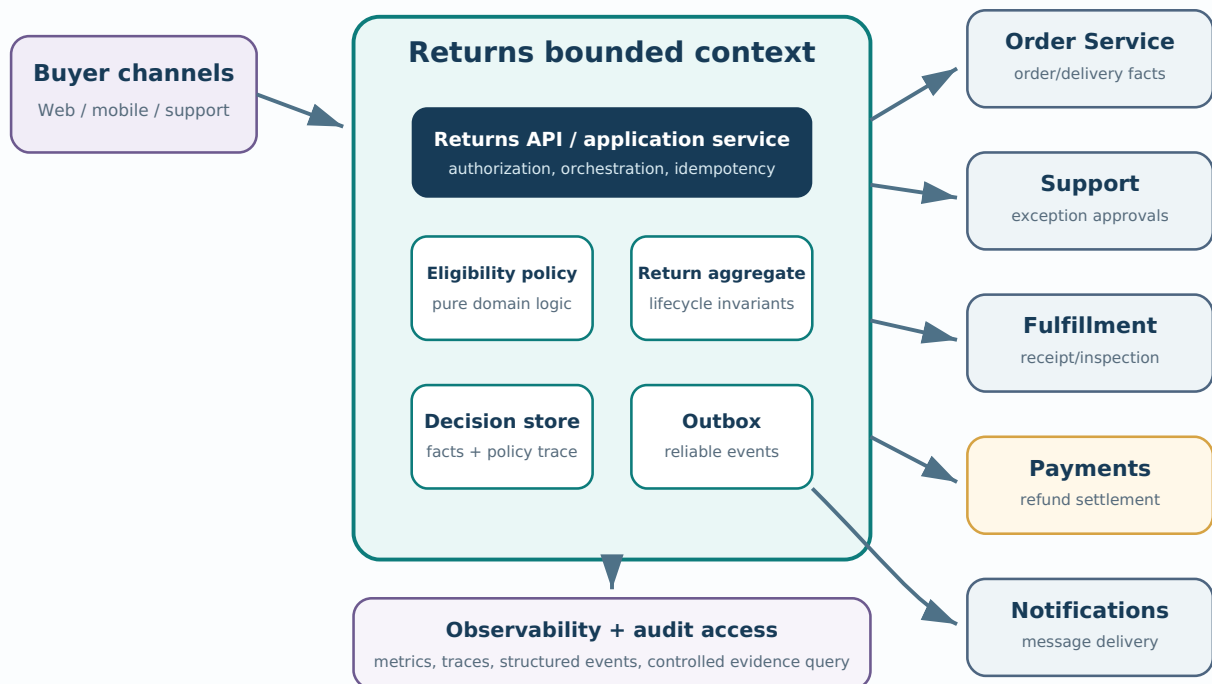
Scenarios describe selected examples; a state model checks lifecycle completeness. Every transition should have an owner, trigger, guard, effect, and allowed next states.

Diagram 12. Return-request state machine

The state model reveals forbidden transitions, recovery states, and terminal states that isolated scenarios can miss.

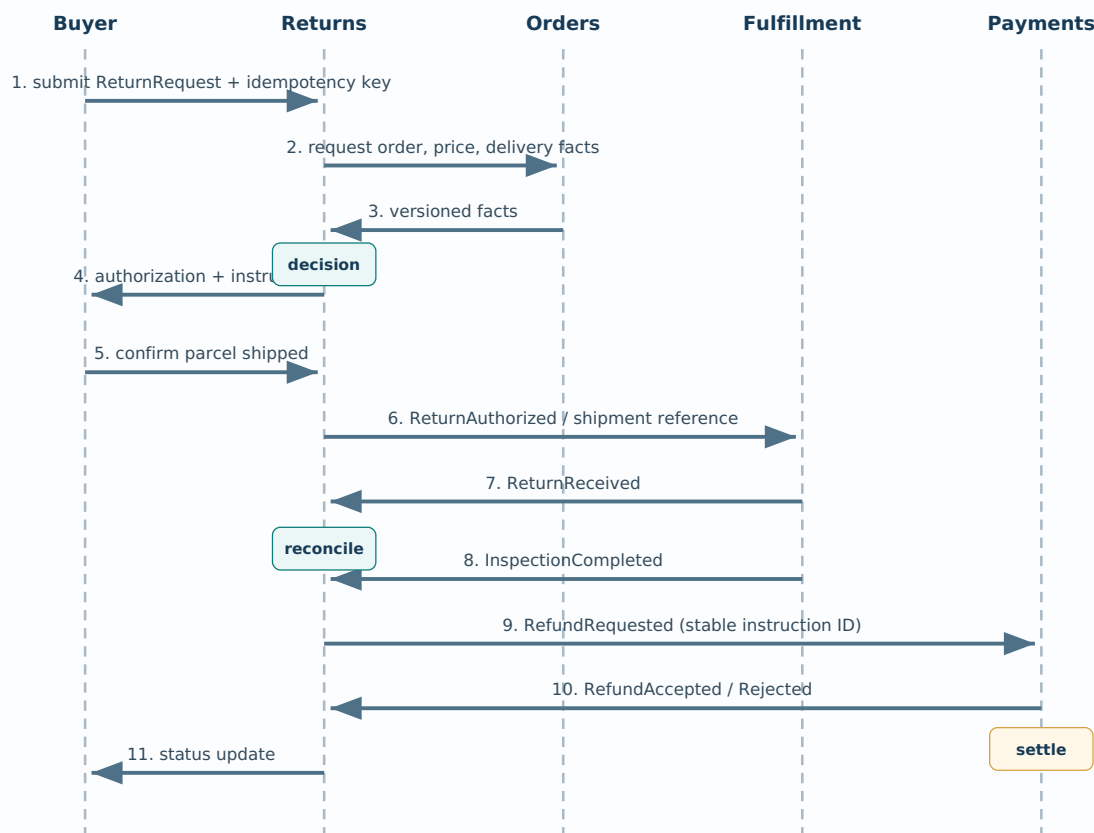
11.10 Architecture and ownership

Diagram 13. Proposed component and contract view



11.11 Critical sequence: from request to refund

Diagram 14. Simplified return/refund sequence



The diagram distinguishes decisions from facts and commands. Delivery, retries, and detailed error branches would be expanded in the technical design.

11.12 Public API sketch

The following contract is illustrative. The specification states semantics and invariants; an OpenAPI document in the design repository would provide the complete machine-readable schema.

POST /v1/returns

Authorization: Bearer <buyer-token>

Idempotency-Key: 8c8e2a58-...

Content-Type: application/json

```
{
  "orderId": "ORD-1729",
  "items": [
    { "orderId": "ITEM-93", "quantity": 1, "reason": "T00_SMALL" }
  ],
  "preferredRemedy": "ORIGINAL_PAYMENT"
}
```

201 Created

```
{
  "returnId": "RET-6501",
  "status": "AUTHORIZED",
  "decision": {
    "evaluatedAt": "2026-03-31T10:00:00Z",
    "policyVersion": "RET-POLICY-EU-2026-01",
    "itemResults": [
      { "orderId": "ITEM-93", "eligible": true, "reason": "WITHIN_RETURN_WINDOW" }
    ]
  },
  "instructions": {
    "shipBy": "2026-04-14T23:59:59Z",
    "labelUrl": "/v1/returns/RET-6501/label"
  },
  "links": { "self": "/v1/returns/RET-6501" }
}
```

Error semantics

Condition	HTTP behavior	Domain behavior	Disclosure rule
Malformed request	400 with field problems	No return request created	May identify submitted invalid fields
Buyer not authorized	404 or policy-approved non-disclosing response	Security rejection recorded with limited data	Do not reveal order/item existence
Business ineligibility	422 or 200 decision representation, according to API convention	Decision recorded with reason	Provide buyer-safe reason and next action
Required facts unavailable	202 with DEFERRED state or 503, according to interaction design	Retryable evaluation recorded	Do not present guessed ineligibility
Same idempotency key, equivalent payload	Return original result	No duplicate aggregate	Same authorized buyer only
Same key, different payload	409 conflict	No mutation of original request	

Condition	HTTP behavior	Domain behavior	Disclosure rule
			Explain key reuse without leaking other data

11.13 Event contract sketch

```
{
  "type": "RefundRequested",
  "schemaVersion": 1,
  "eventId": "EVT-9481",
  "occurredAt": "2026-04-05T12:30:44Z",
  "correlationId": "RET-6501",
  "data": {
    "refundInstructionId": "RFI-7001",
    "returnId": "RET-6501",
    "orderId": "ORD-1729",
    "currency": "EUR",
    "amount": "30.00",
    "allocations": [
      { "orderId": "ITEM-93", "merchandise": "30.00", "tax": "0.00" }
    ],
    "decisionTrace": {
      "policyVersion": "RET-POLICY-EU-2026-01",
      "inspectionId": "INSP-4402",
      "approvedAt": "2026-04-05T12:30:40Z"
    }
  }
}
```

Contract invariants

- `refundInstructionId` is stable across retries and unique for the approved financial instruction.
- Amounts are serialized as decimal strings with an explicit currency; binary floating-point values are not used for money.
- The event contains identifiers and financial allocation, but avoids unnecessary buyer personal data.
- Schema evolution is additive for compatible changes; semantic changes require a versioning decision.
- Consumers must tolerate duplicate delivery and record the result associated with the idempotency key.

11.14 Data and audit model

Record	Key fields	Purpose	Important constraints
ReturnRequest	returnId, buyerRef, orderId, state, createdAt	Aggregate lifecycle and buyer-facing identity	State transitions optimistic/serialized; buyerRef protected

Record	Key fields	Purpose	Important constraints
ReturnItem	orderId, requestedQty, authorizedQty, receivedQty	Item-level quantity control	Cumulative quantities preserve RULE-RET-009
EligibilityDecision	decisionId, inputSnapshotRefs, rules, outcome, reasons	Reproduce why a decision was made	Immutable; includes policy and fact versions
ExceptionApproval	approvalId, scope, actor, reason, expiresAt	Controlled override	Cannot be expanded after use without a new approval
Inspection	inspectionId, receiptId, observed items, outcome	Warehouse evidence	Corrections append a new version and retain history
RefundInstruction	instructionId, amount, allocation, status	Financial command and reconciliation	Idempotent, amount invariant, immutable approved basis
OutboxEvent	eventId, aggregateVersion, payload, publish status	Reliable publication after state change	Transactional write with aggregate changes

11.15 Task decomposition by vertical slices

Horizontal tasks such as “create database,” “build backend,” and “add UI” delay usable evidence. Vertical slices deliver a narrow behavior across the necessary layers.

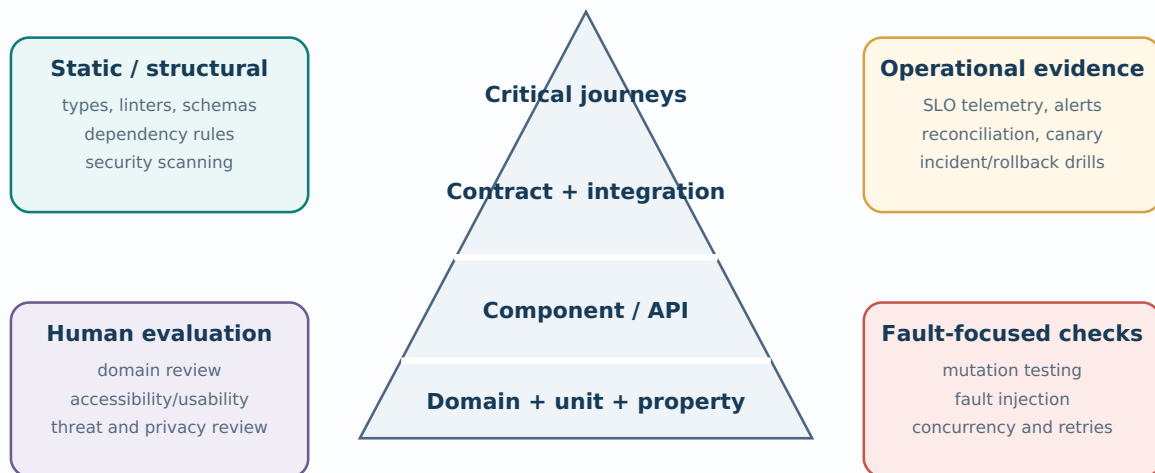
Slice	Behavior delivered	Representative tasks	Exit evidence
S1: policy core	Evaluate return-window and quantity rules from explicit facts	Domain types, policy object, boundary tests, property tests, reason codes	Pure domain suite demonstrates RULE-RET-007 and RULE-RET-009
S2: request API	Authorized buyer submits an eligible request idempotently	Endpoint, authorization, order-facts port, aggregate/store, API tests	SCN happy/boundary/idempotency examples pass
S3: buyer experience	Buyer sees decision and accessible instructions	UI flow, content, label access, accessibility checks, journey test	Keyboard completion and selected end-to-end scenario
S4: receipt and inspection	Warehouse facts update lifecycle and handle mismatch	Event consumer, reconciliation, review state, operator view	Contract and state-transition tests
S5: refund	Approved amount creates one idempotent financial instruction	Calculation, outbox, Payments contract, retry/reconciliation, alerts	Duplicate-delivery and amount-invariant evidence
S6: controlled rollout	Feature enabled for selected market and observed	Flag, migration, dashboards, runbook, rollback drill	Readiness review and canary success criteria

11.16 Detailed tasks for Slice 1

ID	Task	Trace	Notes
TASK-RET-101	Create domain value objects for DeliveryInstant, RequestInstant, ReturnWindow, ItemQuantity, and DecisionReason.	TERM-RET-04, RULE-RET-007	Reject invalid quantities and preserve time-zone semantics.
TASK-RET-102	Implement StandardReturnWindow policy through TDD.	REQ-RET-012, REQ-RET-013	Inclusive exact deadline; clock is explicit.
TASK-RET-103	Add property-based tests for temporal ordering around arbitrary delivery instants.	RULE-RET-007	For any instant d: requests at or before d+30d eligible; after expired, subject to defined precision.
TASK-RET-104	Implement RemainingReturnableQuantity policy.	RULE-RET-009	Include prior authorized/received/refunded quantities according to approved lifecycle rule.
TASK-RET-105	Model Deferred decision when required facts are absent.	REQ-RET-015	Do not use exceptions or null as the business result.
TASK-RET-106	Persist policy version and reason in domain decision representation.	REQ-RET-016	Supports audit and change analysis.
TASK-RET-107	Review domain language and remove synonyms from API/domain names.	P-01, glossary	Explicit human review gate.

11.17 Test portfolio

Diagram 15. Evidence portfolio—not merely a test pyramid



Confidence comes from complementary evidence, not from one layer or one metric.

Representative tests

Test ID	Layer	Behavior/fault	Trace
TEST-RET-001	Domain	Exact day-30 deadline is eligible	RULE-RET-007, SCN-RET-001
TEST-RET-002	Property	Any request strictly after deadline is expired absent exception	RULE-RET-007
TEST-RET-006	Domain	Requested quantity greater than remaining quantity is rejected	RULE-RET-009
TEST-RET-011	API component	Unauthorized buyer receives non-disclosing response	REQ-RET-002, 003
TEST-RET-015	API component	Equivalent repeated idempotency key returns original result	NFR-RET-002
TEST-RET-021	Contract	RefundRequested validates schema and money representation	ADR-RET-003, REQ-RET-030
TEST-RET-022	Integration	Three duplicate deliveries produce one provider refund	NFR-RET-006
TEST-RET-030	Journey	Buyer requests eligible return and obtains instructions with keyboard navigation	REQ-RET-018, NFR-RET-003
TEST-RET-041	Operational	Simulated payment rejection creates alert and recoverable state	REQ-RET-031, NFR-RET-007

11.18 Implementation sketch: idempotent request creation

```
def create_return(command, principal):
    authorize_buyer(principal, command.order_id)

    existing = idempotency_store.find(
        scope=principal.buyer_id,
        key=command.idempotency_key,
    )
    if existing:
        if existing.request.fingerprint != fingerprint(command.payload):
            raise IdempotencyConflict()
        return existing.original_response

    facts = order_facts.load_for_return(command.order_id, command.items)
    decisions = eligibility_policy.evaluate(command, facts)

    request = ReturnRequest.create(
        return_id=return_ids.next(),
        command=command,
        facts=facts,
        decisions=decisions,
    )

    with transaction():
        return_repository.save(request)
        idempotency_store.record(
            scope=principal.buyer_id,
            key=command.idempotency_key,
            request_fingerprint=fingerprint(command.payload),
            original_response=present(request),
            expires_at=clock.now() + hours(24),
        )
        outbox.add_all(request.pending_events())

    return present(request)
```

This pseudocode makes several specification obligations visible: authorization occurs before sensitive lookup; repeated equivalent requests return the original result; key reuse with a different payload conflicts; decisions retain facts; persistence and event publication use a transaction/outbox boundary.

11.19 Implementation sketch: idempotent refund consumer

```
def handle_refund_requested(event):
    instruction_id = event.data.refund_instruction_id

    prior = refund_ledger.find_by_instruction(instruction_id)
    if prior:
        publish(prior.as_result_event(causation_id=event.event_id))
        return

    validate_amount_and_currency(event.data)

    try:
        provider_result = payment_provider.refund(
            provider_payment_ref=resolve_payment(event.data.order_id),
            amount=event.data.amount,
            currency=event.data.currency,
            idempotency_key=instruction_id,
        )
    except TemporaryProviderError:
        schedule_retry(event, stable_key=instruction_id)
        return

    with transaction():
        refund_ledger.record_once(instruction_id, provider_result)
        outbox.add(result_event(event, provider_result))
```

True end-to-end “exactly once” behavior is usually achieved through idempotent effects and reconciliation rather than assuming messages are delivered exactly once. The specification should state the required business outcome—no duplicate refund—not overpromise a particular transport guarantee.

11.20 Threat and abuse cases

Threat / misuse	Potential impact	Specification control	Verification
Enumerate order-item IDs	Disclosure of another buyer’s purchase	REQ-RET-002/003 non-disclosing authorization	Cross-account API tests and log review
Replay request	Duplicate return authorization	NFR-RET-002 idempotency	Repeated/concurrent submission tests
Reuse support exception	Unauthorized policy bypass	RULE-RET-014 scoped, expiring approval	Scope and expiry tests; audit query
Manipulate refund amount in transit	Financial loss	Authenticated contract, amount invariant, policy trace	Schema/signature controls, negative contract tests
Duplicate event delivery	Multiple provider refunds	NFR-RET-006 stable idempotency key	Fault injection and reconciliation

Threat / misuse	Potential impact	Specification control	Verification
Sensitive data in logs	Privacy breach	NFR-RET-005 + logging principle	Automated scanning and sampled inspection

11.21 Release and rollback plan

1. **Shadow evaluation:** compute eligibility in parallel with the existing support process; do not expose decisions to buyers. Compare outcomes and reasons.
2. **Internal pilot:** enable support agents to view proposed decisions and flag disagreements with categorized reasons.
3. **Small market canary:** expose self-service to a controlled share of eligible traffic, excluding high-risk categories.
4. **Progressive expansion:** increase exposure only when decision accuracy, failure rates, latency, support contacts, and refund reconciliation remain within thresholds.
5. **Containment:** disable new self-service authorization through a controlled feature switch while preserving status access for existing returns.
6. **Rollback:** stop new decisions, drain/reconcile pending events, route unresolved requests to support, and preserve audit evidence.

11.22 Definition of done

- ☐ Goals, scope, glossary, business rules, and requirements are approved by named owners.
- ☐ Open questions affecting policy, money, authorization, or compliance are resolved or explicitly excluded from the release.
- ☐ Representative, boundary, negative, exception, and dependency-failure examples exist.
- ☐ Architecture decisions and contracts link to requirements and preserve context ownership.
- ☐ Tasks are complete with reviewable evidence, not merely marked done.
- ☐ Critical functional, security, idempotency, accessibility, and recovery checks pass.
- ☐ Data migration and compatibility behavior are tested against representative production-like data.
- ☐ Dashboards, alerts, runbooks, correlation, and reconciliation are demonstrated.
- ☐ Rollout and containment controls have named owners and decision thresholds.
- ☐ Living documentation and trace links reflect the released behavior.

11.23 A compact traceability view

Goal	Rule / requirement	Scenario	Design / task	Evidence
GOAL-RET-01	RULE-RET-007; REQ-RET-012/013	Boundary and late request	EligibilityPolicy; TASK-RET-102/103	TEST-RET-001/002 + shadow comparison

Goal	Rule / requirement	Scenario	Design / task	Evidence
GOAL-RET-03	RULE-RET-019; REQ-RET-030; NFR-RET-006	Duplicate refund instruction	ADR-RET-003; outbox + ledger tasks	Contract, duplicate-delivery, reconciliation tests
GOAL-RET-02	REQ-RET-018; NFR-RET-001/003	Eligible buyer receives accessible instructions	API/UI slice + observability	Journey, accessibility review, latency SLO

What the case study demonstrates

The same feature is viewed through several lenses. DDD defines meaning and ownership. EARS structures obligations. BDD discovers examples and questions. Gherkin records selected behavior. Specification by Example clarifies calculations and combinations. SDD connects the artifacts to plan and tasks. TDD drives local implementation. Operational evidence closes the loop.

12 Brownfield change and regression control

Most valuable software already exists. Brownfield SDD must describe both the intended change and the behavior that must remain unchanged. It also needs an explicit way to discover the current system rather than assuming existing documentation is accurate.

12.1 Example change request

“Extend the standard return window from 30 to 45 days for loyalty-tier Gold buyers, but do not change final-sale, defect, refund-before-return, or regional statutory policies.”

This appears to be a small constant change. In a mature system it may affect policy configuration, eligibility code, content, analytics, support scripts, cached decisions, existing authorizations, test data, data warehouse logic, and public promises. SDD turns the request into an impact-analysis problem.

12.2 Baseline the existing behavior

Before editing the specification, gather evidence:

- current rule and requirement artifacts;
- production code paths and policy configuration;
- automated tests and their actual execution results;
- API and event contracts;
- user-facing content and support procedures;
- production telemetry and samples of historical decisions;
- known defects, exceptions, and manual workarounds.

Descriptive versus normative truth

The running system shows what currently happens. The approved policy states what should happen. A brownfield analysis must preserve this distinction. Existing code may contain a defect; existing documentation may be stale.

12.3 Express changed and unchanged behavior

REQ-RET-012A / CHANGED BEHAVIOR

WHERE the buyer's loyalty tier is Gold at the request timestamp and the standard loyalty extension is enabled for the applicable market, **WHEN** the buyer submits a return request no later than 45 calendar days after confirmed item delivery, the Returns Service shall evaluate the item as within the loyalty Return Window.

REQ-RET-012B / UNCHANGED BEHAVIOR

WHERE the buyer does not satisfy the active Gold loyalty policy, the Returns Service shall continue to apply the existing market-specific standard Return Window.

REQ-RET-012C / UNCHANGED BEHAVIOR

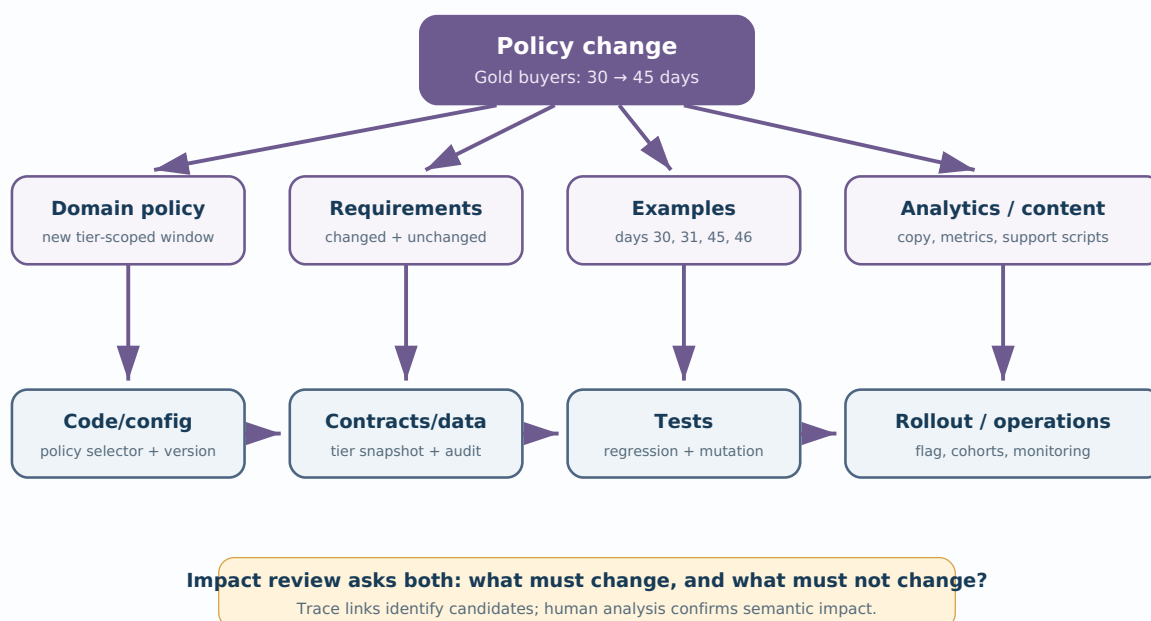
The loyalty Return Window extension shall not override final-sale restrictions, defect-policy routing, quantity limits, or required authorization checks.

REQ-RET-012D / TEMPORAL RULE

The Returns Service shall determine loyalty eligibility using the buyer tier recorded at the return-request timestamp, not the current tier at a later re-evaluation.

12.4 Change-impact graph

Diagram 16. Propagating a 30-day to 45-day loyalty policy change



12.5 Migration and temporal semantics

The policy change creates questions that a simple constant edit does not answer:

- Does the 45-day policy apply to items delivered before the rollout date?
- What if the buyer was Gold at delivery but not at request, or vice versa?
- What policy applies when a deferred request is re-evaluated after tier changes?
- Should previously rejected day-31-to-45 requests be reopened?
- How is the policy version represented in audit, analytics, and support tools?
- Can a market override the loyalty extension?

A recommended rule is to record all time-sensitive decision inputs—including tier snapshot, market, request timestamp, and policy version—so a decision can be reproduced later.

12.6 Regression examples

Case	Buyer / time	Expected after change	Purpose
BC-01	Gold, request on day 31	Within loyalty window	New behavior
BC-02	Gold, exact day 45	Within loyalty window	New boundary
BC-03	Gold, after day 45	Expired absent exception	New negative boundary

Case	Buyer / time	Expected after change	Purpose
BC-04	Standard tier, day 31	Still expired	Unchanged standard policy
BC-05	Gold, day 40, final sale, preference return	Still rejected as final sale	No unintended override
BC-06	Gold, day 40, final sale, verified defect	Route to defect policy	Preserve policy separation
BC-07	Gold at request, downgraded before deferred evaluation	Use request-time tier snapshot	Temporal consistency

12.7 Brownfield delivery strategy

1. **Characterize current behavior** with focused tests and production samples without declaring every behavior correct.
2. **Separate the policy** from scattered conditionals behind a well-named domain interface.
3. **Add the new policy version** without immediately deleting the old version.
4. **Introduce a deterministic selector** based on market, tier snapshot, and effective date.
5. **Run shadow decisions** to compare old and new outcomes for eligible cohorts.
6. **Enable through configuration** for a small cohort, monitor discrepancies, and retain rollback.
7. **Retire old paths** only after historical decisions and outstanding returns no longer require them.

Key principle

Brownfield SDD does not assume that regeneration is safe. It constrains change with characterization evidence, explicit unchanged behavior, compatibility analysis, policy versioning, and controlled rollout.

PART IV

Quality, evidence, and research

How to evaluate alignment, propagate change, design specification-mutation experiments, and adopt SDD without mistaking documentation volume for quality.

13 Traceability and change-impact analysis

Traceability is the ability to follow a development object to its origins, transformations, realization, and evidence. In SDD it is the connective tissue that turns separate documents and tests into a coherent specification system.

13.1 Trace link types

Link type	Meaning	Example	Question supported
Originates from	Records stakeholder, policy, regulation, incident, or research source.	RULE-RET-007 originates from Return Policy v4.2.	Why does this rule exist, and who can change it?
Refines	Provides more precise or lower-level expression.	REQ-RET-013 refines RULE-RET-007 for late requests.	How was an abstract policy translated into system behavior?
Illustrated by	Connects a rule/ requirement to concrete examples.	REQ-RET-013 is illustrated by day-30 and day-30+1ms scenarios.	Which examples explain the intended boundary?
Realized by	Connects behavior to design, task, component, or code.	REQ-RET-030 is realized by ADR-RET-003 and RefundOutboxPublisher.	Where is the requirement implemented?
Verified by	Connects an obligation to evidence.	NFR-RET-006 is verified by duplicate-delivery and reconciliation tests.	What demonstrates satisfaction?
Depends on	Records semantic or execution dependency.	Refund approval depends on a valid inspection outcome.	What else may be affected by a change?
Conflicts with	Records an unresolved or accepted tension.	Immediate refund experience conflicts with fraud-risk constraint.	Which trade-off requires a decision?
Supersedes	Preserves history while replacing a prior version.	RULE-RET-007 v2 supersedes v1 for selected markets.	Which policy applied at a historical time?

13.2 Traceability directions

Forward traceability

From goal or requirement to design, task, code, test, and release. It supports coverage and impact analysis.

Example: Which implementation and tests must change when the Return Window changes?

Backward traceability

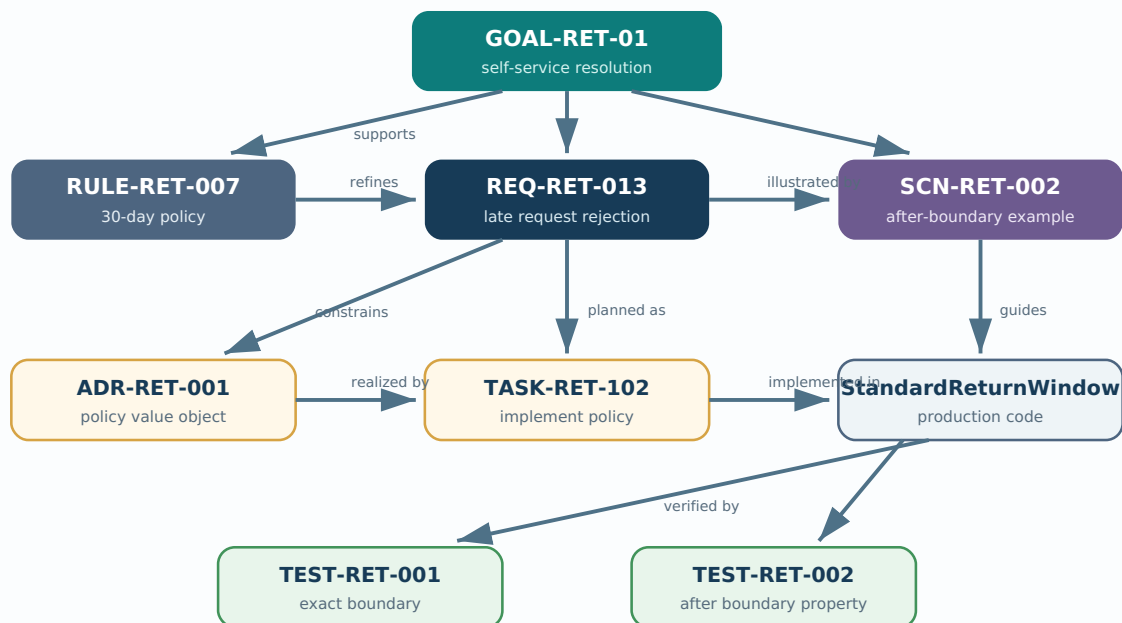
From code, test, or design decision back to the requirement and source. It supports justification and removal.

Example: Why does this policy branch exist, and may we delete it?

13.3 A graph rather than a spreadsheet

A trace matrix is a useful view, but the underlying model is a graph. Nodes are artifacts; typed edges express relationships. Multiple views can be generated from the same graph: requirement coverage, test evidence, release scope, architecture impact, or policy lineage.

Diagram 17. Trace graph for one return-window rule



13.4 Traceability matrix example

Requirement	Source/ rule	Examples	Design	Tasks	Evidence	Status
REQ-RET-013			ADR-RET-001			Covered

Requirement	Source/ rule	Examples	Design	Tasks	Evidence	Status
	RULE-RET-007	SCN-RET-001/002		TASK-RET-102/103	TEST-RET-001/002	
REQ-RET-015	Risk: unreliable delivery facts	SCN-RET-006	DeferredDecision model	TASK-RET-105	Component failure test	Covered
REQ-RET-030	RULE-RET-019	Refund instruction scenario	ADR-RET-003	Refund slice tasks	Contract + integration	Covered
NFR-RET-003	Accessibility policy	Keyboard journey	UI design record	Journey/ accessibility tasks	Scan + human review	Pending human evidence

13.5 Coverage is not binary

“Requirement has a test” is too weak. A critical requirement may need multiple evidence types, while some requirements cannot be fully verified by automated tests.

Coverage dimension	Example question
Scenario coverage	Do examples cover positive, negative, boundary, exception, and failure behavior?
Rule coverage	Is every business rule represented in at least one requirement or model?
Decision coverage	Are architecture decisions connected to the constraints that justify them?
Evidence diversity	Does a critical NFR rely on an appropriate combination of test, review, and production measurement?
Bidirectional integrity	Can every implemented behavior be justified, and can every approved requirement be located in the implementation?
Freshness	Were links revalidated after the last relevant change?

13.6 Automated checks versus human judgment

Good automation candidates

- Missing IDs or duplicate IDs
- Broken links
- Requirement without any task/evidence
- Task without requirement/design link
- Unknown glossary term or forbidden synonym
- Changed contract without version decision
- Stale generated view

Requires semantic review

- Whether a test truly proves a requirement
- Whether an example is representative
- Whether a design satisfies a quality attribute
- Whether a source policy is interpreted correctly
- Whether a change affects behavior indirectly
- Whether a mutation is equivalent

13.7 Change-impact procedure

1. **Identify the changed node.** Policy, requirement, domain term, interface, test oracle, or implementation constraint.
2. **Classify the change.** Editorial, clarification, semantic behavior, scope, quality threshold, design, compatibility, or ownership.
3. **Traverse direct links.** Find refining requirements, examples, design decisions, tasks, components, evidence, dashboards, and documentation.
4. **Search for unlinked semantic references.** Terms, constants, reason codes, thresholds, and copied text may exist outside the graph.
5. **Assess each candidate.** Mark change required, review only, unaffected with rationale, or uncertain.
6. **Update coherent artifact set.** Do not merge a changed requirement while known dependent examples and tests still assert old behavior.
7. **Validate regression and unchanged behavior.** Execute appropriate tests and compare operational or historical cases.
8. **Record the decision.** Preserve why some linked artifacts did not change.

13.8 Traceability smells

- **Link dumping:** every task links to the entire feature spec, providing no useful precision.
- **Coverage theater:** a requirement is marked covered because one test ID exists, although the test asserts only a happy path.
- **Generated-link overconfidence:** semantic similarity is treated as proof of satisfaction.
- **Unversioned policy:** historical decisions cannot identify which rule set applied.
- **Dead requirement:** an approved requirement has no current realization or release disposition.
- **Orphan code:** a business policy branch has no source or current owner.

- **Stale link:** an artifact ID remains but the linked content changed meaning.

13.9 Minimal trace metadata

```
id: REQ-RET-013
status: approved
owner: returns-policy-owner
source:
  - RULE-RET-007@v2
refines:
  - GOAL-RET-01
illustrated_by:
  - SCN-RET-001
  - SCN-RET-002
realized_by:
  - ADR-RET-001
  - src/returns/domain/standard_return_window.py
verified_by:
  - TEST-RET-001
  - TEST-RET-002
introduced_in: release-2026.04
last_semantic_review: 2026-03-18
```

Practical rule

Start with a small set of high-value, typed links. A trace system that nobody maintains is worse than a modest system that reliably supports change decisions.

14 Specification mutation analysis

Mutation testing traditionally evaluates a test suite by introducing small faults into code and checking whether the tests detect them. Specification mutation moves the intervention upstream: deliberately alter a requirement, rule, example, model, or contract and evaluate whether the lifecycle reacts correctly.

14.1 Why mutate specifications?

In an AI-assisted SDD pipeline, a subtle specification defect can propagate consistently into plan, tasks, code, and tests. Everything may agree—and still be wrong. Specification mutation analysis can assess several distinct capabilities:

- **Requirement review strength:** do humans or automated checks recognize the altered meaning?
- **Artifact propagation:** if the change is intentional, do downstream artifacts update consistently?
- **Regression sensitivity:** if the change is accidental, do existing examples and tests reject it?
- **Trace quality:** are affected artifacts identified accurately?
- **Agent reliability:** does an AI agent preserve, ignore, distort, or overgeneralize the mutation?
- **Specification strength:** is the specification precise enough to distinguish correct and faulty implementations?

Related research has applied mutation ideas to requirements inspection and formal specifications. Li et al. proposed scenario generation from mutated requirement models for inspection [26]. Cordy and colleagues proposed mutation model checking to identify formal specifications too weak to reject faulty models [27]. MutDafny, accepted for ICSE 2026 according to its current preprint, evaluates formal specification strength using mutation-based analysis [28]. These works do not directly establish a standard method for natural-language SDD pipelines, but they provide strong conceptual foundations.

14.2 Two fundamentally different experiments

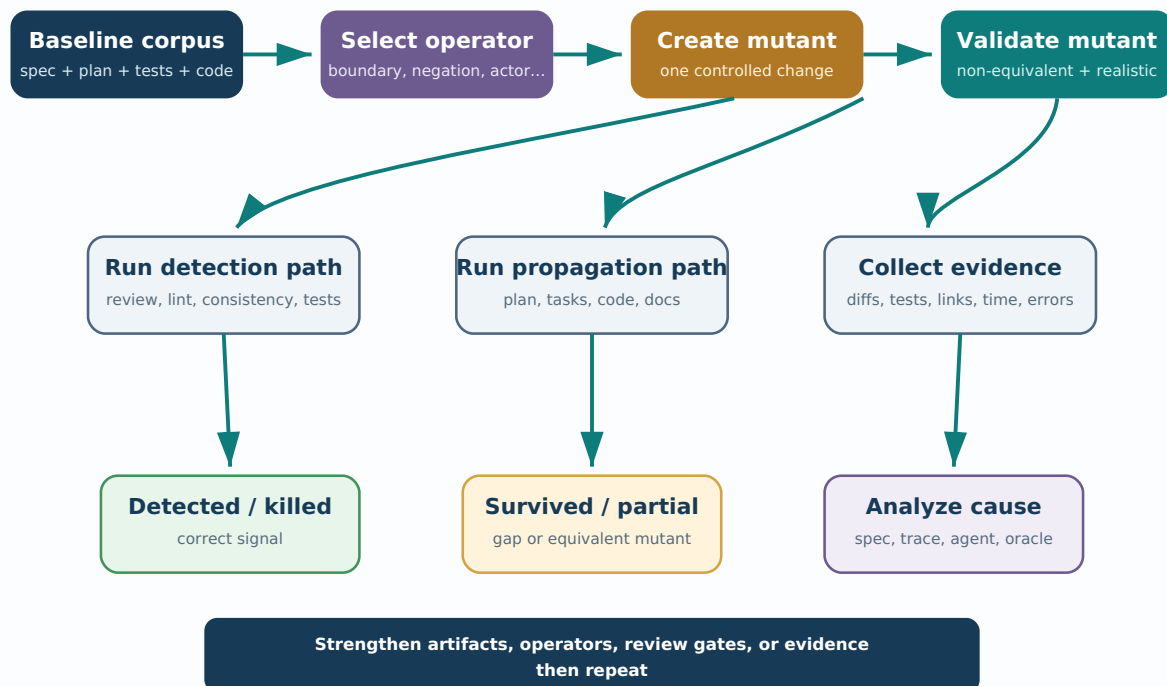
Experiment	Mutation status	Desired response	Question answered
Accidental-defect simulation	The mutant represents an incorrect specification.	Review, consistency checks, examples, or tests should reject/flag it.	Can the lifecycle detect a specification fault?
Intentional-change propagation	The mutant represents an approved change request.	Plan, tasks, code, tests, docs, and operations should update consistently.	Can the lifecycle implement and trace a semantic change?

Do not mix the oracles

In the first experiment, preserving old behavior may be correct because the mutation is a defect. In the second, preserving old behavior is a failure because the change is intended. Label every mutant with its expected semantic status.

14.3 Mutation pipeline

Diagram 18. Specification-mutation analysis pipeline



14.4 Natural-language specification mutation operators

Operator family	Original	Mutant	Potential fault simulated
Boundary replacement	within 30 days	within 45 days	Wrong threshold or approved policy change
Comparator replacement	at or before deadline	before deadline	Inclusive/exclusive boundary error
Negation insertion/deletion	shall reject	shall not reject	Polarity reversal
Deontic strength	shall	should / may	

Operator family	Original	Mutant	Potential fault simulated
			Mandatory behavior weakened
Trigger deletion	WHEN payment fails...	The system shall retry...	Response becomes unconditional or ambiguous
State replacement	WHILE awaiting inspection	WHILE in transit	Wrong lifecycle state
Actor substitution	authorized buyer	user	Authorization scope broadened
Object substitution	order item	order	Granularity or domain object changed
Quantity/unit replacement	5 seconds	5 minutes	Quality threshold or unit fault
Exception deletion	and no active exception exists	clause removed	Approved override ignored
Scope expansion	Gold buyers in enabled markets	all buyers	Feature/configuration condition removed
Response omission	record failure and schedule retry	record failure	Recovery behavior omitted
Temporal-reference shift	tier at request time	current tier	Historical/re-evaluation inconsistency
Reason-code replacement	RETURN_WINDOW_EXPIRED	FINAL_SALE	Incorrect explanation/audit semantics

14.5 Structural mutations

Not all important mutations change a word. Structural operators can simulate common specification defects:

- delete one requirement from a rule's coverage;
- merge two atomic requirements into one compound statement;
- remove a negative or failure scenario;
- detach a requirement from its policy source;
- remove a trace link to a test or design decision;
- move behavior to the wrong bounded context;
- delete an “unchanged behavior” constraint from a brownfield change;
- change scenario data without changing the expected result;
- change an API schema but leave examples and consumers on the old version.

14.6 Worked mutation: return-window boundary

BASELINE REQ-RET-013

IF a request is submitted after 30 calendar days and no active exception exists, THEN the Returns Service shall reject the item with
`RETURN_WINDOW_EXPIRED`.

MUTANT M-BND-01

IF a request is submitted after **45** calendar days and no active exception exists, THEN the Returns Service shall reject the item with
`RETURN_WINDOW_EXPIRED`.

Accidental-defect oracle

The mutation should be detected because it conflicts with RULE-RET-007, existing day-31 rejection examples, policy configuration, and test oracles. A successful lifecycle may signal:

- cross-artifact inconsistency: rule says 30, requirement says 45;
- scenario conflict: day 31 expected ineligible;
- test failure if code is regenerated to implement 45 days;
- policy review rejection because no approved source changed.

Intentional-change oracle

If the mutation is an approved change for Gold buyers, a correct propagation is not a global replacement of 30 with 45. It should add scope, tier-time semantics, policy versioning, new examples, unchanged-behavior constraints, configuration, analytics, and rollout controls. A superficially consistent global change would be a propagation error.

14.7 Expected artifact delta for an intentional change

Artifact	Expected change	Incorrect but plausible agent behavior
Domain rule	Add loyalty-specific rule and relationship to standard rule.	Overwrite standard policy globally.
EARS requirements	Add WHERE condition, tier snapshot semantics, and unchanged constraints.	Replace every “30” with “45.”
Gherkin examples	Add Gold day 31/45/46; preserve Standard day 31 and final-sale cases.	Update only the happy-path example.
Design	Add policy selector, policy version, tier snapshot, configuration/effective date.	Change a constant in one service.
Tasks	Include code, tests, content, analytics, migration, rollout, and audit.	Generate one task: “Update return window.”
Code	Versioned policies and deterministic selection.	

Artifact	Expected change	Incorrect but plausible agent behavior
		Conditional scattered across API/controller/UI.
Evidence	New, unchanged, boundary, temporal, and rollout checks.	Delete or rewrite old tests until green.

14.8 Mutation outcomes

Outcome	Meaning	Next action
Killed correctly	A defect mutant is detected by the intended mechanism.	Record detection layer and time.
Propagated correctly	An intentional change reaches all required artifacts with preserved constraints.	Measure completeness and unnecessary churn.
Survived	No review, check, or test distinguishes the mutant.	Investigate missing oracle, weak spec, trace gap, or equivalent mutation.
Partially propagated	Some artifacts change while others retain old meaning.	Classify omissions and semantic contradictions.
Over-propagated	Unaffected behavior or artifacts change unnecessarily.	Improve scope reasoning and trace precision.
Equivalent / neutral	The mutant does not alter observable intended semantics.	Exclude from score or classify separately after review.
Invalid	The mutant is syntactically broken or unrealistic for the study.	Fix operator or discard according to protocol.

14.9 Metrics for specification mutation

One possible family of metrics follows. These are proposed operationalizations, not established universal standards.

Metric	Illustrative definition	Interpretation caution
Defect mutation detection rate	Detected non-equivalent defect mutants / valid non-equivalent defect mutants	A high rate may reflect easy or unrealistic mutants.
Propagation completeness	Required changed artifacts correctly changed / all required changed artifacts	Requires a trusted gold-standard impact set.
Preservation precision	Required unchanged artifacts preserved / all required unchanged artifacts	Textual differences do not necessarily imply semantic change.
Trace impact recall		

Metric	Illustrative definition	Interpretation caution
	Correctly identified affected artifacts / all truly affected artifacts	Gold impact labels require expert judgment.
Trace impact precision	Correctly identified affected artifacts / all artifacts flagged affected	Review-only candidates may be legitimately broad.
Semantic consistency rate	Mutants with no unresolved contradiction after processing / processed intentional changes	Automated consistency checks may miss deep semantic conflicts.
Detection latency	Time or lifecycle stage at which a defect mutant is first identified	Earlier is not always better if early checks have high false positives.
Repair effort	Human time, agent turns, code changes, or review cycles to restore correctness	Tool familiarity and project complexity strongly influence results.

14.10 Equivalent mutants and semantic ambiguity

Equivalent mutants are a major challenge in mutation testing [25]. In natural-language specifications, equivalence is especially difficult:

- “no later than 30 days” and “within 30 days inclusive” may be equivalent only if the time origin and precision are shared;
- replacing “buyer” with “customer” may be equivalent in one glossary and a scope change in another;
- removing an explicit clause may not change behavior if another governing rule still implies it;
- a threshold change may be neutral for the available dataset but not semantically equivalent.

Use at least two reviewers for difficult cases, record rationale, report inter-rater agreement where appropriate, and keep an “uncertain” class rather than forcing every mutant into killed/survived.

14.11 Designing realistic mutation operators

Operator validity improves when derived from real defect sources:

- requirement review comments and change history;
- bug reports caused by misunderstood rules;
- production incidents and incorrect policy decisions;
- pull-request fixes involving thresholds, scopes, actors, or exceptions;
- known LLM transformation failures;
- regulatory or contract interpretation corrections.

A useful operator catalog should describe the defect class, applicability precondition, transformation, expected semantic effect, examples, and known equivalence risks.

14.12 Safety and ethics

- Run mutation experiments in isolated branches or sandboxes, never against production control artifacts.
- Prevent mutant code, policy, or content from being deployed accidentally.
- Do not include sensitive production data in prompts or study corpora without approved controls.
- Label AI-generated mutants and gold answers; retain human accountability for policy.
- For human-subject studies, consider consent, workload, and organizational evaluation risks.
- Report negative and ambiguous findings rather than optimizing only for a mutation score.

Research opportunity

Natural-language SDD mutation can test something code mutation does not: whether the entire socio-technical lifecycle—people, specifications, AI transformations, architecture, tasks, code, tests, and operations—can distinguish and correctly propagate changes in intent.

15 Metrics and experimental design

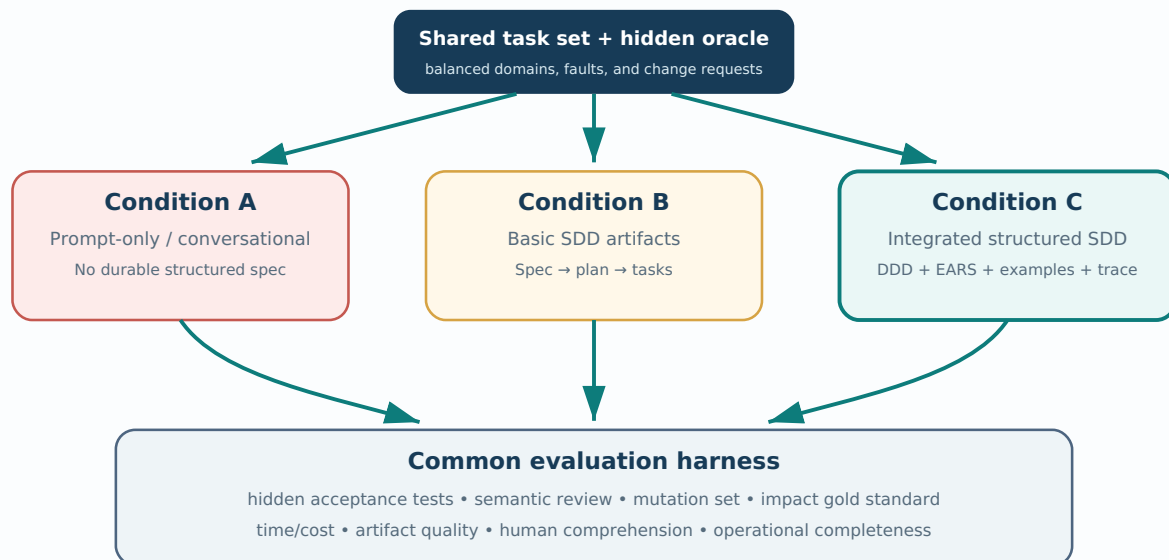
SDD research should measure more than code-generation speed. A fast pipeline that produces plausible but semantically wrong software is not successful. Evaluation should include artifact quality, behavioral correctness, change propagation, human understanding, cost, and operational readiness.

15.1 Research questions

ID	Research question	Example hypothesis
RQ1	Does structured specification improve implementation correctness compared with unstructured prompting?	EARS + examples will reduce boundary and exception errors.
RQ2	Which artifact combination contributes most to reliable change propagation?	Typed trace links and unchanged-behavior examples improve recall and preservation.
RQ3	How well do AI agents detect contradictions among requirements, examples, and tests?	Cross-artifact analysis detects syntactic conflicts more reliably than domain-policy conflicts.
RQ4	Does DDD vocabulary reduce terminology drift in generated plans and code?	An explicit glossary/context map reduces synonym and ownership defects.
RQ5	How does SDD affect human review effort and comprehension?	Structured artifacts increase initial review time but reduce clarification and rework.
RQ6	Which specification mutation operators best predict real downstream faults?	Scope, boundary, exception, and temporal-reference operators will be more revealing than simple lexical replacements.
RQ7	How stable are results across models, tools, domains, and codebases?	Benefits vary strongly by domain complexity and artifact discipline.

15.2 Experimental conditions

Diagram 19. Example comparative study



15.3 Independent variables

Variable	Possible levels	Reason to control or vary
Specification structure	Prose; EARS; EARS + examples; formal/typed constraints	Tests whether syntax and concrete examples affect correctness.
Domain context	No glossary; glossary only; glossary + context map + rule catalog	Measures terminology and ownership alignment.
Traceability	None; untyped links; typed bidirectional links	Evaluates impact analysis and preservation.
Agent workflow	Single prompt; phased prompts; tool-enforced gates	Separates artifact content from orchestration effects.
Review	Agent only; single human; cross-functional review	Measures complementary human/AI effects.
Model/tool	Multiple coding agents or versions	Tests external validity and avoids model-specific claims.
Task type	Greenfield; brownfield; bug fix; policy change; NFR change	Different tasks stress different lifecycle capabilities.

15.4 Dependent variables

Behavioral correctness

- hidden acceptance tests passed;
- domain-expert judgment against a gold specification;
- boundary, negative, exception, and failure-case correctness;
- security, authorization, and data-disclosure violations;
- contract compatibility and state-machine conformance.

Artifact quality

- requirement ambiguity and atomicity ratings;
- coverage of goals, rules, actors, and failure modes;
- contradiction count and severity;
- terminology consistency with the glossary;
- trace-link precision, recall, and freshness;
- task executability and dependency correctness.

Efficiency and human factors

- wall-clock time and active human time;
- agent turns, token/compute cost, and tool executions;
- number and severity of clarification cycles;
- rework after hidden evaluation;
- reviewer comprehension, confidence calibration, and cognitive load;
- time to answer change-impact or “why does this code exist?” questions.

15.5 A sample benchmark task

Task: controlled refund eligibility

Implement an item-level return service with a 30-day inclusive window, scoped support exceptions, quantity limits, missing-fact deferral, idempotent submission, and an idempotent asynchronous refund contract. Then apply a change adding a 45-day Gold-tier window while preserving all unrelated behavior.

The hidden evaluation should include:

- exact and adjacent time boundaries;
- unauthorized order references;
- exception scope and expiry;
- partial quantities and prior refunds;
- missing order facts;
- duplicate and concurrent submissions;

- duplicate refund events;
- Gold and non-Gold regression cases;
- final-sale and defect-policy preservation;
- historical tier snapshot semantics.

15.6 Study protocol

1. **Prepare a gold corpus.** Domain model, authoritative policy, hidden tests, impact labels, and adjudication guide.
2. **Pilot tasks and operators.** Remove impossible, trivial, or highly ambiguous items; estimate workload.
3. **Randomize or counterbalance.** Control learning and order effects for repeated-measures studies.
4. **Standardize environment.** Same repository state, dependencies, model parameters, permissions, and time budget.
5. **Capture complete provenance.** Prompts, outputs, artifact diffs, tool logs, test results, human interventions, and costs.
6. **Blind evaluation where possible.** Reviewers should not know the experimental condition when scoring semantic outputs.
7. **Adjudicate disagreements.** Use independent ratings and a documented resolution procedure.
8. **Report all exclusions.** Invalid/equivalent mutants, environment failures, and incomplete runs must be transparent.
9. **Replicate.** Repeat across tasks, domains, and agent configurations before generalizing.

15.7 Qualitative analysis

Quantitative scores can hide how failures occur. A useful failure taxonomy may include:

Failure class	Description	Example
Requirement omission	An approved behavior is absent downstream.	Missing-fact deferral never reaches tasks or code.
Semantic distortion	The artifact exists but changes meaning.	“Authorized buyer” becomes any authenticated user.
Scope overgeneralization	A conditional rule becomes global.	Gold 45-day extension applies to all buyers.
Constraint loss	Exception or unchanged behavior is dropped.	Final-sale restriction no longer applies.
Oracle adaptation	Tests are changed to agree with incorrect code.	Day-31 expected result updated without policy approval.
Trace hallucination	Links are plausible but unsupported.	A test ID is claimed to cover authorization but checks only format.

Failure class	Description	Example
Implementation leakage	Requirements become tied to accidental design.	Requirement names a table or framework component.
Overproduction	Agent creates large irrelevant changes.	A policy threshold change triggers unrelated refactoring.

15.8 Statistical considerations

The appropriate analysis depends on design and data. Report effect sizes and uncertainty, not only significance tests. Common issues include repeated observations from the same model or developer, clustering by task, non-normal time data, binary correctness outcomes, and many correlated metrics. Mixed-effects models can account for task and participant variation; non-parametric or bootstrap methods may be appropriate for small or skewed samples. Pre-register primary outcomes where feasible to reduce selective reporting.

15.9 Threats to validity

Threat	Risk	Mitigation
Construct validity	Mutation score may not represent real requirement quality.	Derive operators from real defects; combine with expert and hidden behavioral evaluation.
Internal validity	Prompt length, tooling, or reviewer support differs across conditions.	Standardize protocol; log interventions; randomize.
External validity	Toy tasks do not represent large, regulated, or brownfield systems.	Use multiple realistic repositories and industrial case studies.
Conclusion validity	Small samples and noisy metrics lead to unstable claims.	Power analysis, uncertainty intervals, replication, transparent exclusions.
Evaluation leakage	Agent has seen benchmark or hidden answers.	Use new tasks, private variants, and contamination analysis.
Researcher bias	Gold answers reflect one preferred design.	Evaluate behavior separately from design; use multiple experts and alternatives.

15.10 Evidence hierarchy for claims about SDD

Evidence	What it can support	What it cannot safely establish alone
Tool documentation	Current workflow, commands, intended usage	Independent effectiveness or general superiority
Practitioner case report	Feasibility, experience, hypotheses, contextual lessons	General causal effect

Evidence	What it can support	What it cannot safely establish alone
Controlled experiment	Causal effect under defined task and conditions	Broad industrial generalization without replication
Industrial case study	Real-world process and contextual outcomes	Clean causal attribution
Systematic review/meta-analysis	State of evidence and aggregated patterns	More certainty than the included studies permit
Longitudinal multi-site evidence	Adoption, maintenance, and context variation over time	Universal prescription for every team

Interpretive discipline

SDD, especially its modern AI-oriented form, is still an emerging practice. Distinguish established foundations—requirements engineering, BDD, TDD, DDD, formal methods, mutation testing—from current tool claims and research hypotheses.

16 Governance and adoption

SDD adoption succeeds when teams improve decisions and evidence, not when they maximize document count. Governance should be proportional to risk, automate mechanical checks, preserve clear human ownership, and allow lightweight paths for low-risk work.

16.1 Roles and accountabilities

Role	Accountability in SDD	Must not delegate blindly to an AI agent
Domain/product owner	Problem, value, business rules, priorities, policy approval	Whether a plausible rule is actually desired or legally valid
Requirements/business analyst	Structure, completeness, glossary, examples, traceability	Resolution of ambiguous stakeholder intent
Architect/tech lead	Boundaries, quality strategy, contracts, trade-offs, migration	Risk acceptance and irreversible architecture decisions
Developer	Implementation, TDD, refactoring, local evidence, deviation reporting	Claims that code satisfies behavior without appropriate tests/review
Quality engineer	Example discovery, risk-based evidence, mutation/fault strategy	Choice of oracle when policy is unresolved
Security/privacy/compliance	Threats, controls, data obligations, evidence criteria	Approval of regulated or high-impact decisions
Operations/SRE	Reliability, observability, capacity, recovery, rollout	Production readiness based only on pre-production tests
AI workflow/tool owner	Templates, context controls, model evaluation, audit, access	Semantic ownership of the product or policy

16.2 Risk-tiered process

Tier	Typical work	Minimum artifacts	Review level
L1 - Low	Text correction, internal refactor with no behavior change, well-understood maintenance	Change intent, tests/verification, affected files	Normal code review; automated gates
L2 - Moderate	Small feature, non-critical API extension, ordinary bug fix	Compact spec, examples, plan notes, tasks, trace to tests	Product/engineering/quality review

Tier	Typical work	Minimum artifacts	Review level
L3 - High	Money, authorization, personal data, public contract, migration	Full rule/requirement set, failure examples, ADRs, threat model, rollout/recovery, evidence plan	Cross-functional approval gates
L4 - Critical	Safety, regulated decisions, large irreversible change, critical infrastructure	Formalized constraints where appropriate, independent verification, complete traceability, controlled experiment/rehearsal	Independent assurance and accountable sign-off

16.3 Review gates

Gate A - Problem and domain

Is the outcome valuable? Are terms, actors, boundaries, and owners agreed?

Diagram 20. Quality gates and decision ownership



Gate B - Behavior

Are rules, EARS requirements, examples, NFRs, and unresolved questions sufficient?



Gate C - Design

Does the plan satisfy behavior, constraints, security, operability, and migration needs?



Gate D - Implementation readiness

Are tasks traceable, independently demonstrable, and safely ordered?



Gate E - Release evidence

Is the required functional, quality, operational, and rollback evidence present?

16.4 Agent operating rules

1. **Read before write.** Load current project principles, feature artifacts, glossary, contracts, and relevant code.
2. **Declare assumptions.** Separate source-backed facts, inferences, and unresolved questions.
3. **Preserve identifiers.** Do not renumber requirements or silently rename domain concepts.
4. **Do not invent policy.** Escalate material gaps involving money, rights, authorization, safety, privacy, or compliance.
5. **Limit scope.** Change only artifacts and code required by the approved task; report necessary deviations.
6. **Update coherent sets.** A semantic change must consider requirements, examples, design, tasks, code, tests, content, and operations.
7. **Show evidence.** Report commands, test results, remaining failures, and unverified claims accurately.
8. **Protect data.** Do not expose secrets, personal data, or proprietary content to unapproved models or logs.
9. **Keep humans accountable.** AI output is a proposal until accepted through the applicable review gate.

16.5 Suggested repository structure

```

project/
├── constitution.md
├── domain/
│   ├── glossary.md
│   ├── context-map.md
│   ├── rules/
│   │   ├── returns.md
│   │   └── refunds.md
│   └── decisions/
├── specs/
│   └── 014-self-service-returns/
│       ├── spec.md
│       ├── examples.feature
│       ├── decision-tables.md
│       ├── plan.md
│       ├── data-model.md
│       ├── contracts/
│       ├── tasks.md
│       ├── trace.yaml
│       ├── risks.md
│       └── rollout.md
├── architecture/
│   └── decisions/
├── src/
├── tests/
├── ops/
│   ├── dashboards/
│   ├── alerts/
│   └── runbooks/
├── tools/
│   ├── spec-lint/
│   └── trace-check/

```

The structure is illustrative. Co-location with code helps versioning and review, but large organizations may need federated repositories and cross-repository links. The key is reliable identity, ownership, and change coordination.

16.6 Automated policy checks

Check	Example rule	Failure treatment
Requirement syntax	EARS-like requirement has system and mandatory response	Block or warn based on risk tier
Glossary	New capitalized domain term must exist or be proposed	Warn and require review
Trace	Critical requirement must have owner, source, and planned evidence	Block implementation-ready status
Scenario quality	No undefined steps, duplicate names, or vague Then phrases	Block test/living-doc publication

Check	Example rule	Failure treatment
Architecture	Returns code cannot depend directly on provider SDK outside Payments adapter	Block build
Contracts	Breaking schema diff requires approved version/migration decision	Block merge
Evidence	Release candidate must link required test, security, and operational reports	Block release gate

16.7 A 90-day adoption pilot

Period	Focus	Deliverables	Learning question
Days 1-15	Select one meaningful but bounded feature; baseline current process	Metrics, glossary seed, risk tier, pilot charter	Where does ambiguity and rework currently occur?
Days 16-30	Define compact templates and review gates	Constitution, spec template, EARS/BDD guide, trace IDs	What is the minimum useful artifact set?
Days 31-55	Run discovery, specification, planning, and first slices	Reviewed artifacts, agent logs, tasks, tests	Which transformations need strongest human review?
Days 56-70	Exercise change and mutation scenarios	Impact analysis, propagation results, failure taxonomy	Does the artifact graph support safe change?
Days 71-85	Release progressively and observe	Dashboards, operational evidence, user outcome data	Does pre-release evidence predict production behavior?
Days 86-90	Retrospective and decision	Measured benefits/costs, revised workflow, scale/stop decision	Where does SDD add value, and where is it unnecessary?

16.8 Adoption metrics

Leading indicators

- Unresolved material questions before coding
- Requirements with examples and evidence plans
- Trace coverage/freshness
- Review cycle time and clarification count
- Mutation detection and propagation quality

Lagging indicators

- Escaped behavior defects
- Rework due to requirement misunderstanding
- Change lead time
- Incident diagnosis time
- Outcome achievement and support workload

16.9 Common adoption failures

Failure	Symptom	Correction
Documentation inflation	Long generated specs nobody reads	Risk-tier templates, examples, decision-focused reviews
Agent theater	Tools generate artifacts that repeat the same vague prompt	Measure semantic additions, questions, and evidence
Approval bottleneck	Every small change waits for many reviewers	Clear risk tiers and delegated authority
Template worship	Fields are completed without useful decisions	Remove fields with no demonstrated value; add domain-specific checks
Stale parallel truth	Code, tickets, and specs disagree	Coherent change sets and trace/freshness automation
No outcome measurement	Success means documents and generated code	Connect goals to production measures and guardrails
One-size process	Trivial fixes use the same ceremony as regulated decisions	Proportionality by risk, reversibility, and uncertainty

16.10 Decision framework: how much specification?

Factor	Lower-detail signal	Higher-detail signal
Reversibility	Easy rollback, no persistent data impact	Irreversible migration, financial or legal consequences
Uncertainty	Well-understood local behavior	Novel domain, ambiguous policy, new integration
Blast radius	Internal isolated component	Shared platform, public contract, many teams/users
Criticality	Minor inconvenience	Safety, money, rights, privacy, security, compliance
Longevity	Short-lived experiment	Core capability maintained for years
Team distribution	Small co-located team	Many teams, vendors, time zones, ownership boundaries
Evidence difficulty	Simple deterministic function	Emergent performance, resilience, human outcomes

PART V

Reusable material

Copy-ready templates, review checklists, a glossary, and an annotated bibliography for research and implementation.

17 Templates and review checklists

Templates are scaffolding for thinking. Keep fields that repeatedly improve decisions; remove fields that generate ceremony. The following structures can be adapted to Markdown, a requirements tool, or an SDD toolkit.

17.1 Feature specification template

```
# [FEATURE-ID] Feature name

## 1. Status and ownership
- Status: draft | in review | approved | implemented | released | superseded
- Product/domain owner:
- Engineering owner:
- Quality owner:
- Risk tier:
- Target release / effective date:

## 2. Problem and outcomes
- Problem statement:
- Affected actors:
- Desired outcomes and measures:
- Guardrail metrics:
- Evidence/baseline source:

## 3. Scope
### Included
### Explicitly excluded
### Assumptions
### Dependencies

## 4. Domain model
- Bounded context(s):
- Glossary links:
- Business rules and sources:
- State model / decision table:

## 5. Requirements
- Functional requirements (stable IDs; EARS where useful)
- Non-functional requirements (measurable)
- Security/privacy/compliance obligations
- Data lifecycle and audit obligations
- Failure and recovery behavior
- Unchanged behavior for brownfield changes

## 6. Examples and acceptance
- Representative examples
- Boundary values
- Negative and exception examples
- Dependency-failure examples
- Gherkin/decision-table links

## 7. Open questions and decisions
| ID | Question | Materiality | Owner | Needed by | Resolution |
```

```
## 8. Risks
| Risk | Impact | Likelihood | Control | Evidence | Owner |

## 9. Traceability
- Upstream goals/policies:
- Planned design artifacts:
- Planned evidence:

## 10. Release disposition
- Rollout strategy:
- Containment/rollback:
- Success decision:
- Documentation/operational updates:
```

17.2 EARS requirement record

```
id: REQ-XXX-000
status: draft | approved | superseded
pattern: ubiquitous | event | state | unwanted | optional | complex
owner:
source:
priority:
risk:

statement:
  [WHERE feature/configuration,]
  [WHILE state,]
  [WHEN trigger,]
  [IF unwanted condition, THEN]
  the [system/component] SHALL [observable response].

rationale:
terms:
  - TERM-...
preconditions:
postconditions:
examples:
  - SCN-...
quality_measure:
error_and_recovery:
assumptions:
trace:
  refines: [GOAL-..., RULE-...]
  realized_by: []
  verified_by: []
semantic_review_date:
```

17.3 Requirement review card

Check	Pass criterion	Reviewer note
Source and owner	Origin and authority to resolve/change are known.	
Domain language	Terms match glossary and bounded context.	

Check	Pass criterion	Reviewer note
Atomic obligation	One primary response can be traced and verified.	
Trigger/state/condition	Relevant activation context is explicit.	
System boundary	Responsible system/component is named.	
Observable response	Outcome can be demonstrated through defined evidence.	
Quantities	Units, thresholds, inclusivity, population, and period are clear.	
Failure/recovery	Unavailability, invalid input, and partial failure are addressed.	
Examples	Representative, boundary, and negative cases exist.	
Consistency	No unresolved conflict with rule, scenario, NFR, or policy.	

17.4 BDD discovery / Example Mapping template

```
# Story / outcome
As [actor], I want [capability] so that [value].

## Rule R1
[State the business rule]
Examples:
- EX-R1-01: normal representative case
- EX-R1-02: exact boundary
- EX-R1-03: just outside boundary
- EX-R1-04: exception
Questions:
- Q-R1-01: ...

## Rule R2
...

## Cross-cutting questions
- Authorization and disclosure?
- Missing/late/duplicate facts?
- Time and time-zone semantics?
- Quantity/money rounding?
- Idempotency and concurrency?
- Accessibility and localization?
- Observability and recovery?

## Decisions
| Question | Decision | Owner | Date | Affected artifacts |

## Automation candidates
| Example | Best evidence layer | Rationale | Trace ID |
```

17.5 Gherkin feature template

```
@FEATURE-ID
Feature: [Business capability]
  [Who benefits, what behavior matters, and why]

  Rule: [One business rule]

  Background:
    Given [short shared business context only]

  @REQ-... @domain
  Scenario: [Distinguishing condition and outcome]
    Given [relevant domain context]
    And [additional relevant fact]
    When [single central event/action]
    Then [observable business outcome]
    And [additional outcome belonging to same behavior]

  @REQ-... @failure
  Scenario: [Dependency or abnormal condition]
  ...

  @REQ-... @boundary
  Scenario Outline: [Boundary rule]
  ...
  Examples:
    | input | expected outcome | reason |
```

17.6 Technical plan template

```
# [FEATURE-ID] Technical plan

## 1. Inputs and trace
- Approved specification version:
- Requirements/scenarios included:
- Project principles:
- Existing-system baseline:

## 2. Architecture overview
- Context/component ownership
- Data and control flows
- Synchronous/asynchronous boundaries
- Consistency and transaction boundaries
- Diagrams

## 3. Domain realization
- Entities, value objects, aggregates, policies, events
- Invariants and state transitions
- Mapping from domain rules to components

## 4. Contracts
- APIs, events, schemas, error semantics
- Authentication/authorization
- Idempotency, ordering, retries, compatibility

## 5. Data
- Model and migration
- Retention/deletion/audit
- Privacy classification and access
- Reconciliation

## 6. Quality strategy
- Functional test layers
- Performance/capacity
- Security/threat controls
- Accessibility/usability
- Reliability/fault handling
- Observability/runbooks

## 7. Alternatives and decisions
| Decision | Options | Chosen | Rationale | Consequences | Trace |

## 8. Rollout and recovery
- Feature controls
- Shadow/canary/cohort plan
- Success and halt thresholds
- Rollback/containment

## 9. Open technical risks
...
```

17.7 Architecture decision record template

```
# ADR-[ID]: [Decision title]

Status: proposed | accepted | superseded | rejected
Date:
Owners:
Trace: [requirements, NFRs, risks]

## Context
What forces or constraints require a decision?

## Decision drivers
- ...

## Considered options
1. Option A
2. Option B
3. Option C

## Decision
What is chosen? State boundaries and assumptions.

## Consequences
### Positive
### Negative / costs
### Risks

## Verification
How will we know the decision satisfies its drivers?

## Revisit conditions
What change would cause this decision to be reconsidered?
```

17.8 Executable task template

```
- [ ] TASK-[ID] [Clear behavior-oriented title]

Purpose:
  [What behavior or enabling capability this task delivers]

Trace:
  requirements: [REQ-...]
  scenarios: [SCN-...]
  design: [ADR-..., contract...]

Inputs / dependencies:
  - ...

Changes:
  - expected files/components/artifacts

TDD / implementation notes:
  - first meaningful failing test
  - boundary/failure cases
  - refactoring/design consideration

Completion evidence:
  - test/report/review/demonstration

Non-goals:
  - ...

Risks / questions:
  - ...
```

17.9 Traceability record template

```
node:
  id: REQ-RET-013
  type: requirement
  version: 2
  status: approved
  owner: returns-policy-owner

links:
  - type: originates_from
    target: POLICY-RETURNS-4.2
  - type: refines
    target: RULE-RET-007
  - type: illustrated_by
    target: SCN-RET-002
  - type: realized_by
    target: ADR-RET-001
  - type: realized_by
    target: src/returns/domain/standard_return_window.py
  - type: verified_by
    target: TEST-RET-001
  - type: verified_by
    target: TEST-RET-002

review:
  semantic_reviewed_at: 2026-03-18
  reviewed_by: [domain-owner, quality-owner]
  evidence_status: complete
```

17.10 Specification mutation record

```
mutation_id: MUT-RET-BND-001
experiment: accidental_defect | intentional_change
operator: boundary_replacement
source_artifact: REQ-RET-013@v2
location: return window threshold
baseline: "30 calendar days"
mutant: "45 calendar days"
expected_semantic_effect:
  [Describe behavior change]
validity:
  syntactically_valid: true
  realistic: true
  equivalent: false | true | uncertain
  reviewers: [...]
expected_detection_or_propagation:
  - RULE-RET-007 conflict
  - SCN-RET-002 conflict
  - TEST-RET-002 failure
  - [or expected changed artifact set]
observations:
  first_detected_stage:
  artifacts_changed:
  artifacts_missed:
  unnecessary_changes:
  tests_failed:
  human_interventions:
  time_cost:
outcome: killed | survived | propagated | partial | over-propagated | equivalent | invalid
root_cause:
follow_up:
```

17.11 Pull-request checklist

- ☐ The change links to an approved requirement, defect description, or explicit no-behavior-change intent.
- ☐ Changed domain terms and policies have authoritative review.
- ☐ A semantic diff of requirements/examples is included for behavior changes.
- ☐ Impact analysis covers contracts, data, operations, content, and unchanged behavior.
- ☐ Tests were observed failing for the intended missing/changed behavior where TDD is used.
- ☐ Critical scenarios and NFR evidence are reported honestly.
- ☐ Architecture deviations and new risks are documented.
- ☐ No sensitive data, secrets, or unapproved AI-generated artifacts are introduced.
- ☐ Trace links and living documentation are current.
- ☐ Rollout, migration, compatibility, and rollback are addressed when applicable.

17.12 Agent prompt pattern for a specification task

Role and task

Implement TASK-RET-102 only.

Authoritative context

- constitution.md
- domain/glossary.md
- RULE-RET-007
- REQ-RET-012 and REQ-RET-013
- SCN-RET-001 and SCN-RET-002
- ADR-RET-001

Required behavior

Implement the inclusive 30-calendar-day StandardReturnWindow policy and return a typed decision with reason and policy version.

Constraints

- Do not add persistence or API behavior in this task.
- Do not use the system clock inside domain logic.
- Do not rename glossary terms or IDs.
- Do not infer a time-zone rule beyond the instant-based inputs.

Working method

1. Summarize the behavior and identify any contradiction.
2. Write the smallest meaningful failing test and report the failure.
3. Implement, run focused tests, refactor, and run relevant regression tests.
4. Report files changed, evidence, assumptions, and any deviation.

Stop conditions

Stop and request a decision if the approved artifacts disagree about inclusivity, duration, or time origin.

17.13 Full lifecycle review checklist

Domain and purpose

- ☐ Problem, actors, outcomes, and guardrails are explicit.
- ☐ Bounded context and ownership are clear.
- ☐ Terms are defined and used consistently.
- ☐ Rules cite an authoritative source and owner.

Behavior

- ☐ Requirements are necessary, atomic, feasible, measurable, and traceable.
- ☐ Triggers, states, exceptions, and failure responses are explicit.
- ☐ Examples cover representative, boundary, negative, exception, and failure cases.
- ☐ Unchanged behavior is explicit for brownfield changes.

Design and tasks

- ☐ Architecture preserves domain boundaries and quality constraints.

- ☐ Contracts define errors, compatibility, idempotency, and retries.
- ☐ Data migration, privacy, retention, audit, and reconciliation are addressed.
- ☐ Tasks are vertical, dependent, traceable, and demonstrably complete.

Evidence and operations

- ☐ Evidence is appropriate to functional and non-functional claims.
- ☐ Security, accessibility, reliability, and performance are not reduced to generic checkboxes.
- ☐ Dashboards, alerts, correlation, runbooks, and recovery are demonstrated.
- ☐ Rollout and rollback have owners and thresholds.
- ☐ Goals are connected to post-release measurement.

18. Synthesis and glossary

18.1 The central synthesis

The methods in this report are best understood as a coordinated system rather than as competing brands. Each method answers a different question, operates at a different level of abstraction, and produces a different kind of evidence.

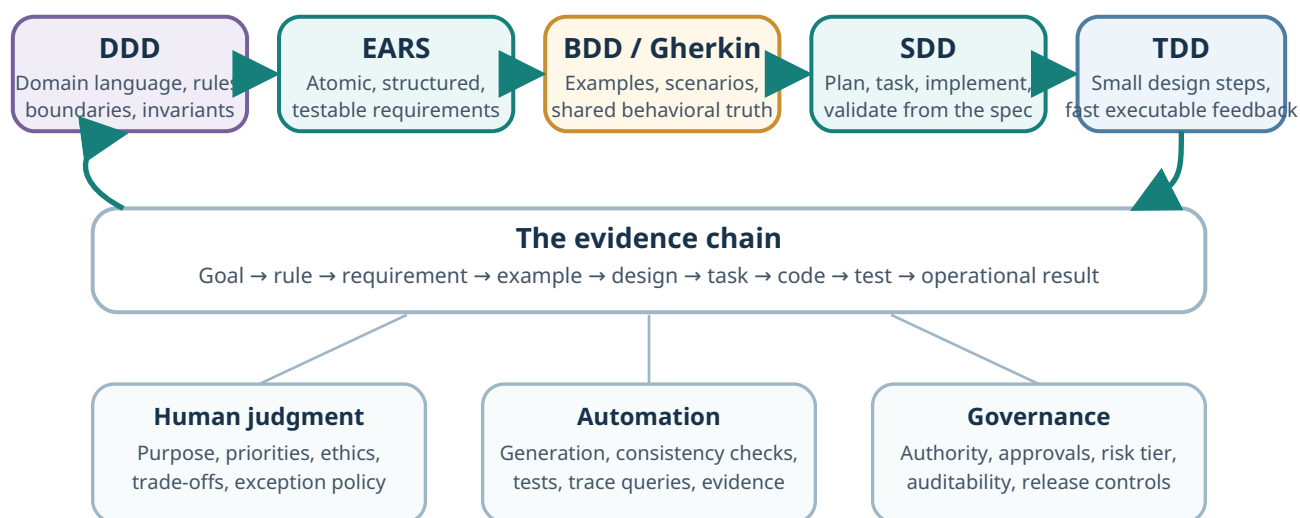


Figure 18.1 - SDD is strongest when domain knowledge, precise requirements, examples, implementation feedback, and governance form one evidence-producing system.

The practical rule

Do not ask whether a team should use DDD, EARS, BDD, Gherkin, TDD, or SDD as if only one can be chosen. Ask which uncertainty each technique must remove, which artifact it should produce, who owns that artifact, and what evidence demonstrates that the resulting system is correct.

18.2 Ten conclusions for practitioners and researchers

1. **A specification is an operational contract, not a prose essay.** It must be precise enough to guide decisions and structured enough to support review, traceability, and change impact analysis.
2. **SDD begins before the feature specification.** Domain boundaries, shared vocabulary, business authority, product goals, and risk constraints determine whether the feature specification has a stable meaning.
3. **EARS improves sentence structure, not domain truth.** A grammatically perfect requirement can still encode the wrong policy. Domain review remains essential.

4. **BDD is primarily a collaborative discovery practice.** Gherkin is a useful representation and automation interface, but it cannot replace conversations that reveal examples, rules, and unresolved questions.
5. **TDD is a local design and feedback technique.** It works inside the larger SDD evidence chain and should not be expected to prove product value, end-to-end correctness, or non-functional fitness alone.
6. **Traceability must be semantic.** A link is valuable only when its type and meaning are clear: derives from, verifies, implements, constrains, observes, supersedes, or conflicts with.
7. **Brownfield specifications must describe preservation as well as change.** Explicit unchanged behavior reduces accidental scope expansion and unnecessary regeneration.
8. **AI coding agents amplify both clarity and ambiguity.** Clear artifacts produce focused plans and tests; contradictions can produce confidently inconsistent output unless gates detect them.
9. **Mutation analysis offers a rigorous way to test the development process itself.** By changing specification semantics intentionally, researchers can measure detection, propagation, over-propagation, and evidence quality.
10. **The final proof exists in operation.** Passing tests are necessary, but successful delivery also requires observability, safe rollout, policy-version evidence, security controls, accessibility, reliability, and measured outcomes.

18.3 Glossary

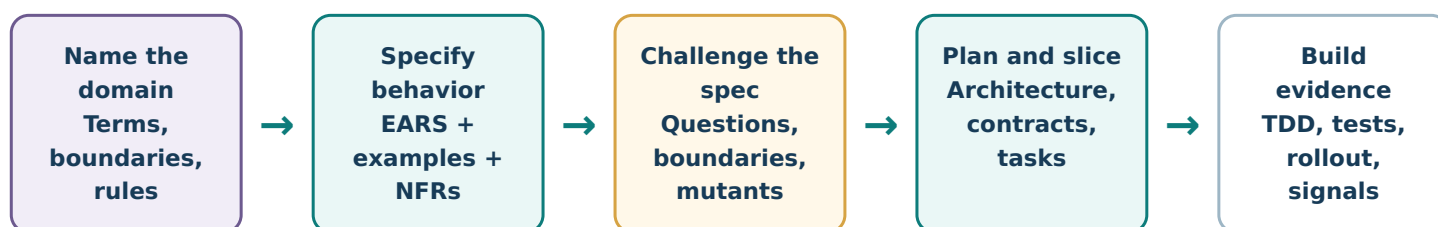
Term	Meaning in this report
Acceptance criterion	A condition that must be demonstrated for a requirement or feature to be accepted. Criteria should be observable and should identify relevant boundaries, errors, and quality constraints.
Aggregate	A DDD consistency boundary containing one aggregate root and associated entities or value objects. External changes enter through the root so invariants can be protected.
Architecture decision record (ADR)	A concise record of an architecturally significant decision, including context, options, decision, rationale, consequences, and status.
Artifact	A durable development object such as a goal, glossary, requirement, scenario, plan, ADR, task, contract, test, dashboard, or runbook.
Behavior-Driven Development (BDD)	A collaborative approach that discovers and communicates behavior through concrete examples, often expressed as Given-When-Then scenarios and sometimes automated.
Bounded context	A boundary inside which a domain model and vocabulary have a consistent meaning. The same word may have a different model in another context.
Business rule	An authoritative constraint, derivation, classification, or obligation that governs business behavior. Requirements frequently operationalize one or more rules.

Term	Meaning in this report
Characterization test	A test that records current behavior before changing a legacy system. It helps distinguish intended change from accidental regression.
Constitution	Project-wide principles and non-negotiable constraints used by an SDD workflow, such as architecture rules, security requirements, test expectations, and review policies.
Context map	A DDD representation of bounded contexts and the relationships or integration patterns between them.
Contract test	A test that verifies an API, message, event, or component boundary against an agreed contract, including compatibility and error semantics.
Decision table	A tabular representation of conditions, combinations, and resulting actions. It is useful when multiple rules interact and prose would conceal combinations.
Domain	The sphere of knowledge, activity, policy, and behavior for which software is built.
Domain event	A meaningful fact that occurred in the domain, named in past tense, such as <code>ReturnApproved</code> or <code>RefundIssued</code> .
Easy Approach to Requirements Syntax (EARS)	A set of structured natural-language patterns for writing requirements using explicit triggers, states, optional features, unwanted conditions, system names, and responses.
Entity	A domain object defined by continuity and identity over time rather than only by its attributes.
Equivalent mutant	A syntactically changed specification or program whose externally relevant semantics are unchanged under the modeled assumptions. Equivalent mutants must not be treated as process failures.
Evidence	An observable result supporting a claim. Examples include review decisions, failing-then-passing tests, contract checks, security findings, performance results, dashboard signals, or reconciliation records.
Example Mapping	A BDD discovery technique that organizes a story into rules, examples, and questions, helping a team find missing cases before automation.
Gherkin	A structured language used by Cucumber and related tools to express features, rules, scenarios, steps, examples, backgrounds, and tags.
Given-When-Then	A scenario structure in which Given establishes relevant context, When describes the event or action, and Then states the observable outcome.
Idempotency	The property that repeating the same operation with the same identity or key does not create additional unintended effects.
Invariant	A condition that must remain true for a domain object, aggregate, process, or system state to be valid.
Living documentation	Documentation kept trustworthy through active use, automation, ownership, traceability, and review rather than by manual publication alone.

Term	Meaning in this report
Mutation operator	A rule for creating a controlled semantic or structural change, such as replacing a threshold, negating a condition, deleting an exception, or changing a temporal boundary.
Non-functional requirement (NFR)	A measurable constraint or quality requirement concerning properties such as performance, security, privacy, accessibility, reliability, operability, maintainability, or compliance.
Oracle	The mechanism used to determine whether an observed result is correct. Human policy decisions, tests, formal properties, contracts, and reference implementations may serve as oracles.
Policy version	An identifier for the business-policy definition used to make a decision. Persisting it supports auditability and reproducibility when rules change.
Property-based testing	A technique that generates many inputs and checks general properties or invariants rather than relying only on manually selected examples.
Requirement	A necessary, verifiable statement of capability, behavior, constraint, interface, or quality. In this report, requirements use stable identifiers and trace links.
Scenario Outline	A Gherkin template executed once for each row of an Examples table, useful for concise representative data variations.
Spec-Driven Development (SDD)	A development approach in which explicit, versioned specifications guide clarification, planning, task decomposition, implementation, validation, and change management.
Specification	The coherent set of authoritative statements, examples, models, constraints, interfaces, and acceptance evidence defining intended behavior.
Specification by Example	A collaborative method that uses concrete examples to discover, specify, verify, and document behavior.
Specification mutation	The controlled alteration of specification semantics or structure to evaluate review, traceability, test adequacy, propagation, and process resilience.
State machine	A model of valid states, transitions, guards, events, and actions for an entity or process lifecycle.
Subdomain	A coherent part of the business domain, often classified as core, supporting, or generic to guide modeling and investment.
Test-Driven Development (TDD)	A short feedback cycle in which a developer writes a meaningful failing test, implements the smallest behavior needed to pass, and then refactors while preserving behavior.
Traceability	The ability to follow meaningful relationships among goals, rules, requirements, examples, design, tasks, implementation, tests, release evidence, and operational results.
Ubiquitous Language	A shared domain language used consistently by domain experts, product staff, engineers, tests, documentation, and code within a bounded context.

Term	Meaning in this report
Validation	Evidence that the system solves the intended stakeholder or business problem: building the right system.
Value object	An immutable domain concept defined by its attributes and semantics rather than a separate identity, such as Money or ReturnWindow.
Verification	Evidence that the implementation conforms to its specified requirements and design: building the system right.
Vertical slice	A small, demonstrable increment crossing the necessary layers to deliver one coherent behavior rather than completing one technical layer in isolation.

18.4 A final implementation heuristic



When a stage cannot be completed honestly, do not compensate by producing more downstream artifacts. Return to the earliest unresolved semantic decision. The most effective SDD workflow is not the one that generates the most documents or code; it is the one that exposes uncertainty early and produces the smallest trustworthy evidence chain.

19. Annotated reading guide and references

This bibliography combines modern AI-oriented SDD material with the older bodies of knowledge that make SDD rigorous: requirements engineering, EARS, BDD, executable specifications, TDD, DDD, formal methods, traceability, and mutation testing. Many foundational publications do not use the label *Spec-Driven Development*; they explain the concepts that a credible SDD process must integrate.

How to use this reading list

For a practitioner introduction, follow Path A. For an academic literature review, follow Path B. For specification-mutation research, follow Path C. Official documentation is useful as primary evidence of current workflows, while peer-reviewed papers and standards are stronger sources for generalizable claims.

19.1 Reading paths

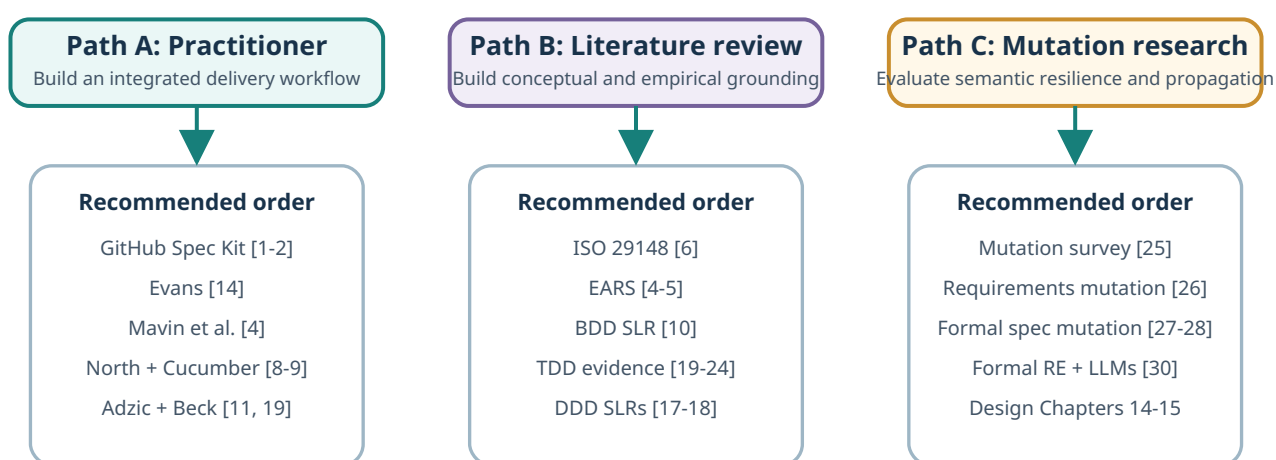


Figure 19.1 - Three reading paths for application, academic synthesis, and specification-mutation research.

19.2 Modern SDD and AI-oriented specification workflows

[1] GitHub. *Spec Kit documentation*. Official documentation for the contemporary Specify → Plan → Tasks → Implement workflow, artifact structure, extensions, and supported agent-oriented practices. **Best for:** understanding a concrete modern implementation of SDD. <https://github.github.com/spec-kit/> (accessed for this edition).

- [2] **GitHub.** *Spec-Driven Development*. Concept document in the Spec Kit repository describing specifications as primary artifacts, greenfield and brownfield use, iterative enhancement, and experimental intent. **Best for:** the philosophy and scope of modern SDD. <https://github.com/github/spec-kit/blob/main/spec-driven.md> (accessed for this edition).
- [3] **Kiro.** *Specs: best practices*. Official guidance on requirements-first and design-first workflows and the requirements, design, and task artifacts used by an AI development environment. **Best for:** comparing another current implementation of specification-led AI development. <https://kiro.dev/docs/specs/best-practices/> (accessed for this edition).

19.3 Requirements engineering and EARS

- [4] **Mavin, A., Wilkinson, P., Harwood, A., and Novak, M. (2009).** "Easy Approach to Requirements Syntax (EARS)." *17th IEEE International Requirements Engineering Conference (RE'09)*, 317-322. DOI: [10.1109/RE.2009.9](https://doi.org/10.1109/RE.2009.9). **Best for:** the original EARS patterns, their motivation, and an early industrial application.
- [5] **Mavin, A. and Wilkinson, P. (2010).** "Big EARS (The Return of Easy Approach to Requirements Engineering)." *18th IEEE International Requirements Engineering Conference*. DOI: [10.1109/RE.2010.39](https://doi.org/10.1109/RE.2010.39). **Best for:** extending EARS beyond sentence templates toward a broader requirements-engineering approach.
- [6] **ISO/IEC/IEEE (2018).** *ISO/IEC/IEEE 29148:2018 - Systems and software engineering - Life cycle processes - Requirements engineering*. **Best for:** normative terminology, characteristics of good requirements, requirements information items, and lifecycle context. <https://www.iso.org/standard/72089.html>.
- [7] **Wiegers, K. and Beatty, J. (2013).** *Software Requirements, 3rd ed.* Microsoft Press. ISBN 9780735679665. **Best for:** elicitation, analysis, specification, validation, prioritization, traceability, and change management in practice.

19.4 BDD, Gherkin, and Specification by Example

- [8] **North, D.** *Introducing BDD*. Foundational practitioner article explaining the motivation for behavior-oriented vocabulary, scenario templates, and outside-in thinking. **Best for:** the historical and conceptual origin of BDD. <https://dannorth.net/introducing-bdd/>.
- [9] **Cucumber.** *Gherkin Reference*. Official grammar and keyword documentation covering Feature, Rule, Scenario, Given, When, Then, Background, Scenario Outline, Examples, Doc Strings, Data Tables, and tags. **Best for:** exact language semantics and supported syntax. <https://cucumber.io/docs/gherkin/reference/>.
- [10] **Farooq, U., et al. (2023).** "Behavior Driven Development: A Systematic Literature Review." *IEEE Access*, 11, 88008-88024. DOI: [10.1109/ACCESS.2023.3302356](https://doi.org/10.1109/ACCESS.2023.3302356). **Best for:** an academic map of BDD definitions, reported benefits, challenges, tools, and research gaps.
- [11] **Adzic, G. (2011).** *Specification by Example: How Successful Teams Deliver the Right Software*. Manning. ISBN 9781617290084. **Best for:** collaborative specification, illustrative examples, executable specifications, and living documentation. <https://www.manning.com/books/specification-by-example>.
- [12] **Nicieja, K. (2017).** *Writing Great Specifications: Using Specification by Example and Gherkin*. Manning. ISBN 9781617294105. **Best for:** concrete guidance for readable, maintainable, domain-oriented executable specifications. <https://www.manning.com/books/writing-great-specifications>.
- [13] **Wynne, M. and Hellesoy, A. (2017).** *The Cucumber Book, 2nd ed.* Pragmatic Bookshelf. **Best for:** the Cucumber ecosystem, collaboration practices, scenario design, automation, and maintenance.

19.5 Domain-Driven Design

- [14] **Evans, E. (2003).** *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley. ISBN 9780321125217. **Best for:** the foundational concepts of ubiquitous language, model-driven design, aggregates, repositories, domain services, bounded contexts, and strategic design. <https://www.informit.com/store/domain-driven-design-tackling-complexity-in-the-heart-9780321125217>.

- [15] **Vernon, V. (2013).** *Implementing Domain-Driven Design*. Addison-Wesley. ISBN 9780321834577. **Best for:** applying strategic and tactical DDD, including bounded contexts, context maps, aggregates, domain events, and architecture.
- [16] **Vernon, V. (2016).** *Domain-Driven Design Distilled*. Addison-Wesley. ISBN 9780134434421. **Best for:** a compact entry point to strategic design and the most important tactical patterns.
- [17] **Ozkan, D., Babur, O., and van den Brand, M. G. J. (2025).** "Domain-Driven Design in software development: A systematic literature review on implementation, challenges, and effectiveness." *Journal of Systems and Software*, 230, 112537. DOI: [10.1016/j.jss.2025.112537](https://doi.org/10.1016/j.jss.2025.112537). **Best for:** current peer-reviewed evidence concerning DDD adoption, implementation, reported outcomes, and limitations.
- [18] **Sangabriel-Alarcon, J. A., et al. (2024).** "Domain-Driven Design in Microservices-Based Systems Development: A Systematic Literature Review and Thematic Analysis." *Programming and Computer Software*, 50, 742-770. DOI: [10.1134/S0361768824700749](https://doi.org/10.1134/S0361768824700749). **Best for:** the intersection of DDD, bounded contexts, and microservice decomposition.

19.6 Test-Driven Development and empirical evidence

- [19] **Beck, K. (2002).** *Test Driven Development: By Example*. Addison-Wesley. ISBN 9780321146533. **Best for:** the foundational Red-Green-Refactor discipline, incremental design, and concrete examples. <https://www.informit.com/store/test-driven-development-by-example-9780321146533>.
- [20] **Freeman, S. and Pryce, N. (2009).** *Growing Object-Oriented Software, Guided by Tests*. Addison-Wesley. ISBN 9780321503626. **Best for:** outside-in TDD, test doubles, ports and adapters, and evolving object-oriented design through tests.
- [21] **George, B. and Williams, L. (2004).** "A structured experiment of test-driven development." *Information and Software Technology*, 46(5), 337-342. DOI: [10.1016/j.infsof.2003.09.011](https://doi.org/10.1016/j.infsof.2003.09.011). **Best for:** an early controlled study of quality and development effort.
- [22] **Rafique, Y. and Misis, V. B. (2013).** "The Effects of Test-Driven Development on External Quality and Productivity: A Meta-Analysis." *IEEE Transactions on Software Engineering*, 39(6), 835-856. DOI: [10.1109/TSE.2012.28](https://doi.org/10.1109/TSE.2012.28). **Best for:** synthesis of empirical TDD results and the importance of context and study design.
- [23] **Romano, S., Fucci, D., Scanniello, G., Turhan, B., and Juristo, N. (2017).** "Findings from a multi-method study on test-driven development." *Information and Software Technology*, 89, 64-77. DOI: [10.1016/j.infsof.2017.03.010](https://doi.org/10.1016/j.infsof.2017.03.010). **Best for:** understanding why TDD outcomes vary and why quantitative and qualitative evidence should be combined.
- [24] **Roman, A. and Mnich, M. (2021).** "Test-driven development with mutation testing - an experimental study." *Software Quality Journal*, 29, 1-38. DOI: [10.1007/s11219-020-09534-x](https://doi.org/10.1007/s11219-020-09534-x). **Best for:** connecting TDD practice with mutation adequacy and experimental evaluation.

19.7 Mutation testing, requirements mutation, and formal specifications

- [25] **Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Le Traon, Y., and Harman, M. (2019).** "Mutation Testing Advances: An Analysis and Survey." *Advances in Computers*, 112, 275-378. DOI: [10.1016/bs.adcom.2018.03.015](https://doi.org/10.1016/bs.adcom.2018.03.015). **Best for:** mutation-testing foundations, operator design, equivalent mutants, subsumption, cost reduction, and empirical practice.
- [26] **Li, J., et al. (2021).** "A requirements inspection method based on scenarios generated by model mutation and the experimental validation." *Empirical Software Engineering*, 26, 108. DOI: [10.1007/s10664-021-10001-9](https://doi.org/10.1007/s10664-021-10001-9). **Best for:** evidence that mutation-generated scenarios can support requirements inspection and defect discovery.

- [27] **Cordy, M., Lazreg, M. B., Legay, A., and Schobbens, P.-Y. (2023).** "Towards Strengthening Formal Specifications with Mutation Model Checking." Research paper on using mutation and model checking to assess and strengthen formal specifications. **Best for:** connecting mutation concepts to specification adequacy rather than only program tests. [Institutional open-access manuscript](#).
- [28] **Amaral, V., Mendes, J., and Campos, J. C. (2025).** "MutDafny: A Mutation-Based Approach to Assess Dafny Specifications." arXiv:2511.15403. **Best for:** a recent mutation-based approach to assessing formal Dafny specifications and detecting specification weaknesses. <https://arxiv.org/abs/2511.15403>.
- [29] **Jackson, D. (2012).** *Software Abstractions: Logic, Language, and Analysis*, 2nd ed. MIT Press. **Best for:** lightweight formal modeling with Alloy, relational logic, scopes, counterexamples, and automated analysis of software abstractions. <https://mitpress.mit.edu/9780262528900/software-abstractions/>.

19.8 LLMs, formal requirements, and AI-assisted software engineering

- [30] **Ferrari, A. and Spoletini, P. (2025).** "Formal requirements engineering and large language models: A two-way roadmap." *Information and Software Technology*, 181, 107697. DOI: [10.1016/j.infsof.2025.107697](https://doi.org/10.1016/j.infsof.2025.107697). **Best for:** understanding how LLMs can support requirements engineering and how formal requirements techniques can constrain and evaluate LLM behavior.
- [31] **GitHub.** *Spec Kit repository*. Source code, templates, command definitions, issues, and release history for the open-source toolkit. **Best for:** reproducible tool inspection and empirical studies of workflow evolution. <https://github.com/github/spec-kit>.

19.9 Suggested source-to-section mapping for an academic paper

Paper section	Recommended primary sources	Purpose
Modern SDD definition	[1], [2], [3], [31]	Define current tool-supported workflows without claiming that the underlying ideas are new.
Requirements quality	[4], [5], [6], [7]	Ground requirement syntax, quality characteristics, validation, and change management.
Executable behavior	[8]-[13]	Explain BDD, Gherkin, examples, collaboration, automation, and living documentation.
Domain semantics	[14]-[18]	Explain domain language, boundaries, models, invariants, and current empirical evidence.
Implementation feedback	[19]-[24]	Describe TDD and report empirical results with appropriate nuance.
Mutation foundation	[25]-[29]	Justify operators, metrics, equivalent-mutant treatment, and specification-strength analysis.
AI and formal rigor	[30], plus [1]-[3]	Position the two-way relationship between AI assistance and requirements/formal constraints.

19.10 Example literature-review argument

A defensible literature review can make the following argument:

1. Requirements engineering established quality characteristics, lifecycle artifacts, validation, and traceability long before modern AI coding agents [6-7].
2. EARS provided controlled natural-language patterns that reduce common sentence-level ambiguity while retaining readability [4-5].
3. BDD and Specification by Example connected domain conversations, concrete examples, acceptance behavior, and living documentation [8-13].
4. DDD supplied strategic boundaries and a shared domain model so the same words have deliberate meanings in specifications and implementation [14-18].
5. TDD supplied a fine-grained implementation feedback cycle, although empirical outcomes vary with context and execution [19-24].
6. Mutation research supplied methods for assessing test and specification strength by introducing controlled faults or semantic changes [25-29].
7. Modern SDD toolchains combine these concerns around versioned artifacts and AI-supported planning and implementation [1-3, 31].
8. The research opportunity is therefore not merely to measure code generation. It is to measure semantic preservation, traceability, validation strength, governance, and operational evidence across the entire specification-driven lifecycle.

Citation discipline

Distinguish practitioner guidance, official tool documentation, standards, controlled experiments, systematic reviews, and conceptual research. Avoid treating a tool's documentation as proof that the workflow improves quality. Use empirical studies for outcome claims, and report conflicting or context-dependent findings rather than presenting one method as universally superior.

19.11 Closing note

Spec-Driven Development becomes technically meaningful when the specification is more than an input prompt. It must function as a versioned, reviewable, traceable, challengeable, and evidence-producing model of intent. DDD supplies semantic boundaries; EARS improves requirement structure; BDD and Gherkin make behavior concrete; Specification by Example turns examples into shared acceptance assets; TDD provides rapid implementation feedback; mutation analysis tests whether the system of artifacts can detect and propagate semantic change. Combined carefully, these practices offer a disciplined path from business meaning to reliable software and a rich agenda for empirical research.

Links and current tool documentation were checked for this edition before publication. Publication metadata should be verified against the publisher or DOI record when preparing a formal thesis or journal submission.

PART VI

Evidence and definitions

Build a common language for claims, sources, examples, standards, and uncertainty before evaluating a development method.

WHAT THIS PART ANSWERS

Which claims describe a process? Which claims report outcomes?
What does a worked example prove—and what does it not prove?

USE IT WHEN

You need to review evidence quality, compare studies, or explain the limits of vendor documentation.

RESEARCH EXPANSION

What counts as Spec-Driven Development?

define SDD as a governed lifecycle in which specification artifacts direct planning, implementation, and verification.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first define SDD as a governed lifecycle in which specification artifacts direct planning, implementation, and verification, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

define SDD as a governed lifecycle in which specification artifacts direct planning, implementation, and verification. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Define the question and the authoritative decision source.
- 2 Create a durable artifact with a stable identifier and owner.
- 3 Link downstream work to the artifact rather than private memory.
- 4 Challenge the result with an independent executable or review check.
- 5 Record evidence, deviations, and the trigger for future revision.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE[\[R1\] GitHub Spec Kit documentation](#)[\[R2\] GitHub, What is SDD?](#)[\[R5\] ISO/IEC/IEEE 29148:2018](#)

RESEARCH EXPANSION

The SDD maturity spectrum

distinguish spec-first, spec-anchored, spec-as-source, and formally constrained variants.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must distinguish spec-first, spec-anchored, spec-as-source, and formally constrained variants; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

distinguish spec-first, spec-anchored, spec-as-source, and formally constrained variants. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Define the question and the authoritative decision source.
- 2 Create a durable artifact with a stable identifier and owner.
- 3 Link downstream work to the artifact rather than private memory.
- 4 Challenge the result with an independent executable or review check.
- 5 Record evidence, deviations, and the trigger for future revision.

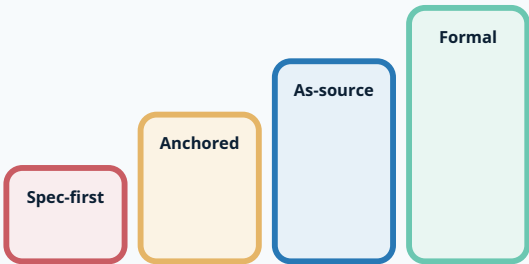
EVIDENCE TO RETAIN

- OWNER
- ARTIFACT
- VERIFICATION
- REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R2] GitHub, What is SDD?

[R3] GitHub, Spec Persistence Models

[R30] Formal RE and LLM roadmap

[R52] Lamport, Specifying Systems

RESEARCH EXPANSION

Sources, assertions, and illustrative numbers

label what is observed, what is inferred, and what is merely a worked demonstration.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To label what is observed, what is inferred, and what is merely a worked demonstration, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

label what is observed, what is inferred, and what is merely a worked demonstration. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Define the question and the authoritative decision source.
- 2 Create a durable artifact with a stable identifier and owner.
- 3 Link downstream work to the artifact rather than private memory.
- 4 Challenge the result with an independent executable or review check.
- 5 Record evidence, deviations, and the trigger for future revision.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

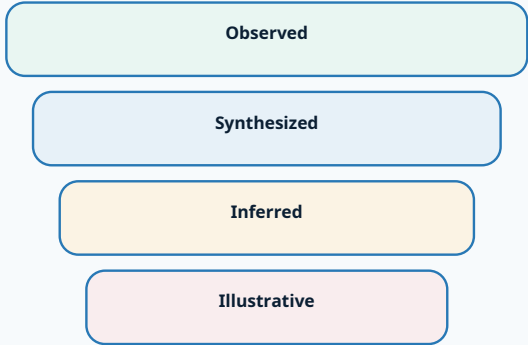
VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R19] Nagappan et al. 2008

[R20] Rafique and Misis meta-analysis

[R33] METR early-2025 RCT

[R34] METR 2026 experiment update

RESEARCH EXPANSION

An evidence hierarchy for an SDD report

rank standards, original research, systematic reviews, official documentation, books, and practitioner commentary by the questions they can answer.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to rank standards, original research, systematic reviews, official documentation, books, and practitioner commentary by the questions they can answer without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

rank standards, original research, systematic reviews, official documentation, books, and practitioner commentary by the questions they can answer. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Define the question and the authoritative decision source.
- 2 Create a durable artifact with a stable identifier and owner.
- 3 Link downstream work to the artifact rather than private memory.
- 4 Challenge the result with an independent executable or review check.
- 5 Record evidence, deviations, and the trigger for future revision.

EVIDENCE TO RETAIN

OWNER

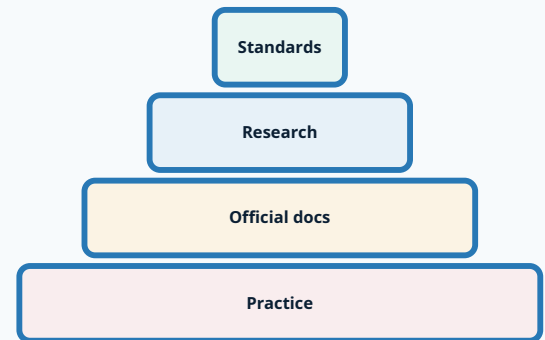
ARTIFACT

VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R5] ISO/IEC/IEEE 29148:2018

[R7] Requirements quality mapping

[R12] BDD systematic review 2023

[R32] LLMs for software engineering SLR

RESEARCH EXPANSION

Established disciplines versus AI-era packaging

show that requirements engineering, DDD, BDD, TDD, traceability, and formal methods predate current agent-oriented SDD toolchains.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to show that requirements engineering, DDD, BDD, TDD, traceability, and formal methods predate current agent-oriented SDD toolchains. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

show that requirements engineering, DDD, BDD, TDD, traceability, and formal methods predate current agent-oriented SDD toolchains. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Define the question and the authoritative decision source.
- 2 Create a durable artifact with a stable identifier and owner.
- 3 Link downstream work to the artifact rather than private memory.
- 4 Challenge the result with an independent executable or review check.
- 5 Record evidence, deviations, and the trigger for future revision.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

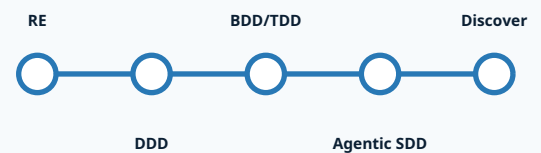
VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

```
ACTIVE_TASK: T3
ALLOWED_PATHS: src/refunds/**, tests/refunds/**
REQUIRED_GATES: unit, BDD, OpenAPI
STOP_ON: ambiguity, contract conflict, scope breach
```

FAILURE MODE / LIMIT

A prompt is advisory. Without executable gates, repository permissions, and independent verification, an agent can satisfy the apparent task while violating scope or hidden intent.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R5] ISO/IEC/IEEE 29148:2018

[R9] Dan North, Introducing BDD

[R15] Evans, Domain-Driven Design

[R18] Beck, TDD by Example

RESEARCH EXPANSION

Corrections to common SDD claims

replace universal claims about defect reduction, productivity, and automation with bounded statements that preserve study context.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first replace universal claims about defect reduction, productivity, and automation with bounded statements that preserve study context, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

replace universal claims about defect reduction, productivity, and automation with bounded statements that preserve study context. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Define the question and the authoritative decision source.
- 2 Create a durable artifact with a stable identifier and owner.
- 3 Link downstream work to the artifact rather than private memory.
- 4 Challenge the result with an independent executable or review check.
- 5 Record evidence, deviations, and the trigger for future revision.

EVIDENCE TO RETAIN

- OWNER
- ARTIFACT
- VERIFICATION
- REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

```
ACTIVE_TASK: T3
ALLOWED_PATHS: src/refunds/**, tests/refunds/**
REQUIRED_GATES: unit, BDD, OpenAPI
STOP_ON: ambiguity, contract conflict, scope breach
```

FAILURE MODE / LIMIT

A prompt is advisory. Without executable gates, repository permissions, and independent verification, an agent can satisfy the apparent task while violating scope or hidden intent.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R19] Nagappan et al. 2008

[R20] Rafique and Misisic meta-analysis

[R33] METR early-2025 RCT

[R34] METR 2026 experiment update

RESEARCH EXPANSION

Spec persistence is a design choice

decide whether a specification is disposable scaffolding, a maintained anchor, or the only human-edited source.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must decide whether a specification is disposable scaffolding, a maintained anchor, or the only human-edited source; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

decide whether a specification is disposable scaffolding, a maintained anchor, or the only human-edited source. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Define the question and the authoritative decision source.
- 2 Create a durable artifact with a stable identifier and owner.
- 3 Link downstream work to the artifact rather than private memory.
- 4 Challenge the result with an independent executable or review check.
- 5 Record evidence, deviations, and the trigger for future revision.

EVIDENCE TO RETAIN

OWNER

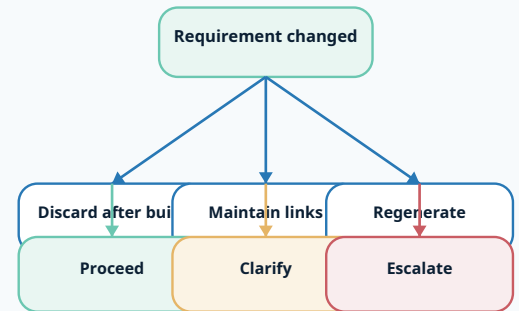
ARTIFACT

VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE[\[R3\] GitHub, Spec Persistence Models](#)[\[R1\] GitHub Spec Kit documentation](#)

RESEARCH EXPANSION

The artifact graph, not the document stack

model semantic relations among goals, rules, requirements, examples, decisions, tasks, code, tests, and operational observations.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To model semantic relations among goals, rules, requirements, examples, decisions, tasks, code, tests, and operational observations, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

model semantic relations among goals, rules, requirements, examples, decisions, tasks, code, tests, and operational observations. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Define the question and the authoritative decision source.
- 2 Create a durable artifact with a stable identifier and owner.
- 3 Link downstream work to the artifact rather than private memory.
- 4 Challenge the result with an independent executable or review check.
- 5 Record evidence, deviations, and the trigger for future revision.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R26] Gotel and Finkelstein 1994

[R27] Requirements traceability book

[R5] ISO/IEC/IEEE 29148:2018

RESEARCH EXPANSION

Lifecycle governance and authority

assign ownership and approval rights so that product policy, architecture, security, and verification decisions do not silently conflict.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to assign ownership and approval rights so that product policy, architecture, security, and verification decisions do not silently conflict without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

assign ownership and approval rights so that product policy, architecture, security, and verification decisions do not silently conflict. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Define the question and the authoritative decision source.
- 2 Create a durable artifact with a stable identifier and owner.
- 3 Link downstream work to the artifact rather than private memory.
- 4 Challenge the result with an independent executable or review check.
- 5 Record evidence, deviations, and the trigger for future revision.

EVIDENCE TO RETAIN

- OWNER
- ARTIFACT
- VERIFICATION
- REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

Product	Domain	Architecture	QA
Security	Ops	Product	Domain
Architecture	QA	Security	Ops
Product	Domain	Architecture	QA

WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

- [R5] ISO/IEC/IEEE 29148:2018
- [R44] OWASP ASVS
- [R49] Scrum Guide 2020

RESEARCH EXPANSION

Research method used in this edition

triangulate official documentation, standards, primary studies, systematic reviews, and the supplied PLT-4821 walkthrough.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to triangulate official documentation, standards, primary studies, systematic reviews, and the supplied PLT-4821 walkthrough. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

triangulate official documentation, standards, primary studies, systematic reviews, and the supplied PLT-4821 walkthrough. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Define the question and the authoritative decision source.
- 2 Create a durable artifact with a stable identifier and owner.
- 3 Link downstream work to the artifact rather than private memory.
- 4 Challenge the result with an independent executable or review check.
- 5 Record evidence, deviations, and the trigger for future revision.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R1] GitHub Spec Kit documentation

[R6] Mavin et al. 2009 EARS

[R17] DDD systematic review 2025

[R30] Formal RE and LLM roadmap

RESEARCH EXPANSION

Evaluation rubric for an SDD implementation

assess semantic clarity, traceability, executability, change handling, governance, and operational evidence instead of counting documents.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first assess semantic clarity, traceability, executability, change handling, governance, and operational evidence instead of counting documents, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

assess semantic clarity, traceability, executability, change handling, governance, and operational evidence instead of counting documents. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Define the question and the authoritative decision source.
- 2 Create a durable artifact with a stable identifier and owner.
- 3 Link downstream work to the artifact rather than private memory.
- 4 Challenge the result with an independent executable or review check.
- 5 Record evidence, deviations, and the trigger for future revision.

EVIDENCE TO RETAIN

- OWNER
- ARTIFACT
- VERIFICATION
- REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

Clarity	Links	Tests	Change
Gates	Ops	Clarity	Links
Tests	Change	Gates	Ops
Clarity	Links	Tests	Change

WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R5] ISO/IEC/IEEE 29148:2018

[R26] Gotel and Finkelstein 1994

[R36] DORA software delivery metrics

[R39] SPACE framework

PART VII

Advanced requirements and EARS

Turn stakeholder intent into controlled, testable requirements without pretending that syntax alone creates domain truth.

WHAT THIS PART ANSWERS

Who has authority? Which trigger, state, response, timing, and failure details are missing? How will each requirement be contradicted?

USE IT WHEN

A ticket sounds clear but cannot yet produce a precise failing test or measurable acceptance decision.

RESEARCH EXPANSION

Anatomy of a high-quality requirement

separate actor, trigger, precondition, response, object, threshold, rationale, source, and verification method.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first separate actor, trigger, precondition, response, object, threshold, rationale, source, and verification method, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

separate actor, trigger, precondition, response, object, threshold, rationale, source, and verification method. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Identify the policy owner and the decision source.
- 2 Separate trigger, state, actor, response, and threshold.
- 3 Write boundary, error, timing, and data examples.
- 4 Derive an independent failing test or review oracle.
- 5 Approve the requirement and link every downstream artifact.

EVIDENCE TO RETAIN

APPROVED CRITERION

DECISION SOURCE

EXAMPLES

TEST MAPPING

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

Authority

Condition

Actor

Response

Measure

WORKED EXAMPLE

WHEN an event is cancelled,
THE Refunds service SHALL create exactly one
refund
for ticket price minus frozen service fees
within five business days.

FAILURE MODE / LIMIT

A well-formed sentence can still encode the wrong policy, omit an actor, or hide an unmeasurable quality boundary. Syntax helps expose gaps; it does not supply domain truth.

WARNING SIGNS

- Words such as fast, automatic, appropriate, or supported remain unquantified.
- One sentence contains several triggers or outcomes.
- The policy owner and measurement method are missing.
- A test can pass without proving the business rule.

DECISION RULE

Do not authorize planning while a reasonable reviewer can derive two materially different behaviors from the same requirement.

EVIDENCE

[R5] ISO/IEC/IEEE 29148:2018

[R7] Requirements quality mapping

RESEARCH EXPANSION

A taxonomy of ambiguity

identify lexical, syntactic, semantic, pragmatic, quantitative, temporal, and cross-artifact ambiguity.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must identify lexical, syntactic, semantic, pragmatic, quantitative, temporal, and cross-artifact ambiguity; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

identify lexical, syntactic, semantic, pragmatic, quantitative, temporal, and cross-artifact ambiguity. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Identify the policy owner and the decision source.
- 2 Separate trigger, state, actor, response, and threshold.
- 3 Write boundary, error, timing, and data examples.
- 4 Derive an independent failing test or review oracle.
- 5 Approve the requirement and link every downstream artifact.

EVIDENCE TO RETAIN

APPROVED CRITERION

DECISION SOURCE

EXAMPLES

TEST MAPPING

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

Lexical	Semantic	Temporal	Scope
Authority	Data	Lexical	Semantic
Temporal	Scope	Authority	Data
Lexical	Semantic	Temporal	Scope

WORKED EXAMPLE

WHEN an event is cancelled,
THE Refunds service SHALL create exactly one
refund
for ticket price minus frozen service fees
within five business days.

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- Words such as fast, automatic, appropriate, or supported remain unquantified.
- One sentence contains several triggers or outcomes.
- The policy owner and measurement method are missing.
- A test can pass without proving the business rule.

DECISION RULE

Do not authorize planning while a reasonable reviewer can derive two materially different behaviors from the same requirement.

EVIDENCE

[R7] Requirements quality mapping

[R8] NaPiRE agile RE survey

RESEARCH EXPANSION

EARS as controlled natural language

use a small family of sentence patterns to make triggers, states, exceptional conditions, and optionality visible.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To use a small family of sentence patterns to make triggers, states, exceptional conditions, and optionality visible, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

use a small family of sentence patterns to make triggers, states, exceptional conditions, and optionality visible. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Identify the policy owner and the decision source.
- 2 Separate trigger, state, actor, response, and threshold.
- 3 Write boundary, error, timing, and data examples.
- 4 Derive an independent failing test or review oracle.
- 5 Approve the requirement and link every downstream artifact.

EVIDENCE TO RETAIN

APPROVED CRITERION

DECISION SOURCE

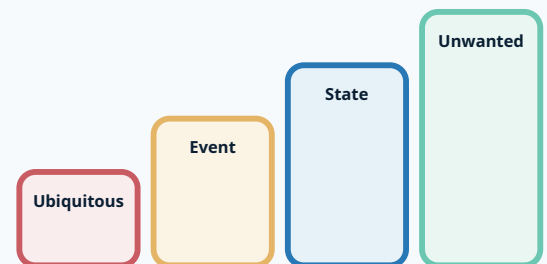
EXAMPLES

TEST MAPPING

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

WHEN an event is cancelled,
THE Refunds service SHALL create exactly one
refund
for ticket price minus frozen service fees
within five business days.

FAILURE MODE / LIMIT

A well-formed sentence can still encode the wrong policy, omit an actor, or hide an unmeasurable quality boundary. Syntax helps expose gaps; it does not supply domain truth.

WARNING SIGNS

- Words such as fast, automatic, appropriate, or supported remain unquantified.
- One sentence contains several triggers or outcomes.
- The policy owner and measurement method are missing.
- A test can pass without proving the business rule.

DECISION RULE

Do not authorize planning while a reasonable reviewer can derive two materially different behaviors from the same requirement.

EVIDENCE

[R6] Mavin et al. 2009 EARS

[R5] ISO/IEC/IEEE 29148:2018

RESEARCH EXPANSION

Ubiquitous requirements for invariants

state behavior that must always hold without inventing an artificial event or state.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to state behavior that must always hold without inventing an artificial event or state without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

state behavior that must always hold without inventing an artificial event or state. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Identify the policy owner and the decision source.
- 2 Separate trigger, state, actor, response, and threshold.
- 3 Write boundary, error, timing, and data examples.
- 4 Derive an independent failing test or review oracle.
- 5 Approve the requirement and link every downstream artifact.

EVIDENCE TO RETAIN

APPROVED CRITERION

DECISION SOURCE

EXAMPLES

TEST MAPPING

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

WHEN an event is cancelled,
THE Refunds service SHALL create exactly one
refund
for ticket price minus frozen service fees
within five business days.

FAILURE MODE / LIMIT

A well-formed sentence can still encode the wrong policy, omit an actor, or hide an unmeasurable quality boundary. Syntax helps expose gaps; it does not supply domain truth.

WARNING SIGNS

- Words such as fast, automatic, appropriate, or supported remain unquantified.
- One sentence contains several triggers or outcomes.
- The policy owner and measurement method are missing.
- A test can pass without proving the business rule.

DECISION RULE

Do not authorize planning while a reasonable reviewer can derive two materially different behaviors from the same requirement.

EVIDENCE

[R6] Mavin et al. 2009 EARS

[R5] ISO/IEC/IEEE 29148:2018

RESEARCH EXPANSION

Event-driven requirements

name the initiating event and the observable response while distinguishing trigger from implementation mechanism.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to name the initiating event and the observable response while distinguishing trigger from implementation mechanism. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

name the initiating event and the observable response while distinguishing trigger from implementation mechanism. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Identify the policy owner and the decision source.
- 2 Separate trigger, state, actor, response, and threshold.
- 3 Write boundary, error, timing, and data examples.
- 4 Derive an independent failing test or review oracle.
- 5 Approve the requirement and link every downstream artifact.

EVIDENCE TO RETAIN

APPROVED CRITERION

DECISION SOURCE

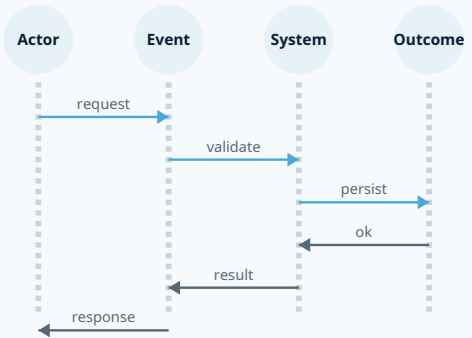
EXAMPLES

TEST MAPPING

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

WHEN an event is cancelled,
THE Refunds service SHALL create exactly one
refund
for ticket price minus frozen service fees
within five business days.

FAILURE MODE / LIMIT

A well-formed sentence can still encode the wrong policy, omit an actor, or hide an unmeasurable quality boundary. Syntax helps expose gaps; it does not supply domain truth.

WARNING SIGNS

- Words such as fast, automatic, appropriate, or supported remain unquantified.
- One sentence contains several triggers or outcomes.
- The policy owner and measurement method are missing.
- A test can pass without proving the business rule.

DECISION RULE

Do not authorize planning while a reasonable reviewer can derive two materially different behaviors from the same requirement.

EVIDENCE

[R6] Mavin et al. 2009 EARS

[R5] ISO/IEC/IEEE 29148:2018

RESEARCH EXPANSION

State-driven requirements

bind obligations to an explicit duration or state and define entry, exit, and competing transitions.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first bind obligations to an explicit duration or state and define entry, exit, and competing transitions, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

bind obligations to an explicit duration or state and define entry, exit, and competing transitions. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Identify the policy owner and the decision source.
- 2 Separate trigger, state, actor, response, and threshold.
- 3 Write boundary, error, timing, and data examples.
- 4 Derive an independent failing test or review oracle.
- 5 Approve the requirement and link every downstream artifact.

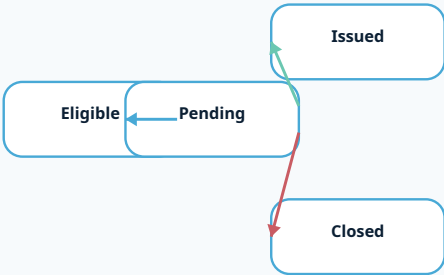
EVIDENCE TO RETAIN

- APPROVED CRITERION
- DECISION SOURCE
- EXAMPLES
- TEST MAPPING

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

WHEN an event is cancelled,
THE Refunds service SHALL create exactly one
refund
for ticket price minus frozen service fees
within five business days.

FAILURE MODE / LIMIT

A well-formed sentence can still encode the wrong policy, omit an actor, or hide an unmeasurable quality boundary. Syntax helps expose gaps; it does not supply domain truth.

WARNING SIGNS

- Words such as fast, automatic, appropriate, or supported remain unquantified.
- One sentence contains several triggers or outcomes.
- The policy owner and measurement method are missing.
- A test can pass without proving the business rule.

DECISION RULE

Do not authorize planning while a reasonable reviewer can derive two materially different behaviors from the same requirement.

EVIDENCE

[R6] Mavin et al. 2009 EARS

[R5] ISO/IEC/IEEE 29148:2018

RESEARCH EXPANSION

Unwanted behavior requirements

specify detection, containment, response, evidence, and recovery for invalid or hazardous conditions.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must specify detection, containment, response, evidence, and recovery for invalid or hazardous conditions; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

specify detection, containment, response, evidence, and recovery for invalid or hazardous conditions. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Identify the policy owner and the decision source.
- 2 Separate trigger, state, actor, response, and threshold.
- 3 Write boundary, error, timing, and data examples.
- 4 Derive an independent failing test or review oracle.
- 5 Approve the requirement and link every downstream artifact.

EVIDENCE TO RETAIN

APPROVED CRITERION

DECISION SOURCE

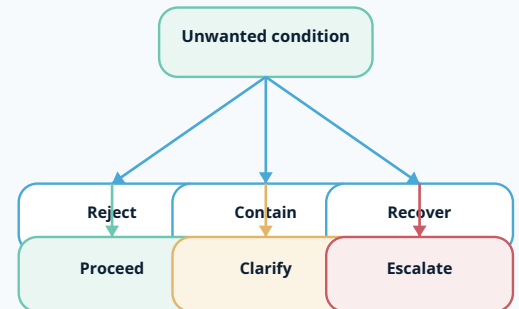
EXAMPLES

TEST MAPPING

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

WHEN an event is cancelled,
THE Refunds service SHALL create exactly one
refund
for ticket price minus frozen service fees
within five business days.

FAILURE MODE / LIMIT

A well-formed sentence can still encode the wrong policy, omit an actor, or hide an unmeasurable quality boundary. Syntax helps expose gaps; it does not supply domain truth.

WARNING SIGNS

- Words such as fast, automatic, appropriate, or supported remain unquantified.
- One sentence contains several triggers or outcomes.
- The policy owner and measurement method are missing.
- A test can pass without proving the business rule.

DECISION RULE

Do not authorize planning while a reasonable reviewer can derive two materially different behaviors from the same requirement.

EVIDENCE

[R6] Mavin et al. 2009 EARS

[R44] OWASP ASVS

RESEARCH EXPANSION

Optional feature requirements

make configuration and feature-flag conditions explicit so optional behavior does not leak into the default path.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To make configuration and feature-flag conditions explicit so optional behavior does not leak into the default path, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

make configuration and feature-flag conditions explicit so optional behavior does not leak into the default path. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Identify the policy owner and the decision source.
- 2 Separate trigger, state, actor, response, and threshold.
- 3 Write boundary, error, timing, and data examples.
- 4 Derive an independent failing test or review oracle.
- 5 Approve the requirement and link every downstream artifact.

EVIDENCE TO RETAIN

APPROVED CRITERION

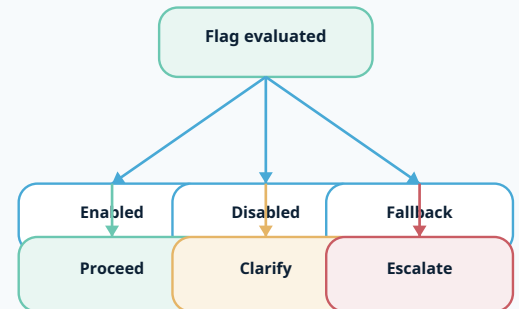
DECISION SOURCE

EXAMPLES

TEST MAPPING

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

WHEN an event is cancelled,
THE Refunds service SHALL create exactly one
refund
for ticket price minus frozen service fees
within five business days.

FAILURE MODE / LIMIT

A well-formed sentence can still encode the wrong policy, omit an actor, or hide an unmeasurable quality boundary. Syntax helps expose gaps; it does not supply domain truth.

WARNING SIGNS

- Words such as fast, automatic, appropriate, or supported remain unquantified.
- One sentence contains several triggers or outcomes.
- The policy owner and measurement method are missing.
- A test can pass without proving the business rule.

DECISION RULE

Do not authorize planning while a reasonable reviewer can derive two materially different behaviors from the same requirement.

EVIDENCE

[R6] Mavin et al. 2009 EARS

[R48] OpenFeature specification

RESEARCH EXPANSION

Composing EARS patterns safely

split multi-trigger, multi-response sentences into atomic requirements while preserving a shared rule and scenario.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to split multi-trigger, multi-response sentences into atomic requirements while preserving a shared rule and scenario without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

split multi-trigger, multi-response sentences into atomic requirements while preserving a shared rule and scenario. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Identify the policy owner and the decision source.
- 2 Separate trigger, state, actor, response, and threshold.
- 3 Write boundary, error, timing, and data examples.
- 4 Derive an independent failing test or review oracle.
- 5 Approve the requirement and link every downstream artifact.

EVIDENCE TO RETAIN

APPROVED CRITERION

DECISION SOURCE

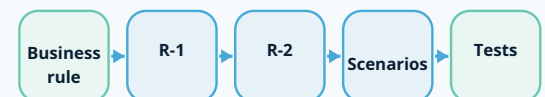
EXAMPLES

TEST MAPPING

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

WHEN an event is cancelled,
THE Refunds service SHALL create exactly one
refund
for ticket price minus frozen service fees
within five business days.

FAILURE MODE / LIMIT

A well-formed sentence can still encode the wrong policy, omit an actor, or hide an unmeasurable quality boundary. Syntax helps expose gaps; it does not supply domain truth.

WARNING SIGNS

- Words such as fast, automatic, appropriate, or supported remain unquantified.
- One sentence contains several triggers or outcomes.
- The policy owner and measurement method are missing.
- A test can pass without proving the business rule.

DECISION RULE

Do not authorize planning while a reasonable reviewer can derive two materially different behaviors from the same requirement.

EVIDENCE

[R6] Mavin et al. 2009 EARS

[R7] Requirements quality mapping

RESEARCH EXPANSION

Non-functional requirements as measurable envelopes

express latency, throughput, reliability, capacity, availability, recovery, and resource constraints with scope and measurement method.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to express latency, throughput, reliability, capacity, availability, recovery, and resource constraints with scope and measurement method. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

express latency, throughput, reliability, capacity, availability, recovery, and resource constraints with scope and measurement method. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Identify the policy owner and the decision source.
- 2 Separate trigger, state, actor, response, and threshold.
- 3 Write boundary, error, timing, and data examples.
- 4 Derive an independent failing test or review oracle.
- 5 Approve the requirement and link every downstream artifact.

EVIDENCE TO RETAIN

APPROVED CRITERION

DECISION SOURCE

EXAMPLES

TEST MAPPING

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

Latency

4.2 d

Availability

8/wk

Capacity

6%

Recovery

40 min

WORKED EXAMPLE

WHEN an event is cancelled,
THE Refunds service SHALL create exactly one
refund
for ticket price minus frozen service fees
within five business days.

FAILURE MODE / LIMIT

A well-formed sentence can still encode the wrong policy, omit an actor, or hide an unmeasurable quality boundary. Syntax helps expose gaps; it does not supply domain truth.

WARNING SIGNS

- Words such as fast, automatic, appropriate, or supported remain unquantified.
- One sentence contains several triggers or outcomes.
- The policy owner and measurement method are missing.
- A test can pass without proving the business rule.

DECISION RULE

Do not authorize planning while a reasonable reviewer can derive two materially different behaviors from the same requirement.

EVIDENCE

[R5] ISO/IEC/IEEE 29148:2018

[R46] Google SRE Workbook

RESEARCH EXPANSION

Data and money semantics

define currency, minor units, precision, rounding point, tax and fee authority, snapshots, and reconciliation evidence.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first define currency, minor units, precision, rounding point, tax and fee authority, snapshots, and reconciliation evidence, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

define currency, minor units, precision, rounding point, tax and fee authority, snapshots, and reconciliation evidence. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Identify the policy owner and the decision source.
- 2 Separate trigger, state, actor, response, and threshold.
- 3 Write boundary, error, timing, and data examples.
- 4 Derive an independent failing test or review oracle.
- 5 Approve the requirement and link every downstream artifact.

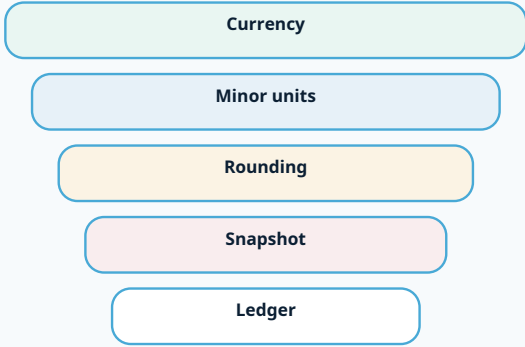
EVIDENCE TO RETAIN

- APPROVED CRITERION
- DECISION SOURCE
- EXAMPLES
- TEST MAPPING

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- Words such as fast, automatic, appropriate, or supported remain unquantified.
- One sentence contains several triggers or outcomes.
- The policy owner and measurement method are missing.
- A test can pass without proving the business rule.

DECISION RULE

Do not authorize planning while a reasonable reviewer can derive two materially different behaviors from the same requirement.

EVIDENCE

[R5] ISO/IEC/IEEE 29148:2018

[R51] Jackson, Software Abstractions

RESEARCH EXPANSION

Time, deadlines, and calendars

distinguish elapsed time, business days, time zones, clock source, grace periods, and eventual consistency.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must distinguish elapsed time, business days, time zones, clock source, grace periods, and eventual consistency; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

distinguish elapsed time, business days, time zones, clock source, grace periods, and eventual consistency. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Identify the policy owner and the decision source.
- 2 Separate trigger, state, actor, response, and threshold.
- 3 Write boundary, error, timing, and data examples.
- 4 Derive an independent failing test or review oracle.
- 5 Approve the requirement and link every downstream artifact.

EVIDENCE TO RETAIN

APPROVED CRITERION

DECISION SOURCE

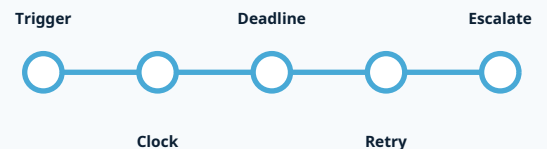
EXAMPLES

TEST MAPPING

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 ->
TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- Words such as fast, automatic, appropriate, or supported remain unquantified.
- One sentence contains several triggers or outcomes.
- The policy owner and measurement method are missing.
- A test can pass without proving the business rule.

DECISION RULE

Do not authorize planning while a reasonable reviewer can derive two materially different behaviors from the same requirement.

EVIDENCE

[R5] ISO/IEC/IEEE 29148:2018

[R46] Google SRE Workbook

RESEARCH EXPANSION

Security and privacy requirements

translate threats, access policy, data minimization, auditability, and failure responses into testable obligations.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To translate threats, access policy, data minimization, auditability, and failure responses into testable obligations, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

translate threats, access policy, data minimization, auditability, and failure responses into testable obligations. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Identify the policy owner and the decision source.
- 2 Separate trigger, state, actor, response, and threshold.
- 3 Write boundary, error, timing, and data examples.
- 4 Derive an independent failing test or review oracle.
- 5 Approve the requirement and link every downstream artifact.

EVIDENCE TO RETAIN

APPROVED CRITERION

DECISION SOURCE

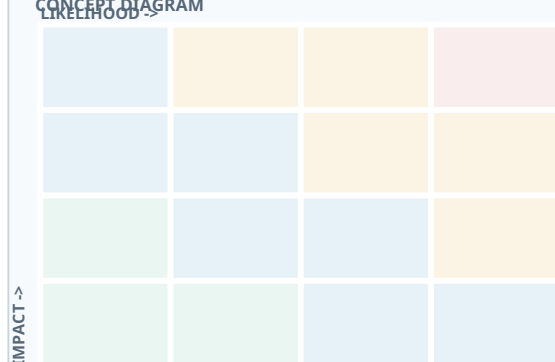
EXAMPLES

TEST MAPPING

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

WHEN an event is cancelled,
THE Refunds service SHALL create exactly one
refund
for ticket price minus frozen service fees
within five business days.

FAILURE MODE / LIMIT

A well-formed sentence can still encode the wrong policy, omit an actor, or hide an unmeasurable quality boundary. Syntax helps expose gaps; it does not supply domain truth.

WARNING SIGNS

- Words such as fast, automatic, appropriate, or supported remain unquantified.
- One sentence contains several triggers or outcomes.
- The policy owner and measurement method are missing.
- A test can pass without proving the business rule.

DECISION RULE

Do not authorize planning while a reasonable reviewer can derive two materially different behaviors from the same requirement.

EVIDENCE

[R44] OWASP ASVS

[R5] ISO/IEC/IEEE 29148:2018

RESEARCH EXPANSION

Accessibility as specification

convert accessibility obligations into acceptance behavior rather than postponing them to a final audit.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to convert accessibility obligations into acceptance behavior rather than postponing them to a final audit without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

convert accessibility obligations into acceptance behavior rather than postponing them to a final audit. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Identify the policy owner and the decision source.
- 2 Separate trigger, state, actor, response, and threshold.
- 3 Write boundary, error, timing, and data examples.
- 4 Derive an independent failing test or review oracle.
- 5 Approve the requirement and link every downstream artifact.

EVIDENCE TO RETAIN

APPROVED CRITERION

DECISION SOURCE

EXAMPLES

TEST MAPPING

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- Words such as fast, automatic, appropriate, or supported remain unquantified.
- One sentence contains several triggers or outcomes.
- The policy owner and measurement method are missing.
- A test can pass without proving the business rule.

DECISION RULE

Do not authorize planning while a reasonable reviewer can derive two materially different behaviors from the same requirement.

EVIDENCE

[R45] WCAG 2.2

[R5] ISO/IEC/IEEE 29148:2018

EXPANDED FROM SUPPLIED WALKTHROUGH

PLT-4821 EARS review laboratory

turn the phrases minus fees, automatic, works, and cannot claim twice into explicit decisions and linked criteria.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to turn the phrases minus fees, automatic, works, and cannot claim twice into explicit decisions and linked criteria. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

turn the phrases minus fees, automatic, works, and cannot claim twice into explicit decisions and linked criteria. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Identify the policy owner and the decision source.
- 2 Separate trigger, state, actor, response, and threshold.
- 3 Write boundary, error, timing, and data examples.
- 4 Derive an independent failing test or review oracle.
- 5 Approve the requirement and link every downstream artifact.

EVIDENCE TO RETAIN

APPROVED CRITERION

DECISION SOURCE

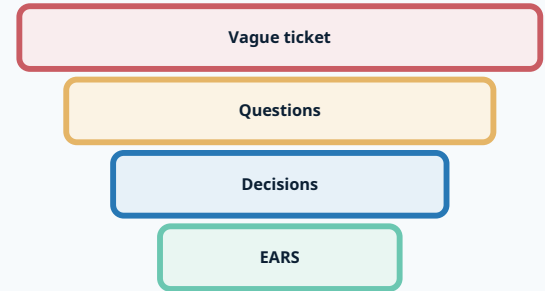
EXAMPLES

TEST MAPPING

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

WHEN an event is cancelled,
THE Refunds service SHALL create exactly one
refund
for ticket price minus frozen service fees
within five business days.

FAILURE MODE / LIMIT

A well-formed sentence can still encode the wrong policy, omit an actor, or hide an unmeasurable quality boundary. Syntax helps expose gaps; it does not supply domain truth.

WARNING SIGNS

- Words such as fast, automatic, appropriate, or supported remain unquantified.
- One sentence contains several triggers or outcomes.
- The policy owner and measurement method are missing.
- A test can pass without proving the business rule.

DECISION RULE

Do not authorize planning while a reasonable reviewer can derive two materially different behaviors from the same requirement.

EVIDENCE

[R6] Mavin et al. 2009 EARS

[R5] ISO/IEC/IEEE 29148:2018

RESEARCH EXPANSION

What EARS cannot guarantee

recognize that controlled syntax improves form but does not prove completeness, correctness, stakeholder agreement, or system feasibility.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first recognize that controlled syntax improves form but does not prove completeness, correctness, stakeholder agreement, or system feasibility, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

recognize that controlled syntax improves form but does not prove completeness, correctness, stakeholder agreement, or system feasibility. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Identify the policy owner and the decision source.
- 2 Separate trigger, state, actor, response, and threshold.
- 3 Write boundary, error, timing, and data examples.
- 4 Derive an independent failing test or review oracle.
- 5 Approve the requirement and link every downstream artifact.

EVIDENCE TO RETAIN

APPROVED CRITERION

DECISION SOURCE

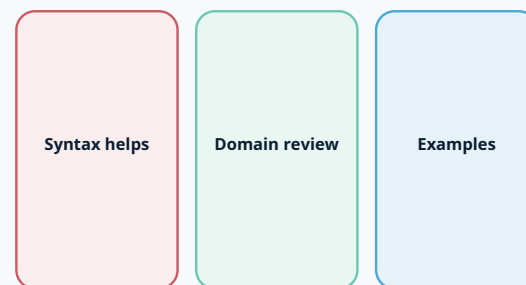
EXAMPLES

TEST MAPPING

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

WHEN an event is cancelled,
THE Refunds service SHALL create exactly one
refund
for ticket price minus frozen service fees
within five business days.

FAILURE MODE / LIMIT

A well-formed sentence can still encode the wrong policy, omit an actor, or hide an unmeasurable quality boundary. Syntax helps expose gaps; it does not supply domain truth.

WARNING SIGNS

- Words such as fast, automatic, appropriate, or supported remain unquantified.
- One sentence contains several triggers or outcomes.
- The policy owner and measurement method are missing.
- A test can pass without proving the business rule.

DECISION RULE

Do not authorize planning while a reasonable reviewer can derive two materially different behaviors from the same requirement.

EVIDENCE

[R6] Mavin et al. 2009 EARS

[R7] Requirements quality mapping

[R30] Formal RE and LLM roadmap

PART VIII

Domain modeling as the semantic foundation

Use Domain-Driven Design to decide what words mean, where rules belong, and which boundaries must remain explicit.

WHAT THIS PART ANSWERS

What is the ubiquitous language? Which bounded context owns the policy? What must be frozen, translated, or published as an event?

USE IT WHEN

A feature crosses teams, services, or vocabularies and plausible technical solutions disagree about business meaning.

RESEARCH EXPANSION

Strategic DDD before tactical patterns

identify core, supporting, and generic subdomains and decide where modeling investment creates differentiation.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first identify core, supporting, and generic subdomains and decide where modeling investment creates differentiation, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

identify core, supporting, and generic subdomains and decide where modeling investment creates differentiation. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Collect the language used by domain experts in real cases.
- 2 Map commands, events, policies, and disputed terminology.
- 3 Assign data and rule authority to explicit bounded contexts.
- 4 Choose the smallest boundary that protects each invariant.
- 5 Trace the model vocabulary into requirements and code names.

EVIDENCE TO RETAIN

GLOSSARY

CONTEXT MAP

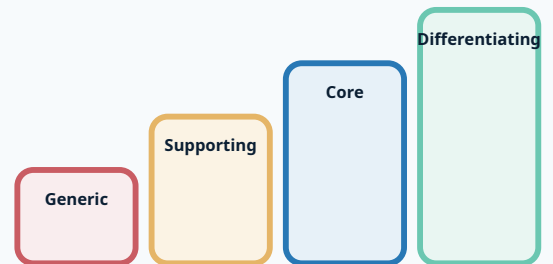
INVARIANT

DOMAIN EVENT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Refund aggregate

- identity: orderId
- invariant: at most one active refund
- value object: Money(minorUnits, currency)
- event: RefundIssued

FAILURE MODE / LIMIT

Treating DDD as a folder convention or a collection of tactical classes creates ceremony without clarifying ownership, language, or consistency boundaries.

WARNING SIGNS

- The same noun means different things across teams.
- Data ownership is confused with business-rule authority.
- Every bounded context is turned into a network service.
- Invariants span several aggregates without an explicit policy.

DECISION RULE

Do not choose a technical boundary until rule authority, language, and consistency needs are explicit.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R16] Vernon, Implementing DDD

RESEARCH EXPANSION

A ubiquitous-language workshop

capture terms, definitions, synonyms, forbidden overloads, examples, and unresolved disagreements in the language of the domain.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must capture terms, definitions, synonyms, forbidden overloads, examples, and unresolved disagreements in the language of the domain; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

capture terms, definitions, synonyms, forbidden overloads, examples, and unresolved disagreements in the language of the domain. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Collect the language used by domain experts in real cases.
- 2 Map commands, events, policies, and disputed terminology.
- 3 Assign data and rule authority to explicit bounded contexts.
- 4 Choose the smallest boundary that protects each invariant.
- 5 Trace the model vocabulary into requirements and code names.

EVIDENCE TO RETAIN

GLOSSARY

CONTEXT MAP

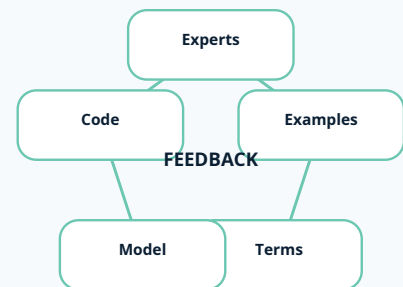
INVARIANT

DOMAIN EVENT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The same noun means different things across teams.
- Data ownership is confused with business-rule authority.
- Every bounded context is turned into a network service.
- Invariants span several aggregates without an explicit policy.

DECISION RULE

Do not choose a technical boundary until rule authority, language, and consistency needs are explicit.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R16] Vernon, Implementing DDD

RESEARCH EXPANSION

Finding bounded contexts

use language shifts, policy ownership, lifecycle differences, data authority, and team boundaries to locate model boundaries.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To use language shifts, policy ownership, lifecycle differences, data authority, and team boundaries to locate model boundaries, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

use language shifts, policy ownership, lifecycle differences, data authority, and team boundaries to locate model boundaries. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Collect the language used by domain experts in real cases.
- 2 Map commands, events, policies, and disputed terminology.
- 3 Assign data and rule authority to explicit bounded contexts.
- 4 Choose the smallest boundary that protects each invariant.
- 5 Trace the model vocabulary into requirements and code names.

EVIDENCE TO RETAIN

GLOSSARY

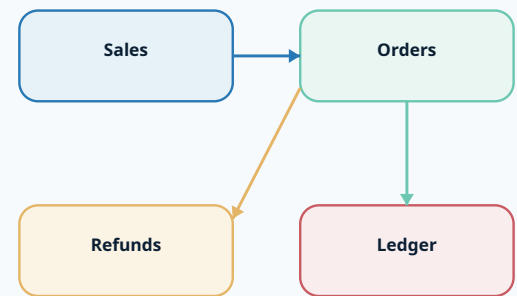
CONTEXT MAP

INVARIANT

DOMAIN EVENT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

```
ACTIVE_TASK: T3
ALLOWED_PATHS: src/refunds/**, tests/refunds/**
REQUIRED_GATES: unit, BDD, OpenAPI
STOP_ON: ambiguity, contract conflict, scope breach
```

FAILURE MODE / LIMIT

A prompt is advisory. Without executable gates, repository permissions, and independent verification, an agent can satisfy the apparent task while violating scope or hidden intent.

WARNING SIGNS

- The same noun means different things across teams.
- Data ownership is confused with business-rule authority.
- Every bounded context is turned into a network service.
- Invariants span several aggregates without an explicit policy.

DECISION RULE

Do not choose a technical boundary until rule authority, language, and consistency needs are explicit.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R17] DDD systematic review 2025

[R37] DORA loosely coupled teams

RESEARCH EXPANSION

Context-map relationships

record upstream-downstream power, contracts, translation, partnership, conformist behavior, and anti-corruption boundaries.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to record upstream-downstream power, contracts, translation, partnership, conformist behavior, and anti-corruption boundaries without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

record upstream-downstream power, contracts, translation, partnership, conformist behavior, and anti-corruption boundaries. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Collect the language used by domain experts in real cases.
- 2 Map commands, events, policies, and disputed terminology.
- 3 Assign data and rule authority to explicit bounded contexts.
- 4 Choose the smallest boundary that protects each invariant.
- 5 Trace the model vocabulary into requirements and code names.

EVIDENCE TO RETAIN

GLOSSARY

CONTEXT MAP

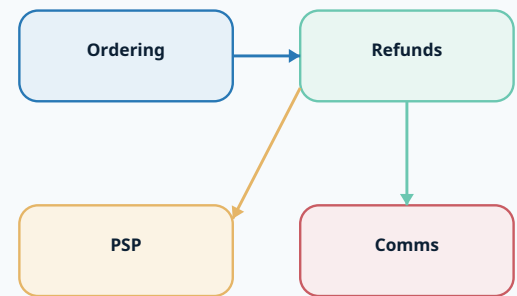
INVARIANT

DOMAIN EVENT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

```
ACTIVE_TASK: T3
ALLOWED_PATHS: src/refunds/**, tests/refunds/**
REQUIRED_GATES: unit, BDD, OpenAPI
STOP_ON: ambiguity, contract conflict, scope breach
```

FAILURE MODE / LIMIT

A prompt is advisory. Without executable gates, repository permissions, and independent verification, an agent can satisfy the apparent task while violating scope or hidden intent.

WARNING SIGNS

- The same noun means different things across teams.
- Data ownership is confused with business-rule authority.
- Every bounded context is turned into a network service.
- Invariants span several aggregates without an explicit policy.

DECISION RULE

Do not choose a technical boundary until rule authority, language, and consistency needs are explicit.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R16] Vernon, Implementing DDD

RESEARCH EXPANSION

Subdomains and investment choices

avoid applying rich modeling uniformly by matching design depth to business differentiation and risk.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to avoid applying rich modeling uniformly by matching design depth to business differentiation and risk. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

avoid applying rich modeling uniformly by matching design depth to business differentiation and risk. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Collect the language used by domain experts in real cases.
- 2 Map commands, events, policies, and disputed terminology.
- 3 Assign data and rule authority to explicit bounded contexts.
- 4 Choose the smallest boundary that protects each invariant.
- 5 Trace the model vocabulary into requirements and code names.

EVIDENCE TO RETAIN

GLOSSARY

CONTEXT MAP

INVARIANT

DOMAIN EVENT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

Value	Volatility	Risk	Complexity
Value	Volatility	Risk	Complexity
Value	Volatility	Risk	Complexity
Value	Volatility	Risk	Complexity

WORKED EXAMPLE

```
Refund aggregate
- identity: orderId
- invariant: at most one active refund
- value object: Money(minorUnits, currency)
- event: RefundIssued
```

FAILURE MODE / LIMIT

Treating DDD as a folder convention or a collection of tactical classes creates ceremony without clarifying ownership, language, or consistency boundaries.

WARNING SIGNS

- The same noun means different things across teams.
- Data ownership is confused with business-rule authority.
- Every bounded context is turned into a network service.
- Invariants span several aggregates without an explicit policy.

DECISION RULE

Do not choose a technical boundary until rule authority, language, and consistency needs are explicit.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R17] DDD systematic review 2025

RESEARCH EXPANSION

Event storming for specification discovery

surface commands, events, policies, actors, hotspots, and external systems before committing to an implementation structure.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first surface commands, events, policies, actors, hotspots, and external systems before committing to an implementation structure, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

surface commands, events, policies, actors, hotspots, and external systems before committing to an implementation structure. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Collect the language used by domain experts in real cases.
- 2 Map commands, events, policies, and disputed terminology.
- 3 Assign data and rule authority to explicit bounded contexts.
- 4 Choose the smallest boundary that protects each invariant.
- 5 Trace the model vocabulary into requirements and code names.

EVIDENCE TO RETAIN

GLOSSARY

CONTEXT MAP

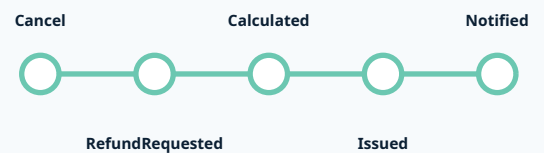
INVARIANT

DOMAIN EVENT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Refund aggregate

- identity: orderId
- invariant: at most one active refund
- value object: Money(minorUnits, currency)
- event: RefundIssued

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The same noun means different things across teams.
- Data ownership is confused with business-rule authority.
- Every bounded context is turned into a network service.
- Invariants span several aggregates without an explicit policy.

DECISION RULE

Do not choose a technical boundary until rule authority, language, and consistency needs are explicit.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R16] Vernon, Implementing DDD

RESEARCH EXPANSION

Entities and value objects

distinguish identity-bearing lifecycles from immutable descriptive values and encode equality accordingly.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must distinguish identity-bearing lifecycles from immutable descriptive values and encode equality accordingly; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

distinguish identity-bearing lifecycles from immutable descriptive values and encode equality accordingly. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Collect the language used by domain experts in real cases.
- 2 Map commands, events, policies, and disputed terminology.
- 3 Assign data and rule authority to explicit bounded contexts.
- 4 Choose the smallest boundary that protects each invariant.
- 5 Trace the model vocabulary into requirements and code names.

EVIDENCE TO RETAIN

GLOSSARY

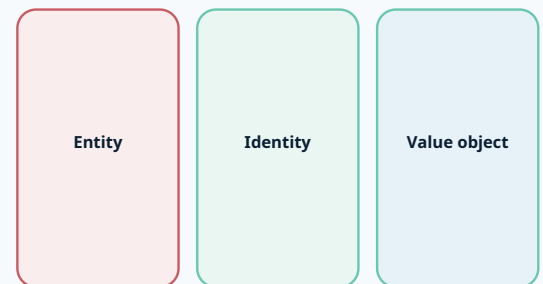
CONTEXT MAP

INVARIANT

DOMAIN EVENT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

```
Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 ->
TEST-12 -> EVID-7
```

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The same noun means different things across teams.
- Data ownership is confused with business-rule authority.
- Every bounded context is turned into a network service.
- Invariants span several aggregates without an explicit policy.

DECISION RULE

Do not choose a technical boundary until rule authority, language, and consistency needs are explicit.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R16] Vernon, Implementing DDD

RESEARCH EXPANSION

Aggregates and invariants

choose the smallest consistency boundary that can protect business rules under concurrent change.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To choose the smallest consistency boundary that can protect business rules under concurrent change, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

choose the smallest consistency boundary that can protect business rules under concurrent change. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Collect the language used by domain experts in real cases.
- 2 Map commands, events, policies, and disputed terminology.
- 3 Assign data and rule authority to explicit bounded contexts.
- 4 Choose the smallest boundary that protects each invariant.
- 5 Trace the model vocabulary into requirements and code names.

EVIDENCE TO RETAIN

GLOSSARY

CONTEXT MAP

INVARIANT

DOMAIN EVENT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

Aggregate root

Entities

Values

Invariant

Transaction

WORKED EXAMPLE

Refund aggregate

- identity: orderId
- invariant: at most one active refund
- value object: Money(minorUnits, currency)
- event: RefundIssued

FAILURE MODE / LIMIT

Treating DDD as a folder convention or a collection of tactical classes creates ceremony without clarifying ownership, language, or consistency boundaries.

WARNING SIGNS

- The same noun means different things across teams.
- Data ownership is confused with business-rule authority.
- Every bounded context is turned into a network service.
- Invariants span several aggregates without an explicit policy.

DECISION RULE

Do not choose a technical boundary until rule authority, language, and consistency needs are explicit.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R16] Vernon, Implementing DDD

RESEARCH EXPANSION

Domain services and policy objects

place cross-entity business calculations in explicit domain concepts without turning every operation into an anemic service.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to place cross-entity business calculations in explicit domain concepts without turning every operation into an anemic service without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

place cross-entity business calculations in explicit domain concepts without turning every operation into an anemic service. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Collect the language used by domain experts in real cases.
- 2 Map commands, events, policies, and disputed terminology.
- 3 Assign data and rule authority to explicit bounded contexts.
- 4 Choose the smallest boundary that protects each invariant.
- 5 Trace the model vocabulary into requirements and code names.

EVIDENCE TO RETAIN

GLOSSARY

CONTEXT MAP

INVARIANT

DOMAIN EVENT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Refund aggregate

- identity: orderId
- invariant: at most one active refund
- value object: Money(minorUnits, currency)
- event: RefundIssued

FAILURE MODE / LIMIT

Treating DDD as a folder convention or a collection of tactical classes creates ceremony without clarifying ownership, language, or consistency boundaries.

WARNING SIGNS

- The same noun means different things across teams.
- Data ownership is confused with business-rule authority.
- Every bounded context is turned into a network service.
- Invariants span several aggregates without an explicit policy.

DECISION RULE

Do not choose a technical boundary until rule authority, language, and consistency needs are explicit.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R16] Vernon, Implementing DDD

RESEARCH EXPANSION

Domain events as business facts

name facts in past tense, define versioned semantics, and separate domain meaning from transport details.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to name facts in past tense, define versioned semantics, and separate domain meaning from transport details. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

name facts in past tense, define versioned semantics, and separate domain meaning from transport details. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Collect the language used by domain experts in real cases.
- 2 Map commands, events, policies, and disputed terminology.
- 3 Assign data and rule authority to explicit bounded contexts.
- 4 Choose the smallest boundary that protects each invariant.
- 5 Trace the model vocabulary into requirements and code names.

EVIDENCE TO RETAIN

GLOSSARY

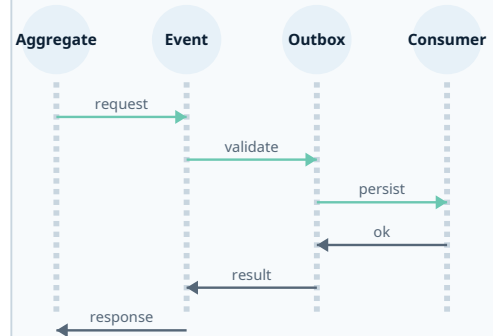
CONTEXT MAP

INVARIANT

DOMAIN EVENT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Refund aggregate
- identity: orderId
- invariant: at most one active refund
- value object: Money(minorUnits, currency)
- event: RefundIssued

FAILURE MODE / LIMIT

Treating DDD as a folder convention or a collection of tactical classes creates ceremony without clarifying ownership, language, or consistency boundaries.

WARNING SIGNS

- The same noun means different things across teams.
- Data ownership is confused with business-rule authority.
- Every bounded context is turned into a network service.
- Invariants span several aggregates without an explicit policy.

DECISION RULE

Do not choose a technical boundary until rule authority, language, and consistency needs are explicit.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R16] Vernon, Implementing DDD

RESEARCH EXPANSION

Anti-corruption layers

translate unstable or foreign models at the boundary so external vocabulary does not infect the core domain model.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first translate unstable or foreign models at the boundary so external vocabulary does not infect the core domain model, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

translate unstable or foreign models at the boundary so external vocabulary does not infect the core domain model. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Collect the language used by domain experts in real cases.
- 2 Map commands, events, policies, and disputed terminology.
- 3 Assign data and rule authority to explicit bounded contexts.
- 4 Choose the smallest boundary that protects each invariant.
- 5 Trace the model vocabulary into requirements and code names.

EVIDENCE TO RETAIN

GLOSSARY

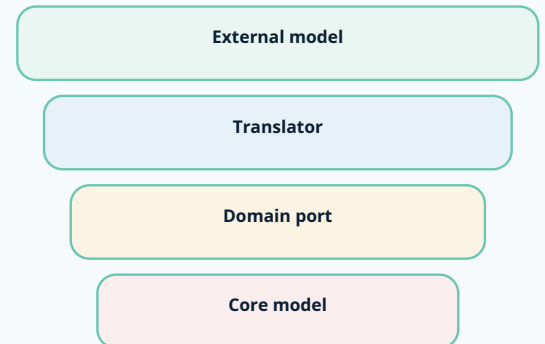
CONTEXT MAP

INVARIANT

DOMAIN EVENT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The same noun means different things across teams.
- Data ownership is confused with business-rule authority.
- Every bounded context is turned into a network service.
- Invariants span several aggregates without an explicit policy.

DECISION RULE

Do not choose a technical boundary until rule authority, language, and consistency needs are explicit.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R16] Vernon, Implementing DDD

[R23] Cockburn, Hexagonal Architecture

RESEARCH EXPANSION

Tracing DDD concepts into specifications

link glossary terms, business rules, bounded contexts, aggregates, and events to requirement and scenario identifiers.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must link glossary terms, business rules, bounded contexts, aggregates, and events to requirement and scenario identifiers; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

link glossary terms, business rules, bounded contexts, aggregates, and events to requirement and scenario identifiers. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Collect the language used by domain experts in real cases.
- 2 Map commands, events, policies, and disputed terminology.
- 3 Assign data and rule authority to explicit bounded contexts.
- 4 Choose the smallest boundary that protects each invariant.
- 5 Trace the model vocabulary into requirements and code names.

EVIDENCE TO RETAIN

GLOSSARY

CONTEXT MAP

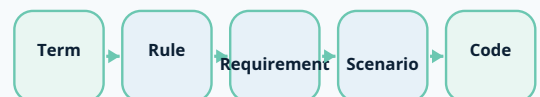
INVARIANT

DOMAIN EVENT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Refund aggregate

- identity: orderId
- invariant: at most one active refund
- value object: Money(minorUnits, currency)
- event: RefundIssued

FAILURE MODE / LIMIT

Treating DDD as a folder convention or a collection of tactical classes creates ceremony without clarifying ownership, language, or consistency boundaries.

WARNING SIGNS

- The same noun means different things across teams.
- Data ownership is confused with business-rule authority.
- Every bounded context is turned into a network service.
- Invariants span several aggregates without an explicit policy.

DECISION RULE

Do not choose a technical boundary until rule authority, language, and consistency needs are explicit.

EVIDENCE

[R26] Gotel and Finkelstein 1994

[R27] Requirements traceability book

[R15] Evans, Domain-Driven Design

RESEARCH EXPANSION

DDD and microservices: a necessary caution

avoid assuming that bounded contexts automatically require separate network services or that microservices guarantee domain integrity.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To avoid assuming that bounded contexts automatically require separate network services or that microservices guarantee domain integrity, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

avoid assuming that bounded contexts automatically require separate network services or that microservices guarantee domain integrity. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Collect the language used by domain experts in real cases.
- 2 Map commands, events, policies, and disputed terminology.
- 3 Assign data and rule authority to explicit bounded contexts.
- 4 Choose the smallest boundary that protects each invariant.
- 5 Trace the model vocabulary into requirements and code names.

EVIDENCE TO RETAIN

GLOSSARY

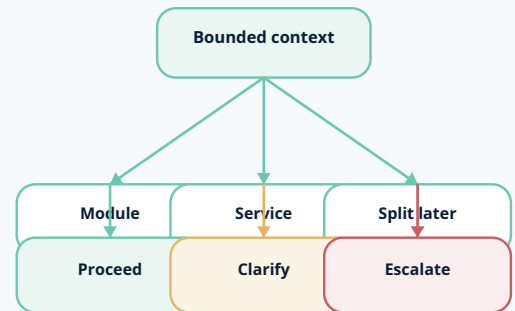
CONTEXT MAP

INVARIANT

DOMAIN EVENT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Refund aggregate

- identity: orderId
- invariant: at most one active refund
- value object: Money(minorUnits, currency)
- event: RefundIssued

FAILURE MODE / LIMIT

Treating DDD as a folder convention or a collection of tactical classes creates ceremony without clarifying ownership, language, or consistency boundaries.

WARNING SIGNS

- The same noun means different things across teams.
- Data ownership is confused with business-rule authority.
- Every bounded context is turned into a network service.
- Invariants span several aggregates without an explicit policy.

DECISION RULE

Do not choose a technical boundary until rule authority, language, and consistency needs are explicit.

EVIDENCE

[R17] DDD systematic review 2025

[R37] DORA loosely coupled teams

EXPANDED FROM SUPPLIED WALKTHROUGH

PLT-4821 domain model

place frozen order totals in Ordering, refund policy and idempotency in Refunds, money movement in Payments, and communication in Notification.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to place frozen order totals in Ordering, refund policy and idempotency in Refunds, money movement in Payments, and communication in Notification without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

place frozen order totals in Ordering, refund policy and idempotency in Refunds, money movement in Payments, and communication in Notification. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Collect the language used by domain experts in real cases.
- 2 Map commands, events, policies, and disputed terminology.
- 3 Assign data and rule authority to explicit bounded contexts.
- 4 Choose the smallest boundary that protects each invariant.
- 5 Trace the model vocabulary into requirements and code names.

EVIDENCE TO RETAIN

GLOSSARY

CONTEXT MAP

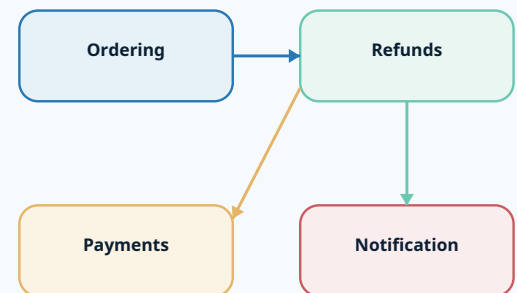
INVARIANT

DOMAIN EVENT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Refund aggregate

- identity: orderId
- invariant: at most one active refund
- value object: Money(minorUnits, currency)
- event: RefundIssued

FAILURE MODE / LIMIT

Treating DDD as a folder convention or a collection of tactical classes creates ceremony without clarifying ownership, language, or consistency boundaries.

WARNING SIGNS

- The same noun means different things across teams.
- Data ownership is confused with business-rule authority.
- Every bounded context is turned into a network service.
- Invariants span several aggregates without an explicit policy.

DECISION RULE

Do not choose a technical boundary until rule authority, language, and consistency needs are explicit.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R16] Vernon, Implementing DDD

RESEARCH EXPANSION

What the DDD evidence supports

present DDD as a rich design discipline with reported benefits and adoption challenges while acknowledging limited and heterogeneous empirical evaluation.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to present DDD as a rich design discipline with reported benefits and adoption challenges while acknowledging limited and heterogeneous empirical evaluation. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

present DDD as a rich design discipline with reported benefits and adoption challenges while acknowledging limited and heterogeneous empirical evaluation. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Collect the language used by domain experts in real cases.
- 2 Map commands, events, policies, and disputed terminology.
- 3 Assign data and rule authority to explicit bounded contexts.
- 4 Choose the smallest boundary that protects each invariant.
- 5 Trace the model vocabulary into requirements and code names.

EVIDENCE TO RETAIN

GLOSSARY

CONTEXT MAP

INVARIANT

DOMAIN EVENT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Refund aggregate

- identity: orderId
- invariant: at most one active refund
- value object: Money(minorUnits, currency)
- event: RefundIssued

FAILURE MODE / LIMIT

Treating DDD as a folder convention or a collection of tactical classes creates ceremony without clarifying ownership, language, or consistency boundaries.

WARNING SIGNS

- The same noun means different things across teams.
- Data ownership is confused with business-rule authority.
- Every bounded context is turned into a network service.
- Invariants span several aggregates without an explicit policy.

DECISION RULE

Do not choose a technical boundary until rule authority, language, and consistency needs are explicit.

EVIDENCE

[R17] DDD systematic review 2025

[R37] DORA loosely coupled teams

PART IX

BDD, Gherkin, and executable examples

Use conversation and concrete examples to expose rules before automating only the scenarios that provide durable evidence.

WHAT THIS PART ANSWERS

What examples reveal the rule? Which boundaries and rejections matter? At what layer should the behavior be demonstrated?

USE IT WHEN

Product, quality, and engineering need one shared dialect for observable behavior.

RESEARCH EXPANSION

Discovery, formulation, and automation

separate the conversation that finds behavior, the notation that records it, and the code that executes it.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first separate the conversation that finds behavior, the notation that records it, and the code that executes it, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

separate the conversation that finds behavior, the notation that records it, and the code that executes it. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Discover rules and questions collaboratively before syntax.
- 2 Choose concrete examples for normal, boundary, and error paths.
- 3 Formulate declarative scenarios in ubiquitous language.
- 4 Automate at the lowest stable seam that proves the behavior.
- 5 Execute, review, prune, and update the living specification.

EVIDENCE TO RETAIN

RULE MAP

SCENARIOS

STEP BINDING

RUN RESULT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- Scenarios describe clicks, selectors, or database setup.
- All examples are happy paths with no boundaries.
- Step definitions contain hidden decisions or assertions.
- Feature files pass but stakeholders no longer review them.

DECISION RULE

Automate only after the example is understood by product, engineering, and quality roles in the same domain language.

EVIDENCE

[R10] Cucumber BDD documentation

[R12] BDD systematic review 2023

RESEARCH EXPANSION

Example Mapping

organize a story into rules, concrete examples, and open questions so missing behavior becomes visible before coding.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must organize a story into rules, concrete examples, and open questions so missing behavior becomes visible before coding; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

organize a story into rules, concrete examples, and open questions so missing behavior becomes visible before coding. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Discover rules and questions collaboratively before syntax.
- 2 Choose concrete examples for normal, boundary, and error paths.
- 3 Formulate declarative scenarios in ubiquitous language.
- 4 Automate at the lowest stable seam that proves the behavior.
- 5 Execute, review, prune, and update the living specification.

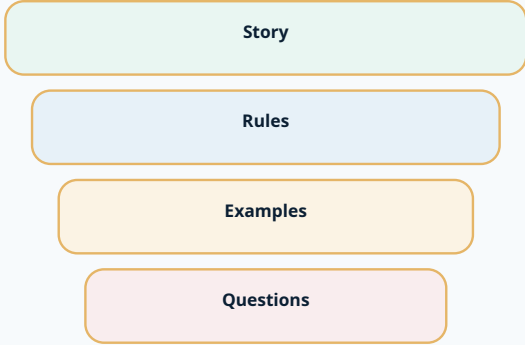
EVIDENCE TO RETAIN

- RULE MAP
- SCENARIOS
- STEP BINDING
- RUN RESULT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Feature: Cancelled-event refund
Scenario: First refund request
Given a paid order with frozen fees
When the event is cancelled
Then one refund is issued
And duplicate requests return 409

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- Scenarios describe clicks, selectors, or database setup.
- All examples are happy paths with no boundaries.
- Step definitions contain hidden decisions or assertions.
- Feature files pass but stakeholders no longer review them.

DECISION RULE

Automate only after the example is understood by product, engineering, and quality roles in the same domain language.

EVIDENCE

- [R10] Cucumber BDD documentation
- [R13] Adzic, Specification by Example

RESEARCH EXPANSION

Gherkin grammar as a representation

use Feature, Rule, Scenario, Given, When, Then, Background, and Scenario Outline deliberately rather than as decorative keywords.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To use Feature, Rule, Scenario, Given, When, Then, Background, and Scenario Outline deliberately rather than as decorative keywords, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

use Feature, Rule, Scenario, Given, When, Then, Background, and Scenario Outline deliberately rather than as decorative keywords. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Discover rules and questions collaboratively before syntax.
- 2 Choose concrete examples for normal, boundary, and error paths.
- 3 Formulate declarative scenarios in ubiquitous language.
- 4 Automate at the lowest stable seam that proves the behavior.
- 5 Execute, review, prune, and update the living specification.

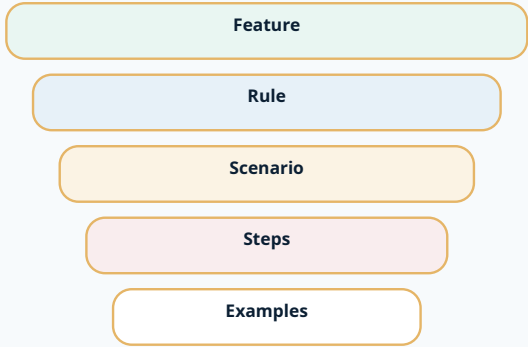
EVIDENCE TO RETAIN

- RULE MAP
- SCENARIOS
- STEP BINDING
- RUN RESULT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Feature: Cancelled-event refund
Scenario: First refund request
Given a paid order with frozen fees
When the event is cancelled
Then one refund is issued
And duplicate requests return 409

FAILURE MODE / LIMIT

Automating examples too early can freeze misunderstanding. UI-level steps, duplicated setup, and implementation language turn living documentation into brittle test scripts.

WARNING SIGNS

- Scenarios describe clicks, selectors, or database setup.
- All examples are happy paths with no boundaries.
- Step definitions contain hidden decisions or assertions.
- Feature files pass but stakeholders no longer review them.

DECISION RULE

Automate only after the example is understood by product, engineering, and quality roles in the same domain language.

EVIDENCE

- [R11] Cucumber Gherkin reference
- [R14] Nicieja, Writing Great Specifications

RESEARCH EXPANSION

Given establishes relevant context

include only state and preconditions needed to understand behavior, not implementation setup or incidental UI details.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to include only state and preconditions needed to understand behavior, not implementation setup or incidental UI details without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

include only state and preconditions needed to understand behavior, not implementation setup or incidental UI details. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Discover rules and questions collaboratively before syntax.
- 2 Choose concrete examples for normal, boundary, and error paths.
- 3 Formulate declarative scenarios in ubiquitous language.
- 4 Automate at the lowest stable seam that proves the behavior.
- 5 Execute, review, prune, and update the living specification.

EVIDENCE TO RETAIN

- RULE MAP
- SCENARIOS
- STEP BINDING
- RUN RESULT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

```
ACTIVE_TASK: T3
ALLOWED_PATHS: src/refunds/**, tests/refunds/**
REQUIRED_GATES: unit, BDD, OpenAPI
STOP_ON: ambiguity, contract conflict, scope breach
```

FAILURE MODE / LIMIT

A prompt is advisory. Without executable gates, repository permissions, and independent verification, an agent can satisfy the apparent task while violating scope or hidden intent.

WARNING SIGNS

- Scenarios describe clicks, selectors, or database setup.
- All examples are happy paths with no boundaries.
- Step definitions contain hidden decisions or assertions.
- Feature files pass but stakeholders no longer review them.

DECISION RULE

Automate only after the example is understood by product, engineering, and quality roles in the same domain language.

EVIDENCE

- [R11] Cucumber Gherkin reference
- [R14] Nicieja, Writing Great Specifications

RESEARCH EXPANSION

When names the business event

describe one meaningful action or event so the scenario has a clear behavioral pivot.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to describe one meaningful action or event so the scenario has a clear behavioral pivot. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

describe one meaningful action or event so the scenario has a clear behavioral pivot. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Discover rules and questions collaboratively before syntax.
- 2 Choose concrete examples for normal, boundary, and error paths.
- 3 Formulate declarative scenarios in ubiquitous language.
- 4 Automate at the lowest stable seam that proves the behavior.
- 5 Execute, review, prune, and update the living specification.

EVIDENCE TO RETAIN

RULE MAP

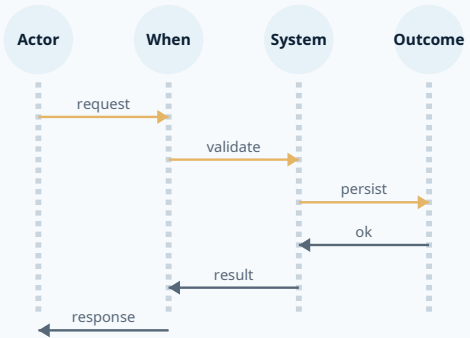
SCENARIOS

STEP BINDING

RUN RESULT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- Scenarios describe clicks, selectors, or database setup.
- All examples are happy paths with no boundaries.
- Step definitions contain hidden decisions or assertions.
- Feature files pass but stakeholders no longer review them.

DECISION RULE

Automate only after the example is understood by product, engineering, and quality roles in the same domain language.

EVIDENCE

[R11] Cucumber Gherkin reference

[R9] Dan North, Introducing BDD

RESEARCH EXPANSION

Then states observable outcomes

assert business-visible results, side effects, messages, and invariants without prescribing private implementation.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first assert business-visible results, side effects, messages, and invariants without prescribing private implementation, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

assert business-visible results, side effects, messages, and invariants without prescribing private implementation. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Discover rules and questions collaboratively before syntax.
- 2 Choose concrete examples for normal, boundary, and error paths.
- 3 Formulate declarative scenarios in ubiquitous language.
- 4 Automate at the lowest stable seam that proves the behavior.
- 5 Execute, review, prune, and update the living specification.

EVIDENCE TO RETAIN

- RULE MAP
- SCENARIOS
- STEP BINDING
- RUN RESULT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- Scenarios describe clicks, selectors, or database setup.
- All examples are happy paths with no boundaries.
- Step definitions contain hidden decisions or assertions.
- Feature files pass but stakeholders no longer review them.

DECISION RULE

Automate only after the example is understood by product, engineering, and quality roles in the same domain language.

EVIDENCE

[R11] Cucumber Gherkin reference

[R13] Adzic, Specification by Example

RESEARCH EXPANSION

Rules and Scenario Outlines

group examples by business rule and use tabular variation only when each row expresses the same behavior shape.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must group examples by business rule and use tabular variation only when each row expresses the same behavior shape; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

group examples by business rule and use tabular variation only when each row expresses the same behavior shape. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Discover rules and questions collaboratively before syntax.
- 2 Choose concrete examples for normal, boundary, and error paths.
- 3 Formulate declarative scenarios in ubiquitous language.
- 4 Automate at the lowest stable seam that proves the behavior.
- 5 Execute, review, prune, and update the living specification.

EVIDENCE TO RETAIN

- RULE MAP
- SCENARIOS
- STEP BINDING
- RUN RESULT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

Rule	Case A	Case B	Boundary
Rule	Case A	Case B	Boundary
Rule	Case A	Case B	Boundary
Rule	Case A	Case B	Boundary

WORKED EXAMPLE

Feature: Cancelled-event refund
Scenario: First refund request
Given a paid order with frozen fees
When the event is cancelled
Then one refund is issued
And duplicate requests return 409

FAILURE MODE / LIMIT

Automating examples too early can freeze misunderstanding. UI-level steps, duplicated setup, and implementation language turn living documentation into brittle test scripts.

WARNING SIGNS

- Scenarios describe clicks, selectors, or database setup.
- All examples are happy paths with no boundaries.
- Step definitions contain hidden decisions or assertions.
- Feature files pass but stakeholders no longer review them.

DECISION RULE

Automate only after the example is understood by product, engineering, and quality roles in the same domain language.

EVIDENCE

- [R11] Cucumber Gherkin reference
- [R14] Nicieja, Writing Great Specifications

RESEARCH EXPANSION

Decision tables for interacting rules

enumerate combinations of conditions and actions when prose or independent scenarios would conceal missing combinations.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To enumerate combinations of conditions and actions when prose or independent scenarios would conceal missing combinations, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

enumerate combinations of conditions and actions when prose or independent scenarios would conceal missing combinations. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Discover rules and questions collaboratively before syntax.
- 2 Choose concrete examples for normal, boundary, and error paths.
- 3 Formulate declarative scenarios in ubiquitous language.
- 4 Automate at the lowest stable seam that proves the behavior.
- 5 Execute, review, prune, and update the living specification.

EVIDENCE TO RETAIN

RULE MAP

SCENARIOS

STEP BINDING

RUN RESULT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

Paid	Cancelled	Already refund	Eligible
Paid	Cancelled	Already refund	Eligible
Paid	Cancelled	Already refund	Eligible
Paid	Cancelled	Already refund	Eligible

WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- Scenarios describe clicks, selectors, or database setup.
- All examples are happy paths with no boundaries.
- Step definitions contain hidden decisions or assertions.
- Feature files pass but stakeholders no longer review them.

DECISION RULE

Automate only after the example is understood by product, engineering, and quality roles in the same domain language.

EVIDENCE

[R13] Adzic, Specification by Example

[R14] Nicieja, Writing Great Specifications

RESEARCH EXPANSION

Scenario quality heuristics

keep scenarios declarative, focused, deterministic, independent, domain-literate, and valuable to a non-programmer reviewer.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to keep scenarios declarative, focused, deterministic, independent, domain-literate, and valuable to a non-programmer reviewer without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

keep scenarios declarative, focused, deterministic, independent, domain-literate, and valuable to a non-programmer reviewer. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Discover rules and questions collaboratively before syntax.
- 2 Choose concrete examples for normal, boundary, and error paths.
- 3 Formulate declarative scenarios in ubiquitous language.
- 4 Automate at the lowest stable seam that proves the behavior.
- 5 Execute, review, prune, and update the living specification.

EVIDENCE TO RETAIN

RULE MAP

SCENARIOS

STEP BINDING

RUN RESULT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

Clarity

4.2 d

Focus

8/wk

Stability

6%

Value

40 min

WORKED EXAMPLE

Feature: Cancelled-event refund
Scenario: First refund request
Given a paid order with frozen fees
When the event is cancelled
Then one refund is issued
And duplicate requests return 409

FAILURE MODE / LIMIT

Automating examples too early can freeze misunderstanding. UI-level steps, duplicated setup, and implementation language turn living documentation into brittle test scripts.

WARNING SIGNS

- Scenarios describe clicks, selectors, or database setup.
- All examples are happy paths with no boundaries.
- Step definitions contain hidden decisions or assertions.
- Feature files pass but stakeholders no longer review them.

DECISION RULE

Automate only after the example is understood by product, engineering, and quality roles in the same domain language.

EVIDENCE

[R10] Cucumber BDD documentation

[R11] Cucumber Gherkin reference

[R14] Nicieja, Writing Great Specifications

RESEARCH EXPANSION

The UI-step anti-pattern

replace click-and-type scripts with domain behavior at the appropriate boundary and reserve end-to-end tests for a small set of critical journeys.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to replace click-and-type scripts with domain behavior at the appropriate boundary and reserve end-to-end tests for a small set of critical journeys. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

replace click-and-type scripts with domain behavior at the appropriate boundary and reserve end-to-end tests for a small set of critical journeys. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Discover rules and questions collaboratively before syntax.
- 2 Choose concrete examples for normal, boundary, and error paths.
- 3 Formulate declarative scenarios in ubiquitous language.
- 4 Automate at the lowest stable seam that proves the behavior.
- 5 Execute, review, prune, and update the living specification.

EVIDENCE TO RETAIN

RULE MAP

SCENARIOS

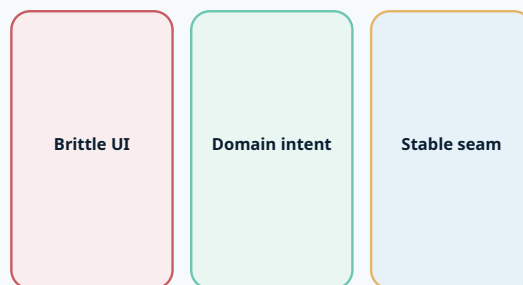
STEP BINDING

RUN RESULT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- Scenarios describe clicks, selectors, or database setup.
- All examples are happy paths with no boundaries.
- Step definitions contain hidden decisions or assertions.
- Feature files pass but stakeholders no longer review them.

DECISION RULE

Automate only after the example is understood by product, engineering, and quality roles in the same domain language.

EVIDENCE

[R10] Cucumber BDD documentation

[R11] Cucumber Gherkin reference

RESEARCH EXPANSION

Living documentation requires maintenance

keep scenario suites trustworthy through ownership, review, execution, pruning, and links to decisions rather than by storing feature files indefinitely.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first keep scenario suites trustworthy through ownership, review, execution, pruning, and links to decisions rather than by storing feature files indefinitely, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

keep scenario suites trustworthy through ownership, review, execution, pruning, and links to decisions rather than by storing feature files indefinitely. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Discover rules and questions collaboratively before syntax.
- 2 Choose concrete examples for normal, boundary, and error paths.
- 3 Formulate declarative scenarios in ubiquitous language.
- 4 Automate at the lowest stable seam that proves the behavior.
- 5 Execute, review, prune, and update the living specification.

EVIDENCE TO RETAIN

RULE MAP

SCENARIOS

STEP BINDING

RUN RESULT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

```
ACTIVE_TASK: T3
ALLOWED_PATHS: src/refunds/**, tests/refunds/**
REQUIRED_GATES: unit, BDD, OpenAPI
STOP_ON: ambiguity, contract conflict, scope breach
```

FAILURE MODE / LIMIT

A prompt is advisory. Without executable gates, repository permissions, and independent verification, an agent can satisfy the apparent task while violating scope or hidden intent.

WARNING SIGNS

- Scenarios describe clicks, selectors, or database setup.
- All examples are happy paths with no boundaries.
- Step definitions contain hidden decisions or assertions.
- Feature files pass but stakeholders no longer review them.

DECISION RULE

Automate only after the example is understood by product, engineering, and quality roles in the same domain language.

EVIDENCE

[R13] Adzic, Specification by Example

[R38] DORA documentation quality

RESEARCH EXPANSION

BDD automation architecture

bind scenarios to stable domain or application seams and keep step definitions thin, composable, and free of hidden assertions.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must bind scenarios to stable domain or application seams and keep step definitions thin, composable, and free of hidden assertions; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

bind scenarios to stable domain or application seams and keep step definitions thin, composable, and free of hidden assertions. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Discover rules and questions collaboratively before syntax.
- 2 Choose concrete examples for normal, boundary, and error paths.
- 3 Formulate declarative scenarios in ubiquitous language.
- 4 Automate at the lowest stable seam that proves the behavior.
- 5 Execute, review, prune, and update the living specification.

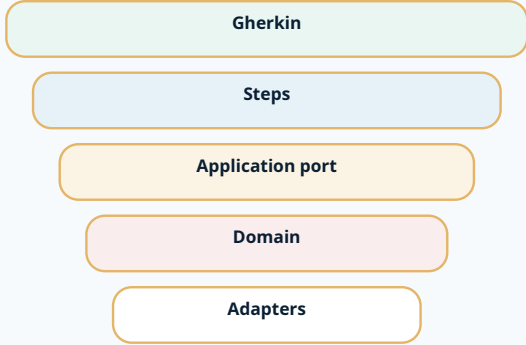
EVIDENCE TO RETAIN

- RULE MAP
- SCENARIOS
- STEP BINDING
- RUN RESULT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Feature: Cancelled-event refund
Scenario: First refund request
Given a paid order with frozen fees
When the event is cancelled
Then one refund is issued
And duplicate requests return 409

FAILURE MODE / LIMIT

Automating examples too early can freeze misunderstanding. UI-level steps, duplicated setup, and implementation language turn living documentation into brittle test scripts.

WARNING SIGNS

- Scenarios describe clicks, selectors, or database setup.
- All examples are happy paths with no boundaries.
- Step definitions contain hidden decisions or assertions.
- Feature files pass but stakeholders no longer review them.

DECISION RULE

Automate only after the example is understood by product, engineering, and quality roles in the same domain language.

EVIDENCE

- [R10] Cucumber BDD documentation
- [R23] Cockburn, Hexagonal Architecture

EXPANDED FROM SUPPLIED WALKTHROUGH

PLT-4821 feature scenarios

express first refund, duplicate request, expired token, frozen fees, and delayed PSP behavior as separate business examples.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To express first refund, duplicate request, expired token, frozen fees, and delayed PSP behavior as separate business examples, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

express first refund, duplicate request, expired token, frozen fees, and delayed PSP behavior as separate business examples. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Discover rules and questions collaboratively before syntax.
- 2 Choose concrete examples for normal, boundary, and error paths.
- 3 Formulate declarative scenarios in ubiquitous language.
- 4 Automate at the lowest stable seam that proves the behavior.
- 5 Execute, review, prune, and update the living specification.

EVIDENCE TO RETAIN

RULE MAP

SCENARIOS

STEP BINDING

RUN RESULT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Feature: Cancelled-event refund
Scenario: First refund request
Given a paid order with frozen fees
When the event is cancelled
Then one refund is issued
And duplicate requests return 409

FAILURE MODE / LIMIT

Automating examples too early can freeze misunderstanding. UI-level steps, duplicated setup, and implementation language turn living documentation into brittle test scripts.

WARNING SIGNS

- Scenarios describe clicks, selectors, or database setup.
- All examples are happy paths with no boundaries.
- Step definitions contain hidden decisions or assertions.
- Feature files pass but stakeholders no longer review them.

DECISION RULE

Automate only after the example is understood by product, engineering, and quality roles in the same domain language.

EVIDENCE

[R11] Cucumber Gherkin reference

[R13] Adzic, Specification by Example

RESEARCH EXPANSION

From examples to acceptance evidence

record the mapping between rule, scenario, test runner, environment, result, and retained evidence so green has an auditable meaning.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to record the mapping between rule, scenario, test runner, environment, result, and retained evidence so green has an auditable meaning without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

record the mapping between rule, scenario, test runner, environment, result, and retained evidence so green has an auditable meaning. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Discover rules and questions collaboratively before syntax.
- 2 Choose concrete examples for normal, boundary, and error paths.
- 3 Formulate declarative scenarios in ubiquitous language.
- 4 Automate at the lowest stable seam that proves the behavior.
- 5 Execute, review, prune, and update the living specification.

EVIDENCE TO RETAIN

- RULE MAP
- SCENARIOS
- STEP BINDING
- RUN RESULT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- Scenarios describe clicks, selectors, or database setup.
- All examples are happy paths with no boundaries.
- Step definitions contain hidden decisions or assertions.
- Feature files pass but stakeholders no longer review them.

DECISION RULE

Automate only after the example is understood by product, engineering, and quality roles in the same domain language.

EVIDENCE

[R26] Gotel and Finkelstein 1994

[R27] Requirements traceability book

[R13] Adzic, Specification by Example

RESEARCH EXPANSION

What BDD research can and cannot claim

report collaboration and specification benefits alongside adoption cost, tool coupling, maintenance, and the uneven quality of empirical evidence.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to report collaboration and specification benefits alongside adoption cost, tool coupling, maintenance, and the uneven quality of empirical evidence. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

report collaboration and specification benefits alongside adoption cost, tool coupling, maintenance, and the uneven quality of empirical evidence. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Discover rules and questions collaboratively before syntax.
- 2 Choose concrete examples for normal, boundary, and error paths.
- 3 Formulate declarative scenarios in ubiquitous language.
- 4 Automate at the lowest stable seam that proves the behavior.
- 5 Execute, review, prune, and update the living specification.

EVIDENCE TO RETAIN

- RULE MAP
- SCENARIOS
- STEP BINDING
- RUN RESULT

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Feature: Cancelled-event refund
Scenario: First refund request
Given a paid order with frozen fees
When the event is cancelled
Then one refund is issued
And duplicate requests return 409

FAILURE MODE / LIMIT

Automating examples too early can freeze misunderstanding. UI-level steps, duplicated setup, and implementation language turn living documentation into brittle test scripts.

WARNING SIGNS

- Scenarios describe clicks, selectors, or database setup.
- All examples are happy paths with no boundaries.
- Step definitions contain hidden decisions or assertions.
- Feature files pass but stakeholders no longer review them.

DECISION RULE

Automate only after the example is understood by product, engineering, and quality roles in the same domain language.

EVIDENCE

- [R12] BDD systematic review 2023
- [R10] Cucumber BDD documentation

PART X

TDD inside the larger evidence chain

Apply red–green–refactor to implementation increments while keeping tests connected to upstream behavior and independent enough to reveal defects.

WHAT THIS PART ANSWERS

Did the test fail for the intended reason? What design pressure does the next example create? Is the oracle derived from the contract or copied from the code?

USE IT WHEN

You need fast feedback, testable design, and proof that selected behavior is absent before implementation.

RESEARCH EXPANSION

The Red-Green-Refactor loop

write the smallest failing test, implement only enough behavior, improve the design while green, and repeat.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first write the smallest failing test, implement only enough behavior, improve the design while green, and repeat, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

write the smallest failing test, implement only enough behavior, improve the design while green, and repeat. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Select one contract-derived expectation and make it fail.
- 2 Confirm the failure is meaningful rather than environmental.
- 3 Implement only the behavior needed for the current example.
- 4 Refactor names and seams while the oracle remains green.
- 5 Add boundaries, properties, contracts, and mutation checks.

EVIDENCE TO RETAIN

RED RESULT

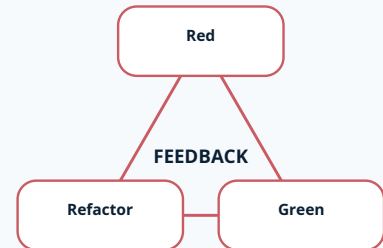
GREEN RESULT

REFACTOR

MUTATION SCORE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The first observed state is green rather than red.
- Tests mirror implementation structure instead of behavior.
- Mocks specify every call while outcomes remain weak.
- Coverage rises while realistic mutants continue to survive.

DECISION RULE

A test is credible only after its failure has been observed for the intended reason and its expectation comes from an upstream contract.

EVIDENCE

[R18] Beck, TDD by Example

[R22] Madeyski, empirical TDD

RESEARCH EXPANSION

Why the Red state is evidence

demonstrate that the test can detect absence or violation of behavior before production code can satisfy it.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must demonstrate that the test can detect absence or violation of behavior before production code can satisfy it; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

demonstrate that the test can detect absence or violation of behavior before production code can satisfy it. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Select one contract-derived expectation and make it fail.
- 2 Confirm the failure is meaningful rather than environmental.
- 3 Implement only the behavior needed for the current example.
- 4 Refactor names and seams while the oracle remains green.
- 5 Add boundaries, properties, contracts, and mutation checks.

EVIDENCE TO RETAIN

RED RESULT

GREEN RESULT

REFACTOR

MUTATION SCORE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 ->
TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The first observed state is green rather than red.
- Tests mirror implementation structure instead of behavior.
- Mocks specify every call while outcomes remain weak.
- Coverage rises while realistic mutants continue to survive.

DECISION RULE

A test is credible only after its failure has been observed for the intended reason and its expectation comes from an upstream contract.

EVIDENCE

[R18] Beck, TDD by Example

[R28] Jia and Harman mutation survey

RESEARCH EXPANSION

Green means minimum, not final architecture

avoid speculative generality by earning abstractions through additional examples and constraints.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To avoid speculative generality by earning abstractions through additional examples and constraints, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

avoid speculative generality by earning abstractions through additional examples and constraints. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Select one contract-derived expectation and make it fail.
- 2 Confirm the failure is meaningful rather than environmental.
- 3 Implement only the behavior needed for the current example.
- 4 Refactor names and seams while the oracle remains green.
- 5 Add boundaries, properties, contracts, and mutation checks.

EVIDENCE TO RETAIN

RED RESULT

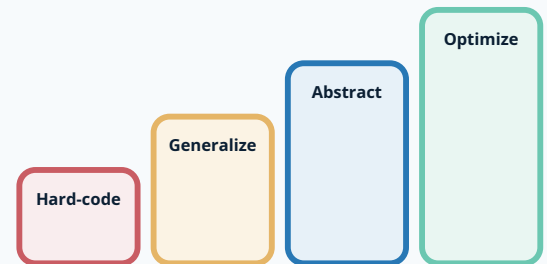
GREEN RESULT

REFACTOR

MUTATION SCORE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The first observed state is green rather than red.
- Tests mirror implementation structure instead of behavior.
- Mocks specify every call while outcomes remain weak.
- Coverage rises while realistic mutants continue to survive.

DECISION RULE

A test is credible only after its failure has been observed for the intended reason and its expectation comes from an upstream contract.

EVIDENCE

[R18] Beck, TDD by Example

[R21] Romano et al. 2017 TDD

RESEARCH EXPANSION

Refactor while the oracle stays green

improve naming, structure, duplication, coupling, and domain alignment without changing externally specified behavior.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to improve naming, structure, duplication, coupling, and domain alignment without changing externally specified behavior without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

improve naming, structure, duplication, coupling, and domain alignment without changing externally specified behavior. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Select one contract-derived expectation and make it fail.
- 2 Confirm the failure is meaningful rather than environmental.
- 3 Implement only the behavior needed for the current example.
- 4 Refactor names and seams while the oracle remains green.
- 5 Add boundaries, properties, contracts, and mutation checks.

EVIDENCE TO RETAIN

RED RESULT

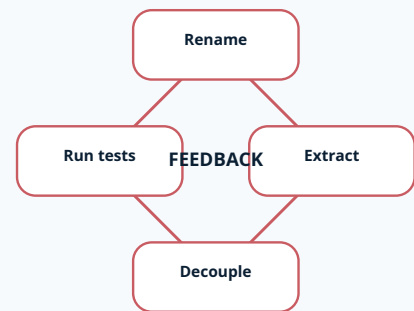
GREEN RESULT

REFACTOR

MUTATION SCORE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The first observed state is green rather than red.
- Tests mirror implementation structure instead of behavior.
- Mocks specify every call while outcomes remain weak.
- Coverage rises while realistic mutants continue to survive.

DECISION RULE

A test is credible only after its failure has been observed for the intended reason and its expectation comes from an upstream contract.

EVIDENCE

[R18] Beck, TDD by Example

[R23] Cockburn, Hexagonal Architecture

RESEARCH EXPANSION

Triangulation and example pressure

add examples that force a general rule only when a single passing case leaves room for a false implementation.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to add examples that force a general rule only when a single passing case leaves room for a false implementation. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

add examples that force a general rule only when a single passing case leaves room for a false implementation. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Select one contract-derived expectation and make it fail.
- 2 Confirm the failure is meaningful rather than environmental.
- 3 Implement only the behavior needed for the current example.
- 4 Refactor names and seams while the oracle remains green.
- 5 Add boundaries, properties, contracts, and mutation checks.

EVIDENCE TO RETAIN

RED RESULT

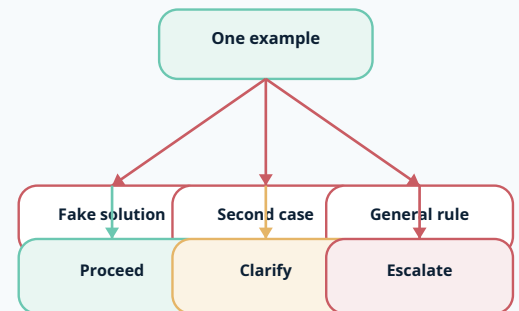
GREEN RESULT

REFACTOR

MUTATION SCORE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The first observed state is green rather than red.
- Tests mirror implementation structure instead of behavior.
- Mocks specify every call while outcomes remain weak.
- Coverage rises while realistic mutants continue to survive.

DECISION RULE

A test is credible only after its failure has been observed for the intended reason and its expectation comes from an upstream contract.

EVIDENCE

[R18] Beck, TDD by Example

[R13] Adzic, Specification by Example

RESEARCH EXPANSION

Test doubles as diagnostic seams

use fakes, stubs, spies, and mocks intentionally to isolate responsibility and diagnose whether failure belongs to logic or integration.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first use fakes, stubs, spies, and mocks intentionally to isolate responsibility and diagnose whether failure belongs to logic or integration, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

use fakes, stubs, spies, and mocks intentionally to isolate responsibility and diagnose whether failure belongs to logic or integration. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1
- Select one contract-derived expectation and make it fail.
- 2
- Confirm the failure is meaningful rather than environmental.
- 3
- Implement only the behavior needed for the current example.
- 4
- Refactor names and seams while the oracle remains green.
- 5
- Add boundaries, properties, contracts, and mutation checks.

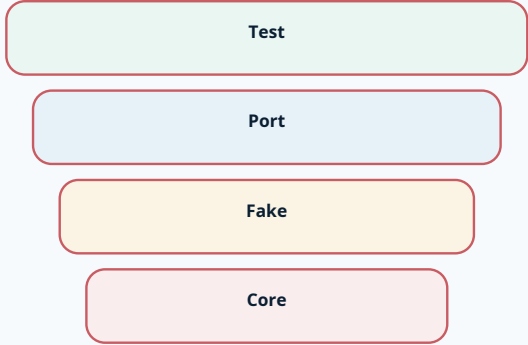
EVIDENCE TO RETAIN

- RED RESULT
- GREEN RESULT
- REFACTOR
- MUTATION SCORE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

```
it('refunds price minus service fees', async () => {
  const order = paidOrder({ total: 12900, fees: 900 });
  expect(await refund(order.id)).toEqual(12000);
});
```

FAILURE MODE / LIMIT

Green tests can coexist with wrong intent when tests are derived from the implementation or share the same mistaken assumption as the code. Independent derivation and mutation checks matter.

WARNING SIGNS

- The first observed state is green rather than red.
- Tests mirror implementation structure instead of behavior.
- Mocks specify every call while outcomes remain weak.
- Coverage rises while realistic mutants continue to survive.

DECISION RULE

A test is credible only after its failure has been observed for the intended reason and its expectation comes from an upstream contract.

EVIDENCE

- [R22] Madeyski, empirical TDD
- [R23] Cockburn, Hexagonal Architecture

RESEARCH EXPANSION

Unit, component, contract, and end-to-end tests

assign each risk to the cheapest layer that can detect it and avoid duplicating the same assertion across every layer.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must assign each risk to the cheapest layer that can detect it and avoid duplicating the same assertion across every layer; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

assign each risk to the cheapest layer that can detect it and avoid duplicating the same assertion across every layer. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Select one contract-derived expectation and make it fail.
- 2 Confirm the failure is meaningful rather than environmental.
- 3 Implement only the behavior needed for the current example.
- 4 Refactor names and seams while the oracle remains green.
- 5 Add boundaries, properties, contracts, and mutation checks.

EVIDENCE TO RETAIN

RED RESULT

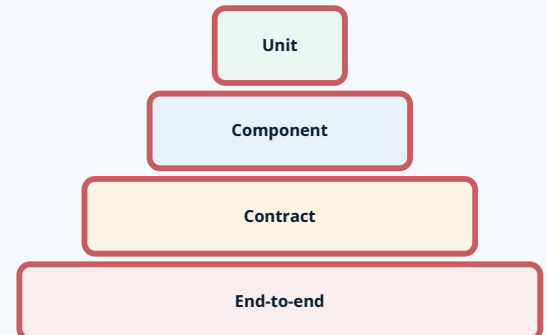
GREEN RESULT

REFACTOR

MUTATION SCORE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

```
it('refunds price minus service fees', async () => {
  const order = paidOrder({ total: 12900, fees: 900 });
  expect(await refund(order.id)).toEqual(12000);
});
```

FAILURE MODE / LIMIT

Green tests can coexist with wrong intent when tests are derived from the implementation or share the same mistaken assumption as the code. Independent derivation and mutation checks matter.

WARNING SIGNS

- The first observed state is green rather than red.
- Tests mirror implementation structure instead of behavior.
- Mocks specify every call while outcomes remain weak.
- Coverage rises while realistic mutants continue to survive.

DECISION RULE

A test is credible only after its failure has been observed for the intended reason and its expectation comes from an upstream contract.

EVIDENCE

[R24] OpenAPI Specification

[R25] Pact consumer-driven contracts

[R46] Google SRE Workbook

RESEARCH EXPANSION

Property-based testing for invariants

generate broad input spaces around algebraic, financial, state, and serialization invariants that examples alone may miss.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To generate broad input spaces around algebraic, financial, state, and serialization invariants that examples alone may miss, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

generate broad input spaces around algebraic, financial, state, and serialization invariants that examples alone may miss. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Select one contract-derived expectation and make it fail.
- 2 Confirm the failure is meaningful rather than environmental.
- 3 Implement only the behavior needed for the current example.
- 4 Refactor names and seams while the oracle remains green.
- 5 Add boundaries, properties, contracts, and mutation checks.

EVIDENCE TO RETAIN

RED RESULT

GREEN RESULT

REFACTOR

MUTATION SCORE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

```
it('refunds price minus service fees', async () => {
  const order = paidOrder({ total: 12900, fees: 900 });
  expect(await refund(order.id)).toEqual(12000);
});
```

FAILURE MODE / LIMIT

Green tests can coexist with wrong intent when tests are derived from the implementation or share the same mistaken assumption as the code. Independent derivation and mutation checks matter.

WARNING SIGNS

- The first observed state is green rather than red.
- Tests mirror implementation structure instead of behavior.
- Mocks specify every call while outcomes remain weak.
- Coverage rises while realistic mutants continue to survive.

DECISION RULE

A test is credible only after its failure has been observed for the intended reason and its expectation comes from an upstream contract.

EVIDENCE

[R28] Jia and Harman mutation survey

[R51] Jackson, Software Abstractions

RESEARCH EXPANSION

Mutation testing for oracle strength

introduce controlled faults and measure whether the suite detects them rather than trusting coverage percentages alone.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to introduce controlled faults and measure whether the suite detects them rather than trusting coverage percentages alone without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

introduce controlled faults and measure whether the suite detects them rather than trusting coverage percentages alone. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Select one contract-derived expectation and make it fail.
- 2 Confirm the failure is meaningful rather than environmental.
- 3 Implement only the behavior needed for the current example.
- 4 Refactor names and seams while the oracle remains green.
- 5 Add boundaries, properties, contracts, and mutation checks.

EVIDENCE TO RETAIN

RED RESULT

GREEN RESULT

REFACTOR

MUTATION SCORE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

```
it('refunds price minus service fees', async () => {
  const order = paidOrder({ total: 12900, fees: 900 });
  expect(await refund(order.id)).toEqual(12000);
});
```

FAILURE MODE / LIMIT

Green tests can coexist with wrong intent when tests are derived from the implementation or share the same mistaken assumption as the code. Independent derivation and mutation checks matter.

WARNING SIGNS

- The first observed state is green rather than red.
- Tests mirror implementation structure instead of behavior.
- Mocks specify every call while outcomes remain weak.
- Coverage rises while realistic mutants continue to survive.

DECISION RULE

A test is credible only after its failure has been observed for the intended reason and its expectation comes from an upstream contract.

EVIDENCE

[R28] Jia and Harman mutation survey

[R29] Papadakis et al. mutation advances

RESEARCH EXPANSION

Designing edge cases systematically

derive boundaries from equivalence classes, state transitions, decision tables, temporal windows, and failure modes.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to derive boundaries from equivalence classes, state transitions, decision tables, temporal windows, and failure modes. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

derive boundaries from equivalence classes, state transitions, decision tables, temporal windows, and failure modes. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

1

Select one contract-derived expectation and make it fail.

2

Confirm the failure is meaningful rather than environmental.

3

Implement only the behavior needed for the current example.

4

Refactor names and seams while the oracle remains green.

5

Add boundaries, properties, contracts, and mutation checks.

EVIDENCE TO RETAIN

- RED RESULT
- GREEN RESULT
- REFACTOR
- MUTATION SCORE

REVIEW QUESTIONS

•

What uncertainty is removed before implementation?

•

Which artifact is authoritative, and who approves it?

•

What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

Boundary	State	Combination	Failure
Boundary	State	Combination	Failure
Boundary	State	Combination	Failure
Boundary	State	Combination	Failure

WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The first observed state is green rather than red.
- Tests mirror implementation structure instead of behavior.
- Mocks specify every call while outcomes remain weak.
- Coverage rises while realistic mutants continue to survive.

DECISION RULE

A test is credible only after its failure has been observed for the intended reason and its expectation comes from an upstream contract.

EVIDENCE

[R5] ISO/IEC/IEEE 29148:2018

[R13] Adzic, Specification by Example

RESEARCH EXPANSION

Concurrency and idempotency tests

exercise duplicate, reordered, simultaneous, timed-out, and retried commands against the same business identity.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first exercise duplicate, reordered, simultaneous, timed-out, and retried commands against the same business identity, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

exercise duplicate, reordered, simultaneous, timed-out, and retried commands against the same business identity. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Select one contract-derived expectation and make it fail.
- 2 Confirm the failure is meaningful rather than environmental.
- 3 Implement only the behavior needed for the current example.
- 4 Refactor names and seams while the oracle remains green.
- 5 Add boundaries, properties, contracts, and mutation checks.

EVIDENCE TO RETAIN

RED RESULT

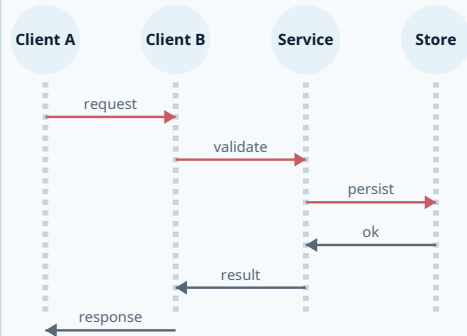
GREEN RESULT

REFACTOR

MUTATION SCORE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

```
it('refunds price minus service fees', async () => {
  const order = paidOrder({ total: 12900, fees: 900 });
  expect(await refund(order.id)).toEqual(12000);
});
```

FAILURE MODE / LIMIT

Green tests can coexist with wrong intent when tests are derived from the implementation or share the same mistaken assumption as the code. Independent derivation and mutation checks matter.

WARNING SIGNS

- The first observed state is green rather than red.
- Tests mirror implementation structure instead of behavior.
- Mocks specify every call while outcomes remain weak.
- Coverage rises while realistic mutants continue to survive.

DECISION RULE

A test is credible only after its failure has been observed for the intended reason and its expectation comes from an upstream contract.

EVIDENCE[\[R24\] OpenAPI Specification](#)[\[R46\] Google SRE Workbook](#)

RESEARCH EXPANSION

Time and clock control

inject a clock and specify calendar policy so tests can cover deadlines, retries, expiry, and time-zone transitions deterministically.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must inject a clock and specify calendar policy so tests can cover deadlines, retries, expiry, and time-zone transitions deterministically; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

inject a clock and specify calendar policy so tests can cover deadlines, retries, expiry, and time-zone transitions deterministically. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Select one contract-derived expectation and make it fail.
- 2 Confirm the failure is meaningful rather than environmental.
- 3 Implement only the behavior needed for the current example.
- 4 Refactor names and seams while the oracle remains green.
- 5 Add boundaries, properties, contracts, and mutation checks.

EVIDENCE TO RETAIN

RED RESULT

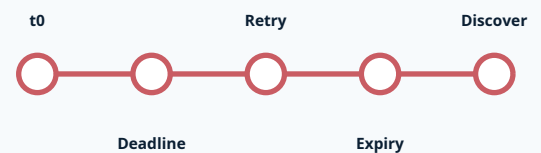
GREEN RESULT

REFACTOR

MUTATION SCORE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The first observed state is green rather than red.
- Tests mirror implementation structure instead of behavior.
- Mocks specify every call while outcomes remain weak.
- Coverage rises while realistic mutants continue to survive.

DECISION RULE

A test is credible only after its failure has been observed for the intended reason and its expectation comes from an upstream contract.

EVIDENCE

[R5] ISO/IEC/IEEE 29148:2018

[R46] Google SRE Workbook

RESEARCH EXPANSION

Money and rounding tests

cover minor units, currency mismatch, half-even ties, negative adjustments, percentage fees, and reconciliation totals.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To cover minor units, currency mismatch, half-even ties, negative adjustments, percentage fees, and reconciliation totals, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

cover minor units, currency mismatch, half-even ties, negative adjustments, percentage fees, and reconciliation totals. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

1

Select one contract-derived expectation and make it fail.

2

Confirm the failure is meaningful rather than environmental.

3

Implement only the behavior needed for the current example.

4

Refactor names and seams while the oracle remains green.

5

Add boundaries, properties, contracts, and mutation checks.

EVIDENCE TO RETAIN

- RED RESULT
- GREEN RESULT
- REFACTOR
- MUTATION SCORE

REVIEW QUESTIONS

•

What uncertainty is removed before implementation?

•

Which artifact is authoritative, and who approves it?

•

What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

2.5	3.5	Fee	Currency
2.5	3.5	Fee	Currency
2.5	3.5	Fee	Currency
2.5	3.5	Fee	Currency

WORKED EXAMPLE

```
it('refunds price minus service fees', async () => {
  const order = paidOrder({ total: 12900, fees: 900 });
  expect(await refund(order.id)).toEqual(12000);
});
```

FAILURE MODE / LIMIT

Green tests can coexist with wrong intent when tests are derived from the implementation or share the same mistaken assumption as the code. Independent derivation and mutation checks matter.

WARNING SIGNS

- The first observed state is green rather than red.
- Tests mirror implementation structure instead of behavior.
- Mocks specify every call while outcomes remain weak.
- Coverage rises while realistic mutants continue to survive.

DECISION RULE

A test is credible only after its failure has been observed for the intended reason and its expectation comes from an upstream contract.

EVIDENCE

[R5] ISO/IEC/IEEE 29148:2018

[R51] Jackson, Software Abstractions

RESEARCH EXPANSION

TDD with coding agents

constrain the agent to a contract-derived failing test, a task scope, and independent gates rather than letting it generate both oracle and solution from one interpretation.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to constrain the agent to a contract-derived failing test, a task scope, and independent gates rather than letting it generate both oracle and solution from one interpretation without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

constrain the agent to a contract-derived failing test, a task scope, and independent gates rather than letting it generate both oracle and solution from one interpretation. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Select one contract-derived expectation and make it fail.
- 2 Confirm the failure is meaningful rather than environmental.
- 3 Implement only the behavior needed for the current example.
- 4 Refactor names and seams while the oracle remains green.
- 5 Add boundaries, properties, contracts, and mutation checks.

EVIDENCE TO RETAIN

RED RESULT

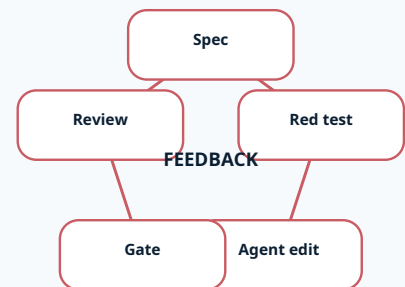
GREEN RESULT

REFACTOR

MUTATION SCORE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

```
it('refunds price minus service fees', async () => {
  const order = paidOrder({ total: 12900, fees: 900 });
  expect(await refund(order.id)).toEqual(12000);
});
```

FAILURE MODE / LIMIT

Green tests can coexist with wrong intent when tests are derived from the implementation or share the same mistaken assumption as the code. Independent derivation and mutation checks matter.

WARNING SIGNS

- The first observed state is green rather than red.
- Tests mirror implementation structure instead of behavior.
- Mocks specify every call while outcomes remain weak.
- Coverage rises while realistic mutants continue to survive.

DECISION RULE

A test is credible only after its failure has been observed for the intended reason and its expectation comes from an upstream contract.

EVIDENCE

[R1] GitHub Spec Kit documentation

[R30] Formal RE and LLM roadmap

[R32] LLMs for software engineering SLR

EXPANDED FROM SUPPLIED WALKTHROUGH

PLT-4821 test portfolio

combine pure calculator tests, service state tests, OpenAPI checks, BDD scenarios, concurrency tests, and reconciliation evidence.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to combine pure calculator tests, service state tests, OpenAPI checks, BDD scenarios, concurrency tests, and reconciliation evidence. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

combine pure calculator tests, service state tests, OpenAPI checks, BDD scenarios, concurrency tests, and reconciliation evidence. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Select one contract-derived expectation and make it fail.
- 2 Confirm the failure is meaningful rather than environmental.
- 3 Implement only the behavior needed for the current example.
- 4 Refactor names and seams while the oracle remains green.
- 5 Add boundaries, properties, contracts, and mutation checks.

EVIDENCE TO RETAIN

RED RESULT

GREEN RESULT

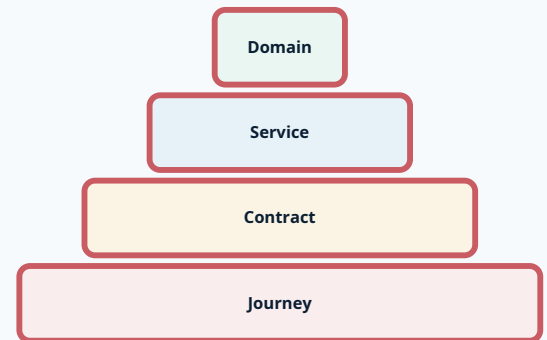
REFACTOR

MUTATION SCORE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

```
it('refunds price minus service fees', async () => {
  const order = paidOrder({ total: 12900, fees: 900 });
  expect(await refund(order.id)).toEqual(12000);
});
```

FAILURE MODE / LIMIT

Green tests can coexist with wrong intent when tests are derived from the implementation or share the same mistaken assumption as the code. Independent derivation and mutation checks matter.

WARNING SIGNS

- The first observed state is green rather than red.
- Tests mirror implementation structure instead of behavior.
- Mocks specify every call while outcomes remain weak.
- Coverage rises while realistic mutants continue to survive.

DECISION RULE

A test is credible only after its failure has been observed for the intended reason and its expectation comes from an upstream contract.

EVIDENCE

[R18] Beck, TDD by Example

[R24] OpenAPI Specification

[R25] Pact consumer-driven contracts

RESEARCH EXPANSION

What the empirical TDD evidence says

synthesize case studies, experiments, and meta-analysis without treating large defect reductions as universal or ignoring implementation cost and context.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first synthesize case studies, experiments, and meta-analysis without treating large defect reductions as universal or ignoring implementation cost and context, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

synthesize case studies, experiments, and meta-analysis without treating large defect reductions as universal or ignoring implementation cost and context. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Select one contract-derived expectation and make it fail.
- 2 Confirm the failure is meaningful rather than environmental.
- 3 Implement only the behavior needed for the current example.
- 4 Refactor names and seams while the oracle remains green.
- 5 Add boundaries, properties, contracts, and mutation checks.

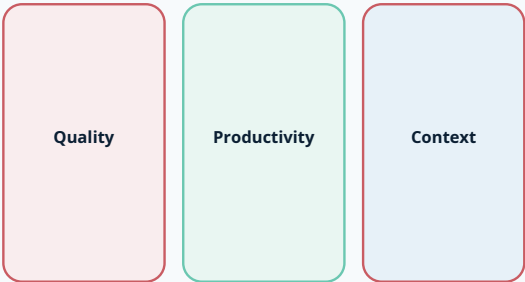
EVIDENCE TO RETAIN

- RED RESULT
- GREEN RESULT
- REFACTOR
- MUTATION SCORE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

```
it('refunds price minus service fees', async () => {
  const order = paidOrder({ total: 12900, fees: 900 });
  expect(await refund(order.id)).toEqual(12000);
});
```

FAILURE MODE / LIMIT

Green tests can coexist with wrong intent when tests are derived from the implementation or share the same mistaken assumption as the code. Independent derivation and mutation checks matter.

WARNING SIGNS

- The first observed state is green rather than red.
- Tests mirror implementation structure instead of behavior.
- Mocks specify every call while outcomes remain weak.
- Coverage rises while realistic mutants continue to survive.

DECISION RULE

A test is credible only after its failure has been observed for the intended reason and its expectation comes from an upstream contract.

EVIDENCE

- [R19] Nagappan et al. 2008
- [R20] Rafique and Misis meta-analysis
- [R21] Romano et al. 2017 TDD
- [R22] Madeyski, empirical TDD

PART XI

Architecture, contracts, and traceability

Make boundaries, dependencies, APIs, events, decisions, and requirement links reduce the search space for change and failure.

WHAT THIS PART ANSWERS

Where can the defect live? Which contract protects the seam? What downstream artifacts and evidence must change when a rule changes?

USE IT WHEN

A feature spans components, consumers, or legacy code and the cost of unnoticed drift is high.

RESEARCH EXPANSION

Hexagonal architecture

place domain and application behavior behind explicit inbound and outbound ports so tests and adapters remain replaceable.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first place domain and application behavior behind explicit inbound and outbound ports so tests and adapters remain replaceable, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

place domain and application behavior behind explicit inbound and outbound ports so tests and adapters remain replaceable. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Name the behavioral boundary and its authoritative contract.
- 2 Place policy behind a narrow port with explicit dependencies.
- 3 Version schemas, events, migrations, and failure semantics.
- 4 Create typed links from requirement to code and evidence.
- 5 Run gates and retain results for release and diagnosis.

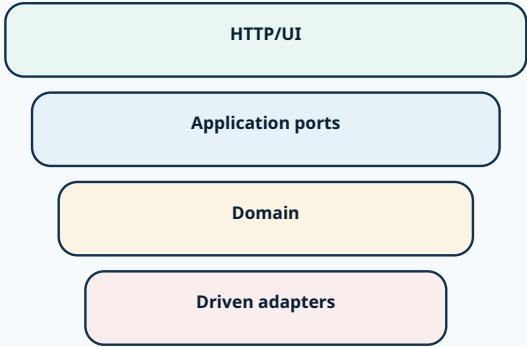
EVIDENCE TO RETAIN

- CONTRACT
- ADR
- TYPED LINKS
- RELEASE EVIDENCE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A feature change requires coordinated edits across many teams.
- Contracts omit errors, compatibility, or ownership.
- Trace links exist but their semantics are unknown.
- Operational failures cannot be mapped back to requirements.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

- [R23] Cockburn, Hexagonal Architecture
- [R16] Vernon, Implementing DDD

RESEARCH EXPANSION

Functional core, imperative shell

keep calculations and decisions pure while containing database, clock, network, and messaging effects at the edges.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must keep calculations and decisions pure while containing database, clock, network, and messaging effects at the edges; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

keep calculations and decisions pure while containing database, clock, network, and messaging effects at the edges. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

1

Name the behavioral boundary and its authoritative contract.

2

Place policy behind a narrow port with explicit dependencies.

3

Version schemas, events, migrations, and failure semantics.

4

Create typed links from requirement to code and evidence.

5

Run gates and retain results for release and diagnosis.

EVIDENCE TO RETAIN

- CONTRACT
- ADR
- TYPED LINKS
- RELEASE EVIDENCE

REVIEW QUESTIONS

•

What uncertainty is removed before implementation?

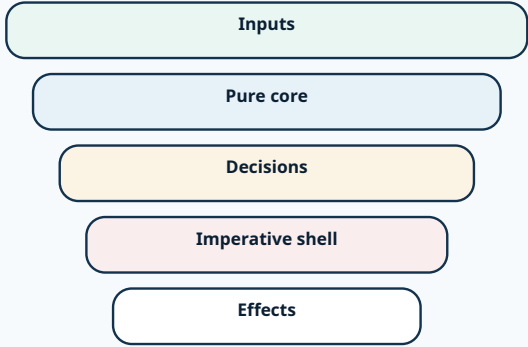
•

Which artifact is authoritative, and who approves it?

•

What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A feature change requires coordinated edits across many teams.
- Contracts omit errors, compatibility, or ownership.
- Trace links exist but their semantics are unknown.
- Operational failures cannot be mapped back to requirements.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

- [R23] Cockburn, Hexagonal Architecture
- [R18] Beck, TDD by Example

RESEARCH EXPANSION

SOLID principles and narrow ports

use responsibility, dependency inversion, and interface segregation to create small seams with explicit meanings.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To use responsibility, dependency inversion, and interface segregation to create small seams with explicit meanings, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

use responsibility, dependency inversion, and interface segregation to create small seams with explicit meanings. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Name the behavioral boundary and its authoritative contract.
- 2 Place policy behind a narrow port with explicit dependencies.
- 3 Version schemas, events, migrations, and failure semantics.
- 4 Create typed links from requirement to code and evidence.
- 5 Run gates and retain results for release and diagnosis.

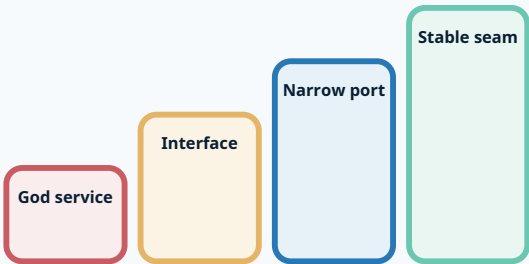
EVIDENCE TO RETAIN

- CONTRACT
- ADR
- TYPED LINKS
- RELEASE EVIDENCE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A feature change requires coordinated edits across many teams.
- Contracts omit errors, compatibility, or ownership.
- Trace links exist but their semantics are unknown.
- Operational failures cannot be mapped back to requirements.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

- [R23] Cockburn, Hexagonal Architecture
- [R16] Vernon, Implementing DDD

RESEARCH EXPANSION

OpenAPI as an HTTP contract

specify resources, operations, schemas, status codes, headers, errors, compatibility, and examples before consumers depend on them.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to specify resources, operations, schemas, status codes, headers, errors, compatibility, and examples before consumers depend on them without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

specify resources, operations, schemas, status codes, headers, errors, compatibility, and examples before consumers depend on them. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Name the behavioral boundary and its authoritative contract.
- 2 Place policy behind a narrow port with explicit dependencies.
- 3 Version schemas, events, migrations, and failure semantics.
- 4 Create typed links from requirement to code and evidence.
- 5 Run gates and retain results for release and diagnosis.

EVIDENCE TO RETAIN

CONTRACT

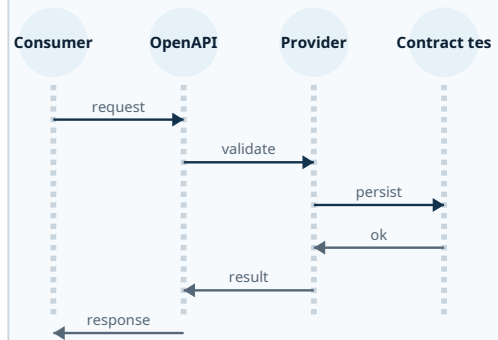
ADR

TYPED LINKS

RELEASE EVIDENCE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

```
POST /orders/{orderId}/refunds
201 RefundIssued
409 AlreadyRefunded
401 InvalidToken
Idempotency-Key: required
```

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A feature change requires coordinated edits across many teams.
- Contracts omit errors, compatibility, or ownership.
- Trace links exist but their semantics are unknown.
- Operational failures cannot be mapped back to requirements.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R24] OpenAPI Specification

[R5] ISO/IEC/IEEE 29148:2018

RESEARCH EXPANSION

Event contracts and versioning

define event meaning, identity, ordering assumptions, schema evolution, replay behavior, and deprecation policy.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to define event meaning, identity, ordering assumptions, schema evolution, replay behavior, and deprecation policy. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

define event meaning, identity, ordering assumptions, schema evolution, replay behavior, and deprecation policy. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Name the behavioral boundary and its authoritative contract.
- 2 Place policy behind a narrow port with explicit dependencies.
- 3 Version schemas, events, migrations, and failure semantics.
- 4 Create typed links from requirement to code and evidence.
- 5 Run gates and retain results for release and diagnosis.

EVIDENCE TO RETAIN

CONTRACT

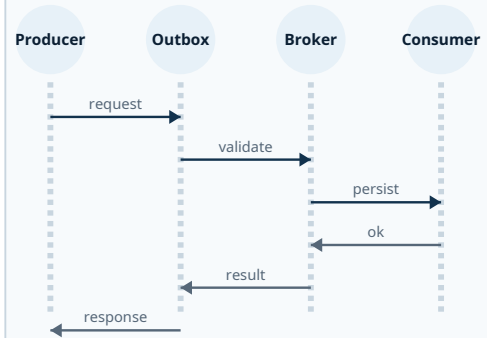
ADR

TYPED LINKS

RELEASE EVIDENCE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

```
POST /orders/{orderId}/refunds
201 RefundIssued
409 AlreadyRefunded
401 InvalidToken
Idempotency-Key: required
```

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A feature change requires coordinated edits across many teams.
- Contracts omit errors, compatibility, or ownership.
- Trace links exist but their semantics are unknown.
- Operational failures cannot be mapped back to requirements.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE[\[R24\] OpenAPI Specification](#)[\[R46\] Google SRE Workbook](#)

RESEARCH EXPANSION

Consumer-driven contracts

capture consumer expectations at the boundary while retaining provider verification and avoiding false confidence about end-to-end business behavior.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first capture consumer expectations at the boundary while retaining provider verification and avoiding false confidence about end-to-end business behavior, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

capture consumer expectations at the boundary while retaining provider verification and avoiding false confidence about end-to-end business behavior. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Name the behavioral boundary and its authoritative contract.
- 2 Place policy behind a narrow port with explicit dependencies.
- 3 Version schemas, events, migrations, and failure semantics.
- 4 Create typed links from requirement to code and evidence.
- 5 Run gates and retain results for release and diagnosis.

EVIDENCE TO RETAIN

CONTRACT

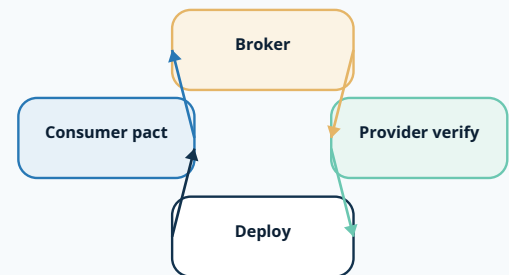
ADR

TYPED LINKS

RELEASE EVIDENCE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

```
POST /orders/{orderId}/refunds
201 RefundIssued
409 AlreadyRefunded
401 InvalidToken
Idempotency-Key: required
```

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A feature change requires coordinated edits across many teams.
- Contracts omit errors, compatibility, or ownership.
- Trace links exist but their semantics are unknown.
- Operational failures cannot be mapped back to requirements.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R25] Pact consumer-driven contracts

[R37] DORA loosely coupled teams

RESEARCH EXPANSION

Database and schema contracts

treat migration order, backward compatibility, defaults, nullability, and rollback behavior as explicit release constraints.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must treat migration order, backward compatibility, defaults, nullability, and rollback behavior as explicit release constraints; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

treat migration order, backward compatibility, defaults, nullability, and rollback behavior as explicit release constraints. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Name the behavioral boundary and its authoritative contract.
- 2 Place policy behind a narrow port with explicit dependencies.
- 3 Version schemas, events, migrations, and failure semantics.
- 4 Create typed links from requirement to code and evidence.
- 5 Run gates and retain results for release and diagnosis.

EVIDENCE TO RETAIN

CONTRACT

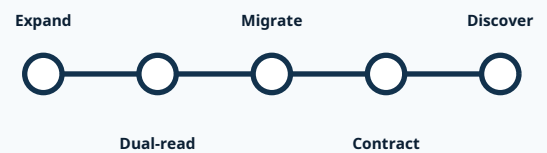
ADR

TYPED LINKS

RELEASE EVIDENCE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

```
POST /orders/{orderId}/refunds
201 RefundIssued
409 AlreadyRefunded
401 InvalidToken
Idempotency-Key: required
```

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A feature change requires coordinated edits across many teams.
- Contracts omit errors, compatibility, or ownership.
- Trace links exist but their semantics are unknown.
- Operational failures cannot be mapped back to requirements.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R36] DORA software delivery metrics

[R46] Google SRE Workbook

RESEARCH EXPANSION

Feature flags as policy seams

separate deployment from release while specifying defaults, targeting, failure behavior, ownership, expiry, and cleanup.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To separate deployment from release while specifying defaults, targeting, failure behavior, ownership, expiry, and cleanup, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

separate deployment from release while specifying defaults, targeting, failure behavior, ownership, expiry, and cleanup. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Name the behavioral boundary and its authoritative contract.
- 2 Place policy behind a narrow port with explicit dependencies.
- 3 Version schemas, events, migrations, and failure semantics.
- 4 Create typed links from requirement to code and evidence.
- 5 Run gates and retain results for release and diagnosis.

EVIDENCE TO RETAIN

CONTRACT

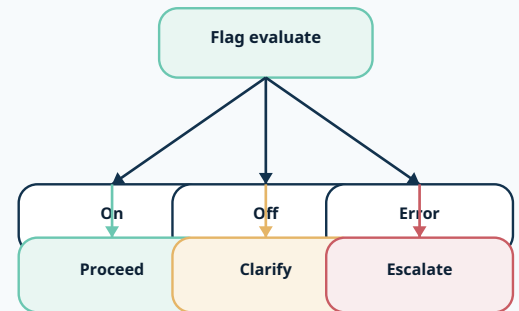
ADR

TYPED LINKS

RELEASE EVIDENCE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A feature change requires coordinated edits across many teams.
- Contracts omit errors, compatibility, or ownership.
- Trace links exist but their semantics are unknown.
- Operational failures cannot be mapped back to requirements.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R48] OpenFeature specification

[R36] DORA software delivery metrics

RESEARCH EXPANSION

Architecture Decision Records

record context, decision, alternatives, rationale, consequences, status, and supersession for architecturally significant choices.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to record context, decision, alternatives, rationale, consequences, status, and supersession for architecturally significant choices without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

record context, decision, alternatives, rationale, consequences, status, and supersession for architecturally significant choices. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Name the behavioral boundary and its authoritative contract.
- 2 Place policy behind a narrow port with explicit dependencies.
- 3 Version schemas, events, migrations, and failure semantics.
- 4 Create typed links from requirement to code and evidence.
- 5 Run gates and retain results for release and diagnosis.

EVIDENCE TO RETAIN

- CONTRACT
- ADR
- TYPED LINKS
- RELEASE EVIDENCE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A feature change requires coordinated edits across many teams.
- Contracts omit errors, compatibility, or ownership.
- Trace links exist but their semantics are unknown.
- Operational failures cannot be mapped back to requirements.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

- [R16] Vernon, Implementing DDD
- [R23] Cockburn, Hexagonal Architecture

RESEARCH EXPANSION

A traceability information model

represent artifacts as typed nodes and semantic links rather than a flat spreadsheet of filenames.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to represent artifacts as typed nodes and semantic links rather than a flat spreadsheet of filenames. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

represent artifacts as typed nodes and semantic links rather than a flat spreadsheet of filenames. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Name the behavioral boundary and its authoritative contract.
- 2 Place policy behind a narrow port with explicit dependencies.
- 3 Version schemas, events, migrations, and failure semantics.
- 4 Create typed links from requirement to code and evidence.
- 5 Run gates and retain results for release and diagnosis.

EVIDENCE TO RETAIN

- CONTRACT
- ADR
- TYPED LINKS
- RELEASE EVIDENCE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A feature change requires coordinated edits across many teams.
- Contracts omit errors, compatibility, or ownership.
- Trace links exist but their semantics are unknown.
- Operational failures cannot be mapped back to requirements.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

- [R26] Gotel and Finkelstein 1994
- [R27] Requirements traceability book

RESEARCH EXPANSION

Requirement-to-test matrices

use matrices as review views generated from richer links and include both coverage and intentional exclusions.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first use matrices as review views generated from richer links and include both coverage and intentional exclusions, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

use matrices as review views generated from richer links and include both coverage and intentional exclusions. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

1

Name the behavioral boundary and its authoritative contract.

2

Place policy behind a narrow port with explicit dependencies.

3

Version schemas, events, migrations, and failure semantics.

4

Create typed links from requirement to code and evidence.

5

Run gates and retain results for release and diagnosis.

EVIDENCE TO RETAIN

- CONTRACT
- ADR
- TYPED LINKS
- RELEASE EVIDENCE

REVIEW QUESTIONS

•

What uncertainty is removed before implementation?

•

Which artifact is authoritative, and who approves it?

•

What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

R-1	R-2	BDD	Unit
Contract	Ops	R-1	R-2
BDD	Unit	Contract	Ops
R-1	R-2	BDD	Unit

WORKED EXAMPLE

WHEN an event is cancelled,
THE Refunds service SHALL create exactly one
refund
for ticket price minus frozen service fees
within five business days.

FAILURE MODE / LIMIT

A well-formed sentence can still encode the wrong policy, omit an actor, or hide an unmeasurable quality boundary. Syntax helps expose gaps; it does not supply domain truth.

WARNING SIGNS

- A feature change requires coordinated edits across many teams.
- Contracts omit errors, compatibility, or ownership.
- Trace links exist but their semantics are unknown.
- Operational failures cannot be mapped back to requirements.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

- [R26] Gotel and Finkelstein 1994
- [R27] Requirements traceability book

RESEARCH EXPANSION

Semantic link types

differentiate derives-from, constrains, satisfies, verifies, implements, observes, supersedes, conflicts-with, and preserves.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must differentiate derives-from, constrains, satisfies, verifies, implements, observes, supersedes, conflicts-with, and preserves; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

differentiate derives-from, constrains, satisfies, verifies, implements, observes, supersedes, conflicts-with, and preserves. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Name the behavioral boundary and its authoritative contract.
- 2 Place policy behind a narrow port with explicit dependencies.
- 3 Version schemas, events, migrations, and failure semantics.
- 4 Create typed links from requirement to code and evidence.
- 5 Run gates and retain results for release and diagnosis.

EVIDENCE TO RETAIN

CONTRACT

ADR

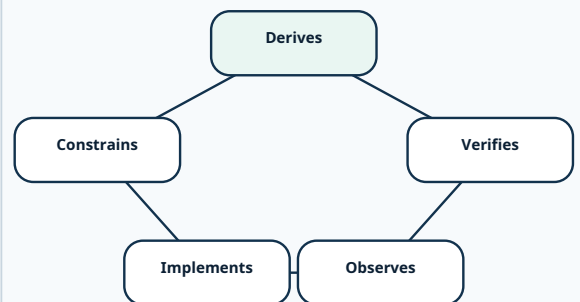
TYPED LINKS

RELEASE EVIDENCE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A feature change requires coordinated edits across many teams.
- Contracts omit errors, compatibility, or ownership.
- Trace links exist but their semantics are unknown.
- Operational failures cannot be mapped back to requirements.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R26] Gotel and Finkelstein 1994

[R27] Requirements traceability book

RESEARCH EXPANSION

Change-impact analysis

traverse forward and backward links, then review semantic relevance instead of assuming every connected artifact must change.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To traverse forward and backward links, then review semantic relevance instead of assuming every connected artifact must change, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

traverse forward and backward links, then review semantic relevance instead of assuming every connected artifact must change. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Name the behavioral boundary and its authoritative contract.
- 2 Place policy behind a narrow port with explicit dependencies.
- 3 Version schemas, events, migrations, and failure semantics.
- 4 Create typed links from requirement to code and evidence.
- 5 Run gates and retain results for release and diagnosis.

EVIDENCE TO RETAIN

CONTRACT

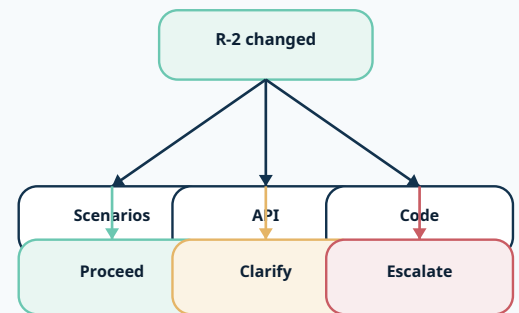
ADR

TYPED LINKS

RELEASE EVIDENCE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A feature change requires coordinated edits across many teams.
- Contracts omit errors, compatibility, or ownership.
- Trace links exist but their semantics are unknown.
- Operational failures cannot be mapped back to requirements.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R26] Gotel and Finkelstein 1994

[R27] Requirements traceability book

RESEARCH EXPANSION

Continuous integration verification gates

make schema, unit, BDD, contract, security, accessibility, performance, and traceability checks explicit merge policies.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to make schema, unit, BDD, contract, security, accessibility, performance, and traceability checks explicit merge policies without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

make schema, unit, BDD, contract, security, accessibility, performance, and traceability checks explicit merge policies. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

1

Name the behavioral boundary and its authoritative contract.

2

Place policy behind a narrow port with explicit dependencies.

3

Version schemas, events, migrations, and failure semantics.

4

Create typed links from requirement to code and evidence.

5

Run gates and retain results for release and diagnosis.

EVIDENCE TO RETAIN

- CONTRACT
- ADR
- TYPED LINKS
- RELEASE EVIDENCE

REVIEW QUESTIONS

•

What uncertainty is removed before implementation?

•

Which artifact is authoritative, and who approves it?

•

What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

•

A feature change requires coordinated edits across many teams.

•

Contracts omit errors, compatibility, or ownership.

•

Trace links exist but their semantics are unknown.

•

Operational failures cannot be mapped back to requirements.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

RESEARCH EXPANSION

Security evidence

retain threat assumptions, test results, dependency findings, access-control checks, and audit-event verification as part of conformance.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to retain threat assumptions, test results, dependency findings, access-control checks, and audit-event verification as part of conformance. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

retain threat assumptions, test results, dependency findings, access-control checks, and audit-event verification as part of conformance. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Name the behavioral boundary and its authoritative contract.
- 2 Place policy behind a narrow port with explicit dependencies.
- 3 Version schemas, events, migrations, and failure semantics.
- 4 Create typed links from requirement to code and evidence.
- 5 Run gates and retain results for release and diagnosis.

EVIDENCE TO RETAIN

CONTRACT

ADR

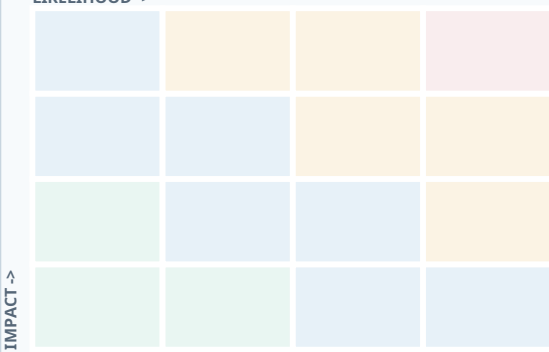
TYPED LINKS

RELEASE EVIDENCE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A feature change requires coordinated edits across many teams.
- Contracts omit errors, compatibility, or ownership.
- Trace links exist but their semantics are unknown.
- Operational failures cannot be mapped back to requirements.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R44] OWASP ASVS

[R5] ISO/IEC/IEEE 29148:2018

RESEARCH EXPANSION

Observability as a specification output

define logs, traces, metrics, correlation identifiers, privacy rules, and operational questions before failures occur.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first define logs, traces, metrics, correlation identifiers, privacy rules, and operational questions before failures occur, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

define logs, traces, metrics, correlation identifiers, privacy rules, and operational questions before failures occur. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Name the behavioral boundary and its authoritative contract.
- 2 Place policy behind a narrow port with explicit dependencies.
- 3 Version schemas, events, migrations, and failure semantics.
- 4 Create typed links from requirement to code and evidence.
- 5 Run gates and retain results for release and diagnosis.

EVIDENCE TO RETAIN

CONTRACT

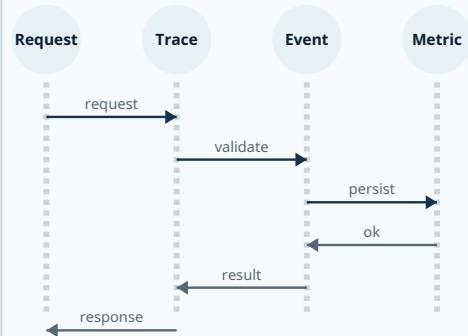
ADR

TYPED LINKS

RELEASE EVIDENCE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A feature change requires coordinated edits across many teams.
- Contracts omit errors, compatibility, or ownership.
- Trace links exist but their semantics are unknown.
- Operational failures cannot be mapped back to requirements.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE[\[R46\] Google SRE Workbook](#)[\[R47\] OpenTelemetry specification](#)

RESEARCH EXPANSION

Release and production evidence

connect rollout, feature flags, canaries, SLOs, reconciliation, incident signals, and rollback decisions to the requirements they protect.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must connect rollout, feature flags, canaries, SLOs, reconciliation, incident signals, and rollback decisions to the requirements they protect; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

connect rollout, feature flags, canaries, SLOs, reconciliation, incident signals, and rollback decisions to the requirements they protect. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Name the behavioral boundary and its authoritative contract.
- 2 Place policy behind a narrow port with explicit dependencies.
- 3 Version schemas, events, migrations, and failure semantics.
- 4 Create typed links from requirement to code and evidence.
- 5 Run gates and retain results for release and diagnosis.

EVIDENCE TO RETAIN

CONTRACT

ADR

TYPED LINKS

RELEASE EVIDENCE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

Canary

4.2 d

SLO

8/wk

Errors

6%

Rollback

40 min

WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 ->
TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A feature change requires coordinated edits across many teams.
- Contracts omit errors, compatibility, or ownership.
- Trace links exist but their semantics are unknown.
- Operational failures cannot be mapped back to requirements.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R36] DORA software delivery metrics

[R46] Google SRE Workbook

[R48] OpenFeature specification

EXPANDED FROM SUPPLIED WALKTHROUGH

PLT-4821 architecture evidence pack

show how one requirement maps to a Money value object, Refund aggregate, service port, API response, event, tests, and production reconciliation.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To show how one requirement maps to a Money value object, Refund aggregate, service port, API response, event, tests, and production reconciliation, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

show how one requirement maps to a Money value object, Refund aggregate, service port, API response, event, tests, and production reconciliation. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Name the behavioral boundary and its authoritative contract.
- 2 Place policy behind a narrow port with explicit dependencies.
- 3 Version schemas, events, migrations, and failure semantics.
- 4 Create typed links from requirement to code and evidence.
- 5 Run gates and retain results for release and diagnosis.

EVIDENCE TO RETAIN

CONTRACT

ADR

TYPED LINKS

RELEASE EVIDENCE

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A feature change requires coordinated edits across many teams.
- Contracts omit errors, compatibility, or ownership.
- Trace links exist but their semantics are unknown.
- Operational failures cannot be mapped back to requirements.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R23] Cockburn, Hexagonal Architecture

[R24] OpenAPI Specification

[R26] Gotel and Finkelstein 1994

PART XII

AI-era SDD and agent governance

Treat coding agents as bounded engineering participants: powerful at transformation and search, fallible at authority, semantics, security, and independent judgment.

WHAT THIS PART ANSWERS

How should an agent be briefed? Which actions may it take? How are requirements, tests, tools, reviews, and risk controls kept independent?

USE IT WHEN

You are moving from autocomplete or chat toward delegated, multi-step software-engineering work.

AI-ERA SDD · OPERATING THESIS

Why AI changes the bottleneck

The engineering question is no longer “Can a model produce code?” It is “Can the team preserve authority, intent, and evidence while the model transforms artifacts at speed?”

AI changes the economics of ambiguity. A weak requirement no longer produces only a slow conversation; it can generate a complete plan, test suite, and implementation before anyone notices that the interpretation was wrong. The output may be fluent, internally consistent, and still violate the business rule.

That does not make specification-first thinking new. Requirements engineering, DDD, BDD, TDD, formal methods, and traceability already address different forms of uncertainty. The modern AI-oriented SDD contribution is an *assembly*: versioned intent, staged transformations, coding agents, deterministic controls, and retained evidence.

The new bottleneck. When generation becomes cheap, review, verification, risk analysis, and decision authority become the scarce resources. Speed therefore depends on better task boundaries and better evidence—not on asking the model to write more code.

AMPLIFICATION MODEL

Weak intent

Vague rule,
hidden policy,
missing
boundary

×

Agent speed

Rapid search,
drafting,
editing, and
retries

Large output

Plan, tests,
code, docs,
migrations

→

Review load

More surface
area to
validate

Strong intent

Stable IDs,
examples,
contracts,
exclusions

×

Independent gates

Tests,
permissions,
mutation,
human
approval

AI drafts

Artifacts decide

Gates enforce

Humans accept risk

EVIDENCE

AI-01 · GitHub Spec Kit

AI-02 · Kiro Specs

AI-03 · OpenAI Codex

AI-12 · Formal RE and LLMs roadmap

AI-13 · LLMs for Requirements Engineering review

AI-ERA SDD · CONTROL MODEL

The AI-assisted SDD control loop

A safe workflow assigns the model useful transformation work while keeping domain authority, approval, and risk acceptance outside the model.



Agent contribution

STAGE	USEFUL AI WORK	AUTHORITY STAYS WITH
Discover	Summarize sources, cluster terms, propose questions	Stakeholders and domain experts
Specify	Draft EARS lines, detect smells, generate examples	Product, engineering, risk owners
Plan	Explore options, identify affected files, draft ADRs	Architects and maintainers
Implement	Edit code, run tools, repair failures, document diffs	Task contract and repository policy
Verify	Run suites, compare artifacts, summarize evidence	Independent oracle and human reviewer

Control loop

- 1 · Establish authority**
Name the approved spec, rule IDs, ADRs, and owners. Conversation history is not the source of truth.
- 2 · Bound the task**
Declare allowed paths, tools, commands, time, network access, and stop conditions.
- 3 · Execute visibly**
Retain the diff, tool calls, test output, assumptions, and unresolved uncertainty.
- 4 · Challenge independently**
Use contract-derived tests, mutation, static analysis, security checks, and human review.
- 5 · Reopen upstream**
When evidence conflicts with intent, repair the spec or plan before patching code silently.

AI-ERA SDD · CONTEXT ENGINEERING

Context engineering is briefing design

A task packet should be authoritative, minimal, local to the decision, and explicit about what the agent must not infer.

The context packet

A model cannot read “the repository” in the same way a long-tenured engineer remembers it. Context engineering is the deliberate selection, ordering, and compression of information needed for one decision. More context is not automatically better: stale files, duplicated rules, irrelevant tool definitions, and contradictory notes compete for attention.

LAYER	INCLUDE ONLY WHAT CHANGES THE TASK
Constitution	Non-negotiable architecture, security, testing, and review rules
Feature intent	Goal, actors, scope, exclusions, EARS IDs, NFRs
Domain model	Terms, owner context, invariants, state transitions
Contracts	Gherkin, OpenAPI, events, consumer promises
Task	One vertical slice, allowed paths, dependencies, done evidence
Repository map	Relevant files and symbols—not a blind dump
Commands	Exact tests, linters, type checks, and build steps

Briefing order matters

1. State the goal and authority first.
2. List constraints and exclusions before solution hints.
3. Provide the contract and failing oracle.
4. Point to the smallest relevant code surface.
5. End with stop conditions and required evidence.

Context smells

- A 500-line “constitution” containing tutorials and examples.
- Every MCP tool loaded for every task.
- Ticket, wiki, prompt, and spec disagree without precedence.
- Generated summaries replace the original policy source.
- Old architectural decisions are present but not marked superseded.

Design rule: build the smallest packet that lets an agent make the next decision correctly and lets a reviewer reconstruct why it was allowed.

AI-ERA SDD · WORKED PACKET

PLT-4821: a complete implementation packet

The prompt is only one field. The real control surface is the combination of authority, scope, tools, stop conditions, and required evidence.

Agent task packet

```
task: PLT-4821 / T3-refund-calculation
goal: issue a refund for ticket price minus frozen service fees
authority:
  spec: specs/checkout-refunds/spec.md#EARS-2
  terms: docs/ddd.md#service-fee
  decision: adr/ADR-017-rounding.md
allowed_paths:
  - src/refunds/**
  - tests/refunds/**
forbidden:
  - notification wiring (T4)
  - checkout UI (T5)
  - live fee configuration
required_oracles:
  - npm test -- refunds.service.spec.ts
  - npm run bdd -- contracts/refunds.feature
  - npm run contract -- refunds.openapi.yaml
stop_on:
  - ambiguous fee ownership
  - contract conflict
  - edit outside allowed_paths
  - missing order snapshot
report:
  changed_files, tests_run, assumptions, unresolved_risk
```

Why every field exists

-  **Authority**
Prevents the model from treating its own generated interpretation as the requirement.
-  **Allowed paths**
Makes a vertical slice enforceable and reduces accidental refactoring.
-  **Forbidden scope**
Stops “helpful” adjacent work from expanding review and rollback risk.
-  **Independent oracles**
Defines success outside the implementation prompt.
-  **Stop conditions**
Converts uncertainty into escalation instead of silent invention.
-  **Report schema**
Makes review repeatable and keeps evidence in the pull request.

Worked decision: if the order snapshot does not contain the service fee, the agent must stop. Recomputing from current fee configuration would create a plausible but historically incorrect refund.

AI-ERA SDD · ENFORCEMENT

From prompts to enforceable controls

A prompt can request behavior; only permissions and executable gates can prevent or reject behavior.



Four different control types

CONTROL	EXAMPLE	FAILURE MODE
Prompt instruction	“Do not edit payment code.”	The model ignores, forgets, or rationalizes around it.
Context	Provide the refund spec and context map.	The source is stale or contradictory.
Permission	Allow edits only under <code>src/refunds/**</code> .	The permitted area is still semantically wrong.
Gate	CI rejects scope breach or failed contracts.	The gate covers the wrong behavior.

No single layer is sufficient. Instructions communicate intent; context supplies facts; permissions bound capability; gates decide whether observable evidence is acceptable.

Deterministic path gate

```
#!/usr/bin/env bash
set -euo pipefail
allowed='^(src/refunds/|tests/refunds/)'

while IFS= read -r file; do
  if ! [[ "$file" =~ $allowed ]]; then
    echo "BLOCKED: T3 cannot modify $file" >&2
    exit 2
  fi
done < <(git diff --name-only)

npm test -- refunds.service.spec.ts
npm run bdd -- contracts/refunds.feature
```

A hook or CI job can invoke this script before completion. The model receives a clear failure and cannot “persuade” the operating system to accept an out-of-scope edit.

Important: deterministic does not mean correct. The allowlist and tests are policy artifacts and require review, versioning, and ownership.

AI-ERA SDD · ROLES

Agent roles across the lifecycle

Assign agents narrow responsibilities and explicit non-authorities. Role names are useful only when they change inputs, outputs, permissions, and review.

AGENT ROLE	INPUT	OUTPUT	MAY NOT DECIDE	BEST GATE
Explorer	Ticket, repository map, domain sources	Affected concepts, files, questions	Final scope or policy	Human triage
Spec critic	Draft requirements and glossary	Ambiguities, smells, counterexamples	Business authority	PO/QA review
Planner	Approved spec, architecture rules	Options, risks, ADR draft, tasks	Risk acceptance	Architecture review
Implementer	One task packet and failing oracle	Patch, test output, change report	Requirement changes	Scope + CI gates
Test adversary	Spec, contracts, implementation diff	Boundary tests, mutants, failure probes	Merge verdict alone	Mutation + reviewer
Verifier	Artifacts, patch, execution evidence	Coverage matrix and deviations	Waive unresolved red evidence	Named human owner

Why roles help

Roles reduce prompt overload, clarify outputs, and make conflicts visible. An explorer can propose that Payments owns a rule; a domain owner can reject it before the planner builds on the mistake.

Why roles do not guarantee independence

Two role prompts sent to the same model with the same context may repeat the same blind spot. Independence must be designed through different evidence, hidden tests, mutation, alternate models, or human/domain review.

Operating rule: separate proposal from approval. The same agent may perform several roles for low-risk work, but it should never silently promote its own assumption into approved domain policy.

AI-ERA SDD · SINGLE-AGENT

Single-agent flow: implement T3 safely

The safest autonomous loop is small: observe red, make a bounded change, run declared evidence, inspect scope, and stop when the specification is insufficient.

Execution trace

- Read only**
Load EARS-2, ADR-017, the refund aggregate, ports, and the failing T3 test. Produce an implementation plan before editing.
- Confirm the oracle**
Run the targeted test and observe NotImplemented – T3. A red failure proves the detector can see absence.
- Implement minimally**
Add integer-cent subtraction in the pure calculator. Do not wire notifications or change the API.
- Triangulate**
Run minor-unit, duplicate-request, and boundary tests to prevent an example-specific implementation.
- Inspect the diff**
Check paths, public contracts, dependencies, and generated files. Explain every changed line.
- Report uncertainty**
If fee data is unavailable or types disagree, stop and reopen the task rather than invent a fallback.

Agent completion report

STATUS: GREEN WITHIN T3 SCOPE

Changed:

```
src/refunds/domain/refund-calculator.ts
tests/refunds/refund-calculator.spec.ts
```

Evidence:

```
unit: 18 passed
bdd: 2 passed
openapi: unchanged / valid
path_gate: passed
```

Interpretation:

```
serviceFee comes from OrderSnapshot
calculation uses integer minor units
ADR-017 remains the single rounding authority
```

Not changed:

```
refund.issued event (T4)
checkout UI (T5)
```

Residual risk:

```
percentage-fee products require an additional
boundary example before enabling the feature.
```

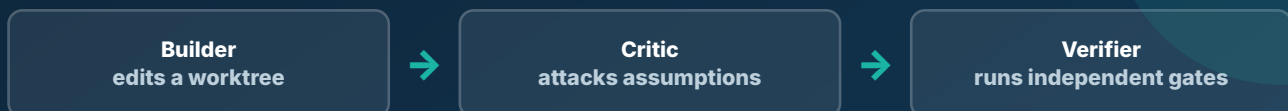
This report is not proof by itself. It is a structured index into the diff and machine-generated evidence that a reviewer can reproduce.

AI-ERA SDD · MULTI-AGENT

Multi-agent orchestration without groupthink

Multiple agents create value when work is genuinely independent or when roles challenge one another with different evidence.

BUILDER-CRITIC-VERIFIER TOPOLOGY



Parallelism that helps

- Independent tasks run in isolated worktrees with non-overlapping Touches : globs.
- One agent researches affected code while another drafts counterexamples; neither edits production code yet.
- Alternative plans are compared against the same constraints before an architecture decision is accepted.
- A verifier receives the approved spec and patch, but not the builder's chain of reasoning, reducing anchoring.

Parallelism that hurts

- Several agents edit the same files and create merge conflict as an operating model.
- Agents vote on a domain fact none of them is authorized to know.
- All agents receive the same flawed summary and repeat the same omission.
- The team counts generated variants instead of review cost, integration risk, and evidence quality.

Anti-groupthink design: vary evidence, not only prompts. Give the critic original policy text, the builder the approved task packet, and the verifier hidden boundary tests or mutants. A second voice is not independent when it shares the same blind spot.

AI-ERA SDD · VERIFICATION

The independent-oracle problem

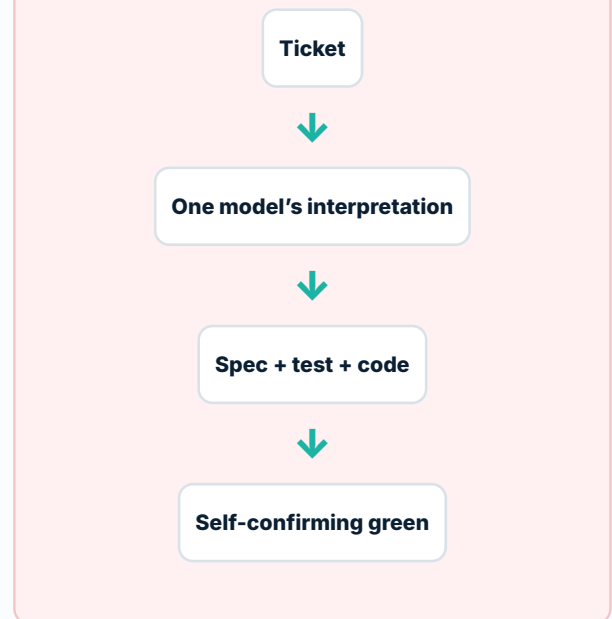
Agent-produced artifacts can agree perfectly and still be wrong together. SDD therefore needs evidence that is capable of disagreeing with the agent's interpretation.

The leakage pattern

If one model converts a vague ticket into a requirement, generates a test from its own requirement, writes the implementation, and then explains why the result is correct, every artifact can share the same misunderstanding. Green tests demonstrate internal consistency—not alignment with stakeholder intent.

A test is independent only to the extent that its expected result comes from a source that could disagree with the implementation.

High-risk chain



Source independence

Derive expected values from approved policy tables, regulations, recorded examples, or domain-owner decisions.

Method independence

Use property-based checks, metamorphic relations, model checking, static analysis, or a separate test harness.

Reviewer independence

Have a reviewer inspect requirement coverage and mutants without being anchored to the builder's rationale.

Practical minimum: for business-critical behavior, the builder may generate candidate tests, but at least one boundary oracle and the final acceptance matrix should be supplied or approved independently.

EVIDENCE

[AI-07 · SWE-bench](#)
[AI-08 · SWE-agent](#)
[AI-12 · Formal RE and LLMs roadmap](#)
[AI-13 · LLMs for Requirements Engineering review](#)

AI-ERA SDD · REQUIREMENTS

AI-generated requirements: useful drafts, human authority

LLMs are strong at restructuring language and proposing missing cases; they are not an authoritative source for domain policy.

From vague ticket to reviewable draft

INPUT

“When an event is cancelled, refund buyers automatically. Refund the ticket price minus fees. Make sure it cannot happen twice.”

AI-ASSISTED EARS DRAFT

EARS-2: WHEN an event is marked CANCELLED, the Refunds context shall create one refund per paid order for the captured ticket amount minus the service fee stored in the order snapshot.

EARS-3: WHILE a refund for an order is PENDING or ISSUED, the API shall reject another request with HTTP 409 and code ALREADY_REFUNDED.

The draft is better because it exposes actor, trigger, source of fee data, uniqueness, state, and observable response. It is not yet approved: “service fee,” cancellation authority, timing, currency, and rounding still require named owners.

Quality rubric for generated requirements

DIMENSION	REVIEW QUESTION
Authority	Who can decide this rule, and is that source cited?
Completeness	Are trigger, precondition, response, timing, failure, and unchanged behavior present?
Consistency	Does the line conflict with glossary, ADR, API, regulation, or another requirement?
Testability	Can a reviewer state the failing observation before code exists?
Boundary coverage	Are thresholds, empty values, duplicates, retries, and partial failures considered?
Traceability	Does the requirement have a stable ID and downstream evidence target?

Best use of AI: generate questions, variants, smells, and counterexamples. Do not use model confidence as evidence that the requirement is true.

EVIDENCE

[AI-12 · Formal RE and LLMs roadmap](#)[AI-13 · LLMs for Requirements Engineering review](#)[AI-01 · GitHub Spec Kit](#)[AI-02 · Kiro Specs](#)

AI-ERA SDD · TEST GENERATION

AI-generated tests: detect absence, not mirror code

A green test suite is meaningful only when its oracles represent behavior independently and can kill realistic semantic mutations.

A mirrored test can be useless

```
// Production code
const refund = total - fees;

// Weak generated test: repeats the implementation
expect(calculate(total, fees))
  .toBe(total - fees);
```

This catches gross wiring failures but does not challenge fee ownership, rounding, state, duplicate behavior, or historical snapshot semantics.

A stronger evidence set

```
it('uses the frozen service fee', ...)
it('rejects an issued refund with 409', ...)
it('preserves integer minor units', ...)
it('does not read live fee configuration', ...)

property: 0 <= refund <= capturedAmount
metamorphic: increasing serviceFee by 1 cent
               decreases refund by exactly 1 cent
```

Mutation as an adversarial review

MUTATION	EXPECTED DETECTOR
Use live fee config instead of snapshot	Historical-fee fixture
Allow second refund	EARS-3 / 409 scenario
Round half-up instead of half-even	ADR-017 boundary examples
Refund provider fee as well	Policy table and exact amount oracle
Publish event before persistence	Failure-injection component test

Generated tests become more valuable when the model is asked to *attack* the current suite: identify semantic changes that should fail, propose mutants, and report which survive. The score is not a vanity number; each survivor is a concrete statement about what the evidence cannot currently distinguish.

Separation rule: have the test adversary work from the approved specification and public contracts, not from the builder’s implementation explanation.

AI-ERA SDD · TOOLS

Repository tools, MCP, and least privilege

Connecting more tools expands capability and attack surface. Treat every resource, prompt, and action as a typed trust decision.

MCP in an SDD workflow

The Model Context Protocol standardizes how an AI host connects to servers that expose **resources**, **prompts**, and **tools**. In SDD, those categories should map to different trust decisions.

PRIMITIVE	SDD USE	PRIMARY RISK
Resource	Read Jira, policy, architecture, logs, schemas	Stale, overbroad, or sensitive context
Prompt	Reusable spec review or task workflow	Hidden assumptions presented as policy
Tool	Create issue, run query, deploy, modify file	Side effects, privilege escalation, irreversible action

Trust boundary: tool descriptions and retrieved content are inputs, not instructions. A malicious issue comment or document can attempt to redirect the agent.

Least-privilege profile for T3

```
resources:
  allow:
    - specs/checkout-refunds/**
    - docs/ddd.md#refunds
    - src/refunds/**
  deny:
    - .env
    - secrets/**
    - customer-data/**

tools:
  allow:
    - read_file
    - search_code
    - run_test[target=refunds]
    - edit_file[path=src/refunds/**]
  deny:
    - deploy_production
    - rotate_secret
    - delete_repository

network: off
human_approval: external_write | destructive
```

The profile should be versioned beside the task or repository policy. A broad “developer” role is usually too powerful for a one-file domain change.

AI-ERA SDD · SECURITY

Security for coding agents

A coding agent combines code generation, repository access, command execution, and external tools. Security must therefore be specified as behavior and enforced as capability.

Threat model for a coding agent

THREAT	EXAMPLE	CONTROL
Prompt injection	Issue text says to ignore policy and export secrets	Untrusted-content boundaries, tool allowlists, approval
Secret exposure	Agent reads .env while debugging	Deny paths, secret broker, redaction, no log echo
Dependency attack	Generated fix installs an unreviewed package	Lockfile policy, provenance, allowlist, SCA scan
Destructive action	Cleanup command deletes data or branches	Sandbox, dry-run, human confirmation, backups
Cross-task contamination	Agent reuses confidential context in another repo	Isolated sessions, tenant boundaries, retention policy
Evidence tampering	Agent weakens tests to make CI green	Protected tests, diff rules, mutation, reviewer ownership

Security requirements as EARS

- SEC-AI-01: WHEN an agent requests a tool that can modify an external system, the host shall display the target, operation, and affected resource before execution unless a reviewed policy explicitly permits that exact operation.
- SEC-AI-02: WHILE a task is classified as high risk, the agent shall operate without production credentials and shall not merge or deploy without named human approval.
- SEC-AI-03: IF retrieved content contains instructions unrelated to the user-approved task, THEN the agent shall treat those instructions as untrusted data and shall not expand permissions.

Governance connection: NIST’s AI RMF and GenAI profile emphasize governance, mapping, measurement, and management; NIST’s SSDF profile adds secure development practices for AI systems. OWASP provides concrete threat categories for generative and agentic applications.

EVIDENCE

- AI-14 · NIST Generative AI Profile
- AI-15 · NIST SP 800-218A
- AI-16 · OWASP GenAI Security Project
- AI-06 · Model Context Protocol specification

AI-ERA SDD · EVALUATION

Evaluate agents on conformance, not code volume

The unit of evaluation is the governed task: did the agent preserve intent, respect boundaries, produce independent evidence, and reduce total human effort?

100%

REQ. COVERAGE

0

SCOPE BREACHES

92%

MUTANTS KILLED

18m

REVIEW TIME

Conformance scorecard

MEASURE	DEFINITION
Requirement coverage	In-scope requirement IDs with named executable evidence ÷ all in-scope IDs
First-pass conformance	Tasks accepted without spec- or behavior-related rework ÷ completed tasks
Scope adherence	Declared-path and tool-policy violations per task
Mutation detection	Meaningful seeded semantic changes detected ÷ applicable mutations
Repair count	Builder-review cycles after the first submitted patch
Reviewer effort	Active minutes to understand, validate, and approve or reject the patch
Cost and latency	Model/tool spend and elapsed task time, including waiting

Example verdict for T3

Coverage: EARS-1, EARS-2, EARS-3 all mapped to named tests.

Scope: two files changed; both allowed; no new dependency.

Mutations: 11/12 killed; surviving mutation changes event ordering and belongs to T4.

Review: 18 active minutes; one clarification on percentage-fee products.

Verdict: CONFORMS for T3. Record the survivor as a future contract obligation rather than hiding it.

Avoid the wrong target: lines of code, prompts sent, or tasks “completed” can all increase while delivery value falls. Measure outcomes, evidence quality, and human effort together.

AI-ERA SDD · EVIDENCE

What the productivity evidence actually says

Research results differ because they measure different tasks, users, repositories, tools, and definitions of success. Treat productivity as an empirical question, not a product slogan.

Fixed-task experiment

55.8% faster

COPILOT TREATMENT GROUP

Peng and colleagues asked developers to implement a JavaScript HTTP server. The treatment group completed the defined task faster. The result supports productivity benefit in that controlled task; it does not establish the same effect for every codebase, user, quality bar, or agentic workflow.

Experienced maintainers

19% longer

EARLY-2025 METR RCT

Experienced open-source developers worked on real issues in familiar, mature repositories. AI-allowed tasks took longer in that sample. METR explicitly cautioned against generalizing beyond the studied setting.

Why both can be true

- Task clarity: a fixed implementation task differs from ambiguous product work.
- Repository familiarity: experts already know implicit constraints that the agent must rediscover.
- Quality bar: production review includes style, compatibility, documentation, and hidden expectations.
- Interaction mode: autocomplete, chat, and long-running agents impose different waiting and review costs.
- Selection: developers choose tasks where AI helps and avoid tasks where it does not.

What changed by 2026

METR's later experiment reported weak evidence suggesting improved speedup, but the organization changed its design because participant and task selection made the central estimate unreliable. The responsible conclusion is not "AI is always faster" or "AI is slower." It is that productivity is conditional and must be measured in the team's actual task distribution.

SDD hypothesis: better specifications should increase AI usefulness by reducing hidden requirements and review rework. That is testable: baseline lead time, first-pass verification, reviewer effort, defect escape, and cost before the pilot.

EVIDENCE

AI-09 · Copilot productivity experiment

AI-10 · METR early-2025 developer RCT

AI-11 · METR 2026 experiment update

AI-07 · SWE-bench

AI-ERA SDD · GOVERNANCE

Governance: match autonomy to consequence

Governance should change the allowed actions and evidence burden according to risk; it should not be a generic approval meeting applied equally to every task.

Risk-tiered delegation

TIER	EXAMPLES	AGENT AUTONOMY	REQUIRED APPROVAL
Low	Formatting, docs, deterministic refactor, test data	May edit and open PR in sandbox	Normal code review
Medium	Feature logic, schema-compatible API change	Bounded task packet, no production access	Owner + independent tests
High	Payments, auth, privacy, destructive migration	Research and draft only; protected execution	Named domain, security, and change owners
Prohibited	Unreviewed secret export, bypassing controls, autonomous prod deletion	Blocked by policy	Not delegable as specified

NIST-aligned control cycle



ARTIFACT OWNER
Approves requirement and decision authority.

AGENT OPERATOR
Configures context, tools, environment, and task.

INDEPENDENT REVIEWER
Checks evidence, exceptions, and residual risk.

Audit trail: retain model/tool version where available, task packet, permissions, source references, diff, commands, test results, approvals, and exceptions. Do not retain sensitive prompts or data without a purpose and policy.

AI-ERA SDD · ADOPTION

A 30/60/90-day AI-ready SDD rollout

Pilot the smallest complete control loop, automate the repeated gates, and scale only after measuring both delivery outcomes and human review cost.

First 30 days

Goal: create one governed pilot.

- Select a medium-risk, testable feature.
- Define the feature folder and stable IDs.
- Create one task-packet template.
- Restrict paths, tools, network, and secrets.
- Capture baseline reviewer effort and lead time.

Exit: one end-to-end verified slice with reproducible evidence.

Days 31–60

Goal: make the workflow repeatable.

- Add spec and contract review checklists.
- Automate path, test, lint, and security gates.
- Separate builder and verifier roles on critical rules.
- Introduce requirement and mutation coverage.
- Document exceptions as ADRs.

Exit: two sprints of comparable data and no hidden scope breaches.

Days 61–90

Goal: scale only what has evidence.

- Define risk tiers and approved agent profiles.
- Curate skills, MCP servers, and tool allowlists.
- Add incident and rollback playbooks.
- Train reviewers on conformance, not style alone.
- Review cost, quality, and human experience together.

Exit: a decision to expand, revise, or stop based on measured outcomes.

Minimum viable AI policy: approved artifacts outrank prompts; one active task per agent; least privilege by default; no silent requirement changes; independent evidence for material behavior; human approval for external or destructive action; and every exception recorded.

Use AI to reduce the cost of transformation. Do not let it become the unreviewed source of domain truth, the author of its own exam, and the judge of its own answer.

EVIDENCE

[AI-01 · GitHub Spec Kit](#)
[AI-02 · Kiro Specs](#)
[AI-03 · OpenAI Codex](#)
[AI-14 · NIST Generative AI Profile](#)
[AI-15 · NIST SP 800-218A](#)

PART XIII

PLT-4821: the end-to-end research case

Follow one partial-refund feature from ambiguous Jira ticket through domain decisions, EARS, contracts, architecture, task execution, AI controls, and verification evidence.

WHAT THIS PART ANSWERS

Which assumptions were hidden? How does each artifact resolve one? What changes when an agent performs the implementation work?

USE IT WHEN

You want a concrete reference implementation of the full method rather than an isolated technique.

BASED ON SUPPLIED WALKTHROUGH

The incoming ticket and ambiguity inventory

list every phrase that permits multiple defensible interpretations before anybody chooses one in code.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first list every phrase that permits multiple defensible interpretations before anybody chooses one in code, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

list every phrase that permits multiple defensible interpretations before anybody chooses one in code. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

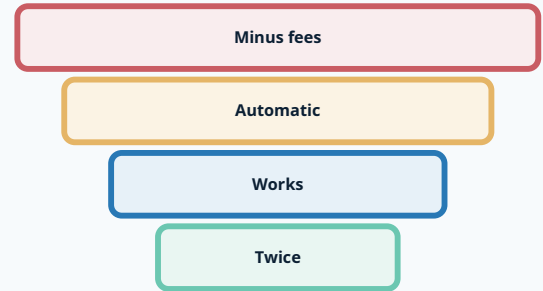
VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

WHEN an event is cancelled,
THE Refunds service SHALL create exactly one
refund
for ticket price minus frozen service fees
within five business days.

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R7] Requirements quality mapping

[R8] NaPIRE agile RE survey

EXPANDED CASE

Stakeholder questions and decision ownership

route fee policy, timing, duplicate behavior, rounding, partiality, and notification questions to named authorities.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must route fee policy, timing, duplicate behavior, rounding, partiality, and notification questions to named authorities; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

route fee policy, timing, duplicate behavior, rounding, partiality, and notification questions to named authorities. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

PO	Finance	Support	Security
Engineering	PO	Finance	Support
Security	Engineering	PO	Finance
Support	Security	Engineering	PO

WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 ->
TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R5] ISO/IEC/IEEE 29148:2018

[R15] Evans, Domain-Driven Design

BASED ON SUPPLIED WALKTHROUGH

The approved EARS specification

state atomic requirements for minor units, cancellation trigger, duplicate states, expired tokens, timing, and optional line-item refunds.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To state atomic requirements for minor units, cancellation trigger, duplicate states, expired tokens, timing, and optional line-item refunds, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

state atomic requirements for minor units, cancellation trigger, duplicate states, expired tokens, timing, and optional line-item refunds. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

WHEN an event is cancelled,
THE Refunds service SHALL create exactly one
refund
for ticket price minus frozen service fees
within five business days.

FAILURE MODE / LIMIT

A well-formed sentence can still encode the wrong policy, omit an actor, or hide an unmeasurable quality boundary. Syntax helps expose gaps; it does not supply domain truth.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R6] Mavin et al. 2009 EARS

[R5] ISO/IEC/IEEE 29148:2018

EXPANDED CASE

Glossary and ubiquitous language

define service fee, payment-provider fee, frozen order snapshot, refund, refund request, issued, pending, and line item.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to define service fee, payment-provider fee, frozen order snapshot, refund, refund request, issued, pending, and line item without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

define service fee, payment-provider fee, frozen order snapshot, refund, refund request, issued, pending, and line item. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

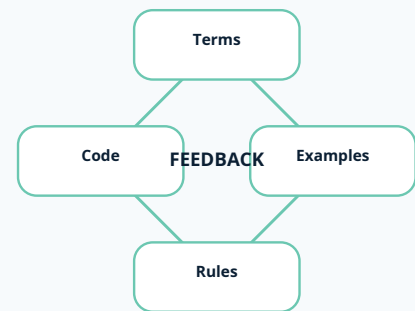
VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R5] ISO/IEC/IEEE 29148:2018

BASED ON SUPPLIED WALKTHROUGH

The bounded-context map

assign totals to Ordering, refund policy to Refunds, settlement to Payments, and outbound messages to Notification.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to assign totals to Ordering, refund policy to Refunds, settlement to Payments, and outbound messages to Notification. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

assign totals to Ordering, refund policy to Refunds, settlement to Payments, and outbound messages to Notification. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

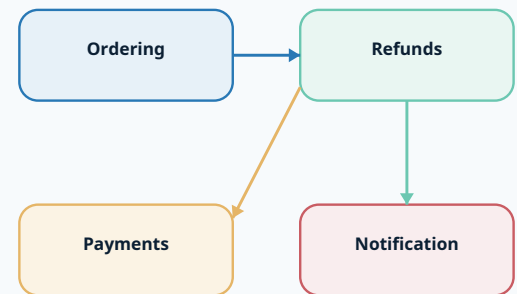
VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

```
ACTIVE_TASK: T3
ALLOWED_PATHS: src/refunds/**, tests/refunds/**
REQUIRED_GATES: unit, BDD, OpenAPI
STOP_ON: ambiguity, contract conflict, scope breach
```

FAILURE MODE / LIMIT

A prompt is advisory. Without executable gates, repository permissions, and independent verification, an agent can satisfy the apparent task while violating scope or hidden intent.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R16] Vernon, Implementing DDD

BASED ON SUPPLIED WALKTHROUGH

Event storm and policy discovery

walk EventCancelled through RefundRequested, RefundCalculated, RefundIssued, BuyerNotified, and failure or retry branches.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first walk EventCancelled through RefundRequested, RefundCalculated, RefundIssued, BuyerNotified, and failure or retry branches, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

walk EventCancelled through RefundRequested, RefundCalculated, RefundIssued, BuyerNotified, and failure or retry branches. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

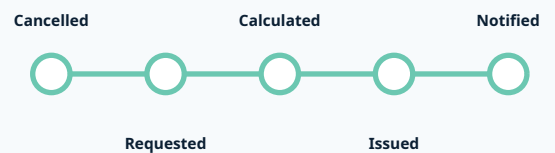
VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Refund aggregate

- identity: orderId
- invariant: at most one active refund
- value object: Money(minorUnits, currency)
- event: RefundIssued

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R16] Vernon, Implementing DDD

EXPANDED CASE

Refund aggregate state machine

protect at-most-one active refund while defining legal transitions, duplicate responses, retry behavior, and terminal states.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must protect at-most-one active refund while defining legal transitions, duplicate responses, retry behavior, and terminal states; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

protect at-most-one active refund while defining legal transitions, duplicate responses, retry behavior, and terminal states. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

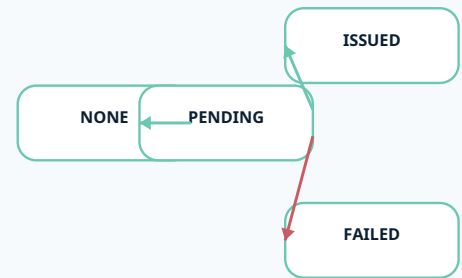
ARTIFACT

VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Refund aggregate

- identity: orderId
- invariant: at most one active refund
- value object: Money(minorUnits, currency)
- event: RefundIssued

FAILURE MODE / LIMIT

Treating DDD as a folder convention or a collection of tactical classes creates ceremony without clarifying ownership, language, or consistency boundaries.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R24] OpenAPI Specification

EXPANDED CASE

HTTP API contract

define the refund command, idempotency identity, success representation, 401, 404, 409, 422, and 503 semantics.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To define the refund command, idempotency identity, success representation, 401, 404, 409, 422, and 503 semantics, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

define the refund command, idempotency identity, success representation, 401, 404, 409, 422, and 503 semantics. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

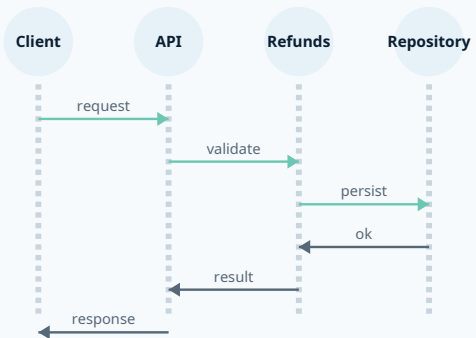
VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

```
POST /orders/{orderId}/refunds
201 RefundIssued
409 AlreadyRefunded
401 InvalidToken
Idempotency-Key: required
```

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R24] OpenAPI Specification

[R44] OWASP ASVS

EXPANDED CASE

Event contract and outbox

version RefundIssued, specify business identifiers and money fields, and ensure publication is atomic with the aggregate change.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to version RefundIssued, specify business identifiers and money fields, and ensure publication is atomic with the aggregate change without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

version RefundIssued, specify business identifiers and money fields, and ensure publication is atomic with the aggregate change. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

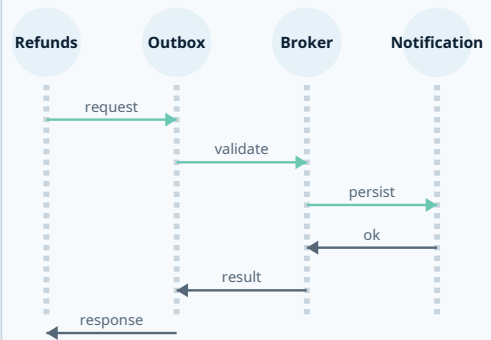
VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

```
POST /orders/{orderId}/refunds
201 RefundIssued
409 AlreadyRefunded
401 InvalidToken
Idempotency-Key: required
```

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R24] OpenAPI Specification

[R46] Google SRE Workbook

BASED ON SUPPLIED WALKTHROUGH

Vertical task slicing

derive small slices from one requirement and one contract each, declare dependencies and allowed paths, and reserve verification capacity.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to derive small slices from one requirement and one contract each, declare dependencies and allowed paths, and reserve verification capacity. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

derive small slices from one requirement and one contract each, declare dependencies and allowed paths, and reserve verification capacity. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

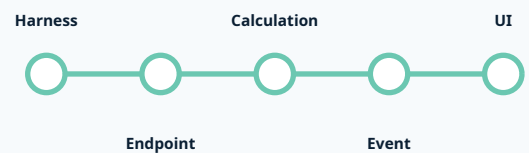
VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R1] GitHub Spec Kit documentation

[R50] Little's Law

BASED ON SUPPLIED WALKTHROUGH

Hexagonal implementation design

keep Money and RefundCalculator pure, orchestrate state in RefundsService, and place HTTP, storage, PSP, and events behind ports.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first keep Money and RefundCalculator pure, orchestrate state in RefundsService, and place HTTP, storage, PSP, and events behind ports, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

keep Money and RefundCalculator pure, orchestrate state in RefundsService, and place HTTP, storage, PSP, and events behind ports. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

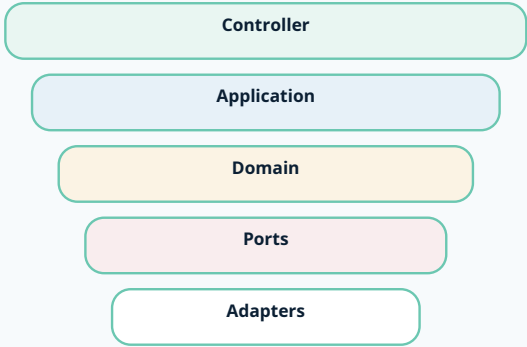
EVIDENCE TO RETAIN

- OWNER
- ARTIFACT
- VERIFICATION
- REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

- [R23] Cockburn, Hexagonal Architecture
- [R16] Vernon, Implementing DDD

EXPANDED CASE

BDD feature and examples

bind EARS criteria to readable cancellation, duplicate, frozen-fee, expired-token, and delayed-provider scenarios.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must bind EARS criteria to readable cancellation, duplicate, frozen-fee, expired-token, and delayed-provider scenarios; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

bind EARS criteria to readable cancellation, duplicate, frozen-fee, expired-token, and delayed-provider scenarios. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Feature: Cancelled-event refund
Scenario: First refund request
Given a paid order with frozen fees
When the event is cancelled
Then one refund is issued
And duplicate requests return 409

FAILURE MODE / LIMIT

Automating examples too early can freeze misunderstanding. UI-level steps, duplicated setup, and implementation language turn living documentation into brittle test scripts.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R11] Cucumber Gherkin reference

[R13] Adzic, Specification by Example

EXPANDED CASE

TDD unit and service tests

start with a failing calculation test, triangulate rounding and currency, then add state, repository, and event tests.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To start with a failing calculation test, triangulate rounding and currency, then add state, repository, and event tests, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

start with a failing calculation test, triangulate rounding and currency, then add state, repository, and event tests. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

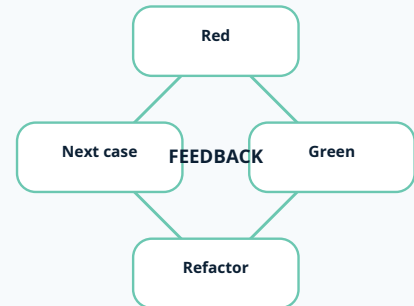
ARTIFACT

VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

```
it('refunds price minus service fees', async () => {
  const order = paidOrder({ total: 12900, fees: 900 });
  expect(await refund(order.id)).toEqual(12000);
});
```

FAILURE MODE / LIMIT

Green tests can coexist with wrong intent when tests are derived from the implementation or share the same mistaken assumption as the code. Independent derivation and mutation checks matter.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R18] Beck, TDD by Example

[R20] Rafique and Misic meta-analysis

EXPANDED CASE

Idempotency under concurrency

prove that simultaneous and retried cancellation events create no more than one financial effect and return stable outcomes.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to prove that simultaneous and retried cancellation events create no more than one financial effect and return stable outcomes without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

prove that simultaneous and retried cancellation events create no more than one financial effect and return stable outcomes. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

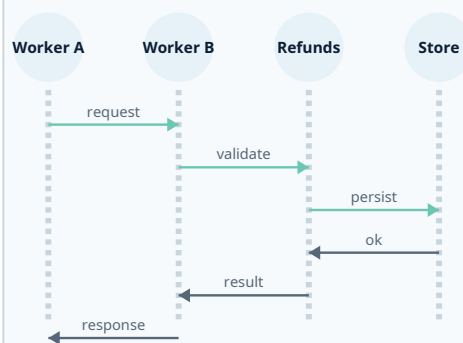
VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R24] OpenAPI Specification

[R46] Google SRE Workbook

EXPANDED CASE

Error handling and security

differentiate authentication, authorization, invalid state, provider outage, policy rejection, and internal failure while preserving audit evidence.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to differentiate authentication, authorization, invalid state, provider outage, policy rejection, and internal failure while preserving audit evidence. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

differentiate authentication, authorization, invalid state, provider outage, policy rejection, and internal failure while preserving audit evidence. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1
- Start from the observable business problem and ambiguity list.
- 2
- Resolve policy, domain, data, timing, and failure decisions.
- 3
- Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4
- Implement in small slices with failing tests and hard gates.
- 5
- Verify behavior in tests and in operational observations.

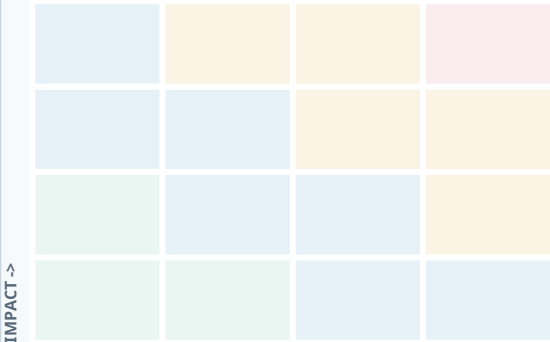
EVIDENCE TO RETAIN

- OWNER
- ARTIFACT
- VERIFICATION
- REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

- [R44] OWASP ASVS
- [R24] OpenAPI Specification

EXPANDED CASE

Verification and traceability matrix

show criterion-by-criterion links to scenarios, tests, code, ADRs, and evidence, including documented exclusions.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first show criterion-by-criterion links to scenarios, tests, code, ADRs, and evidence, including documented exclusions, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

show criterion-by-criterion links to scenarios, tests, code, ADRs, and evidence, including documented exclusions. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

- OWNER
- ARTIFACT
- VERIFICATION
- REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

EARS	BDD	Unit	Contract
Ops	EARS	BDD	Unit
Contract	Ops	EARS	BDD
Unit	Contract	Ops	EARS

WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

- [R26] Gotel and Finkelstein 1994
- [R27] Requirements traceability book

EXPANDED CASE

Operational rollout and measurement

release behind a flag, monitor issuance and reconciliation, define rollback, and label the example dashboard as illustrative rather than empirical proof.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must release behind a flag, monitor issuance and reconciliation, define rollback, and label the example dashboard as illustrative rather than empirical proof; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

release behind a flag, monitor issuance and reconciliation, define rollback, and label the example dashboard as illustrative rather than empirical proof. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

Lead time

4.2 d

Failure

8/wk

Recovery

6%

Reconciliation

40 min

WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 ->
TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R36] DORA software delivery metrics

[R39] SPACE framework

[R48] OpenFeature specification

PART XIV

Additional case studies

Test the method against different domains, failure modes, data sensitivity, operational constraints, and change patterns.

WHAT THIS PART ANSWERS

Which elements generalize? Which controls must vary by risk, regulation, legacy constraints, or interaction style?

USE IT WHEN

You need to avoid overfitting SDD to the refund example or to greenfield web development.

RESEARCH EXPANSION

Healthcare appointment rescheduling

specify eligibility, patient identity, clinician availability, consent, notification, and audit behavior for appointment changes.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first specify eligibility, patient identity, clinician availability, consent, notification, and audit behavior for appointment changes, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

specify eligibility, patient identity, clinician availability, consent, notification, and audit behavior for appointment changes. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

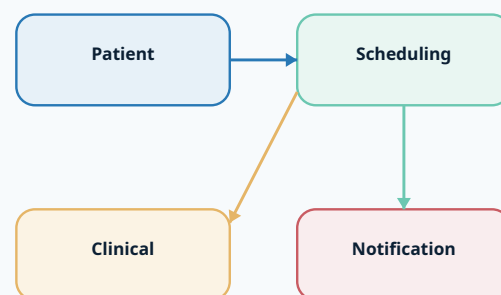
VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R5] ISO/IEC/IEEE 29148:2018

[R44] OWASP ASVS

RESEARCH EXPANSION

Healthcare EARS and safety constraints

write event, state, and unwanted-behavior requirements for locked clinical windows, emergency overrides, and duplicated notifications.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must write event, state, and unwanted-behavior requirements for locked clinical windows, emergency overrides, and duplicated notifications; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

write event, state, and unwanted-behavior requirements for locked clinical windows, emergency overrides, and duplicated notifications. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

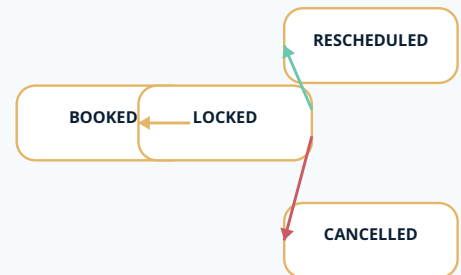
VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

WHEN an event is cancelled,
THE Refunds service SHALL create exactly one
refund
for ticket price minus frozen service fees
within five business days.

FAILURE MODE / LIMIT

A well-formed sentence can still encode the wrong policy, omit an actor, or hide an unmeasurable quality boundary. Syntax helps expose gaps; it does not supply domain truth.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R6] Mavin et al. 2009 EARS

[R5] ISO/IEC/IEEE 29148:2018

RESEARCH EXPANSION

Healthcare BDD and evidence

use examples to distinguish patient request, clinician cancellation, emergency override, and no-capacity outcomes.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To use examples to distinguish patient request, clinician cancellation, emergency override, and no-capacity outcomes, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

use examples to distinguish patient request, clinician cancellation, emergency override, and no-capacity outcomes. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

Actor	Window	Capacity	Outcome
Actor	Window	Capacity	Outcome
Actor	Window	Capacity	Outcome
Actor	Window	Capacity	Outcome

WORKED EXAMPLE

Feature: Cancelled-event refund
 Scenario: First refund request
 Given a paid order with frozen fees
 When the event is cancelled
 Then one refund is issued
 And duplicate requests return 409

FAILURE MODE / LIMIT

Automating examples too early can freeze misunderstanding. UI-level steps, duplicated setup, and implementation language turn living documentation into brittle test scripts.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R11] Cucumber Gherkin reference

[R13] Adzic, Specification by Example

RESEARCH EXPANSION

Fintech transfer initiation

define account authority, balance policy, limits, sanctions screening, idempotency, settlement state, and auditability.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to define account authority, balance policy, limits, sanctions screening, idempotency, settlement state, and auditability without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

define account authority, balance policy, limits, sanctions screening, idempotency, settlement state, and auditability. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R5] ISO/IEC/IEEE 29148:2018

[R44] OWASP ASVS

RESEARCH EXPANSION

Fintech idempotency and ledger integrity

separate command identity from business transfer identity and prove double-entry invariants under retries.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to separate command identity from business transfer identity and prove double-entry invariants under retries. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

separate command identity from business transfer identity and prove double-entry invariants under retries. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

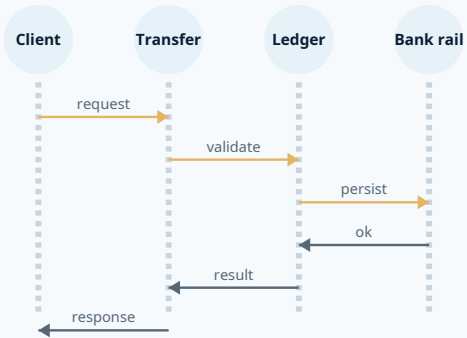
VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R24] OpenAPI Specification

[R51] Jackson, Software Abstractions

RESEARCH EXPANSION

Retail inventory reservation

model sellable stock, reservation, allocation, release, oversell policy, and channel-specific authority.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first model sellable stock, reservation, allocation, release, oversell policy, and channel-specific authority, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

model sellable stock, reservation, allocation, release, oversell policy, and channel-specific authority. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

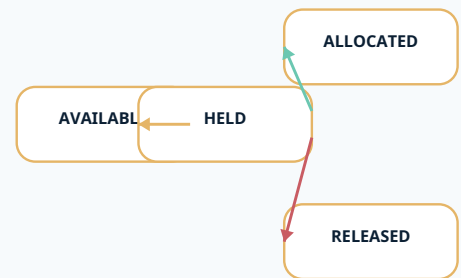
ARTIFACT

VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

```
ACTIVE_TASK: T3
ALLOWED_PATHS: src/refunds/**, tests/refunds/**
REQUIRED_GATES: unit, BDD, OpenAPI
STOP_ON: ambiguity, contract conflict, scope breach
```

FAILURE MODE / LIMIT

A prompt is advisory. Without executable gates, repository permissions, and independent verification, an agent can satisfy the apparent task while violating scope or hidden intent.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R5] ISO/IEC/IEEE 29148:2018

RESEARCH EXPANSION

Retail concurrency and eventual consistency

test simultaneous carts, delayed stock events, partial allocation, and reconciliation without pretending every boundary is strongly consistent.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must test simultaneous carts, delayed stock events, partial allocation, and reconciliation without pretending every boundary is strongly consistent; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

test simultaneous carts, delayed stock events, partial allocation, and reconciliation without pretending every boundary is strongly consistent. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

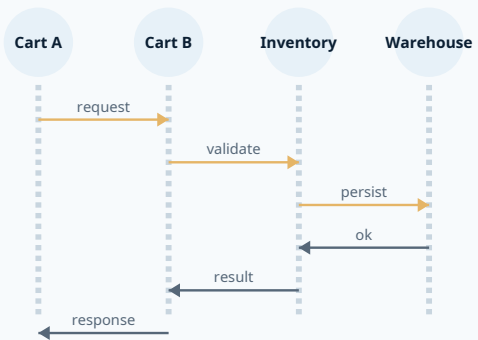
VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

```

ACTIVE_TASK: T3
ALLOWED_PATHS: src/refunds/**, tests/refunds/**
REQUIRED_GATES: unit, BDD, OpenAPI
STOP_ON: ambiguity, contract conflict, scope breach
  
```

FAILURE MODE / LIMIT

A prompt is advisory. Without executable gates, repository permissions, and independent verification, an agent can satisfy the apparent task while violating scope or hidden intent.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R46] Google SRE Workbook

[R24] OpenAPI Specification

RESEARCH EXPANSION

SaaS entitlement changes

specify plan, feature, seat, tenant, billing period, grace window, downgrade, and revocation semantics.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To specify plan, feature, seat, tenant, billing period, grace window, downgrade, and revocation semantics, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

specify plan, feature, seat, tenant, billing period, grace window, downgrade, and revocation semantics. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

Tenant

Subscription

Entitlement

Feature

WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 ->
TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R5] ISO/IEC/IEEE 29148:2018

RESEARCH EXPANSION

SaaS bounded contexts

separate Catalog, Billing, Subscription, Identity, and Entitlement models so the word plan is not overloaded.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to separate Catalog, Billing, Subscription, Identity, and Entitlement models so the word plan is not overloaded without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

separate Catalog, Billing, Subscription, Identity, and Entitlement models so the word plan is not overloaded. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

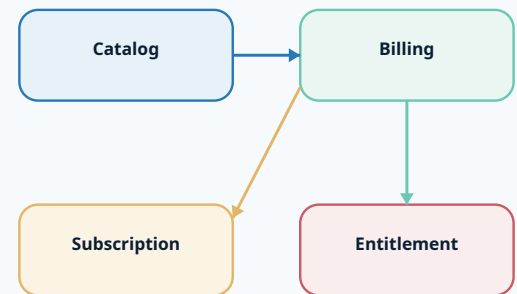
VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

```

ACTIVE_TASK: T3
ALLOWED_PATHS: src/refunds/**, tests/refunds/**
REQUIRED_GATES: unit, BDD, OpenAPI
STOP_ON: ambiguity, contract conflict, scope breach
  
```

FAILURE MODE / LIMIT

A prompt is advisory. Without executable gates, repository permissions, and independent verification, an agent can satisfy the apparent task while violating scope or hidden intent.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R15] Evans, Domain-Driven Design

[R16] Vernon, Implementing DDD

RESEARCH EXPANSION

Logistics tracking events

define event identity, location authority, ordering, late arrival, correction, and customer-facing status.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to define event identity, location authority, ordering, late arrival, correction, and customer-facing status. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

define event identity, location authority, ordering, late arrival, correction, and customer-facing status. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

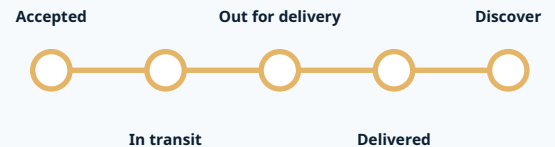
ARTIFACT

VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE[\[R24\] OpenAPI Specification](#)[\[R46\] Google SRE Workbook](#)

RESEARCH EXPANSION

Logistics event-contract evolution

version producer and consumer expectations while preserving replay and handling unknown event fields safely.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first version producer and consumer expectations while preserving replay and handling unknown event fields safely, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

version producer and consumer expectations while preserving replay and handling unknown event fields safely. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

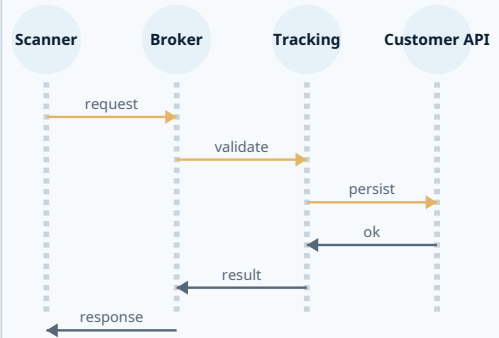
VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

```

POST /orders/{orderId}/refunds
201 RefundIssued
409 AlreadyRefunded
401 InvalidToken
Idempotency-Key: required
  
```

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R24] OpenAPI Specification

[R25] Pact consumer-driven contracts

RESEARCH EXPANSION

Brownfield legacy migration

specify preserved behavior, create characterization tests, introduce an anti-corruption layer, and migrate slices behind a stable contract.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must specify preserved behavior, create characterization tests, introduce an anti-corruption layer, and migrate slices behind a stable contract; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

specify preserved behavior, create characterization tests, introduce an anti-corruption layer, and migrate slices behind a stable contract. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

OWNER

ARTIFACT

VERIFICATION

REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R2] GitHub, What is SDD?

[R3] GitHub, Spec Persistence Models

[R23] Cockburn, Hexagonal Architecture

RESEARCH EXPANSION

Cross-case comparison

compare domain risk, requirement style, consistency boundary, dominant contracts, test emphasis, and operational evidence across cases.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To compare domain risk, requirement style, consistency boundary, dominant contracts, test emphasis, and operational evidence across cases, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

compare domain risk, requirement style, consistency boundary, dominant contracts, test emphasis, and operational evidence across cases. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Start from the observable business problem and ambiguity list.
- 2 Resolve policy, domain, data, timing, and failure decisions.
- 3 Produce requirements, scenarios, contracts, tasks, and ADRs.
- 4 Implement in small slices with failing tests and hard gates.
- 5 Verify behavior in tests and in operational observations.

EVIDENCE TO RETAIN

- OWNER
- ARTIFACT
- VERIFICATION
- REVISION TRIGGER

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

Risk	Boundary	Contract	Evidence
Risk	Boundary	Contract	Evidence
Risk	Boundary	Contract	Evidence
Risk	Boundary	Contract	Evidence

WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- The artifact has no owner or approval state.
- Downstream work does not cite it.
- No independent check can contradict it.
- The artifact is never revised after production learning.

DECISION RULE

Reopen the specification instead of patching code silently whenever evidence exposes a materially different interpretation.

EVIDENCE

[R5] ISO/IEC/IEEE 29148:2018

[R17] DDD systematic review 2025

[R36] DORA software delivery metrics

Specification mutation research

Move mutation upstream to evaluate whether semantic changes in requirements are detected and propagated into plans, contracts, tests, code, and evidence.

WHAT THIS PART ANSWERS

Which mutation operators represent realistic ambiguity? Where does semantic change disappear? How should mutation score be interpreted?

USE IT WHEN

You are designing a research study, evaluating traceability, or testing the independence of an AI-generated evidence chain.

RESEARCH EXPANSION

Goals of specification mutation

measure detection strength, semantic propagation, over-propagation, stale links, and the independence of generated tests.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first measure detection strength, semantic propagation, over-propagation, stale links, and the independence of generated tests, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

measure detection strength, semantic propagation, over-propagation, stale links, and the independence of generated tests. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Choose a realistic semantic fault and define the operator.
- 2 Generate one controlled mutant while preserving syntax.
- 3 Run the full artifact and verification pipeline independently.
- 4 Classify detection, propagation, over-change, and equivalence.
- 5 Retain artifacts and uncertainty for replication and review.

EVIDENCE TO RETAIN

- OPERATOR
- MUTANT
- DETECTIONS
- ADJUDICATION

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Original: duplicate refund -> 409
Mutant: duplicate refund -> 200
Expected detector: BDD scenario + contract test
Result: killed only if both behavior and API shape fail

FAILURE MODE / LIMIT

Equivalent or unrealistic mutants distort conclusions. Operators must represent plausible semantic mistakes, and each surviving mutant needs expert adjudication.

WARNING SIGNS

- Mutants are grammatical but implausible in the domain.
- Equivalent mutants are counted as process failures.
- Generated tests share the same altered interpretation.
- Only kill rate is reported, without propagation precision or effort.

DECISION RULE

Do not interpret a surviving mutant until an expert has ruled on realism, equivalence, and the expected propagation path.

EVIDENCE

- [R28] Jia and Harman mutation survey
- [R30] Formal RE and LLM roadmap

RESEARCH EXPANSION

A mutation-operator catalog

classify value, boundary, trigger, state, temporal, actor, modality, negation, error, optionality, and omission mutations.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must classify value, boundary, trigger, state, temporal, actor, modality, negation, error, optionality, and omission mutations; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

classify value, boundary, trigger, state, temporal, actor, modality, negation, error, optionality, and omission mutations. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Choose a realistic semantic fault and define the operator.
- 2 Generate one controlled mutant while preserving syntax.
- 3 Run the full artifact and verification pipeline independently.
- 4 Classify detection, propagation, over-change, and equivalence.
- 5 Retain artifacts and uncertainty for replication and review.

EVIDENCE TO RETAIN

- OPERATOR
- MUTANT
- DETECTIONS
- ADJUDICATION

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

Value	Boundary	State	Time
Error	Omit	Value	Boundary
State	Time	Error	Omit
Value	Boundary	State	Time

WORKED EXAMPLE

Original: duplicate refund -> 409
Mutant: duplicate refund -> 200
Expected detector: BDD scenario + contract test
Result: killed only if both behavior and API shape fail

FAILURE MODE / LIMIT

Equivalent or unrealistic mutants distort conclusions. Operators must represent plausible semantic mistakes, and each surviving mutant needs expert adjudication.

WARNING SIGNS

- Mutants are grammatical but implausible in the domain.
- Equivalent mutants are counted as process failures.
- Generated tests share the same altered interpretation.
- Only kill rate is reported, without propagation precision or effort.

DECISION RULE

Do not interpret a surviving mutant until an expert has ruled on realism, equivalence, and the expected propagation path.

EVIDENCE

- [R28] Jia and Harman mutation survey
- [R29] Papadakis et al. mutation advances

RESEARCH EXPANSION

EARS requirement mutations

change a trigger, threshold, state qualifier, system response, or exception while retaining grammatical validity.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To change a trigger, threshold, state qualifier, system response, or exception while retaining grammatical validity, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

change a trigger, threshold, state qualifier, system response, or exception while retaining grammatical validity. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Choose a realistic semantic fault and define the operator.
- 2 Generate one controlled mutant while preserving syntax.
- 3 Run the full artifact and verification pipeline independently.
- 4 Classify detection, propagation, over-change, and equivalence.
- 5 Retain artifacts and uncertainty for replication and review.

EVIDENCE TO RETAIN

- OPERATOR
- MUTANT
- DETECTIONS
- ADJUDICATION

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

WHEN an event is cancelled,
THE Refunds service SHALL create exactly one
refund
for ticket price minus frozen service fees
within five business days.

FAILURE MODE / LIMIT

A well-formed sentence can still encode the wrong policy, omit an actor, or hide an unmeasurable quality boundary. Syntax helps expose gaps; it does not supply domain truth.

WARNING SIGNS

- Mutants are grammatical but implausible in the domain.
- Equivalent mutants are counted as process failures.
- Generated tests share the same altered interpretation.
- Only kill rate is reported, without propagation precision or effort.

DECISION RULE

Do not interpret a surviving mutant until an expert has ruled on realism, equivalence, and the expected propagation path.

EVIDENCE

[R6] Mavin et al. 2009 EARS

[R28] Jia and Harman mutation survey

RESEARCH EXPANSION

Gherkin scenario mutations

alter Given state, When action, Then outcome, example row, tag, or scenario omission and observe whether automation exposes the difference.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to alter Given state, When action, Then outcome, example row, tag, or scenario omission and observe whether automation exposes the difference without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

alter Given state, When action, Then outcome, example row, tag, or scenario omission and observe whether automation exposes the difference. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Choose a realistic semantic fault and define the operator.
- 2 Generate one controlled mutant while preserving syntax.
- 3 Run the full artifact and verification pipeline independently.
- 4 Classify detection, propagation, over-change, and equivalence.
- 5 Retain artifacts and uncertainty for replication and review.

EVIDENCE TO RETAIN

OPERATOR

MUTANT

DETECTIONS

ADJUDICATION

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Feature: Cancelled-event refund
Scenario: First refund request
Given a paid order with frozen fees
When the event is cancelled
Then one refund is issued
And duplicate requests return 409

FAILURE MODE / LIMIT

Automating examples too early can freeze misunderstanding. UI-level steps, duplicated setup, and implementation language turn living documentation into brittle test scripts.

WARNING SIGNS

- Mutants are grammatical but implausible in the domain.
- Equivalent mutants are counted as process failures.
- Generated tests share the same altered interpretation.
- Only kill rate is reported, without propagation precision or effort.

DECISION RULE

Do not interpret a surviving mutant until an expert has ruled on realism, equivalence, and the expected propagation path.

EVIDENCE

[R11] Cucumber Gherkin reference

[R28] Jia and Harman mutation survey

RESEARCH EXPANSION

Traceability propagation experiments

track whether a spec mutation changes the expected plan, tasks, contracts, tests, code, and evidence without unrelated churn.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to track whether a spec mutation changes the expected plan, tasks, contracts, tests, code, and evidence without unrelated churn. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

track whether a spec mutation changes the expected plan, tasks, contracts, tests, code, and evidence without unrelated churn. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Choose a realistic semantic fault and define the operator.
- 2 Generate one controlled mutant while preserving syntax.
- 3 Run the full artifact and verification pipeline independently.
- 4 Classify detection, propagation, over-change, and equivalence.
- 5 Retain artifacts and uncertainty for replication and review.

EVIDENCE TO RETAIN

OPERATOR

MUTANT

DETECTIONS

ADJUDICATION

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- Mutants are grammatical but implausible in the domain.
- Equivalent mutants are counted as process failures.
- Generated tests share the same altered interpretation.
- Only kill rate is reported, without propagation precision or effort.

DECISION RULE

Do not interpret a surviving mutant until an expert has ruled on realism, equivalence, and the expected propagation path.

EVIDENCE

[R26] Gotel and Finkelstein 1994

[R27] Requirements traceability book

[R30] Formal RE and LLM roadmap

RESEARCH EXPANSION

Equivalent and invalid mutants

distinguish semantically neutral reformulation, impossible domain states, duplicate mutants, and underspecified equivalence.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first distinguish semantically neutral reformulation, impossible domain states, duplicate mutants, and underspecified equivalence, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

distinguish semantically neutral reformulation, impossible domain states, duplicate mutants, and underspecified equivalence. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1
- Choose a realistic semantic fault and define the operator.
- 2
- Generate one controlled mutant while preserving syntax.
- 3
- Run the full artifact and verification pipeline independently.
- 4
- Classify detection, propagation, over-change, and equivalence.
- 5
- Retain artifacts and uncertainty for replication and review.

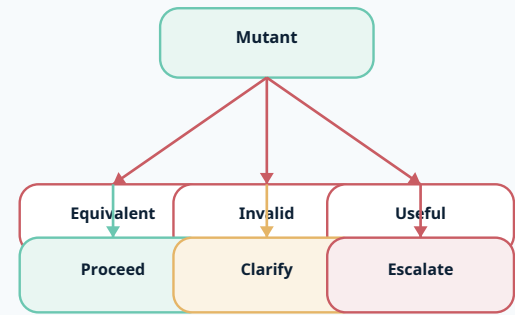
EVIDENCE TO RETAIN

- OPERATOR
- MUTANT
- DETECTIONS
- ADJUDICATION

REVIEW QUESTIONS

-
- What uncertainty is removed before implementation?
-
- Which artifact is authoritative, and who approves it?
-
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

-
- Mutants are grammatical but implausible in the domain.
-
- Equivalent mutants are counted as process failures.
-
- Generated tests share the same altered interpretation.
-
- Only kill rate is reported, without propagation precision or effort.

DECISION RULE

Do not interpret a surviving mutant until an expert has ruled on realism, equivalence, and the expected propagation path.

EVIDENCE

- [R28] Jia and Harman mutation survey
- [R29] Papadakis et al. mutation advances

RESEARCH EXPANSION

Controlled experimental design

define units, treatments, baselines, randomization, task sampling, reviewer blinding, stopping criteria, and retained artifacts.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must define units, treatments, baselines, randomization, task sampling, reviewer blinding, stopping criteria, and retained artifacts; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

define units, treatments, baselines, randomization, task sampling, reviewer blinding, stopping criteria, and retained artifacts. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Choose a realistic semantic fault and define the operator.
- 2 Generate one controlled mutant while preserving syntax.
- 3 Run the full artifact and verification pipeline independently.
- 4 Classify detection, propagation, over-change, and equivalence.
- 5 Retain artifacts and uncertainty for replication and review.

EVIDENCE TO RETAIN

- OPERATOR
- MUTANT
- DETECTIONS
- ADJUDICATION

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- Mutants are grammatical but implausible in the domain.
- Equivalent mutants are counted as process failures.
- Generated tests share the same altered interpretation.
- Only kill rate is reported, without propagation precision or effort.

DECISION RULE

Do not interpret a surviving mutant until an expert has ruled on realism, equivalence, and the expected propagation path.

EVIDENCE

[R20] Rafique and Misic meta-analysis

[R28] Jia and Harman mutation survey

[R30] Formal RE and LLM roadmap

RESEARCH EXPANSION

Metrics and statistical interpretation

report kill rate, propagation recall, propagation precision, review effort, repair cycles, confidence intervals, and qualitative failure explanations.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To report kill rate, propagation recall, propagation precision, review effort, repair cycles, confidence intervals, and qualitative failure explanations, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

report kill rate, propagation recall, propagation precision, review effort, repair cycles, confidence intervals, and qualitative failure explanations. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Choose a realistic semantic fault and define the operator.
- 2 Generate one controlled mutant while preserving syntax.
- 3 Run the full artifact and verification pipeline independently.
- 4 Classify detection, propagation, over-change, and equivalence.
- 5 Retain artifacts and uncertainty for replication and review.

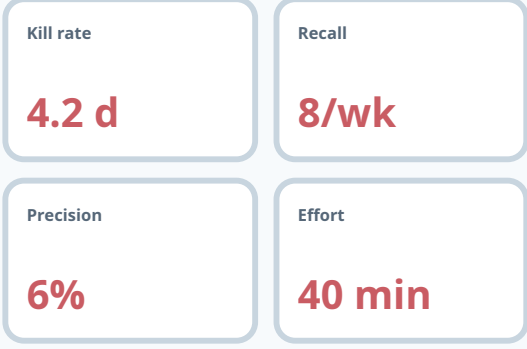
EVIDENCE TO RETAIN

- OPERATOR
- MUTANT
- DETECTIONS
- ADJUDICATION

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Goal: reduce ambiguity-driven rework
Signal: verified tasks rarely reopen
Metric: first-pass verification rate
Guardrail: team pulse + change failure rate

FAILURE MODE / LIMIT

Turning a proxy into an individual target invites gaming. Use trends at the service or team level, pair delivery measures with perception, and inspect the causal story.

WARNING SIGNS

- Mutants are grammatical but implausible in the domain.
- Equivalent mutants are counted as process failures.
- Generated tests share the same altered interpretation.
- Only kill rate is reported, without propagation precision or effort.

DECISION RULE

Do not interpret a surviving mutant until an expert has ruled on realism, equivalence, and the expected propagation path.

EVIDENCE

- [R20] Rafique and Misis meta-analysis
- [R39] SPACE framework

RESEARCH EXPANSION

A research agenda for AI-oriented SDD

study semantic preservation, artifact persistence, adversarial ambiguity, independent oracle generation, brownfield validity, and human-agent governance.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to study semantic preservation, artifact persistence, adversarial ambiguity, independent oracle generation, brownfield validity, and human-agent governance without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

study semantic preservation, artifact persistence, adversarial ambiguity, independent oracle generation, brownfield validity, and human-agent governance. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 Choose a realistic semantic fault and define the operator.
- 2 Generate one controlled mutant while preserving syntax.
- 3 Run the full artifact and verification pipeline independently.
- 4 Classify detection, propagation, over-change, and equivalence.
- 5 Retain artifacts and uncertainty for replication and review.

EVIDENCE TO RETAIN

OPERATOR

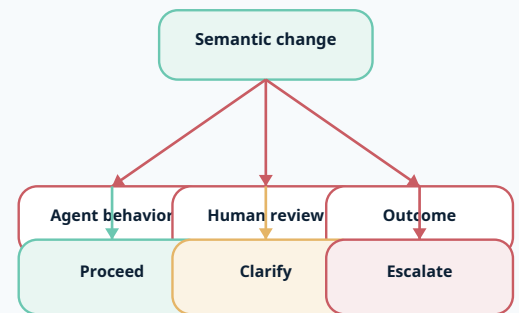
MUTANT

DETECTIONS

ADJUDICATION

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM**WORKED EXAMPLE**

```
ACTIVE_TASK: T3
ALLOWED_PATHS: src/refunds/**, tests/refunds/**
REQUIRED_GATES: unit, BDD, OpenAPI
STOP_ON: ambiguity, contract conflict, scope breach
```

FAILURE MODE / LIMIT

A prompt is advisory. Without executable gates, repository permissions, and independent verification, an agent can satisfy the apparent task while violating scope or hidden intent.

WARNING SIGNS

- Mutants are grammatical but implausible in the domain.
- Equivalent mutants are counted as process failures.
- Generated tests share the same altered interpretation.
- Only kill rate is reported, without propagation precision or effort.

DECISION RULE

Do not interpret a surviving mutant until an expert has ruled on realism, equivalence, and the expected propagation path.

EVIDENCE

[R30] Formal RE and LLM roadmap

[R31] LLMs in requirements engineering SLR

[R32] LLMs for software engineering SLR

PART XVI

Metrics, adoption, and governance

Connect practices to a causal story, combine delivery and human measures, and keep proxies from becoming targets that damage the system.

WHAT THIS PART ANSWERS

What is the baseline? Which mechanism should move each metric?
What evidence would cause the team to stop or change the pilot?

USE IT WHEN

You are planning organizational adoption, comparing teams over time, or presenting evidence to leadership.

RESEARCH EXPANSION

DORA's current five metrics

measure throughput and instability with change lead time, deployment frequency, failed deployment recovery time, change fail rate, and deployment rework rate.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first measure throughput and instability with change lead time, deployment frequency, failed deployment recovery time, change fail rate, and deployment rework rate, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

measure throughput and instability with change lead time, deployment frequency, failed deployment recovery time, change fail rate, and deployment rework rate. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 State the improvement goal without embedding a target number.
- 2 Describe the mechanism and observable signal of progress.
- 3 Choose a small balanced set of team- or service-level metrics.
- 4 Add human, quality, and risk guardrails against optimization.
- 5 Review trends, context, and unintended behavior regularly.

EVIDENCE TO RETAIN

GOAL

SIGNAL

METRIC

GUARDRAIL

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

Lead time

4.2 d

Frequency

8/wk

Failure

6%

Recovery

40 min

WORKED EXAMPLE

Goal: reduce ambiguity-driven rework

Signal: verified tasks rarely reopen

Metric: first-pass verification rate

Guardrail: team pulse + change failure rate

FAILURE MODE / LIMIT

Turning a proxy into an individual target invites gaming. Use trends at the service or team level, pair delivery measures with perception, and inspect the causal story.

WARNING SIGNS

- A single number is used to rank individuals or teams.
- Activity is treated as outcome and context is ignored.
- Targets change behavior faster than the system improves.
- Illustrative pilot numbers are presented as measured evidence.

DECISION RULE

Retire or redesign a metric when people optimize the proxy while the stated goal or human experience deteriorates.

EVIDENCE

[R36] DORA software delivery metrics

[R43] Accelerate

RESEARCH EXPANSION

SPACE as a multidimensional lens

select a small balanced set across satisfaction, performance, activity, communication, and efficiency, including a perceptual measure.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must select a small balanced set across satisfaction, performance, activity, communication, and efficiency, including a perceptual measure; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

select a small balanced set across satisfaction, performance, activity, communication, and efficiency, including a perceptual measure. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1
- State the improvement goal without embedding a target number.
- 2
- Describe the mechanism and observable signal of progress.
- 3
- Choose a small balanced set of team- or service-level metrics.
- 4
- Add human, quality, and risk guardrails against optimization.
- 5
- Review trends, context, and unintended behavior regularly.

EVIDENCE TO RETAIN

- GOAL
- SIGNAL
- METRIC
- GUARDRAIL

REVIEW QUESTIONS

-
- What uncertainty is removed before implementation?
-
- Which artifact is authoritative, and who approves it?
-
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

S	P	A	C
E	S	P	A
C	E	S	P
A	C	E	S

WORKED EXAMPLE

Goal: reduce ambiguity-driven rework
Signal: verified tasks rarely reopen
Metric: first-pass verification rate
Guardrail: team pulse + change failure rate

FAILURE MODE / LIMIT

Turning a proxy into an individual target invites gaming. Use trends at the service or team level, pair delivery measures with perception, and inspect the causal story.

WARNING SIGNS

-
- A single number is used to rank individuals or teams.
-
- Activity is treated as outcome and context is ignored.
-
- Targets change behavior faster than the system improves.
-
- Illustrative pilot numbers are presented as measured evidence.

DECISION RULE

Retire or redesign a metric when people optimize the proxy while the stated goal or human experience deteriorates.

EVIDENCE

[R39] SPACE framework

RESEARCH EXPANSION

DevEx: feedback, cognitive load, and flow

pair developer perceptions with workflow measures and avoid reducing experience to tool telemetry.

DETAILED INTERPRETATION

This is a boundary-management problem as much as a writing problem. To pair developer perceptions with workflow measures and avoid reducing experience to tool telemetry, define what is authoritative, what may change, which assumptions are provisional, and which failures reopen the specification rather than being patched silently in code.

HOW IT OPERATES

pair developer perceptions with workflow measures and avoid reducing experience to tool telemetry. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 State the improvement goal without embedding a target number.
- 2 Describe the mechanism and observable signal of progress.
- 3 Choose a small balanced set of team- or service-level metrics.
- 4 Add human, quality, and risk guardrails against optimization.
- 5 Review trends, context, and unintended behavior regularly.

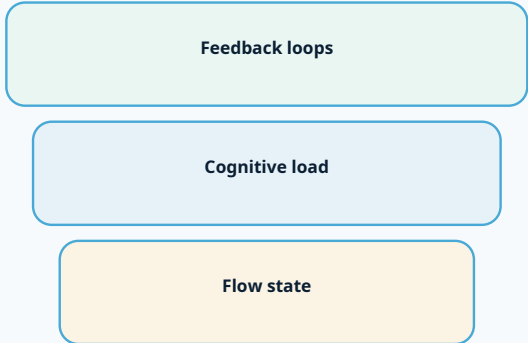
EVIDENCE TO RETAIN

- GOAL
- SIGNAL
- METRIC
- GUARDRAIL

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Goal: reduce ambiguity-driven rework
Signal: verified tasks rarely reopen
Metric: first-pass verification rate
Guardrail: team pulse + change failure rate

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A single number is used to rank individuals or teams.
- Activity is treated as outcome and context is ignored.
- Targets change behavior faster than the system improves.
- Illustrative pilot numbers are presented as measured evidence.

DECISION RULE

Retire or redesign a metric when people optimize the proxy while the stated goal or human experience deteriorates.

EVIDENCE

- [R40] DevEx framework
- [R39] SPACE framework

RESEARCH EXPANSION

GSM and Goodhart-aware design

state the goal in words, identify a signal, choose a proxy, add guardrails, and revisit behavior when the metric becomes a target.

DETAILED INTERPRETATION

The operational test is simple: can a reviewer use this artifact to state the goal in words, identify a signal, choose a proxy, add guardrails, and revisit behavior when the metric becomes a target without relying on private memory? If not, the page may be informative, but it is not yet dependable enough to drive SDD.

HOW IT OPERATES

state the goal in words, identify a signal, choose a proxy, add guardrails, and revisit behavior when the metric becomes a target. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 State the improvement goal without embedding a target number.
- 2 Describe the mechanism and observable signal of progress.
- 3 Choose a small balanced set of team- or service-level metrics.
- 4 Add human, quality, and risk guardrails against optimization.
- 5 Review trends, context, and unintended behavior regularly.

EVIDENCE TO RETAIN

GOAL

SIGNAL

METRIC

GUARDRAIL

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A single number is used to rank individuals or teams.
- Activity is treated as outcome and context is ignored.
- Targets change behavior faster than the system improves.
- Illustrative pilot numbers are presented as measured evidence.

DECISION RULE

Retire or redesign a metric when people optimize the proxy while the stated goal or human experience deteriorates.

EVIDENCE

[R41] Software Engineering at Google GSM

[R42] Strathern on Goodhart effects

RESEARCH EXPANSION

A staged adoption roadmap

pilot one feature, establish templates and gates, instrument the baseline, expand by risk, and retire process steps that do not create evidence.

DETAILED INTERPRETATION

In this part of the lifecycle, the practical objective is to pilot one feature, establish templates and gates, instrument the baseline, expand by risk, and retire process steps that do not create evidence. The value to SDD is not the presence of another document; it is the reduction of a specific uncertainty before that uncertainty is multiplied through plans, code, tests, and release decisions.

HOW IT OPERATES

pilot one feature, establish templates and gates, instrument the baseline, expand by risk, and retire process steps that do not create evidence. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 State the improvement goal without embedding a target number.
- 2 Describe the mechanism and observable signal of progress.
- 3 Choose a small balanced set of team- or service-level metrics.
- 4 Add human, quality, and risk guardrails against optimization.
- 5 Review trends, context, and unintended behavior regularly.

EVIDENCE TO RETAIN

GOAL

SIGNAL

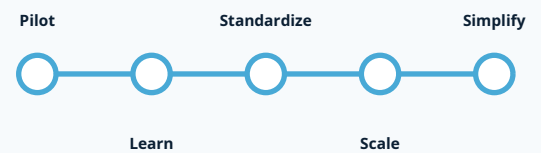
METRIC

GUARDRAIL

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A single number is used to rank individuals or teams.
- Activity is treated as outcome and context is ignored.
- Targets change behavior faster than the system improves.
- Illustrative pilot numbers are presented as measured evidence.

DECISION RULE

Retire or redesign a metric when people optimize the proxy while the stated goal or human experience deteriorates.

EVIDENCE

[R36] DORA software delivery metrics

[R49] Scrum Guide 2020

[R50] Little's Law

RESEARCH EXPANSION

Governance and RACI

assign accountability for domain policy, requirement approval, architecture, security, test evidence, release, and specification maintenance.

DETAILED INTERPRETATION

The concept should be treated as an evidence-producing practice. Teams first assign accountability for domain policy, requirement approval, architecture, security, test evidence, release, and specification maintenance, then attach an owner, a review rule, and an observable check. This turns intent into a control surface that humans and coding agents can both challenge.

HOW IT OPERATES

assign accountability for domain policy, requirement approval, architecture, security, test evidence, release, and specification maintenance. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1
- State the improvement goal without embedding a target number.
- 2
- Describe the mechanism and observable signal of progress.
- 3
- Choose a small balanced set of team- or service-level metrics.
- 4
- Add human, quality, and risk guardrails against optimization.
- 5
- Review trends, context, and unintended behavior regularly.

EVIDENCE TO RETAIN

- GOAL
- SIGNAL
- METRIC
- GUARDRAIL

REVIEW QUESTIONS

-
- What uncertainty is removed before implementation?
-
- Which artifact is authoritative, and who approves it?
-
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM

PO	Engineer	QA	Security
Ops	PO	Engineer	QA
Security	Ops	PO	Engineer
QA	Security	Ops	PO

WORKED EXAMPLE

Trace: R-2 -> BR-4 -> SC-3 -> API-1 -> T3 -> TEST-12 -> EVID-7

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

-
- A single number is used to rank individuals or teams.
-
- Activity is treated as outcome and context is ignored.
-
- Targets change behavior faster than the system improves.
-
- Illustrative pilot numbers are presented as measured evidence.

DECISION RULE

Retire or redesign a metric when people optimize the proxy while the stated goal or human experience deteriorates.

RESEARCH EXPANSION

An honest pilot dashboard

label demonstration values as illustrative, compare trends within the same service, and pair first-pass verification with delivery and human measures.

DETAILED INTERPRETATION

A strong implementation makes this concern visible in the artifact graph. The team must label demonstration values as illustrative, compare trends within the same service, and pair first-pass verification with delivery and human measures; downstream tasks should cite the relevant identifier, and verification should show whether the promised behavior actually survived implementation.

HOW IT OPERATES

label demonstration values as illustrative, compare trends within the same service, and pair first-pass verification with delivery and human measures. This concern should be visible at the point where the decision is made, not reconstructed later from code or conversation history. Assign a stable identifier, record the authority behind the decision, and define what downstream artifact or runtime observation can contradict it.

RELATIONSHIP TO SDD

Within a disciplined SDD workflow, the artifact is reviewed before implementation, referenced by the plan and task, and checked again during verification. For AI-assisted work, the same information belongs in the agent's bounded context packet; the agent may propose changes, but it must stop when policy or artifacts conflict.

PRACTICAL PROCEDURE

- 1 State the improvement goal without embedding a target number.
- 2 Describe the mechanism and observable signal of progress.
- 3 Choose a small balanced set of team- or service-level metrics.
- 4 Add human, quality, and risk guardrails against optimization.
- 5 Review trends, context, and unintended behavior regularly.

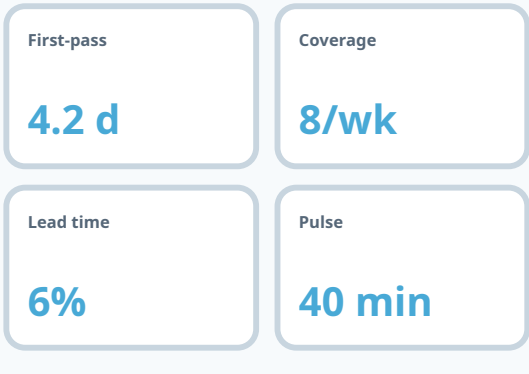
EVIDENCE TO RETAIN

- GOAL
- SIGNAL
- METRIC
- GUARDRAIL

REVIEW QUESTIONS

- What uncertainty is removed before implementation?
- Which artifact is authoritative, and who approves it?
- What independent evidence would expose a wrong interpretation?

CONCEPT DIAGRAM



WORKED EXAMPLE

Goal: reduce ambiguity-driven rework
Signal: verified tasks rarely reopen
Metric: first-pass verification rate
Guardrail: team pulse + change failure rate

FAILURE MODE / LIMIT

The artifact may exist but still be unauthoritative, stale, unlinked, or too vague to drive a decision. Ownership and verification are part of the design.

WARNING SIGNS

- A single number is used to rank individuals or teams.
- Activity is treated as outcome and context is ignored.
- Targets change behavior faster than the system improves.
- Illustrative pilot numbers are presented as measured evidence.

DECISION RULE

Retire or redesign a metric when people optimize the proxy while the stated goal or human experience deteriorates.

EVIDENCE

- [R36] DORA software delivery metrics
- [R39] SPACE framework
- [R40] DevEx framework

Reusable templates

Turn the method into small, reviewable artifacts that can be copied into a repository and adapted to local risk and workflow.

WHAT THIS PART ANSWERS

Which fields are mandatory? Which gates are automated? How are authority, scope, tests, agent permissions, and deviations recorded?

USE IT WHEN

You are ready to pilot one feature and need practical starting material.

RESEARCH EXPANSION

Task, traceability, and verification template

define one criterion, one contract, scope, exclusions, dependencies, allowed resources, failing test, evidence, deviations, and verdict.

Field	Example for PLT-4821
Task	T3 - implement cancellation refund calculation
From	EARS-2; Business Rule BR-4; ADR-017
Contract	POST /orders/{id}/refunds; scenario SC-3
In	Money, RefundCalculator, service orchestration
Out	Notification template, UI, line-item refunds
Allowed paths	src/refunds/**; tests/refunds/**
Red evidence	TEST-12 fails with NotImplemented
Done	unit + BDD + OpenAPI green; trace links complete
Deviation	none, or linked ADR/change request
Verdict	CONFORMS / DOES NOT CONFORM / BLOCKED

EVIDENCE

[R26] Gotel and Finkelstein 1994

[R27] Requirements traceability book

[R1] GitHub Spec Kit documentation

RESEARCH EXPANSION

Review checklists and release gate

provide separate concise checks for product, domain, architecture, QA, security, agent execution, and production readiness.

Gate	Questions
Product	Is the policy authoritative? Are in/out lists and examples agreed?
Domain	Do terms, ownership, invariants, and context boundaries remain coherent?
Architecture	Do ports, data, contracts, migrations, and ADRs support the requirement?
Quality	Did tests fail first? Are errors, boundaries, concurrency, and compatibility covered?
Security	Are identity, authorization, data handling, audit, and threat assumptions tested?
Agent	Did work stay inside scope? Were conflicts escalated? Was the oracle independent?
Operations	Are rollout, telemetry, SLOs, rollback, and reconciliation ready?
Release	Does verification show conformance for every in-scope criterion?

EVIDENCE

[R5] ISO/IEC/IEEE 29148:2018

[R44] OWASP ASVS

[R36] DORA software delivery metrics

Sources and closing synthesis

Return every major claim to its source class, preserve uncertainty, and finish with a compact operating model for professional practice.

WHAT THIS PART ANSWERS

Which sources define methods? Which report outcomes? Which describe current tools? Which claims remain hypotheses for future research?

USE IT WHEN

You need citations, further reading, or a defensible summary of what is established and what is emerging.

RESEARCH EXPANSION

Annotated bibliography N-Z

R27

Cleland-Huang, J., Gotel, O., and Zisman, A. (eds.). Software and Systems Traceability. Springer, 2012. DOI 10.1007/978-1-4471-2239-5.

doi.org/10.1007/978-1-4471-2239-5

R28

Jia, Y. and Harman, M. An Analysis and Survey of the Development of Mutation Testing. IEEE TSE 37(5), 2011, 649-678. DOI 10.1109/TSE.2010.62.

doi.org/10.1109/TSE.2010.62

R29

Papadakis, M. et al. Mutation Testing Advances: An Analysis and Survey. Advances in Computers 112 (2019), 275-378. DOI 10.1016/bs.adcom.2018.03.015.

doi.org/10.1016/bs.adcom.2018.03.015

R30

Ferrari, A. and Spoletini, P. Formal requirements engineering and large language models: A two-way roadmap. Information and Software Technology 181 (2025), 107697. DOI 10.1016/j.infsof.2025.107697.

doi.org/10.1016/j.infsof.2025.107697

R31

Hemmat, A. et al. Research directions for using LLMs in software requirement engineering: a systematic review. Frontiers in Computer Science (2025). DOI 10.3389/fcomp.2025.1519437.

doi.org/10.3389/fcomp.2025.1519437

R32

Systematic literature review of large language models for software engineering. ACM Transactions on Software Engineering and Methodology. DOI 10.1145/3695988.

doi.org/10.1145/3695988

R33

METR. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. Randomized controlled trial; reported 19 percent longer task completion in its studied setting.

metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-s...

R34

METR. We Are Changing Our Developer Productivity Experiment Design, 2026. Discusses selection effects and weak evidence for later speedup estimates.

metr.org/blog/2026-02-24-uplift-update/

R35

Randomized controlled trial of generative AI coding assistance in an enterprise setting; preprint arXiv:2410.12944. Reported an estimated productivity improvement with uncertainty and contextual limitations.

arxiv.org/abs/2410.12944

R36

DORA. Software delivery performance metrics. Current five-metric model: change lead time, deployment frequency, failed deployment recovery time, change fail rate, and deployment rework rate.

dora.dev/guides/dora-metrics/

R37

DORA. Loosely coupled teams capability: independent change, test, and deployment enabled by technical and organizational architecture.

dora.dev/capabilities/loosely-coupled-teams/

R38

DORA. Documentation quality capability: clarity, findability, reliability, and the relationship between documentation and technical practices.

dora.dev/capabilities/documentation-quality/

R39

Forsgren, N. et al. The SPACE of Developer Productivity: There's more to it than you think. ACM Queue 19(1), 2021. DOI 10.1145/3454122.3454124.

doi.org/10.1145/3454122.3454124

R40

Noda, A., Storey, M.-A., Forsgren, N., and Greiler, M. DevEx: What Actually Drives Productivity? Communications of the ACM 66(11), 2023. DOI 10.1145/3610285.

doi.org/10.1145/3610285

R41

Winters, T., Manshreck, T., and Wright, H. Software Engineering at Google. Chapter on measuring engineering productivity using Goals, Signals, and Metrics.

abseil.io/resources/swe-book/html/ch07.html

R42

Strathern, M. Improving ratings: audit in the British University system. European Review 5(3), 1997. DOI 10.1002/(SICI)1234-981X(199707)5:3<305::AID-EURO184>3.0.CO;2-4.

doi.org/10.1002/(SICI)1234-981X(199707)5:3%3C305::AID-EURO1...

R43

Forsgren, N., Humble, J., and Kim, G. Accelerate: The Science of Lean Software and DevOps. IT Revolution, 2018. ISBN 9781942788331.

itrevolution.com/product/accelerate/

R44

OWASP. Application Security Verification Standard (ASVS): testable security requirements for web applications and services.

owasp.org/www-project-application-security-verification-sta...

R45

W3C. Web Content Accessibility Guidelines (WCAG) 2.2, W3C Recommendation.

www.w3.org/TR/WCAG22/

R46

Beyer, B. et al. The Site Reliability Workbook. O'Reilly/Google, 2018; practical SLOs, monitoring, and safe change.

sre.google/workbook/table-of-contents/

R47

OpenTelemetry. Specifications for traces, metrics, logs, context propagation, and semantic conventions.

opentelemetry.io/docs/specs/

R48

OpenFeature. Vendor-neutral feature flagging specification and evaluation model.

openfeature.dev/specification/

R49

Schwaber, K. and Sutherland, J. The Scrum Guide, 2020.

scrumguides.org/scrum-guide.html

R50

Little, J. D. C. A Proof for the Queuing Formula $L = \lambda W$. Operations Research 9(3), 1961, 383-387. DOI 10.1287/opre.9.3.383.

doi.org/10.1287/opre.9.3.383

R51

Jackson, D. Software Abstractions: Logic, Language, and Analysis, 2nd ed. MIT Press, 2012. ISBN 9780262528900.

mitpress.mit.edu/9780262528900/software-abstractions/

R52

Lampert, L. Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley, 2002. Free author-hosted edition.

lampert.azurewebsites.net/tla/book.html

Spec first. Verify independently.

Let AI accelerate search, drafting, implementation, and repair—without allowing generated output to redefine the requirement, weaken the oracle, or silently expand the task.

